ECE 1140 · WORK PACKAGE 2 · DESIGN REVIEW

Software Design Description

For the Railway Simulation Project

Version 1.0

Issued for the design review of 2026-10-22

IEEE Std 1016-2009

MODULE	OWNER
System integration	Braden Bosilovich
CTC Office	Landis
Track Controller (SW)	Harry Tye
Track Controller (HW)	Braden Bosilovich
Track Model	Deni Ephraim
Train Model	Kevin Schillinger
Train Controller (SW and HW)	Jonathan Tsang, Ivan Zheng

Team 3, Track Overflow
University of Pittsburgh · ECE 1140
Prepared for the Port Authority of Allegheny County

Table of Contents

- Revision History
- 1. Introduction
  - 1.1 Document Purpose
  - 1.2 Subject Scope
  - 1.3 Definitions, Acronyms, and Abbreviations
  - 1.4 References
  - 1.5 Document Overview
- 2. Design Overview
  - 2.1 Stakeholder Concerns
  - 2.2 Selected Viewpoints
- 3. Design Views
  - 3.1 001-context: System Context
  - 3.2 002-composition: Software Architecture
  - 3.3 003-logical: Boundary Types
  - 3.4 004-interface: Harness Protocol and Edges
  - 3.5 005-interaction: Representative Interaction, Movement Authority
  - 3.6 006-deployment: Deployment
  - 3.7 Key System-Wide Design Concepts
    - 3.7.1 Route
    - 3.7.2 Authority
    - 3.7.3 Distribution of wayside controllers
    - 3.7.4 Simulation timing
    - 3.7.5 Vitality
    - 3.7.6 Wayside language
    - 3.7.7 Hardware/software interface
- 4. Decisions
  - 4.1 Interface Contract Mechanism
  - 4.2 Shared Boundary Data Types
  - 4.3 Contract Enforcement
  - 4.4 Stand-ins and Stub-First Development
  - 4.5 Contracts Ownership
  - 4.6 External Input Validation
  - 4.7 Knowledge Capture and Decision Logging
  - 4.8 Rejected Tooling: Summary Table
  - 4.9 Open Items: Not Yet Decided
- 5. Module Designs
  - 5.1 CTC Office
    - 5.1.1 Scope

        - 5.1.2 Requirements coverage
        - 5.1.3 Use cases
        - 5.1.4 CTC Office architecture (component view)
        - 5.1.5 Interface
        - 5.1.6 Class diagrams
        - 5.1.7 Sequence diagrams
        - 5.1.8 Open items
    - 5.2 Track Controller (SW)
        - 5.2.1 Scope
        - 5.2.2 Requirements coverage
        - 5.2.3 Use cases
        - 5.2.4 Track Controller (SW) architecture (component view)
        - 5.2.5 Class diagrams
        - 5.2.6 Sequence diagrams
        - 5.2.7 Open items
    - 5.3 Track Controller (HW)
        - 5.3.1 Scope
        - 5.3.2 Requirements coverage
        - 5.3.3 Use cases
        - 5.3.4 Track Controller (HW) architecture (component view)
        - 5.3.5 Class diagrams
        - 5.3.6 Sequence diagrams
        - 5.3.7 Open items
    - 5.4 Track Model
        - 5.4.1 Stakeholder concerns
        - 5.4.2 Use cases
        - 5.4.3 Track Model architecture (component view)
        - 5.4.4 Boundary types
        - 5.4.5 Class diagrams
        - 5.4.6 Interface
        - 5.4.7 Interface dictionary
        - 5.4.8 Data design
        - 5.4.9 Algorithms
        - 5.4.10 User interface
        - 5.4.11 Test interface
        - 5.4.12 Sequence diagrams
        - 5.4.13 State machine
        - 5.4.14 Design decisions
        - 5.4.15 Requirements traceability
    - 5.5 Train Model
        - 5.5.1 Scope

        - 5.5.2 Stakeholder concerns
        - 5.5.3 Train Model in the system
        - 5.5.4 Use cases
        - 5.5.5 Train Model architecture (component view)
        - 5.5.6 Class diagrams
        - 5.5.7 Interface
        - 5.5.8 Interaction: one simulation tick
        - 5.5.9 Design decisions
        - 5.5.10 Review items and deferred work
        - 5.5.11 Requirements traceability
        - 5.5.12 Verification references
    - 5.6 Train Controller (SW and HW)
        - 5.6.1 Scope
        - 5.6.2 Stakeholder concerns
        - 5.6.3 Train Controller in the system
        - 5.6.4 Use cases
        - 5.6.5 Train Controller architecture (component view)
        - 5.6.6 Class diagrams
        - 5.6.7 Interface
        - 5.6.8 Interaction: one tick
        - 5.6.9 Speed law and safety path
        - 5.6.10 Failure inference
        - 5.6.11 Automatic behaviour
        - 5.6.12 Design decisions
        - 5.6.13 Open items
- 6. Appendixes
    - 6.1 Figure Register
    - 6.2 Requirements Traceability Index
    - 6.3 Module Figures Replaced by System Figures

Revision History

Name	Date	Reason for changes	Version
Each module owner	2026-10-20 to 2026-10-22	Module contributions: CTC Office (Landis), Track Controller SW (Harry Tye), Track Controller HW (Braden Bosilovich), Track Model (Deni Ephraim), Train Model (Kevin Schillinger), Train Controller (Jonathan Tsang, Ivan Zheng)	0.x
Ivan Zheng	2026-10-21	System architecture figure	0.9
Braden Bosilovich	2026-10-22	Combined SDD: system views, software architecture, deployment, representative interaction and system-wide concepts written once for the system; module contributions brought in as Section 5	1.0

1. Introduction

1.1 Document Purpose

This document describes the design of the Railway Simulation: how the system is divided into modules, how they exchange data, the concepts every module relies on, and the design of each module. It follows IEEE Std 1016-2009. Its readers are the team, the integrator of the central harness, and the course staff who review the design.

1.2 Subject Scope

The system being designed is the Railway Simulation, version 1.0: a simulation of a light rail system for the Port Authority of Allegheny County, run on one Windows 11 PC with two Raspberry Pi 5 boards attached by USB. The system has five modules, two of which have a software and a hardware variant, so seven module processes run:

Module	Variant	Line	Runs on	Owner
CTC Office	one	Green and Red	Host PC	Landis
Track Controller	SW	Green	Host PC	Harry Tye
Track Controller	HW	Red	Raspberry Pi 5, window on the host PC	Braden Bosilovich
Track Model	one	Green and Red	Host PC	Deni Ephraim
Train Model	one	Green and Red	Host PC	Kevin Schillinger
Train Controller	SW	Green	Host PC	Jonathan Tsang, Ivan Zheng
Train Controller	HW	Red	Raspberry Pi 5, window on the host PC	Jonathan Tsang, Ivan Zheng

A central harness, which is not a module, starts every module process, owns the one simulation clock, maps every module's outputs to its consumers' inputs and writes the event log (D005, D017).

In scope:

- The system's composition, its interfaces and the central harness (Section 3).
- The key system-wide concepts: route, authority, wayside distribution, timing, vitality, the wayside language and the hardware/software interface (Section 3.7).
- The design of every module: use cases, architecture, classes, interfaces, sequences and the module's own decisions (Section 5).

Out of scope:

- The Blue Line, which is not part of the final product (D023).
- Moving Block Overlay (D001).
- The requirements themselves (the SRS), the source code, and the test plans and results.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
Authority	The number of blocks ahead of a train's current block that it may still enter before it must stop; a non-negative integer (D013). 0 means stop before leaving the current block.
Beacon	Station data heard near a station: station name, platform side and whether the track is underground.
Block	The smallest unit of track. At most one train occupies it. Named by its line and number.
Commanded speed	The speed, in whole m/s, the Track Controller sends down a block's track circuit.
CTC	Centralized Traffic Control: the CTC Office, which dispatches trains and suggests speed and authority.
Edge	The connection from a producing module to a consuming one. The harness owns one mapping per edge.
Fail-safe	The most restrictive state, which a fault drives a vital module to.
Harness	The central harness: the process that starts and steps every module, owns the clock and maps every edge (D005).
HW, SW	The hardware and software variants of the Track Controller and Train Controller (D021).
Line	A complete track layout: Green or Red.
Murphy	The tester persona who injects failures.

Term	Definition
PLC	Programmable logic controller. Each wayside runs a Boolean PLC program.
Protocol	A `typing.Protocol`: a Python structural interface type, used for module contracts.
Route	The sequence of blocks a train runs from the yard to its stations and back, in the order the track reaches them.
SDD, SRS	Software Design Description (this document); Software Requirements Specification.
Suggested speed	The CTC Office's speed for a block, before the wayside decides what to send.
Tick	One simulation step. Its length, dt, is fixed at 0.1 s (D006, D017).
Track circuit	The circuit of one block. It detects a train and carries the commanded speed and authority to it.
Truth	The `truth` branch: the team's record of normative facts (decisions D001 to D027, signals, conventions).
Vital	Safety-critical: must be correct and must fail to a safe state.
Wayside	One wayside controller of the Track Controller. It governs a contiguous run of blocks and its devices.
Yard	Where a line's trains enter and leave service (D022).

Each module chapter adds the terms specific to it.

1.4 References

Reference	Owner / Location	Normative?
IEEE Std 1016-2009, Systems Design: Software Design Descriptions	IEEE	Structure of this document
Software Requirements Specification v1.0	Team / `documents/srs-filled.md`	Yes
Truth store: decisions D001 to D027, signals, arbitration rules, conventions	Team / `truth/` on the `truth` branch	Yes

Reference	Owner / Location	Normative?
System integration design	Kevin Schillinger / `documents/system-integration-design.md`	No (how the harness was built)
SDD template	Team / `documents/Railway_Sim_SDD_Template.md`	Structure of Sections 1 to 4
Module SDD contributions	Each owner / `documents/sdd/*.md`, `documents/Railway_Sim_SDD_Train_*.md`	Sources of Section 5
Final Project deck; Deliverable Descriptions; Track Layout & Vehicle Data	Course / `documents/Project_Information/`	Yes
Coding standard	Team / `documents/PYTHON_STYLE_GUIDE.md`	Yes
UI style guide	Team / `truth/ui/style-guide.md`	Yes, for every window

1.5 Document Overview

Section 2 lists the stakeholders, their concerns and the viewpoints that answer them. Section 3 gives the system-level views and the system-wide concepts. Section 4 records the system-wide design decisions. Section 5 holds one chapter per module, each with its use cases, architecture, classes, interface and sequence diagrams. Section 6 holds the appendixes: the figure register and the requirements traceability index.

Conformance to IEEE 1016-2009.

IEEE 1016 content	Where
Identification: date, status, scope, organization, authorship, change history (4.2)	Cover page, Revision History, 1.2
References, glossary, context (4.2)	1.3, 1.4, 3.1
Stakeholders and design concerns (4.3)	2.1
Design viewpoints (4.4)	2.2
Design views (4.5)	Section 3 (system), Section 5 (modules)

IEEE 1016 content	Where
Design rationale (4.8)	Section 4; each module's design decisions in Section 5

2. Design Overview

2.1 Stakeholder Concerns

Stakeholder	Concern	Addressed by
Customer: Port Authority of Allegheny County	Trains never meet, never pass a stop, never exceed their commanded speed; failures end in a safe state	3.7 (authority, vitality); 5.2, 5.3, 5.6
Dispatcher	Dispatch by hand or by schedule; see occupancy, throughput and notices	5.1
Wayside programmer and maintainer	Load a territory and a PLC program per wayside; see the wayside's state	3.7 (wayside language); 5.2, 5.3
Track Builder	Load a line and inspect its blocks	5.4
Murphy	Inject and clear failures on track and trains	5.4, 5.5
Train driver and engineer	Clear Manual control; gains set once before a train moves	5.6
Passengers	Emergency brake, doors on the platform side, announcements	5.5, 5.6
Module owners	Stable, module-owned boundary types; one place where two modules' types meet	3.3, 3.4; 4.1, 4.2, 4.5
Harness integrator	One call per tick per module; the same behaviour at 1x and 10x; clear rejection	3.4, 3.5, 3.7 (timing); 4.6
Course staff	Requirements traced; the rubric's diagrams present	Section 5; 6.2

2.2 Selected Viewpoints

IEEE 1016 viewpoint	Concerns	System view	Module views
Context	What the system and each module do, and for whom	3.1 System context	Use case diagrams and descriptions
Composition	Parts and their responsibilities	3.2 Software architecture	Module architecture (component view)

IEEE 1016 viewpoint	Concerns	System view	Module views
Logical	Classes and types	3.3 Boundary types	Class diagrams
Interface	What crosses each boundary	3.4 Harness protocol and edges	Interface sections and dictionaries
Interaction	How the parts cooperate over time	3.5 Representative interaction	One sequence diagram per use case
Deployment (user-defined, added for the rubric)	Where each process runs and how it is linked	3.6 Deployment	(system view only)
Algorithm, State dynamics	How the key decisions are computed	3.7 System-wide concepts	CTC routing, Train Controller speed law, Track Model block states

Omitted: Physical, Structure, Dependency, Information, Concurrency, Patterns and Resources; concurrency is reduced to the harness's single tick (3.5).

---

3. Design Views

3.1 001-context: System Context

- **Viewpoint:** Context
- **Representation:**

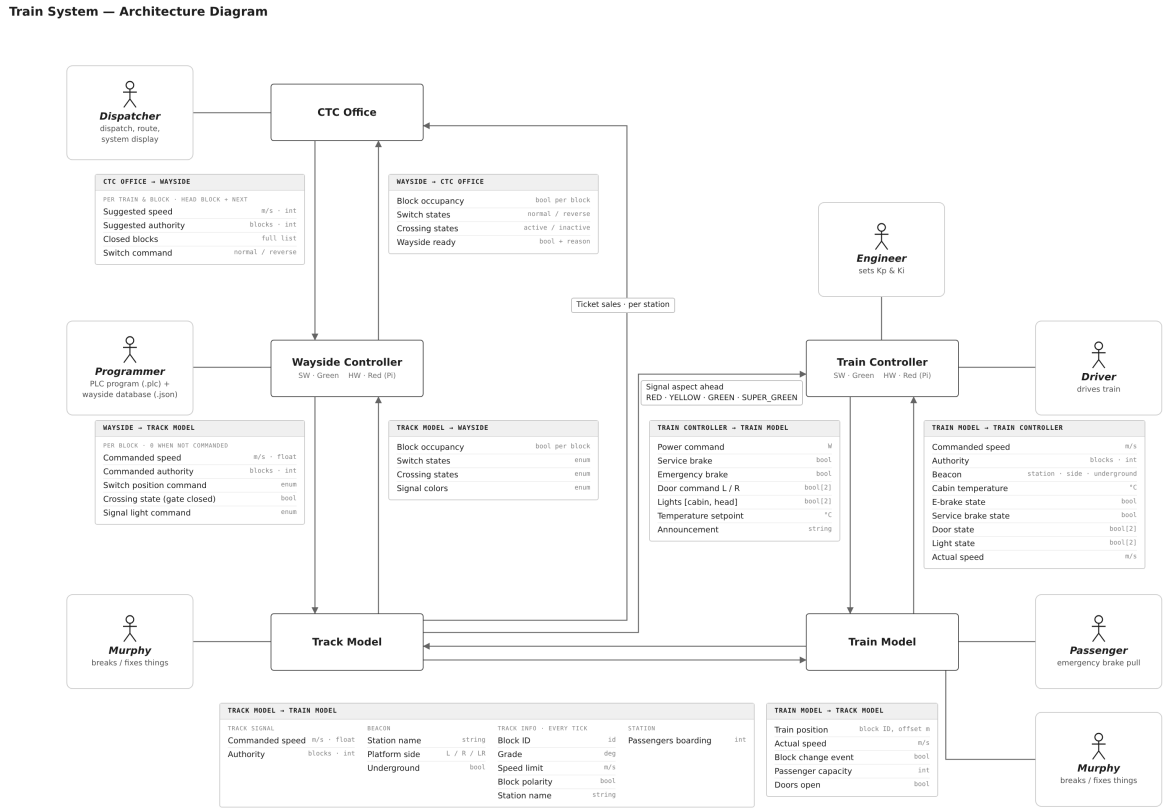

Figure 1. System architecture: modules, users and the information between them

Commands pass down from the CTC Office through the waysides and the track to the train, and status passes back up the same path. The Track Model also sends the signal light ahead straight to the Train Controller, and ticket sales straight to the CTC Office. In the running system every arrow passes through the central harness (D005).

Module	Its users	It receives from	It sends to
CTC Office	Dispatcher	Track Controller (occupancy, switches, crossings, readiness); Track Model (ticket sales)	Track Controller (suggested speed and authority, closures, maintenance switch commands); harness (clock pause and speed)

Module	Its users	It receives from	It sends to
Track Controller SW / HW	Wayside programmer	CTC Office; Track Model (occupancy, switch read-back)	Track Model (switch, signal and crossing commands; speed and authority per track circuit); CTC Office
Track Model	Track Builder, Murphy	Track Controller; Train Model (position)	Track Controller; Train Model (track circuit, grade, beacon, boarding); Train Controller (signal light ahead); CTC Office (ticket sales)
Train Model	Passenger, Murphy	Train Controller (commands); Track Model	Train Controller (speed, brake, door, light, temperature feedback; track circuit and beacon passed through); Track Model (block and offset)
Train Controller SW / HW	Driver, engineer	Train Model; Track Model	Train Model (power, brakes, doors, lights, temperature, announcement)

3.2 002-composition: Software Architecture

- **Viewpoint:** Composition
- **Representation:** the top-level class diagram of the whole system, in two parts.

Part 1: harness and module processes. One executable. Run without arguments it is the central harness; run with `--module <name>` it is one module process. Each module process holds its module and a `ModuleLink` to the harness.

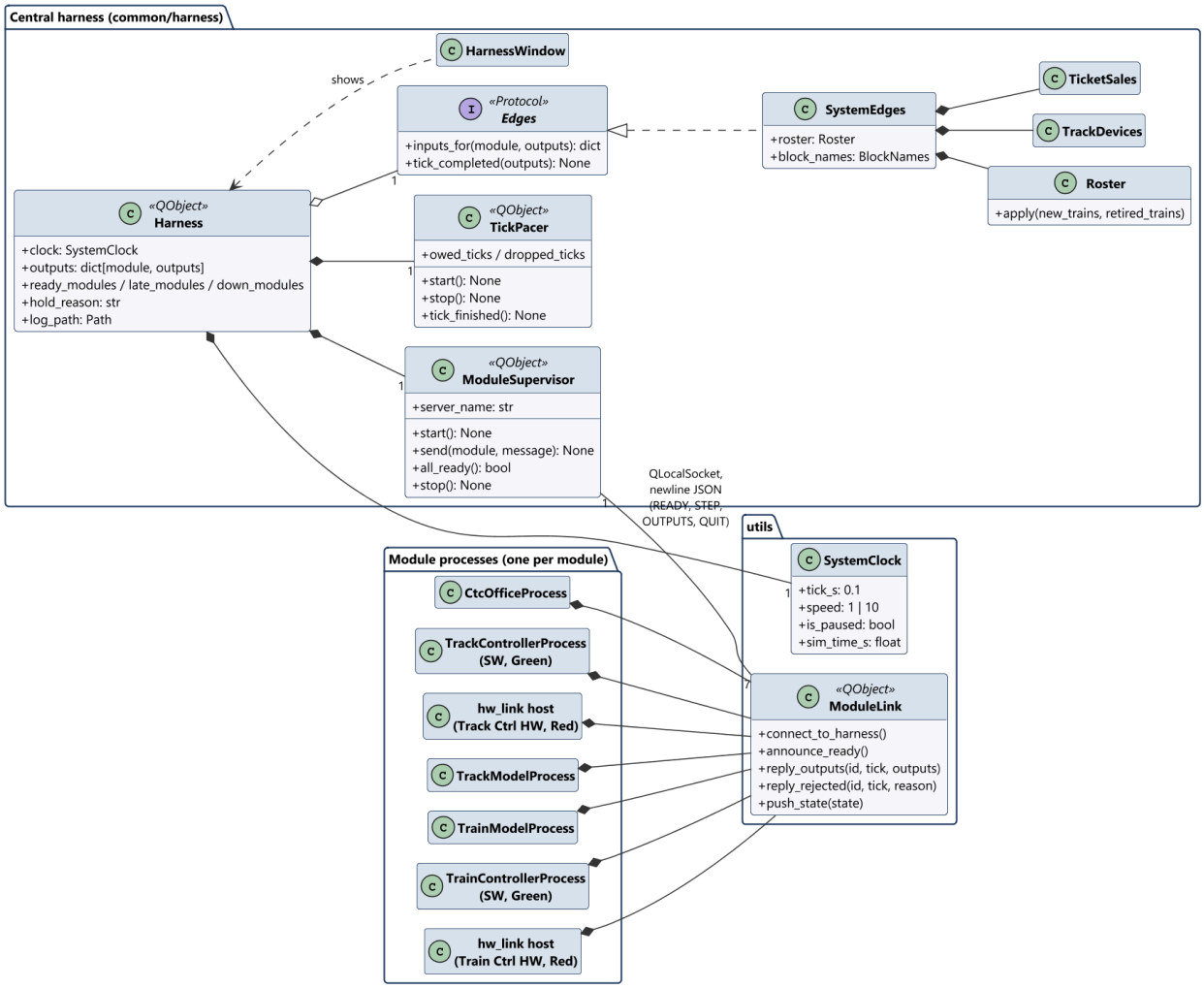

Figure 2. Software architecture, part 1: the central harness, shared utilities and the seven module processes

Part 2: module contracts and their implementations. Each module owns its boundary types and its contract (a `typing.Protocol`); each variant implements it. A HW variant's host process holds a remote proxy, and the module's core runs on its Raspberry Pi behind `PiLinkServer`.

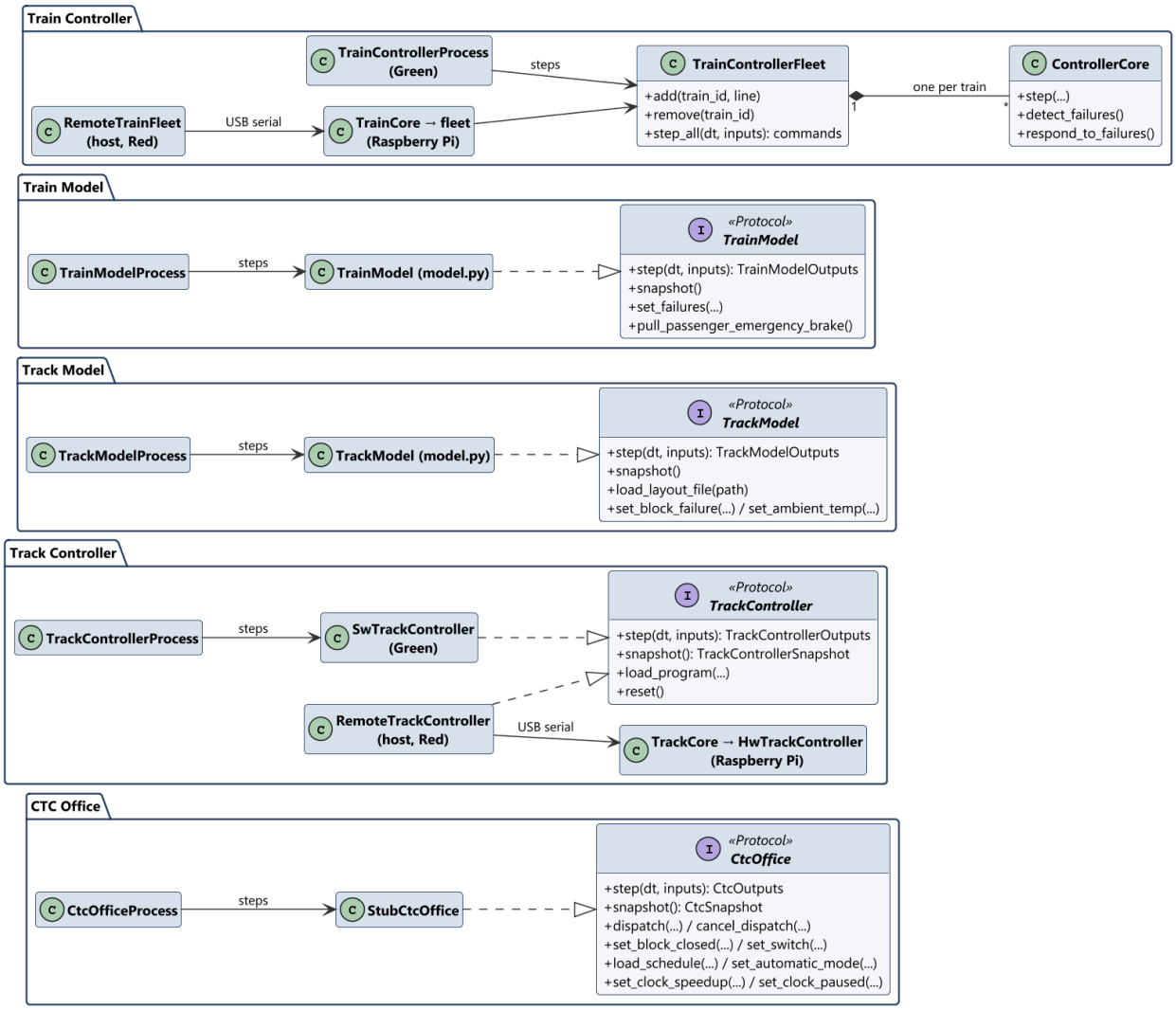

Figure 3. Software architecture, part 2: module contracts, implementations, remote proxies and Pi cores

- **More information:** the parts of each module are in its chapter, under "architecture (component view)". Design patterns used throughout: Mediator (the harness maps every edge), Proxy (the HW hosts), Observer (Qt signals push snapshots to each QML window), and a Qt-free module core under every window.

3.3 003-logical: Boundary Types

- **Viewpoint:** Logical
- **Representation:**
  - **Each module owns its boundary types** and never imports another module's (D005, D008). The stale shared catalog `common/interfaces.py` is not used.
  - Every boundary type is a frozen dataclass or an enumeration, so a value cannot change after it is sent.
  - **Backend units are SI**: m, m/s, m/s^2, W, °C; the windows show imperial units (D002). Authority is a whole number of blocks (D013); commanded speed a whole number of

m/s.
    - **Every reference carries its line**, since block numbers repeat across lines. A switch, signal or crossing is named by the block the layout lists it on.
    - Enumerated values crossing a boundary are in capitals (`conventions/naming`).
    - Speed and authority are **per block**, never per train: the waysides know which blocks are occupied, not which train is in them.
- **More information:** each module's boundary class diagram is in its chapter.

3.4 004-interface: Harness Protocol and Edges

- **Viewpoint:** Interface
- **Representation:** every module process speaks the same protocol to the harness: newline-delimited JSON over a local socket (`QLocalSocket`), each message carrying a request ID and the tick number (D005).

Message	From	Meaning
`READY`	Module	The module is loaded and can step; names the module and the protocol version
`STEP`	Harness	One tick: dt, the unwrapped simulation time at the end of the step, and the module's inputs, mapped from its producers' last outputs
`OUTPUTS`	Module	The reply to a `STEP`: the module's outputs for that tick
`REJECTED`	Module	The reply to a `STEP` the module refused, with a reason; its state did not change
`STATE`	Module	A change made between ticks, such as the CTC Office pausing the clock
`CLOCK_STATE`	Harness	Paused or running, 1x or 10x, sent to the HW hosts initially and on every change
`SHOW`	Harness	Reopen the module's window
`QUIT`	Harness	End the process

The same messages travel over USB serial to a Raspberry Pi (D027). The protocol is `utils/harness_protocol.py`.

For each module, the modules whose outputs it reads (`PRODUCERS`, `common/harness/modules.py`):

Consumer	Producers
CTC Office	Track Controller SW, Track Controller HW, Track Model
Track Controller SW, HW	CTC Office, Track Model
Track Model	Track Controller SW, Track Controller HW, Train Model
Train Model	Track Model, Train Controller SW, Train Controller HW
Train Controller SW, HW	Train Model, Track Model

- **More information:** the harness's edge functions are pure: one per producer and consumer pair, translating one module's types into the other's without changing units. Every field of every edge is listed in the module chapters' interface sections.

3.5 005-interaction: Representative Interaction, Movement Authority

- **Viewpoint:** Interaction
- **Representation:** a train is given authority, across ticks k to $k+5$. Each module steps on the inputs of its tick, which are the outputs of the tick before, so a decision reaches the train a few ticks after it is made.

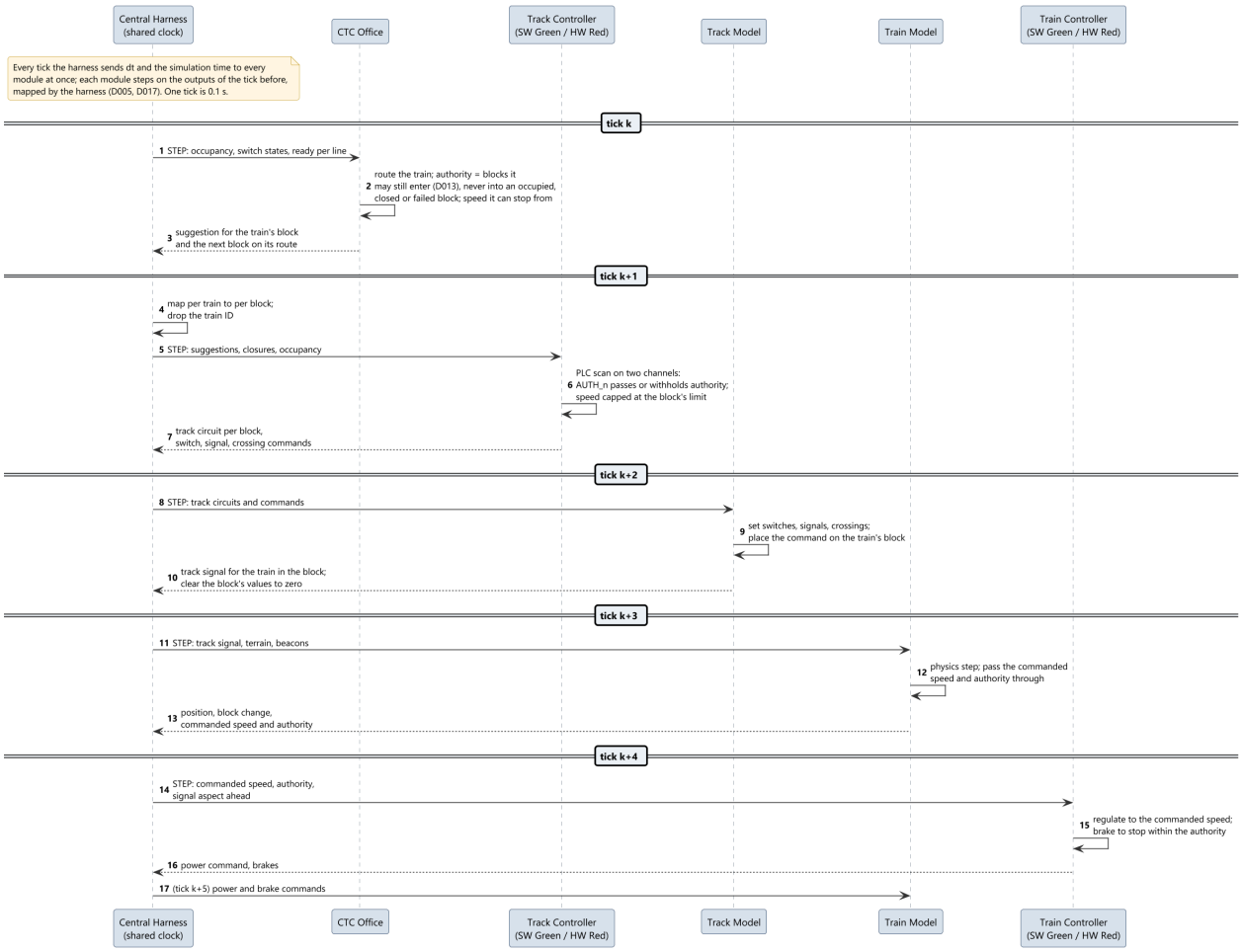

Figure 4. Representative interaction: movement authority from the CTC Office to the train, tick by tick

- **More information:** the harness steps every module on each tick and waits for every reply before the next (D017). Each module's own sequence diagrams, one per use case, are in its chapter.

3.6 006-deployment: Deployment

- **Viewpoint:** Deployment (user-defined)
- **Representation:** one Windows 11 host PC runs the executable as the harness and as each module process; two Raspberry Pi 5 boards run the HW variants' cores and are linked to their host processes by USB (D021, D027).

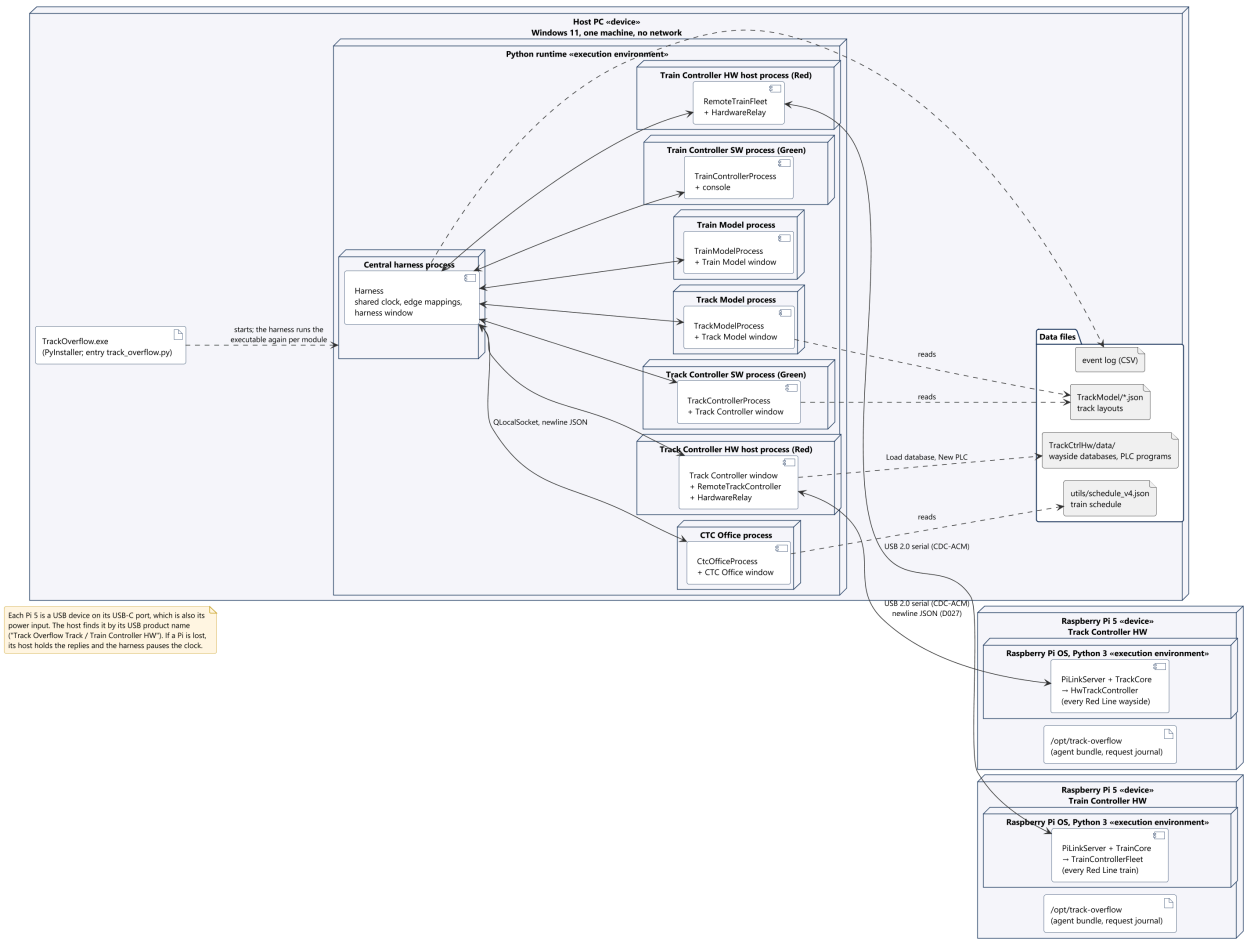

Figure 5. Deployment: the host PC, its processes and data files, and the two Raspberry Pi 5 boards

3.7 Key System-Wide Design Concepts

The concepts every module's design rests on: route, authority, the distribution of wayside controllers, simulation timing, vitality, the wayside language and the hardware/software interface. Each states the design and the decision behind it; a module's own detail is in its chapter.

3.7.1 Route

A train's route is the sequence of blocks it runs from its line's yard to its scheduled stations, in the order the track reaches them, and back to the yard (D020, D022).

- **The CTC Office plans it.** It finds the shortest route the train can run on a graph built from each block's `next_blocks` and each switch's two legs. A route never turns a train back. It goes round closed, closing and failed blocks and trains that are not moving, preferring a way round both, then round unusable blocks only; no detour is more than twice the shortest. When only reversing would get round, the train waits and the dispatcher is alerted (`arbitration/reroute-around-blockage`).

- **The route is never sent as one object.** The CTC Office expresses it as a suggested speed and authority for the block the train is in and the next block on its route.
- **The waysides set the switches.** Outside maintenance mode only the waysides' PLC programs move switches; the CTC Office commands a switch only in maintenance mode, and never one whose block is occupied (`arbitration/no-switch-under-train`). Authority stops at the block before a switch not reported set for the route, and grows once the wayside sets it.
- **The Track Model never routes.** A train follows the rails as the switches lie; a switch set wrong sends it where the track leads, as on a real railway.

3.7.2 Authority

Authority is **the number of blocks ahead of the train's current block that it may still enter** before it must stop, a non-negative integer, everywhere in the system (D013). 0 means stop before leaving the current block.

Module	Role
CTC Office	Computes it along the route, never into a block that is occupied, closed, closing, failed or within another train's authority (`arbitration/exclusive-authority`). Suggests it per block, with a speed the train can stop from: $\sqrt{(2 \times 1.2\ \text{m/s}^2 \times \text{d})}$, and 1 m/s under the block's limit.
Harness	Maps each train's suggestions to per-block values and drops the train ID.
Track Controller	The PLC program's `AUTH_n` passes block n's authority down its track circuit or sends 0; every speed bit is ANDed with it, so withheld authority sends no speed.
Track Model	Holds each block's speed and authority as that block's track-circuit signal, delivers it to the train in the block, then clears it to zero. A block never commanded, or with a failed circuit, carries 0 and 0.
Train Controller	Uses the count it receives and never counts down itself. At 0 it targets 0 and brakes while moving; at 3 it announces arrival. A drop of speed and authority to 0 together from 2 or more blocks is read as a signal pickup failure.

Speed and authority are per block, not per train, like the signal in the rails.

3.7.3 Distribution of wayside controllers

Line	Variant	Runs on	Waysides
Green	Track Controller SW	Host PC	6 by default; the count is set when the line is loaded (2 up to one per block)
Red	Track Controller HW	One Raspberry Pi 5 over USB	3, proposed: Red-1 blocks 1–24 and 73–76, Red-2 25–45 and 67–72, Red-3 46–66 (awaiting approval)

- **One variant per line** (D021). The Train Controller follows the same split: a train on the Green Line gets the software Train Controller, on the Red Line the hardware one.
- **A wayside owns a contiguous run of blocks**, with every switch, signal and crossing listed on them; a switch's point stays in the same wayside. Each wayside has its own database and its own PLC program.
- **A line runs only when every wayside on it is ready**: a person has loaded its database and its PLC program, and its channels agree. While any wayside of a line is not ready, every wayside of that line holds fail-safe and the CTC Office dispatches nothing on it (D022). The other line keeps running.
- **Nothing outside the Track Controller depends on the split.** Every other module's inputs and outputs are keyed by block, switch or crossing, never by wayside, so the boundaries can change without touching them.

3.7.4 Simulation timing

- **One clock, in the harness** (D017). `SystemClock` is the only clock. Every tick the harness sends dt and the simulation time to every module; time counts from 05:00:00 without wrapping. A module steps on the inputs of that tick, which are the outputs of the tick before.
- **Fixed step** (D006, D017): dt is 0.1 s at every speed. 1x is 10 ticks per real second and 10x is 100, so the same inputs give the same results at both.
- **Pause and speed** are the CTC Office's, sent through the harness.
- **The harness waits** for every module's reply before the next tick, and pauses the clock and names the module if a reply takes longer than about 1 s.
- **Consequences.** A command reaches the train a few ticks after the decision that caused it (system interaction view). At 10x a hardware variant's USB round trip plus its scan must stay under 10 ms. The PLC scans once per tick and keeps no memory between scans. Time-of-day rules (announcements held 5 s; train lights on at 18:00 and off at 06:00) read the shared clock.

3.7.5 Vitality

The channels between the CTC Office, the waysides and the trains are non-vital (REQ-INTF-021); the Track Controller and the Train Controller are vital (REQ-FUNC-058, REQ-FUNC-037). Each vital module therefore checks what it is given, and fails to a safe state.

Track Controller (wayside)

1. **The operating system only holds and parses the PLC program**; every protection is PLC code the programmer can read (the professor, Iteration 2; Kevin Schillinger, K3b).
2. **Read-only vital block** (design assumption, pending Harry Tye): a fixed PLC block generated per territory and run after the user's program, which can only make outputs more restrictive. The software variant implements the same rules today as a Python supervisor, to be moved into PLC code.
3. **Diverse execution** (hardware variant): every statement is compiled twice, by different algorithms, and both run every scan; a disagreement holds every output until a new program is loaded.
4. **Diverse implementations**: the two variants share a language and an interface, not code.
5. **Fail-safe outputs**: no program, a program that has not scanned yet, a channel fault, or another wayside not ready: every signal red, every crossing active, speed and authority 0, switches left where they stand.

Train Controller

- Every input every tick, validated all or nothing; a bad value leaves every train unchanged.
- Failures inferred from returned signals latch the emergency brake and set the target to 0.
- No traction while any brake is commanded (`arbitration/traction-cut-under-braking`); service brake at authority 0 while moving; the commanded speed is the only cap in both modes.
- No power until the engineer commissions the gains (D024); the emergency brake is released only by the driver, at a stand.

System

- A failure reads as occupancy, so it is protected like a train; its track circuit carries 0.
- The CTC Office never gives two trains one block (`arbitration/exclusive-authority`).
- Every module rejects an invalid tick whole, before any state changes.

3.7.6 Wayside language

A Boolean-only PLC language shared by both Track Controller variants (REQ-FUNC-051; pending proposal D016), each with its own implementation.

```
// a comment
VAR_IN   OCC_12 OCC_13 SWREV_12 CTCSW_12 CTCREV_12
VAR_OUT  SW_12
LOCK_12 := OCC_12 OR OCC_13
WANT_12 := CTCSW_12 AND CTCREV_12
SW_12   := (LOCK_12 AND SWREV_12) OR (NOT LOCK_12 AND WANT_12)
```

- **Grammar.** `VAR_IN`, `VAR_OUT` and `NAME := expression`; operators `NOT`, `AND`, `XOR`, `OR` (in that binding order), constants `0` and `1`, parentheses; ranges such as `OCC_1..20`. There are no numbers, so a scan always ends in bounded time.
- **Scan.** Assignments run top to bottom, once per tick. A name not yet set reads 0, the restrictive value, and the compiler warns about it.
- **Inputs**, per block n or switch s: `OCC_n`, `CLOSED_n`, `SPD_n_0..4` (the CTC Office's speed as five bits, 1 to 16 m/s), `SWREV_s`, and the maintenance command `CTCSW_s`, `CTCREV_s`.
- **Outputs**: `AUTH_n`, `CMD_n_0..4` (the speed sent, five bits), `SW_s`, signal aspects `SIG_s_R` / `_Y` / `_G` / `_SG`, crossing `XING_c`.
- **Checks at load.** Every line with an error is reported with its line number; a program that names anything its wayside does not have is refused.
- **Open.** The variants' names are not yet identical (the software variant keeps `MAINT`), so one program file does not yet load on both (REQ-INTF-009).

3.7.7 Hardware/software interface

- **Same interface.** A module's software and hardware variants implement the same Protocol and exchange the same inputs and outputs; the harness cannot tell them apart (D021).
- **Where the hardware runs.** Each hardware variant's core runs on its own Raspberry Pi 5: one for the Track Controller (every Red Line wayside), one for the Train Controller (every Red Line train). Its window runs on the host PC (D021, D027).
- **The link.** One USB cable. Each Pi runs in USB device mode as a standard USB serial (CDC-ACM) device, which Windows drives with its built-in driver. The host finds it by its USB product name, never a fixed COM number (D027).
- **The messages.** The harness's newline JSON, each with a request ID and the tick (D005). The module's host process owns the serial port: `HardwareRelay` relays STEP to the Pi and the reply back, in order, and carries the window's actions; `PiLinkServer` on the Pi journals every request, so a restarted Pi rebuilds its state (`common/hw_link/`).
- **If a Pi is lost**, its host holds the replies and the harness pauses the clock and names the module.
- **Open.** Power for a Pi 5 through the same USB-C port.

---

4. Decisions

The system-wide decisions, in the template's order. D-numbers are the team's decisions on `truth`. Each module's own decisions are in its chapter.

4.1 Interface Contract Mechanism

- **ID:** D005: a central harness sits between the modules
- **Context:** each module needs data from others. If modules imported each other's structures, every pair would be coupled and one field change would break the neighbours.
- **Options:**
  - Each module defines its own boundary types and a `typing.Protocol`; the harness owns one pure mapping per edge.
  - A shared catalog of types every module imports (`common/interfaces.py`).
  - Modules call each other directly.
- **Outcome:** the first. The harness is the single driver of every module, one process per module, newline JSON. The Train Controller's Protocol is still to be added; under D005 its absence breaks nothing at integration.

4.2 Shared Boundary Data Types

- **ID:** D002, D013, D008
- **Context:** a value that crosses a boundary must mean the same to every module, units included.
- **Options:**
  - Frozen dataclasses per module, SI units, converted only in the display layer.
  - Mixed units chosen per module.
  - Shared types imported across modules (ruled out by D005).
- **Outcome:** the first. Authority is a block count. The stale catalog is superseded (D008). The Train Controller reads a validated `dict` per train today; a frozen inputs dataclass is its open option.

4.3 Contract Enforcement

- **ID:** none on record; module practice
- **Context:** an unchecked contract drifts.
- **Options:**
  - A static type check plus contract tests per module.
  - Each module's own self-test, run by hand.
  - No enforcement.
- **Outcome:** each module tests its own contract (Protocol conformance, edge mappings, rejection without change); the harness has its own tests of every edge. A shared CI job is not set up; this is an open item (4.9).

4.4 Stand-ins and Stub-First Development

- **ID:** D010, D018
- **Context:** each module had to run before its neighbours existed.
- **Options:**
  - A stub at the real import path from day one.
  - The real module, with a separate test UI standing in for its neighbours.
  - No stand-ins; integrate only when every module is real.
- **Outcome:** the second. Every module has a test UI in a separate process that reaches it only through its public boundary. The harness can also run any module as a stub (`common/harness/stub_module.py`). The test UIs stay for the whole project; the harness never starts them (D005).

4.5 Contracts Ownership

- **ID:** D005
- **Context:** the boundary types are what other modules depend on.
- **Options:**
  - Each module owns its types; only the harness's edge functions know two modules' types.
  - One shared contracts package with one owner.
- **Outcome:** the first. A module with two variants has one contract shard on `truth` with a single named owner, and one shard per variant; the Track Controller contract is owned by Harry Tye.

4.6 External Input Validation

- **ID:** none on record; REQ-NFR-003, REQ-NFR-004
- **Context:** inputs come from other modules, from windows and from files. A wrong value must not corrupt state.
- **Options:**
  - Check the whole tick first and reject it whole, so nothing changes.
  - Check each value as it is used and keep the valid part.
  - Trust the caller.
- **Outcome:** the first, at every module's boundary. The one deliberate exception is the Train Model's process adapter, which rejects a bad train's inputs and still steps the other trains (5.5).

4.7 Knowledge Capture and Decision Logging

- **ID:** `AGENTS.md`: the `truth` branch is the record of decisions
- **Context:** the reasons for design choices get lost between work packages.
- **Options:**
  - A `truth` branch of normative facts, one file per entry, changed only by proposals a person promotes.
  - Decisions only in chat and issue comments.

- **Outcome:** the first. Status (what is built, what is stubbed) is read from the repository, never written into truth.

4.8 Rejected Tooling: Summary Table

Tool or approach	Why it was rejected
`common/interfaces.py` shared catalog	Stale: authority in metres, failures reported by the Train Model; superseded (D008)
Direct imports between modules	Couples every pair of modules; replaced by the harness's edge mappings (D005)
One process for the whole system	A module's window or fault would stall the others; one process per module instead (D005)
A fixed COM port number for the Pis	Changes between PCs; the Pis are found by their USB product name (D027)
A variable time step	1x and 10x would give different results; dt is fixed at 0.1 s (D006, D017)

4.9 Open Items: Not Yet Decided

Item	Owner
The read-only vital PLC block: whether every wayside runs a fixed generated block after the user's program; the SW variant's Python supervisor to be moved into PLC code	Harry Tye
One PLC language, identical names in both Track Controller variants (the SW variant keeps `MAINT`)	Harry Tye, Braden Bosilovich
The Red Line's three wayside territories (proposed in 3.7)	Braden Bosilovich
Powering a Raspberry Pi 5 through the USB-C port that carries the link	Braden Bosilovich, Jonathan Tsang, Ivan Zheng
A 10x tick on a HW variant: USB round trip plus scan under 10 ms	Braden Bosilovich, Jonathan Tsang, Ivan Zheng
The Train Controller's Protocol and typed inputs (4.1, 4.2)	Jonathan Tsang, Ivan Zheng
A shared CI job for every module's contract tests (4.3)	Team

Item	Owner
Spelling of the fourth signal aspect: `SUPER_GREEN` on truth, `SUPER GREEN` in the Train Controller	Deni Ephraim, Jonathan Tsang

Each module's own open items are at the end of its chapter.

---

5. Module Designs

One chapter per module, from its owner's contribution. Each has the module's use cases, its architecture (component view), class diagrams, interface and one sequence diagram per use case. System figures are in Section 3 and are not repeated here.

5.1 CTC Office

Owner: Landis. **Source:** sdd/ctc_office.md.

Module	CTC Office (`CTC-Office/`, packages `ctc` and `ctc_ui`)
Owner	Landis
Status	Draft for WP2 and the design review, 2026-10-22
Basis	`development` at `de7a98f` (PRs #40, #42), plus the CTC Office commits on the integration branch `claude/quirky-fermi-sickg4` (`9c440d6`, `97a2fd1`, `4cc873b`): next-block suggestions, train events, return to yard, the harness process and the harness's clock. Decisions D005, D007, D013, D017, D020, D022, D023, D025.

Reading the figures. Everything shown exists on the basis above; nothing is marked «planned». The integration branch is not yet merged into `development`.

5.1.1 Scope

The CTC Office is the dispatcher's console for the **Green and Red Lines** (the Blue Line is out of scope, D023). It is one module in one process on the host PC, with no hardware variant. It:

- creates every train, when it releases it from the yard, and gives the train ID every module uses (D022);
- dispatches trains by hand, or runs the loaded schedule in automatic mode (D020);
- works out each train's route, authority and suggested speed every step, and sends them to the Track Controller for the block the train is in and the next block on its route;
- closes blocks and sets switches in maintenance mode;
- shows occupancy and throughput to the dispatcher;
- commands the shared clock: pause, run, 1x and 10x (D017).

The CTC Office knows trains **only from block occupancy**: no train IDs, positions within a block or speeds reach it, and a failed block reports occupied like a train. It follows its own trains from that.

Not the CTC Office's job: moving switches outside maintenance mode, signals and crossings (the waysides' PLC programs), the station dwell itself (the Train Controller, D007), and the physics (the Train Model).

5.1.2 Requirements coverage

Every CTC Office item of the functional rubric, and the SRS requirements behind it, is covered by at least one use case (section 3).

Rubric	Points	Requirement	SRS	Use cases
1.2	1	Transit system operating at wall clock speed	REQ-FUNC-001	UC-01
1.3	1	Pause simulation	REQ-FUNC-003	UC-01
1.4	1	Transit system operating at 10 times wall clock speed	REQ-FUNC-002	UC-01
2.0	2	Dispatcher enters destination station and arrival time and dispatches a train	REQ-FUNC-004, 004.1, 004.2	UC-03, UC-20
2.1	2	CTC determines and sends speed limit and authority to the track controller within safety limits	REQ-FUNC-005, 005.1, 005.2	UC-21, UC-13
2.2	1	Switches between manual and automatic mode	REQ-FUNC-006, 007, 008	UC-02
2.3	1	Dispatcher manually dispatches a train to a block	REQ-FUNC-009	UC-04, UC-05
2.4	1	Close block for maintenance	REQ-FUNC-011	UC-09, UC-22
2.5	3	Monitor train occupancy	REQ-FUNC-013	UC-11, UC-12, UC-17, UC-18
2.6	2	Loading and running a schedule	REQ-FUNC-010	UC-07, UC-08, UC-19
2.7	2	In maintenance mode, set switch positions manually, sending the command to the track controller	REQ-FUNC-012	UC-10, UC-22
2.8	1	Throughput metrics displayed	REQ-FUNC-014	UC-14, UC-23

Rubric	Points	Requirement	SRS	Use cases
9.0	5	Each sub-system has a user interface	REQ-INTF-001.1	UC-01 to UC-15
—	—	Live system map	REQ-INTF-002.1, 002.2	UC-11
—	—	Track layout loaded from JSON	REQ-INTF-012	UC-11, UC-16
—	—	Channels with the Track Controller	REQ-INTF-019, 020	UC-17, UC-21, UC-22
—	—	One shared simulation clock	REQ-INTF-013, REQ-NFR-002	UC-01, UC-16
—	—	Validate user input; reject it without changing state	REQ-NFR-003, 004	UC-03 to UC-10, UC-16

REQ-INTF-002.3 and 002.4 (switches and signals on the map) are only partly met; see section 9.

5.1.3 Use cases

Actors:

- **Dispatcher** — the person at the CTC Office window. Every interactive control of the window belongs to one of UC-01 to UC-15.
- **Central Harness** — owns the shared clock and steps the module once per tick (D005, D017). It carries everything the Track Controller and the Track Model send and receive, and keeps the roster of trains the CTC Office creates and retires.
- **Track Controller** and **Track Model** — the neighbouring modules, reached only through the harness.

Use case diagrams

One diagram per actor. A use case in grey belongs to another actor and is shown only for its include or extend link.

Dispatcher (UC-01 to UC-15):

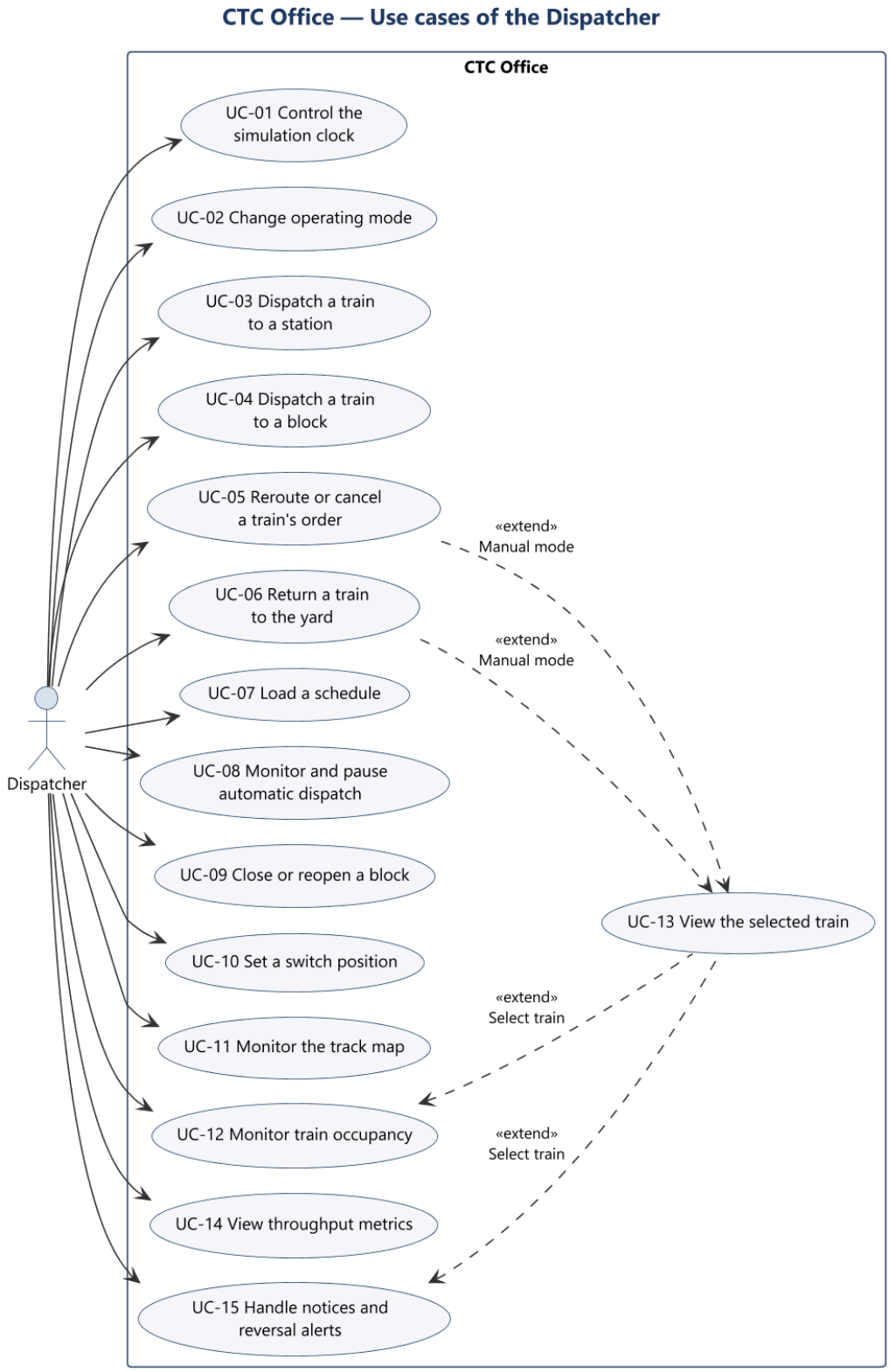

Figure 6. Use cases of the Dispatcher

Central Harness (UC-01, UC-16 and the use cases it includes, UC-24):

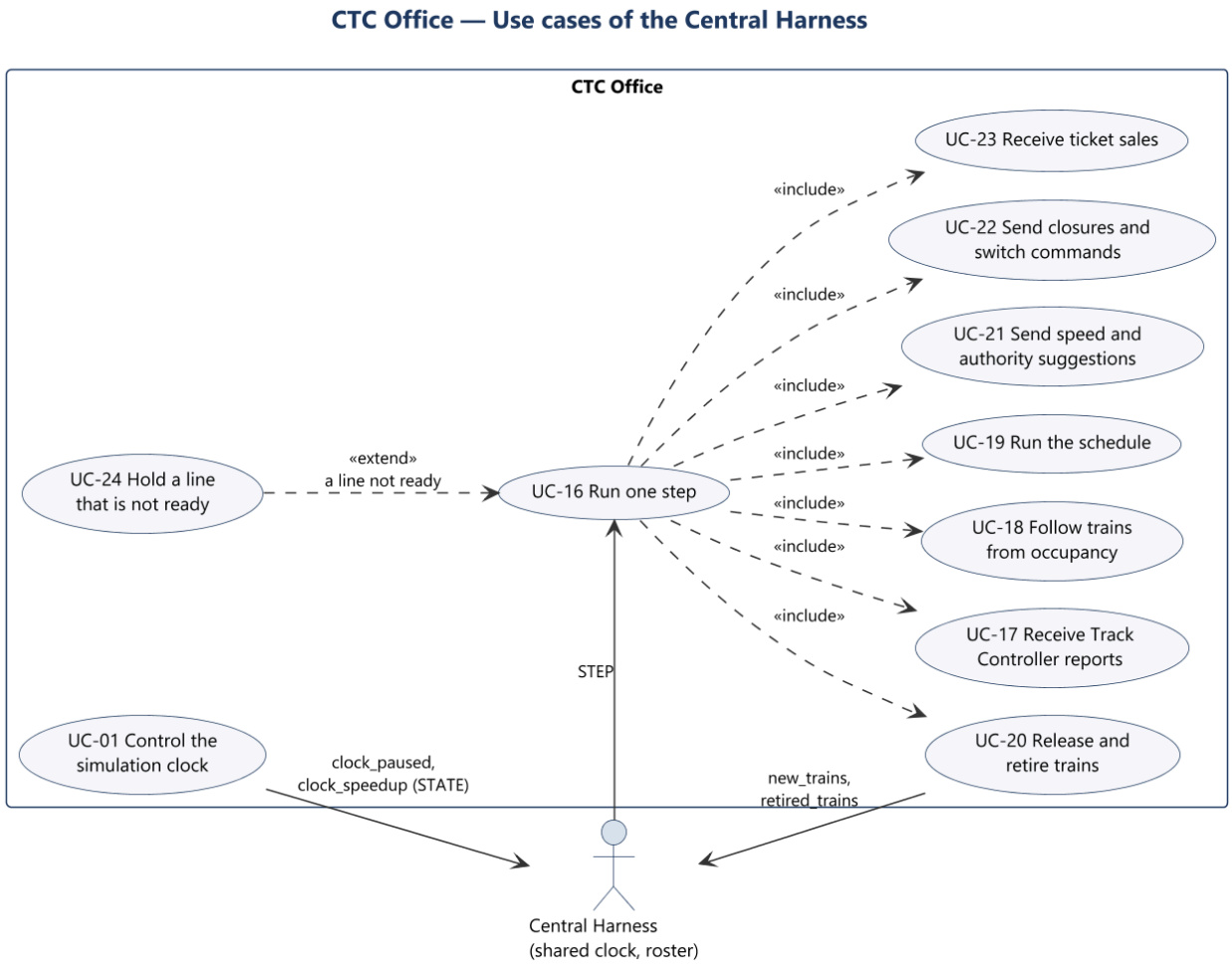

Figure 7. Use cases of the Central Harness

Track Controller (UC-17, UC-18, UC-21, UC-22, UC-24):

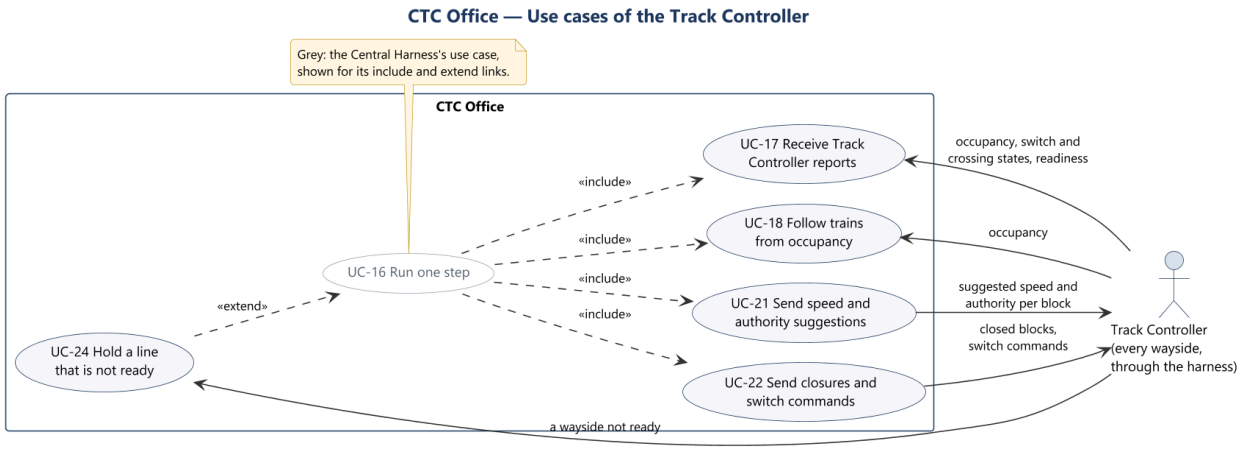

Figure 8. Use cases of the Track Controller

Track Model (UC-23):

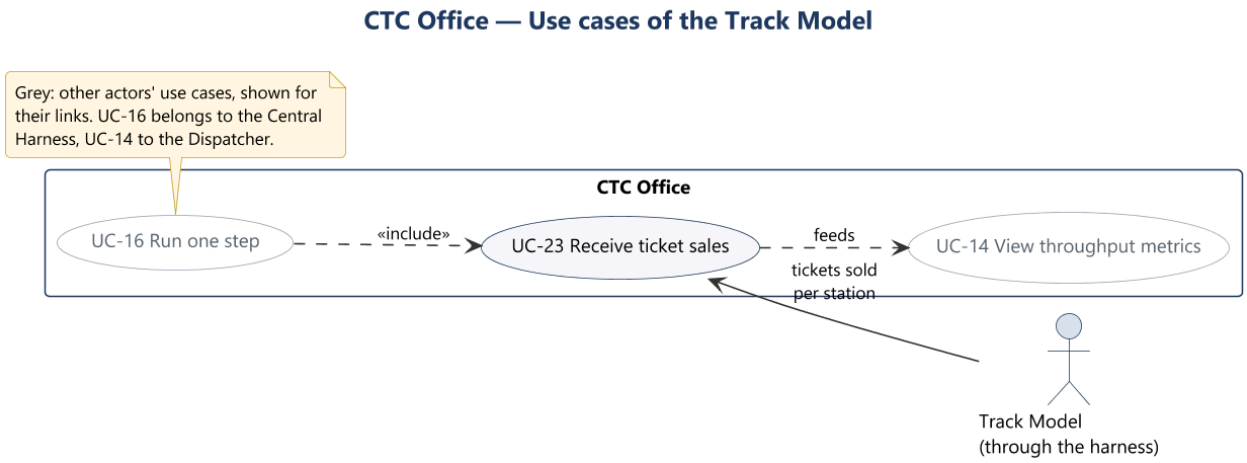

Figure 9. Use cases of the Track Model

User stories

One user story per actor. The dispatcher is the only person who uses the CTC Office; the other three actors are modules, and their stories say what they need from it. Together the stories cover every use case.

Actor	User story	Accepted when	Use cases
Dispatcher	As the dispatcher, I want to dispatch trains by hand or run them from a schedule, close blocks and set switches for maintenance, and see where every train is and how many passengers each line carries, so that trains reach their stations safely and on time.	A train dispatched to a station with an arrival time leaves the yard and stops there; a loaded schedule runs by itself in Automatic mode; a closed block gets no train; every train's block, speed and authority are on screen; every refusal says why.	UC-01 to UC-15, UC-19
Central Harness	As the central harness, I want the CTC Office to answer every step with its outputs, to tell me at once when the dispatcher pauses, runs or speeds up the clock, and to tell me once about each train it creates or retires, so that every module runs on one clock with the same trains.	Every valid step gets a reply and an invalid one a rejection that changes nothing; Run, Pause and 1x / 10x reach the harness while paused; each new and retired train appears in exactly one step's outputs.	UC-01, UC-16, UC-20

Actor	User story	Accepted when	Use cases
Track Controller	As the Track Controller, I want a suggested speed and authority for each block a train is in or about to enter, the closed blocks and the maintenance switch commands, and nothing for a line until every wayside on it is ready, so that my waysides can protect the track.	No two trains' suggestions name one block; no authority runs into an occupied, closed or failed block; a line with a wayside not ready gets nothing.	UC-17, UC-18, UC-21, UC-22, UC-24
Track Model	As the Track Model, I want the tickets I sell at each station counted, so that the dispatcher sees each line's throughput.	Each sale is counted once, in its line and its clock hour.	UC-23, UC-14

UC-01 Control the simulation clock

Actors	Dispatcher; Central Harness
Description	Runs, pauses and speeds up the one shared clock, for every module.
Data	`clock_paused` (true at start), `clock_speedup` (true for 10x). The header shows the harness's time, 24-hour, from 05:00:00.
Stimulus	**Run** / **Pause** and **1x** / **10x** under SIMULATION CLOCK in the header.
Response	The module's outputs change at once and go to the harness as `STATE`, since no step runs while the clock is paused. The harness starts, stops or speeds the ticks of every module. At 10x ticks come ten times as often; dt stays 0.1 s.
Alternatives	The harness allows Run only once every module has said it is ready (D005). Pressing the control that is already set changes nothing.
Requirements	Rubric 1.2 to 1.4; REQ-FUNC-001, 002, 003; REQ-INTF-013; REQ-NFR-002
Sequence	SD-01

UC-02 Change operating mode

Actors	Dispatcher
Description	Chooses Automatic, Manual or Maintenance mode.
Data	One mode at a time.
Stimulus	OPERATING MODE toggle in the header.
Response	The right-hand column shows that mode's view. Automatic: the loaded schedule's runs dispatch themselves. Manual: Dispatch train, and reroute, cancel or return to yard for the selected train. Maintenance: Close block, Active closures and Set switch position. Leaving Automatic holds every scheduled run where it is; leaving Maintenance drops every switch command, while closed blocks stay closed. The mode itself is sent to no module: the Track Controller gets the closed blocks instead.
Requirements	Rubric 2.2; REQ-FUNC-006, 007, 008
Sequence	SD-02

UC-03 Dispatch a train to a station

Actors	Dispatcher
Description	Orders a train to a station of its line, with an optional arrival time.
Data	Line; train: one already on the line, or **New train (T*n*)** with the next free ID; destination station (the line's station blocks); arrival `HH:MM`, optional.

Stimulus	**Dispatch train** (Manual view). It reads **Reroute train** when the train already has an order.
Response	The order is recorded, replacing any earlier one. A new train waits in the yard until the CTC Office releases it (UC-20). From then on the train gets authority toward the station every step (UC-21). The panel shows the suggested speed and authority sent for it. The arrival time ranks the train for contested blocks: the train most behind schedule goes first.
Alternatives	Refused with a message, and nothing changes: the line's waysides are not all ready (the message names the wayside and why); the destination is closed or closing; the train is on the other line; the train ID is empty, unprintable or longer than 32 characters; the arrival is not a time of day. A destination the CTC Office takes for a failure is accepted: the train waits short of it until it clears.
Requirements	Rubric 2.0; REQ-FUNC-004, 004.1, 004.2, 006; REQ-NFR-003, 004
Sequence	SD-03

UC-04 Dispatch a train to a block

Actors	Dispatcher
Description	Sends a train straight to any block of its line.
Data	Line, train, block.
Stimulus	**Set authority** ("or dispatch directly to a block"), Manual view.
Response	An order to that block with no arrival time. The CTC Office turns it into a count of blocks every step (D013).
Alternatives	As UC-03.
Requirements	Rubric 2.3; REQ-FUNC-009
Sequence	SD-04

UC-05 Reroute or cancel a train's order

Actors	Dispatcher
Description	Changes where the selected train is going, or withdraws its order.
Data	Train; new destination.
Stimulus	**Reroute** or **Cancel order** in Selected train (Manual mode).
Response	A reroute replaces the order; the route is worked out again from where the train is, carrying on the way it is going. A cancel drops the order, so the train's authority is 0 from the next step and it stops.
Alternatives	A reroute is refused as UC-03 refuses.
Requirements	Rubric 2.3; REQ-FUNC-009
Sequence	SD-05

UC-06 Return a train to the yard

Actors	Dispatcher
Description	Sends a manual train on the track back to the yard, where it is retired (D022).
Data	Train.
Stimulus	**Return to yard** in Selected train (Manual mode).
Response	An order to the line's yard entry block (Green 57, Red 9). Once the train has stopped there with authority 0 it is retired (UC-20). A later order or a cancel withdraws it.
Alternatives	Refused for a train not on the track (cancel its order instead), for a run of the loaded schedule (the schedule returns it), and as UC-03 refuses.
Requirements	D022

Sequence	SD-06

UC-07 Load a schedule

Actors	Dispatcher
Description	Loads the schedule that automatic mode runs.
Data	Schedule JSON in the `utils/schedule_v4.json` format: per run, the line, run number and timed stops.
Stimulus	**Load schedule** (Automatic view) and a file.
Response	Each run is planned at once (D020). Its train ID is the line's initial and run number (`G1`, `R1`). It stops only at its scheduled stations, in the order the track reaches them from the yard, then returns to the yard. It is due out of the yard at 05:00 plus its departure, and at each stop after the run time and a 45 s dwell at each stop before (D007). DEPARTURES lists every run.
Alternatives	A malformed file, a stop not on the layout, a run listed twice, or two runs with one train ID: "Schedule not loaded: …", and the current schedule is kept.
Requirements	Rubric 2.6; REQ-FUNC-010; REQ-NFR-003, 004
Sequence	SD-07

UC-08 Monitor and pause automatic dispatch

Actors	Dispatcher
Description	Shows how far each run has got, and holds or releases them all.

Data	DEPARTURES: train, due time, status (Queued, Held, In yard, To *station*, At *station*, To yard, Done).
Stimulus	**Pause dispatch** / **Resume dispatch**; any step.
Response	While paused, no run leaves the yard or a stop; a run already on its way carries on to its next stop.
Requirements	Rubric 2.6; REQ-FUNC-007, 010
Sequence	SD-08

UC-09 Close or reopen a block

Actors	Dispatcher
Description	Takes a block out of service for maintenance, and back.
Data	Line, block.
Stimulus	**Close block**, then **Yes, close block** (Maintenance view); **Reopen block** or **Cancel closing** under Active closures.
Response	A clear block closes at once. An occupied block is listed as "Closing — train in block" and closes by itself once the train has left. Either way, orders into it are cancelled at once with a notice, no authority is given into it, and it is in `closed_blocks` from the moment it is requested until it is reopened.
Alternatives	Already closed or closing: says so. Outside maintenance mode: refused.
Requirements	Rubric 2.4; REQ-FUNC-011
Sequence	SD-09

UC-10 Set a switch position

Actors	Dispatcher
Description	Commands a switch, in maintenance mode.
Data	Line; switch, named by its block; **Normal** (the first connection the layout file lists) or **Reverse** (the second). The panel shows the position the Track Controller reports and the one commanded.
Stimulus	**Send to track controller** or **Release** (Maintenance view).
Response	The command is in `switch_commands` every step while maintenance mode lasts; the wayside's PLC program carries it out, and the reported position follows.
Alternatives	The switch's block is occupied: refused. Outside maintenance mode: refused.
Requirements	Rubric 2.7; REQ-FUNC-012
Sequence	SD-10

UC-11 Monitor the track map

Actors	Dispatcher
Description	Shows both lines and what is on them.
Data	Every block from the layout files, coloured by line; occupied, closed and closing blocks; each train the CTC Office follows, as a car tagged with its ID on its head block; an untagged car on any other occupied block (an inferred failure among them); active crossings.
Stimulus	Any step; **+** / **−** zoom, drag to pan, **Fit**.
Response	The map redraws. A selected train's route is highlighted (UC-13).

Requirements	Rubric 2.5; REQ-FUNC-013; REQ-INTF-002.1, 002.2
Sequence	SD-11

UC-12 Monitor train occupancy

Actors	Dispatcher
Description	Lists every train the CTC Office follows or has an order for.
Data	Train, line, block (both blocks while it crosses into the next), suggested speed (mph; its actual speed is never known), authority (blocks, and the block it ends at), destination, requested arrival, status (En route, In yard, Not located, No order).
Stimulus	Train occupancy window, or its dock ("Keep open at bottom"); filter by line, status or train ID.
Response	Rows update after each step. **Select train** loads the row into Selected train (UC-13).
Requirements	Rubric 2.5; REQ-FUNC-013
Sequence	SD-12

UC-13 View the selected train

Actors	Dispatcher
Description	Shows one train in detail.

Data	Current block, next station, suggested speed, authority and the block it runs to, why it ends there (destination, a block occupied, closed or closing, a switch not set, a block reserved by another train), destination, arrival, dwell at the next stop, the route, and where it would have to reverse.
Stimulus	**Select train** in the occupancy window or on a reversal alert.
Response	The panel fills, and the route is highlighted on the map, within authority and beyond it. In Manual mode the panel adds UC-05 and UC-06.
Requirements	Rubric 2.1 (display); REQ-FUNC-005, 013
Sequence	SD-13

UC-14 View throughput metrics

Actors	Dispatcher
Description	Shows passenger throughput per line, from ticket sales.
Data	Tickets per hour on each line since 05:00; tickets sold on each line in each clock hour, last 12 hours.
Stimulus	Any step; hover a bar for its value.
Response	The tiles, table and charts update.
Requirements	Rubric 2.8; REQ-FUNC-014
Sequence	SD-14

UC-15 Handle notices and reversal alerts

Actors	Dispatcher
Description	Tells the dispatcher what needs their attention.

Data	Notices: each wayside not ready and why; orders the CTC Office cancelled itself (block closed or closing); Track Controller reports staged while paused. Reversal alert: the train, what holds it, and the block where it would have to reverse.
Stimulus	**Dismiss** (notices); **Select train** or **Dismiss** (alert).
Response	The CTC Office never reverses a train: it waits, and the alert says so. A dismissed alert does not come back unless the situation clears and happens again.
Requirements	REQ-INTF-004
Sequence	SD-15

UC-16 Run one step

Actors	Central Harness (primary); Track Controller and Track Model through it
Description	Once per tick, takes that tick's inputs, moves every train and run on, and returns the outputs.
Data	Tick number; dt (fixed, 0.1 s); simulation time (unwrapped seconds, 05:00:00 is 18 000); the tick's inputs (UC-17, UC-23).
Stimulus	A STEP message from the harness.
Response	In order: follow the trains (UC-18), add ticket sales (UC-23), run the schedule (UC-19), close pending closures whose block is clear, retire trains (UC-20), work out authority and commit it, release a waiting train (UC-20), and return the outputs (UC-21, UC-22, clock commands, train events). The reply goes before the window redraws, which it does at most every 200 ms.
Alternatives	Malformed or invalid input, a dt that is not finite and positive, or a time not after the module's: the whole tick is refused before anything changes, and the harness logs it and keeps the last outputs. The module's time is the harness's, so a refused tick does not put the schedule behind. A line not ready: UC-24.

Requirements	REQ-INTF-013; REQ-NFR-002, 003, 004; REQ-DSN-004
Sequence	SD-16

UC-17 Receive Track Controller reports

Actors	Track Controller (through the harness)
Description	Takes what every wayside reports.
Data	Occupied blocks; switch positions; crossing states; each wayside's ID, line, ready and why not. The harness maps them from every wayside's `CtcReport`.
Stimulus	Every step.
Response	Occupancy drives tracking (UC-18). A switch not reported set for a train's route cuts its authority short. Crossings show on the map. Readiness gates each line (UC-24).
Alternatives	An unknown block, switch or crossing, or a switch reported in two positions: the tick is refused.
Requirements	Rubric 2.5; REQ-FUNC-013; REQ-INTF-020
Sequence	SD-17

UC-18 Follow trains from occupancy

Actors	Central Harness (included in UC-16); Track Controller (its occupancy)
Description	Works out where each of its trains is, and which occupied blocks are failures, from occupancy alone.

Data	Occupancy now and at the last step; each train's route, authority and suggested speeds; the trains it has released.
Stimulus	Every step.
Response	The train in a yard exit block (Green 63, Red 9) is the one the CTC Office last released there. A train moves into the next block of its route when that block becomes occupied, and leaves a block when it clears; it holds both while crossing.
Alternatives	The next block occupied sooner than half the time the train would need, or an occupancy no train explains: a failure. No authority is given into it and routes go round it, but it is shown only as occupied, and orders into it are kept. A train whose blocks all clear outside the yard is dropped quietly ("Not located"); its order stays.
Requirements	Rubric 2.5; REQ-FUNC-013
Sequence	SD-18

UC-19 Run the schedule

Actors	Central Harness (included in UC-16)
Description	In automatic mode, gives the orders that carry the loaded schedule out (D020).
Data	Each run's plan and progress; the time; ready lines; where each train is.
Stimulus	Every step, in Automatic mode with dispatch not paused.
Response	A run that is due is dispatched from the yard and released (UC-20). It is sent to its next scheduled station, where its authority ends; once its head reaches that block it is held 45 s, then sent on. After its last stop it returns to the yard and is retired. Runs are never sent to meet head-on: where runs use the same track in both directions (Green 77 to 100 and 1 to 28; Red 16 to 66) one scheduled train at a time is let in, and a run due to enter a busy stretch waits at its last stop or in the yard as Held.

Alternatives	A closed or closing stop is skipped. A line not ready: its runs wait.
Requirements	Rubric 2.6; REQ-FUNC-007, 010
Sequence	SD-19

UC-20 Release and retire trains

Actors	Central Harness (included in UC-16; applies the events to its roster)
Description	Lets trains out of the yard and takes them back (D022).
Data	`new_trains`: train ID and line; `retired_trains`: train ID. Each appears in exactly one step's outputs.
Stimulus	Every step.
Response	Release: per line, one train at a time, the waiting train most behind schedule first, and only when every wayside on the line is ready, the yard exit block is clear and not closed or closing, no train is authorized into it, and the scheduler is not holding the run. The harness adds the train to the Train Model and the line's Train Controller at the yard exit block. Retire: a train sent to the yard that has stopped at the yard entry block (Green 57, Red 9) with authority 0 at its destination. The harness removes it; its block clears; its run is done and its order dropped.

Requirements	Rubric 2.0; REQ-FUNC-004; D022
Sequence	SD-20

UC-21 Send speed and authority suggestions

Actors	Track Controller (through the harness)
Description	Suggests each train's speed and authority, for the block it is in and the next block on its route.
Data	Up to two per train: train ID, line, block, speed (whole m/s), authority (blocks). The second is for the next block, while the authority is at least 1, with one block less.
Stimulus	Every step.
Response	Authority and speed are worked out as in section 4.1. The harness sends each line's suggestions to its Track Controller by block, dropping the train ID; the next block always lies within the train's own exclusive authority, so no two trains name one block.
Alternatives	A train still in the yard gets neither. A line not ready: UC-24.
Requirements	Rubric 2.1; REQ-FUNC-005, 005.1, 005.2; REQ-INTF-019
Sequence	SD-21

UC-22 Send closures and switch commands

Actors	Track Controller (through the harness)

Description	Tells the waysides which blocks are closed and, in maintenance mode, where to set switches.
Data	`closed_blocks`: the full set every step, closing blocks included; `switch_commands`: only in maintenance mode.
Stimulus	Every step.
Response	The Track Controller may not override a listed block; the PLC program carries out a switch command.
Alternatives	A line not ready: nothing is sent for it; closures and commands are kept and go out once it is ready (UC-24).
Requirements	Rubric 2.4, 2.7; REQ-FUNC-011, 012; REQ-INTF-019
Sequence	SD-22

UC-23 Receive ticket sales

Actors	Track Model (through the harness)
Description	Takes the tickets sold at each station.
Data	Tickets sold since the previous step, by station name as the layout files write it (`GLENBURY`). The harness delivers each tick's sales exactly once.
Stimulus	Every step.
Response	Added to each station's count since the start, which gives each line's throughput (UC-14).
Alternatives	An unknown station, or a count that is negative or not a whole number: the tick is refused.
Requirements	Rubric 2.8; REQ-FUNC-014
Sequence	SD-23

UC-24 Hold a line that is not ready

Actors	Central Harness (extends UC-16); Track Controller (a wayside not ready)
Description	Keeps a line still while any wayside on it is not ready (no PLC program loaded). The waysides are vital, so nothing is sent to a line until every wayside on it runs its program.
Data	Each wayside's ready and reason. A line is ready once a wayside on it has reported and every wayside on it is ready.
Stimulus	Any step on which a line is not ready.
Response	For that line only: no suggestions, closed blocks or switch commands are sent; no authority is granted; no train is released; automatic dispatch waits; a dispatch is refused, naming the wayside and why; the window lists each wayside not ready. Closures and switch commands are kept. The other line keeps running. Everything resumes on the first step the line is ready.
Requirements	REQ-FUNC-005; D022 (release conditions)
Sequence	SD-24

5.1.4 CTC Office architecture (component view)

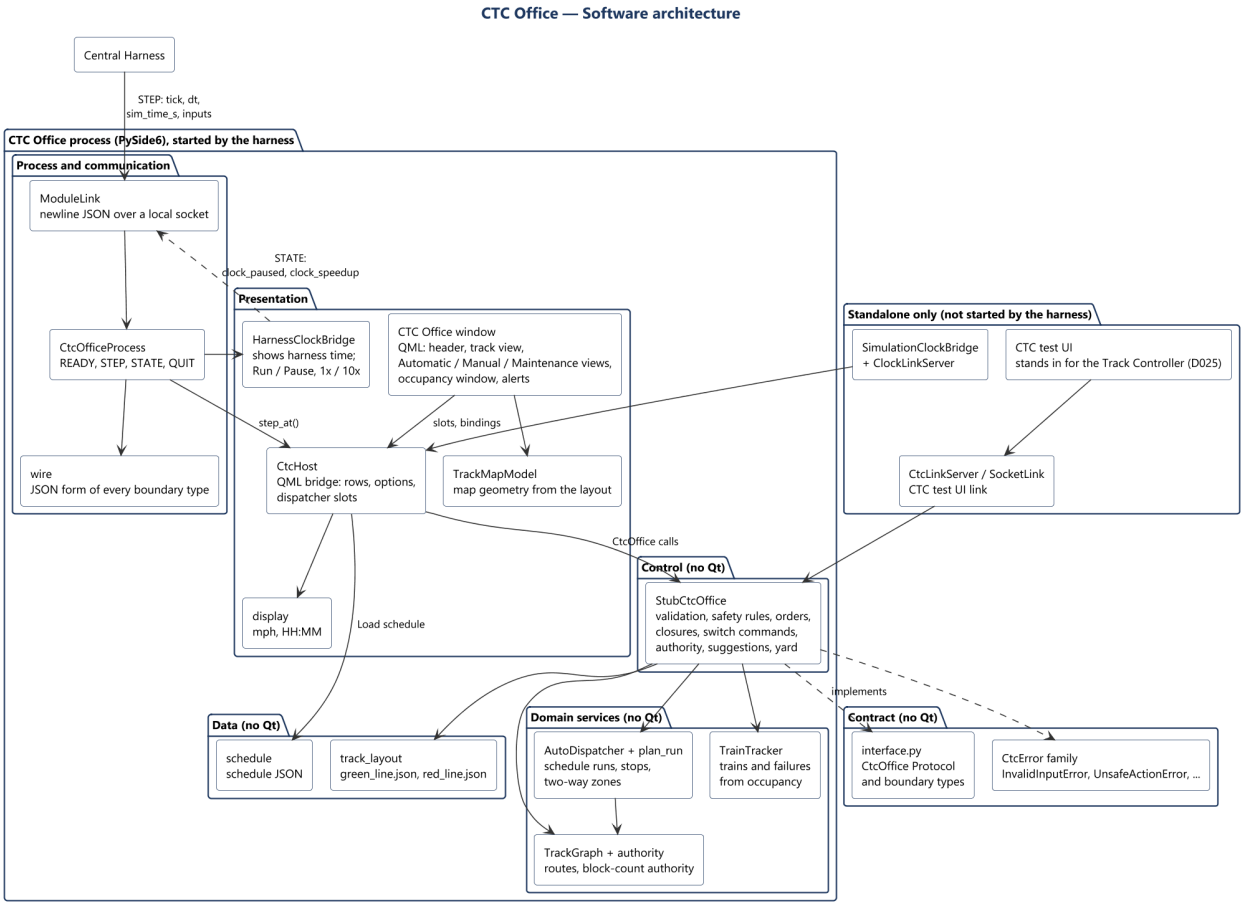

Figure 10. CTC Office architecture (component view)

The module is layered so that every decision is made in Qt-free code:

Layer	Parts	Responsibility
Presentation	QML window (`Main.qml`, `CtcHeader`, the Automatic, Manual and Maintenance views, panels), `CtcHost`, `TrackMapModel`, `display`, `HarnessClockBridge`	Shows the railway in imperial units and 24-hour time; turns clicks into `CtcOffice` calls and shows each refusal's text. Never decides anything.
Process and communication	`CtcOfficeProcess`, `ModuleLink` (shared `utils`), `wire`	`READY`, `STEP`, `STATE` and `QUIT` with the harness; decodes inputs and encodes outputs as JSON.

Layer	Parts	Responsibility
Control	`StubCtcOffice` (`ctc/model.py`)	Validation, safety rules, orders, closures, switch commands, authority, suggestions, yard release and retirement, outputs.
Domain services	`TrainTracker`, `AutoDispatcher` and `plan_run`, `TrackGraph` and `authority`	Tracking from occupancy; schedule plans and progress; routes and block-count authority.
Data	`track_layout`, `schedule`	Reads the layout files at start and a schedule file on request.
Contract	`interface.py`, the `CtcError` family	The `CtcOffice` Protocol and the boundary types.

Design choices:

- **One interface, three users.** The harness (through `CtcOfficeProcess`), the window and the CTC test UI drive the module only through `CtcOffice`. Dispatcher actions are not cross-module inputs.
- **The core is Qt-free.** `interface`, `model`, `tracking`, `scheduler`, `routing`, `track_layout`, `schedule`, `wire` and `actions` import no Qt, and are unit-tested headlessly.
- **Validate, then change.** `step` rejects a bad tick before any state changes; an action is refused with a message before anything changes.
- **Recompute every step.** Routes and authority are worked out again from the current reports every step, never carried from a stored plan. Only the blocks already granted to a train are kept, until it has passed them.
- **The harness's time.** Under the harness the module's time is each step's simulation time, not a sum of dt values; standalone, it moves on by dt.
- `StubCtcOffice` keeps its first name: it is the full implementation.

How the CTC Office decides

Concern	Design
Route	Shortest route a train can run, on a graph built from each block's `next_blocks` and each switch's two legs. It never turns a train back: each route carries on from the block the train came from. It goes round closed, closing and failed blocks and trains that are not moving, preferring a way round both, then round unusable blocks only, then the shortest route, on which the train waits. No detour more than twice the shortest is taken. When only reversing would get round, the train waits and the dispatcher is alerted.
Switches	Outside maintenance mode the CTC Office never commands a switch. Authority stops at the block before a switch not reported set for the route, and grows once the wayside sets it.
Authority	Blocks ahead of the train's head block along its route, to the destination, ending before the first block that is occupied, closed, closing, failed or within another train's authority, or before a switch not set.
Contested blocks	A block no train holds yet goes to the train most behind schedule: the least slack between its requested arrival and the time it needs at each block's limit. A train with no arrival comes after; ties go to the lower ID.
Suggested speed	The block's limit in m/s, rounded down, less 1 m/s; never more than $\sqrt{(2 \times 1.2\ \text{m/s}^2 \times d)}$, d being the length within authority; 0 at authority 0. The next block's value is worked out for that block, with one block less.
Tracking	UC-18.
Automatic dispatch	UC-19. Track order is a first approach and may change.
Yard	UC-20.

Concern	Design
Readiness	UC-24.

5.1.5 Interface

Units are backend units: speed in m/s, distance in m; the window shows mph. Authority is a count of blocks (D013). Every block, switch and crossing reference carries its line, since block numbers repeat across lines; a switch or crossing is named by the block the layout file lists it on. Line names are `"Green"` and `"Red"`; a train ID is a trimmed, printable string of at most 32 characters.

Inputs from the Track Controller (every step, mapped by the harness from every wayside's report):

Field	Type	Meaning
`occupancy`	(line, block, occupied) per block	A train or a failure; a block not listed is clear.
`switches`	(line, switch, `normal` | `reverse`)	Position each switch reports.
`crossings`	(line, crossing, `inactive` | `active`)	State each crossing reports.
`waysides`	(wayside ID, line, ready, reason)	Each wayside's readiness, and why not.

Input from the Track Model:

Field	Type	Meaning
`ticket_sales`	{station name: whole number ≥ 0}	Tickets sold since the previous step.

Outputs to the Track Controller:

Field	Type	Meaning
`suggestions`	(train ID, line, block, speed m/s int, authority blocks int)	Up to two per train: its head block and the next block.
`closed_blocks`	(line, block), full set	Closed and closing blocks; the Track Controller may not override them.
`switch_commands`	(line, switch, `normal` | `reverse`)	Maintenance mode only.

Outputs to the central harness:

Field	Type	Meaning
`clock_speedup`	bool	True for 10x, false for 1x.
`clock_paused`	bool, true at start	True while the dispatcher has the clock paused; also sent at once as `STATE`.
`new_trains`	(train ID, line)	Trains released from the yard this step.
`retired_trains`	(train ID)	Trains retired this step.

5.1.6 Class diagrams

Core: the module and train tracking

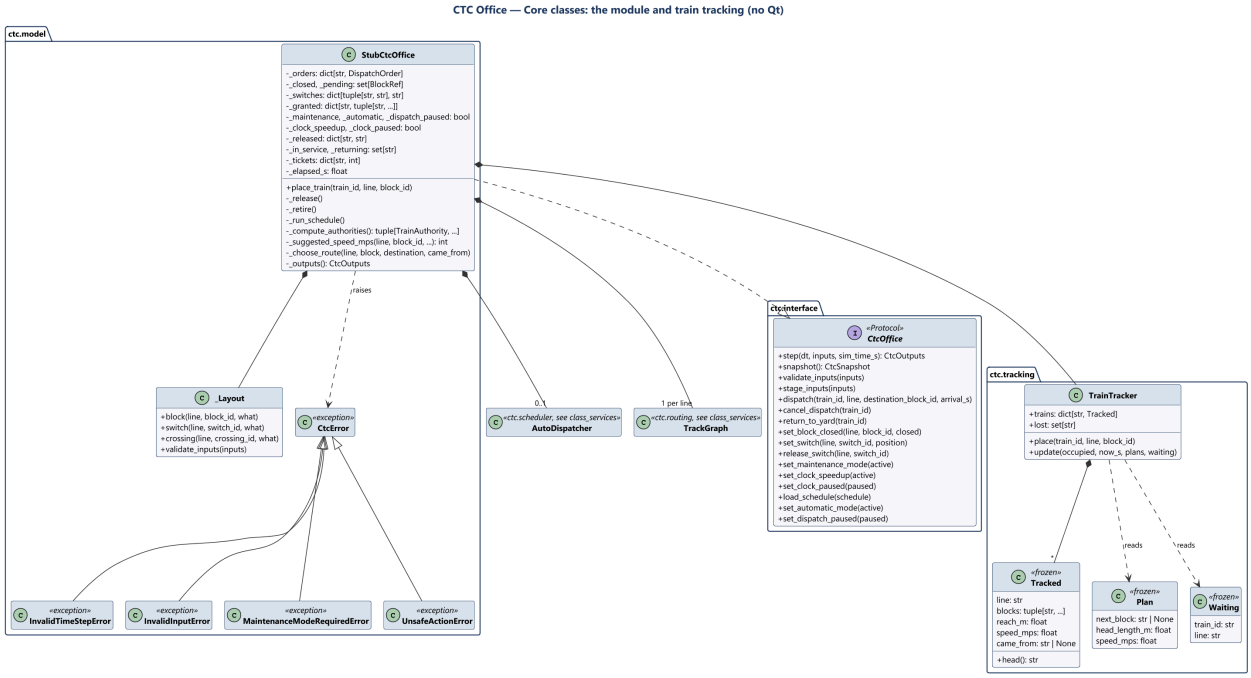

Figure 11. Core classes: module and tracking

- `StubCtcOffice` implements `CtcOffice`. It holds the orders, closures, switch commands and granted blocks, and owns one `TrackGraph` per line, the `TrainTracker` and, once a schedule is loaded, the `AutoDispatcher`.
- `_Layout` checks every reference and input against the layout files.
- Every refusal is a `CtcError`: `InvalidInputError` (bad value), `InvalidTimeStepError`, `MaintenanceModeRequiredError`, `UnsafeActionError` (a safety rule).
- `TrainTracker` keeps one `Tracked` per train: its blocks, how far it may have got, the fastest it may be going and the block it came from. Each step it reads the last step's `Plan` per train and

the trains released (`Waiting`).

Core: scheduling, routing and data

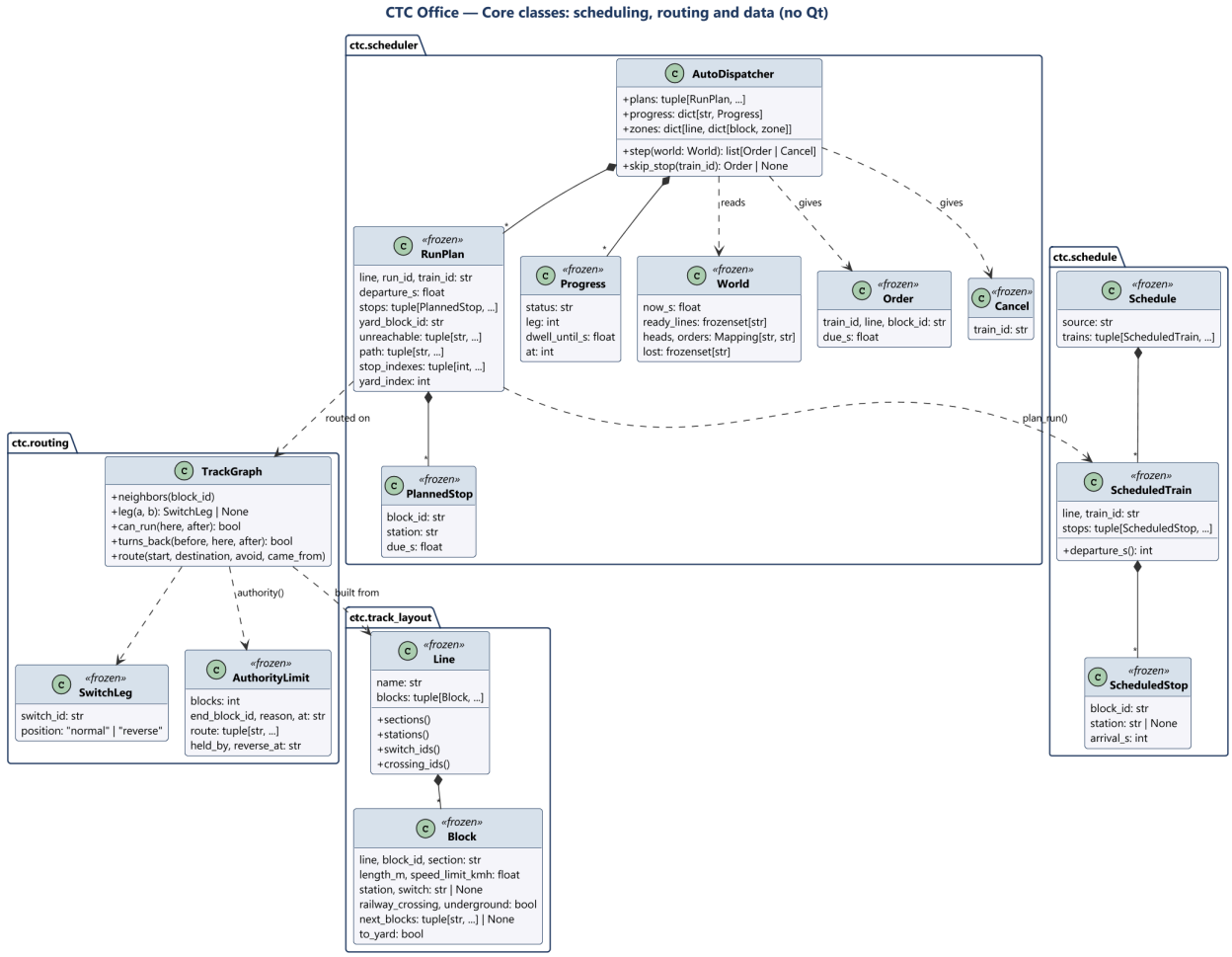

Figure 12. Core classes: scheduling, routing and data

- `plan_run` turns each `ScheduledTrain` into a `RunPlan`: its stops in track order with due times, the path out and back, and where on it each stop is.
- `AutoDispatcher` keeps a `Progress` per run and the two-way zones per line. Each step it reads a `World` and gives `Order` and `Cancel` actions, which the model carries out with the dispatcher's own safety checks.
- `TrackGraph` is built from a `Line`'s blocks; `authority()` counts an `AuthorityLimit` along a route.

Boundary types

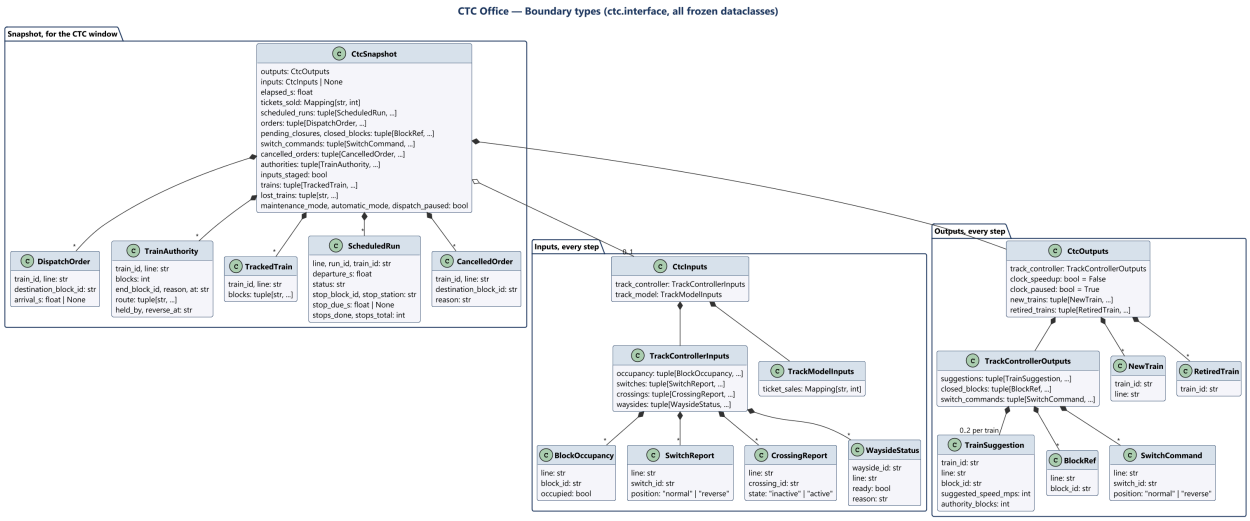

Figure 13. CTC Office: boundary types

All are frozen dataclasses in `ctc/interface.py`, owned by the CTC Office (D005); other modules' types never appear in them, and `common/interfaces.py` is not used (D008). `CtcSnapshot` is for the CTC window only and crosses no module boundary.

Process and window classes

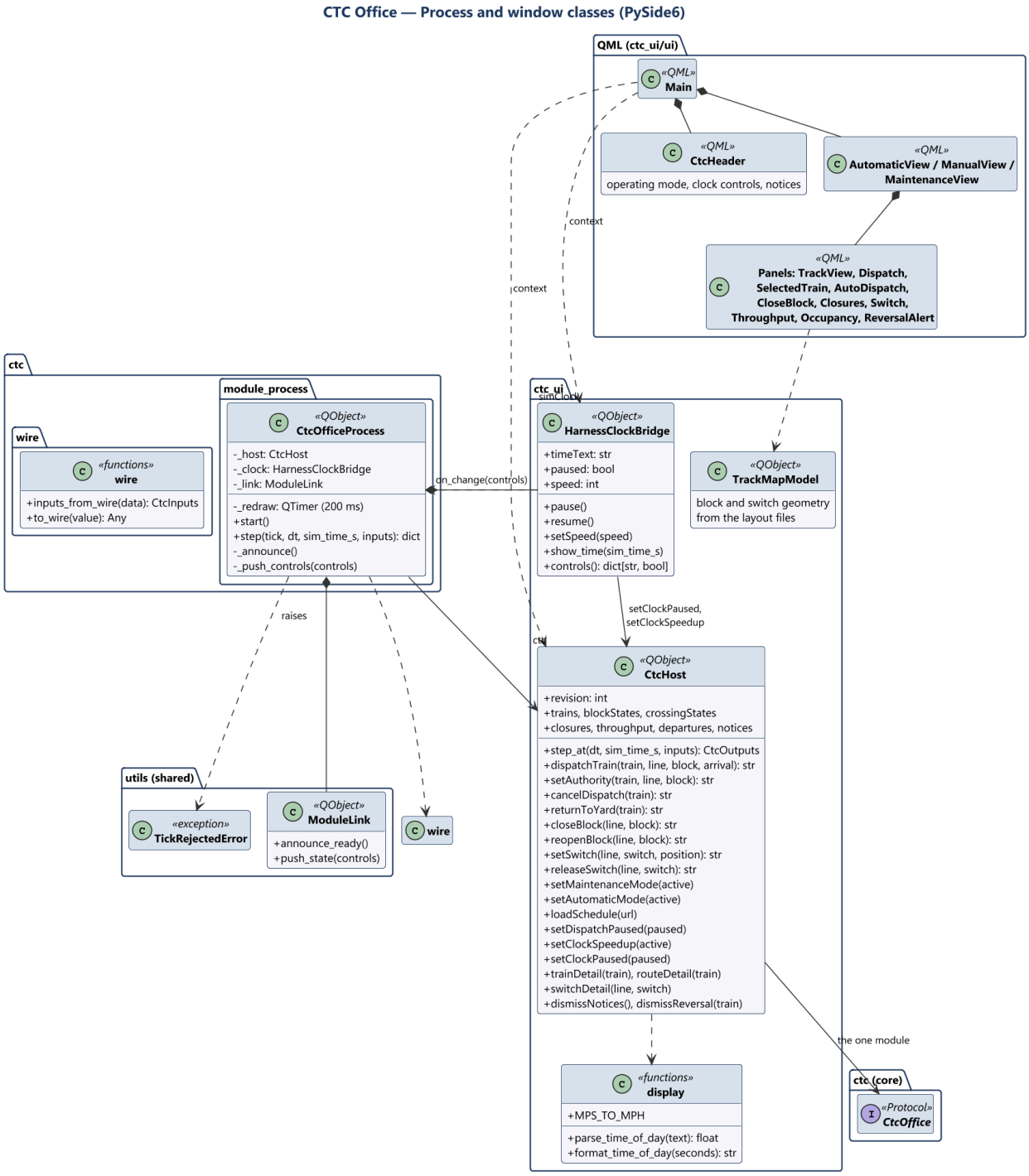

Figure 14. Process and window classes

- `CtcOfficeProcess` answers the harness through `ModuleLink` and steps the module through `CtcHost`.
- `CtcHost` is the only object QML talks to: it reads snapshots into rows, and its slots return an empty string or the refusal's text.

- `HarnessClockBridge` gives QML the same clock face as the standalone clock, but shows the harness's time, and hands Run, Pause and 1x / 10x to the module and the harness.

5.1.7 Sequence diagrams

One sequence diagram per use case, numbered to match: SD-*n* shows UC-*n*. SD-16, one whole step, refers to the diagrams of the use cases it includes.

Diagram	Use case
SD-01	UC-01 Control the simulation clock
SD-02	UC-02 Change operating mode
SD-03	UC-03 Dispatch a train to a station
SD-04	UC-04 Dispatch a train to a block
SD-05	UC-05 Reroute or cancel a train's order
SD-06	UC-06 Return a train to the yard
SD-07	UC-07 Load a schedule
SD-08	UC-08 Monitor and pause automatic dispatch
SD-09	UC-09 Close or reopen a block
SD-10	UC-10 Set a switch position
SD-11	UC-11 Monitor the track map
SD-12	UC-12 Monitor train occupancy
SD-13	UC-13 View the selected train
SD-14	UC-14 View throughput metrics
SD-15	UC-15 Handle notices and reversal alerts
SD-16	UC-16 Run one step
SD-17	UC-17 Receive Track Controller reports
SD-18	UC-18 Follow trains from occupancy
SD-19	UC-19 Run the schedule
SD-20	UC-20 Release and retire trains
SD-21	UC-21 Send speed and authority suggestions

Diagram	Use case
SD-22	UC-22 Send closures and switch commands
SD-23	UC-23 Receive ticket sales
SD-24	UC-24 Hold a line that is not ready

SD-01 Control the simulation clock

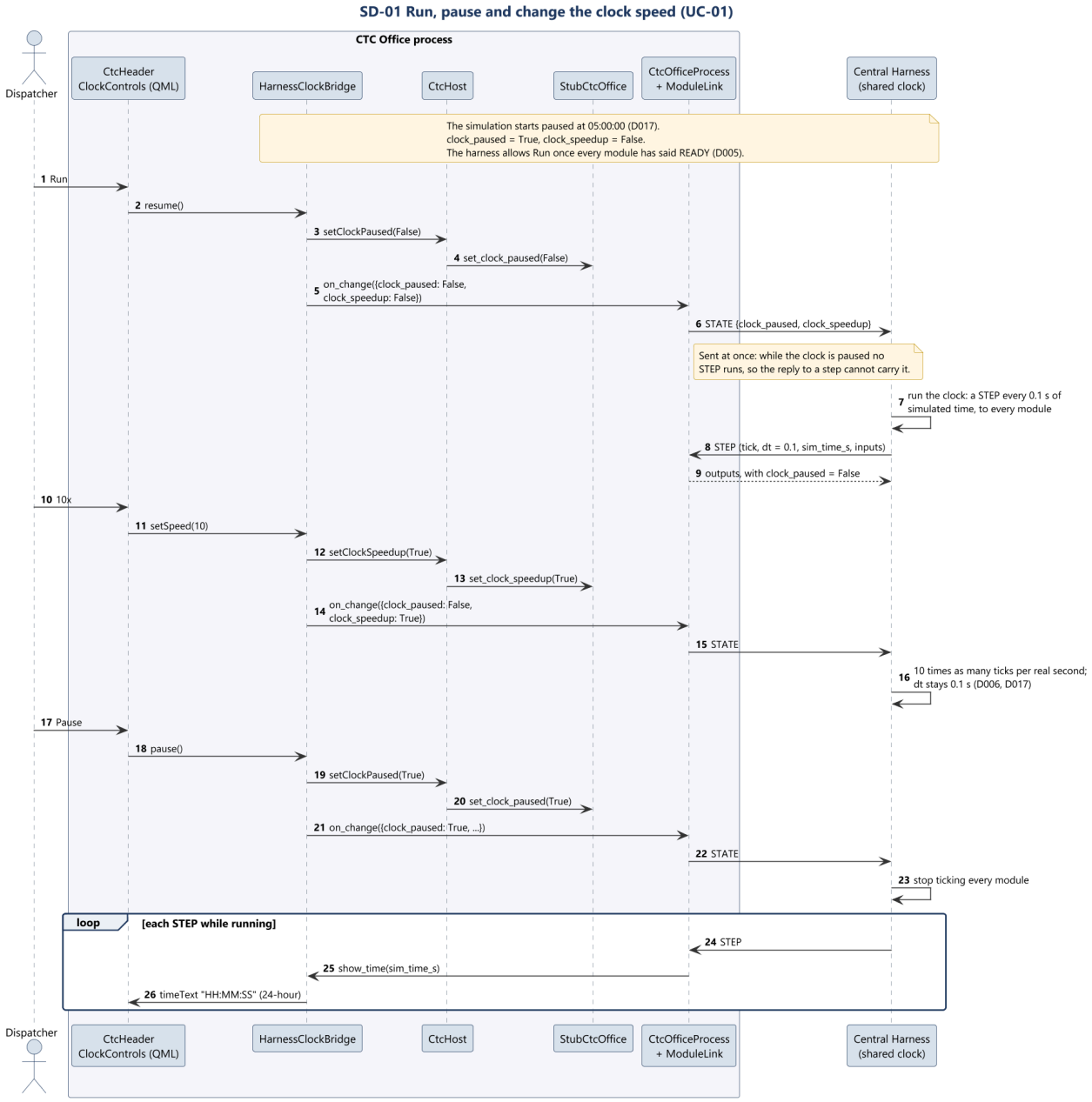

Figure 15. SD-01 Control the simulation clock

SD-02 Change operating mode

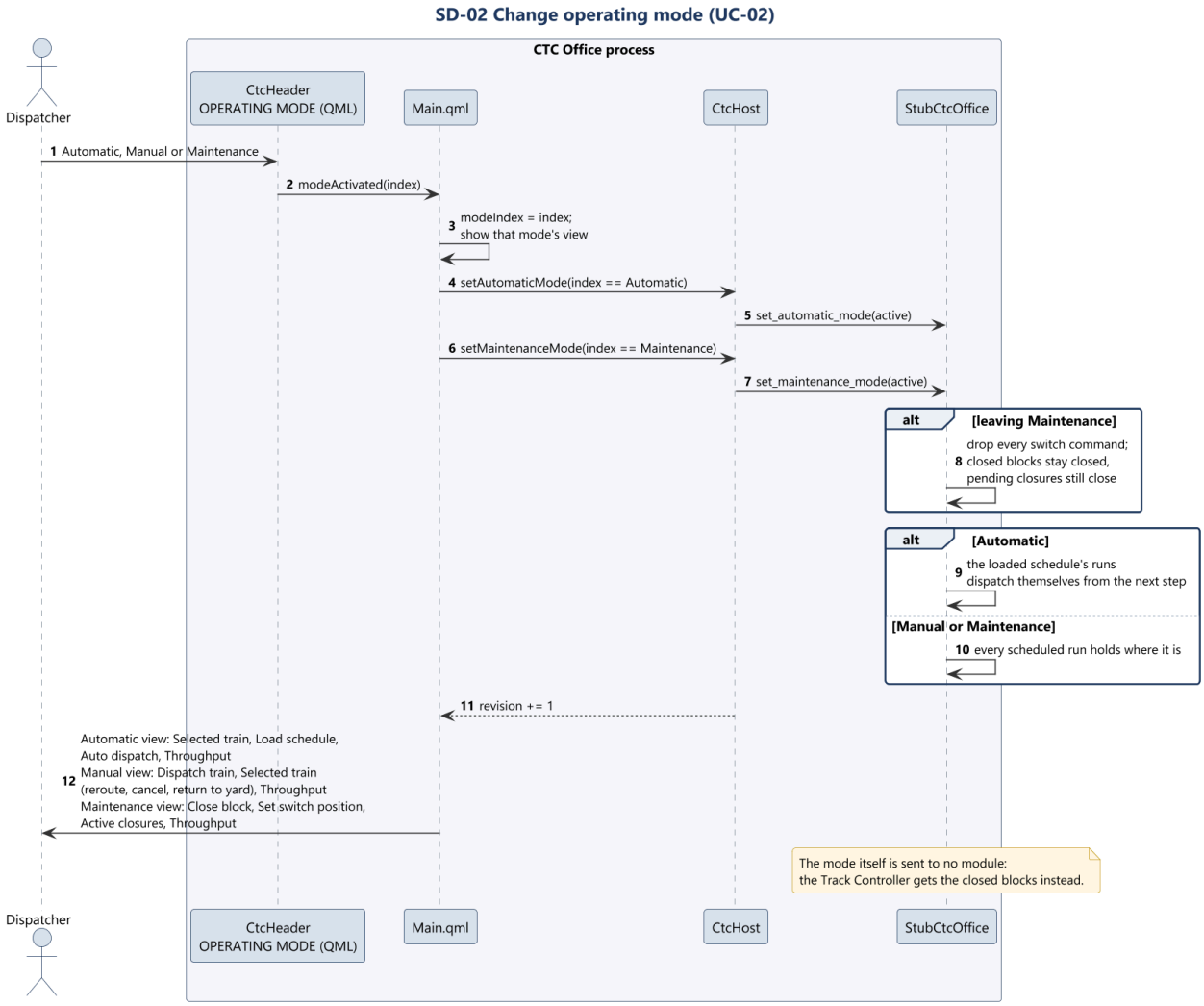

Figure 16. SD-02 Change operating mode

SD-03 Dispatch a train to a station

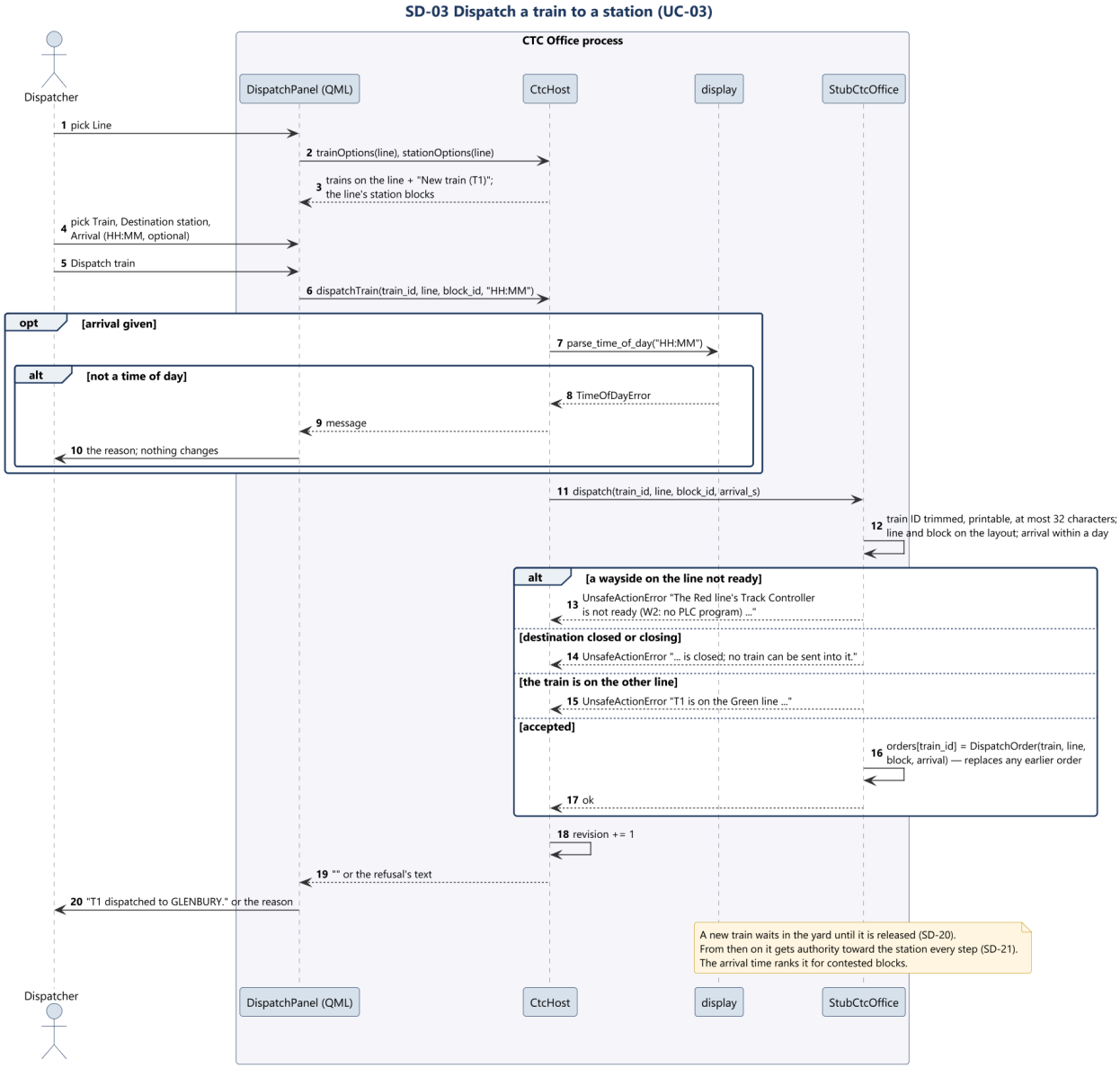

Figure 17. SD-03 Dispatch a train to a station

SD-04 Dispatch a train to a block

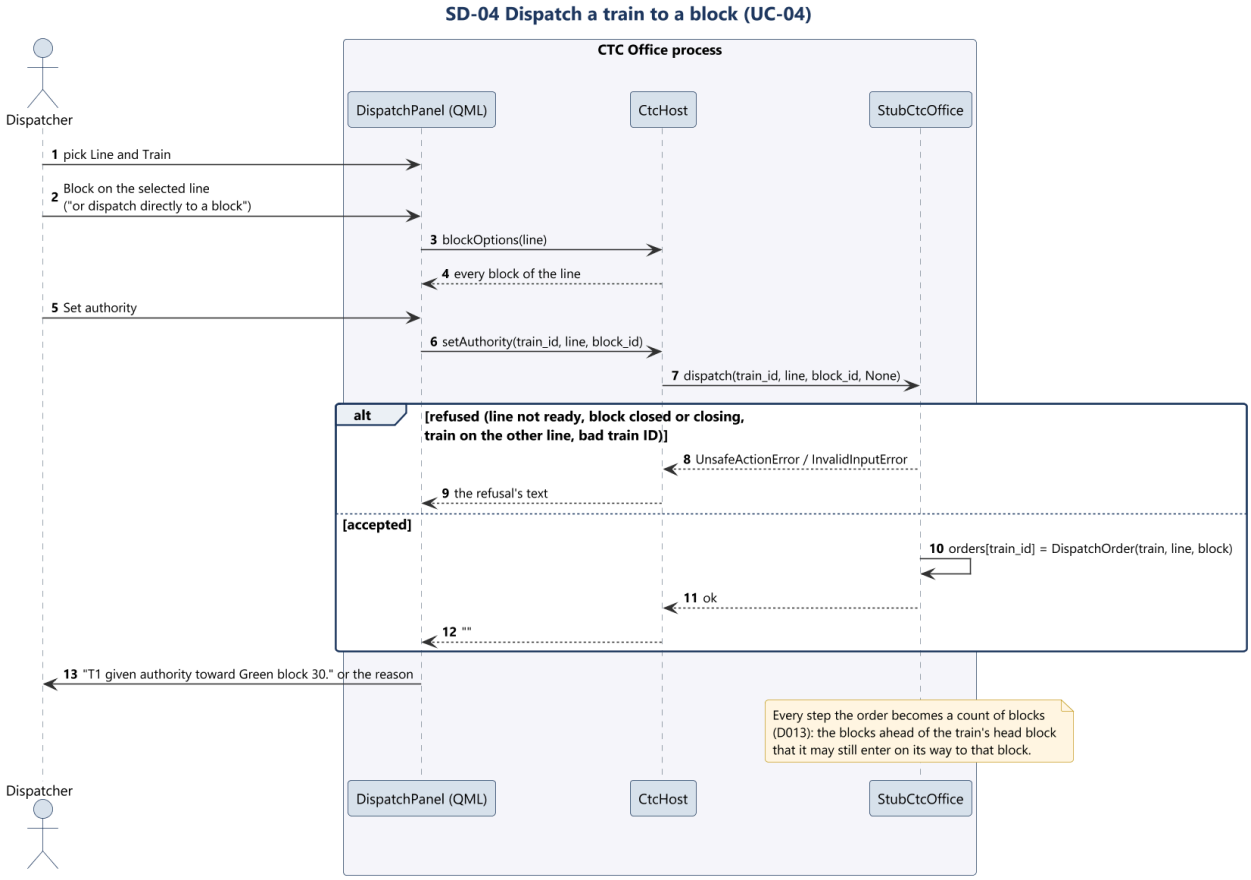

Figure 18. SD-04 Dispatch a train to a block

SD-05 Reroute or cancel a train's order

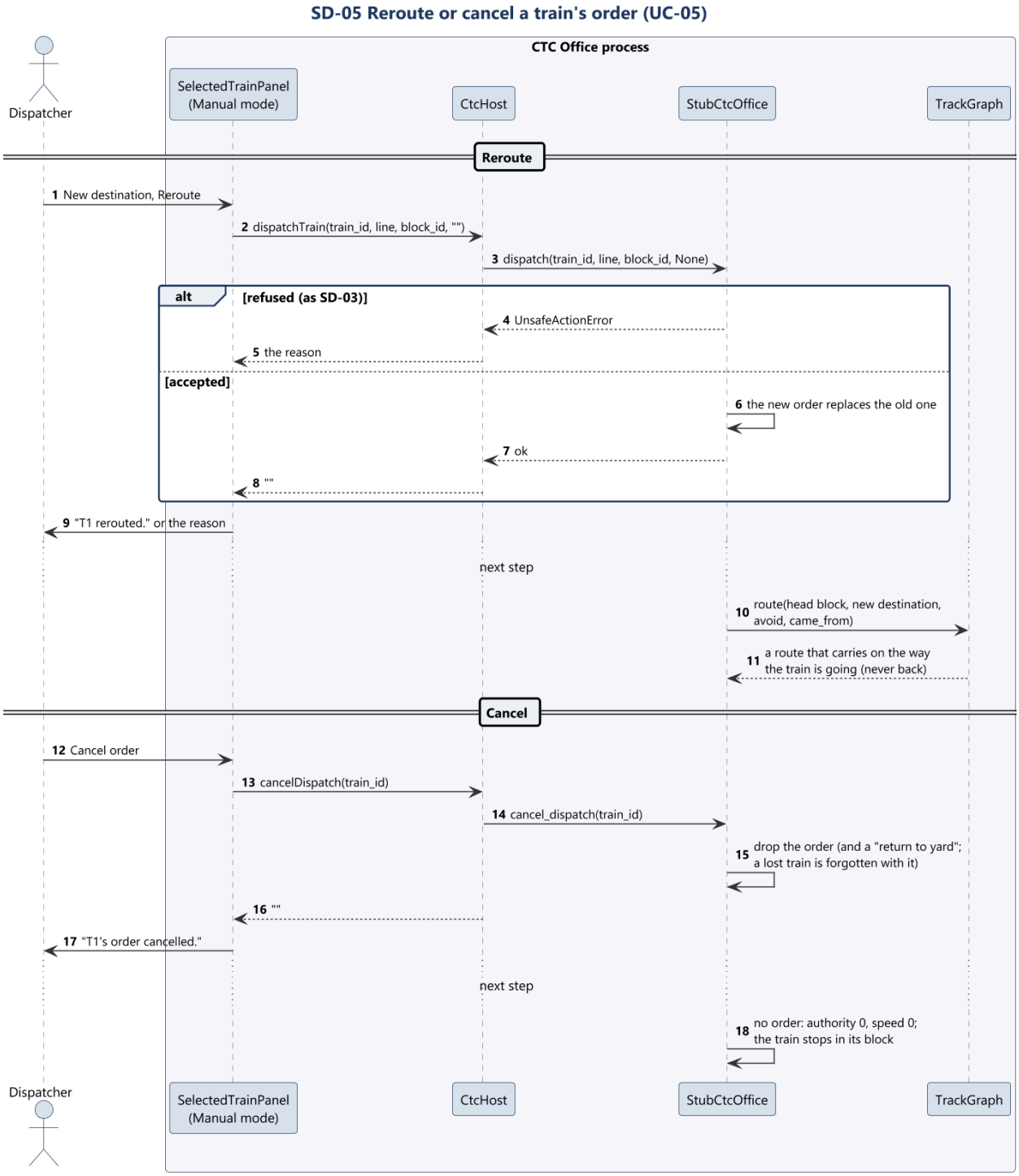

Figure 19. SD-05 Reroute or cancel a train's order

SD-06 Return a train to the yard

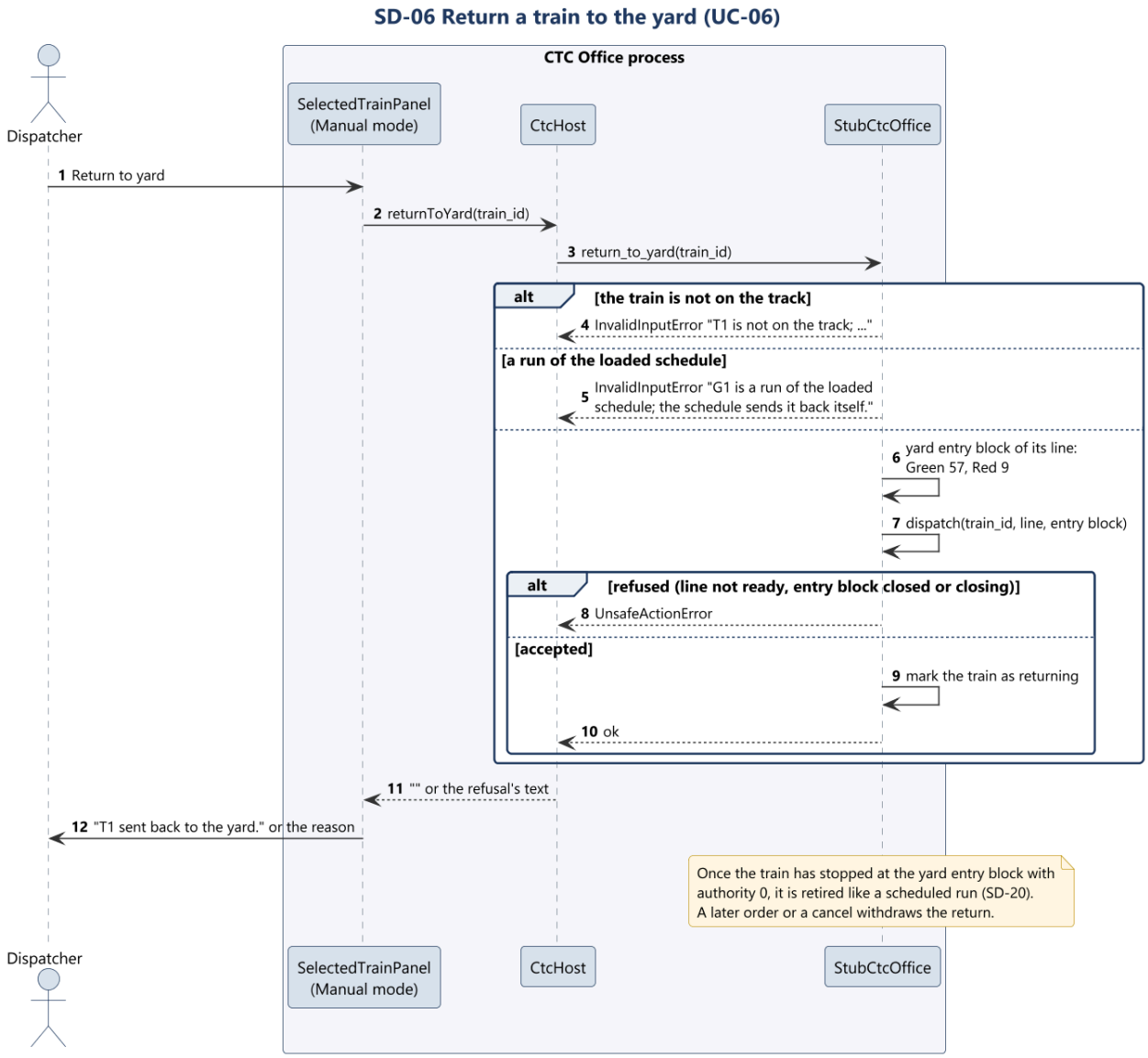

Figure 20. SD-06 Return a train to the yard

SD-07 Load a schedule

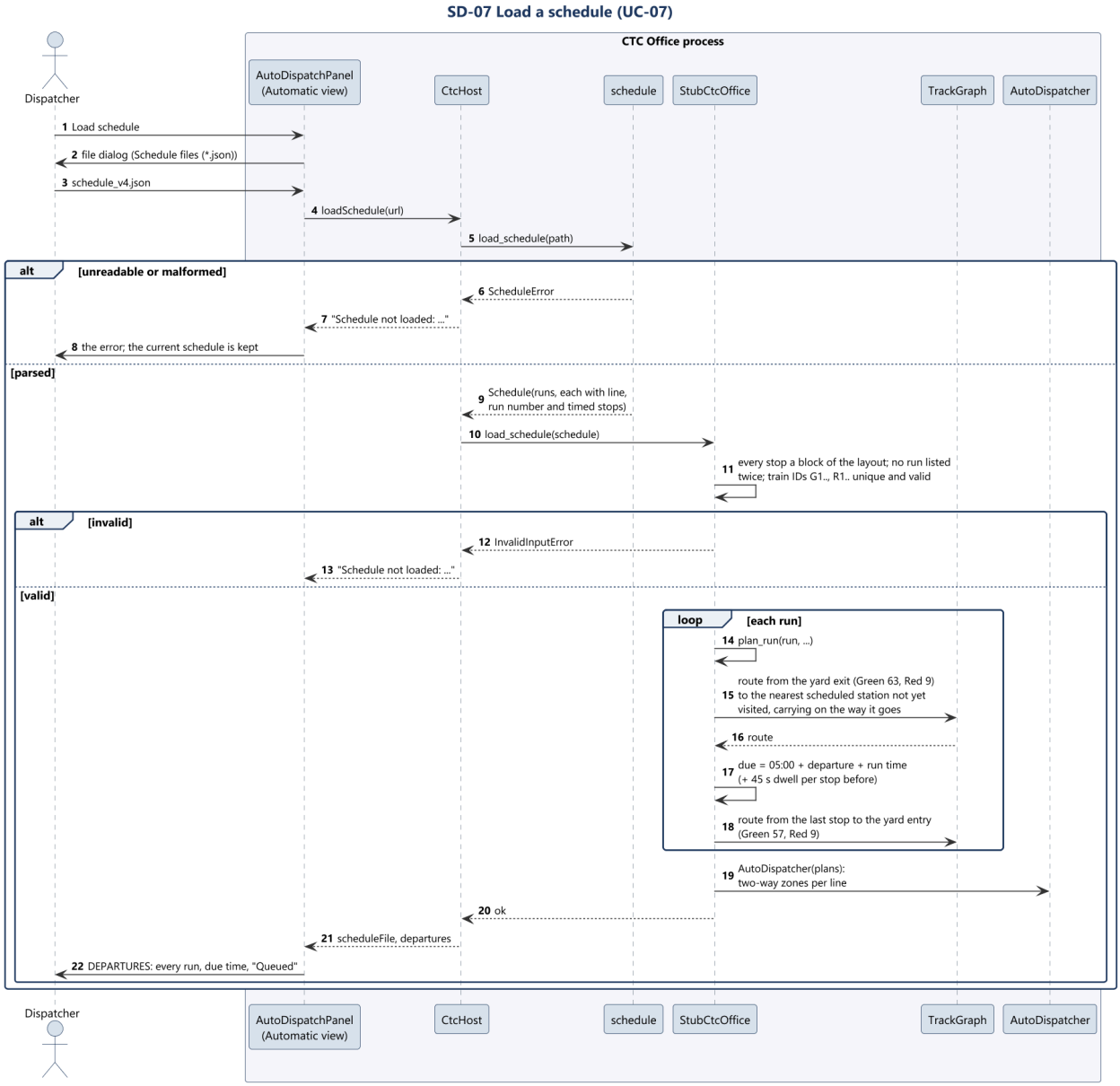

Figure 21. SD-07 Load a schedule

SD-08 Monitor and pause automatic dispatch

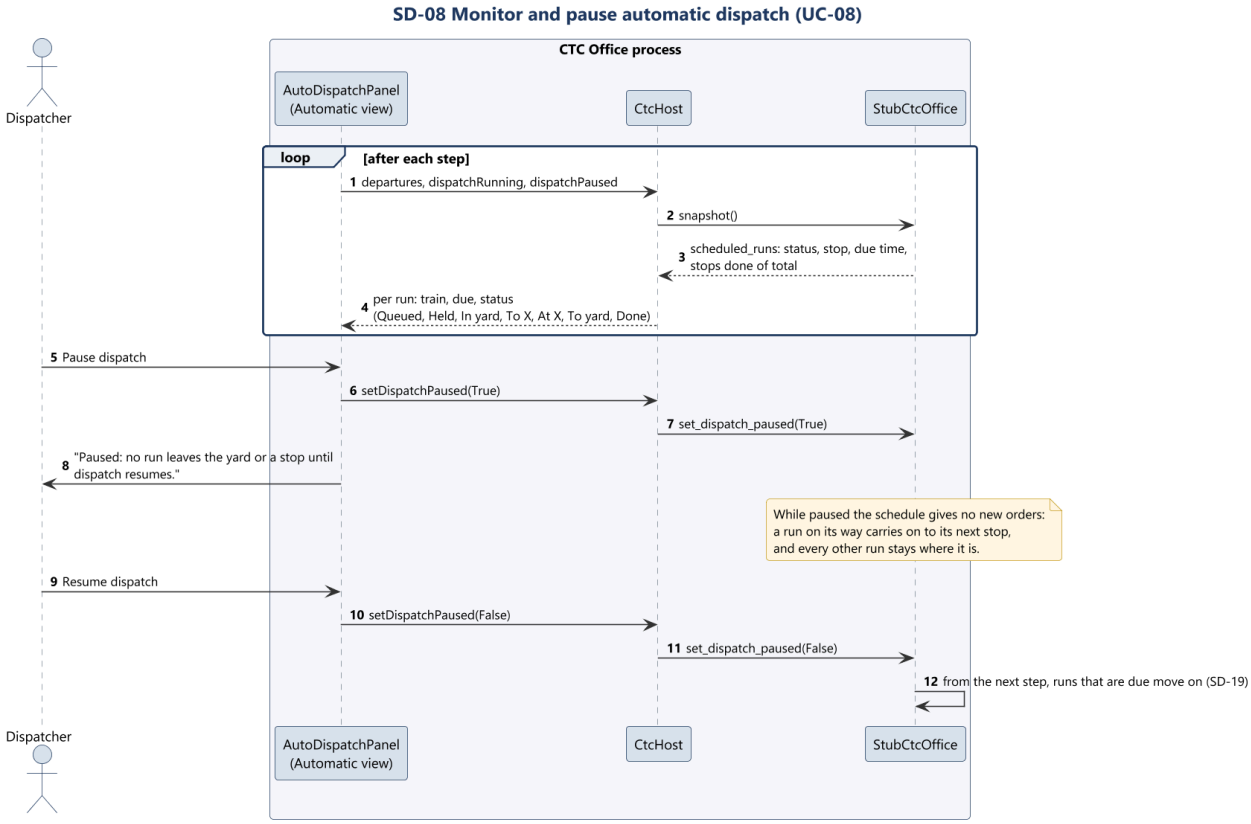

Figure 22. SD-08 Monitor and pause automatic dispatch

SD-09 Close or reopen a block

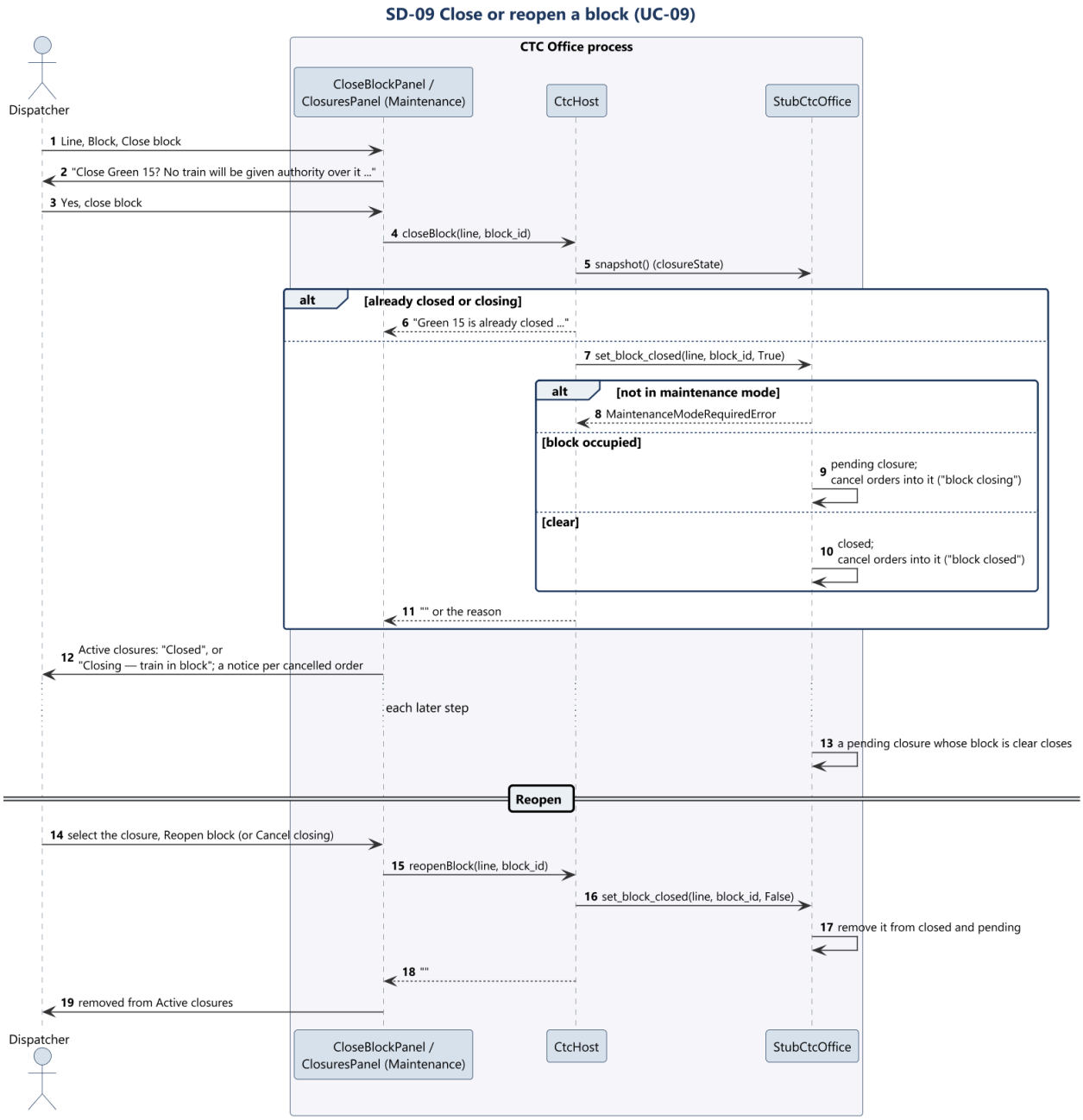

Figure 23. SD-09 Close or reopen a block

SD-10 Set a switch position

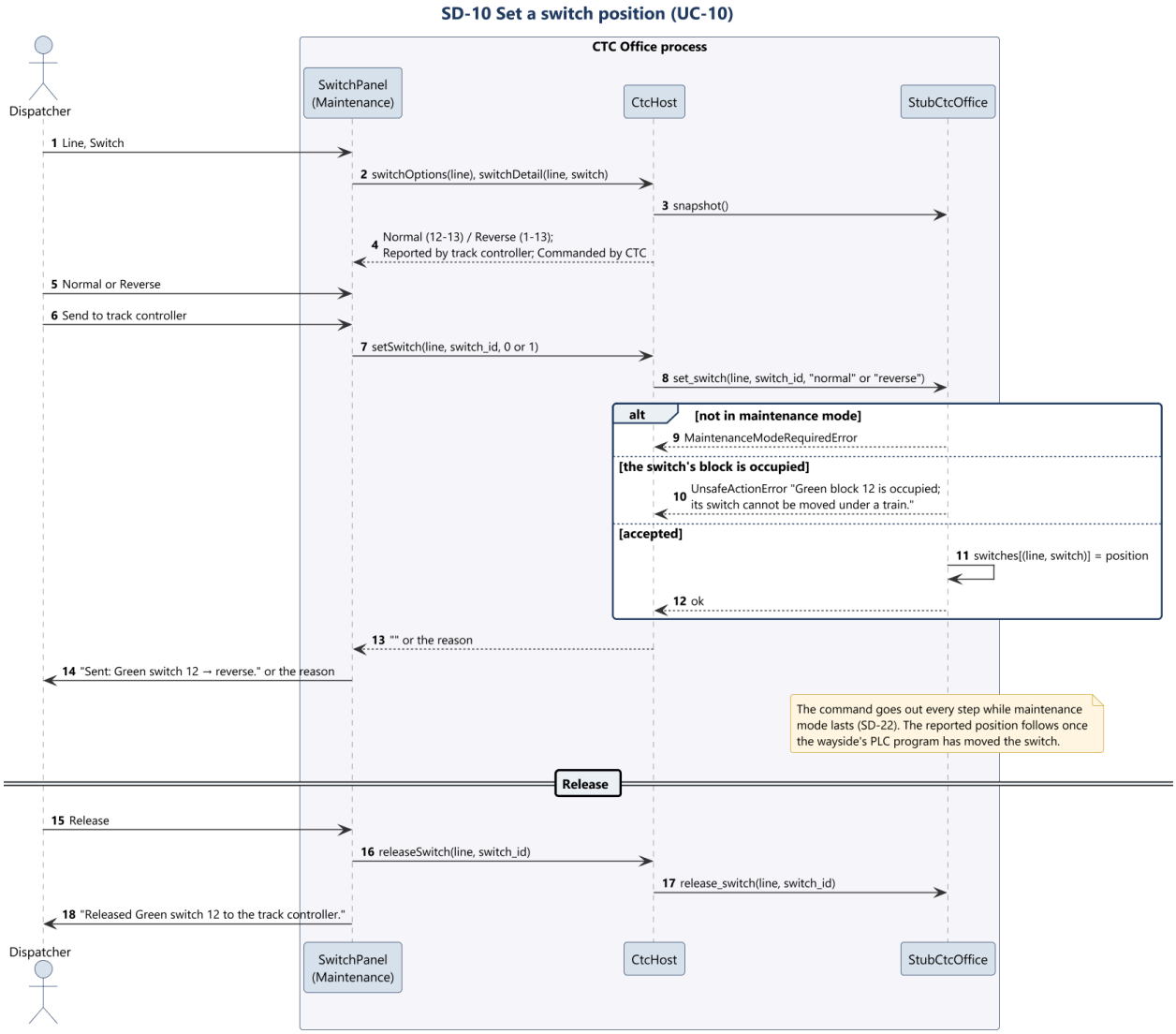

Figure 24. SD-10 Set a switch position

SD-11 Monitor the track map

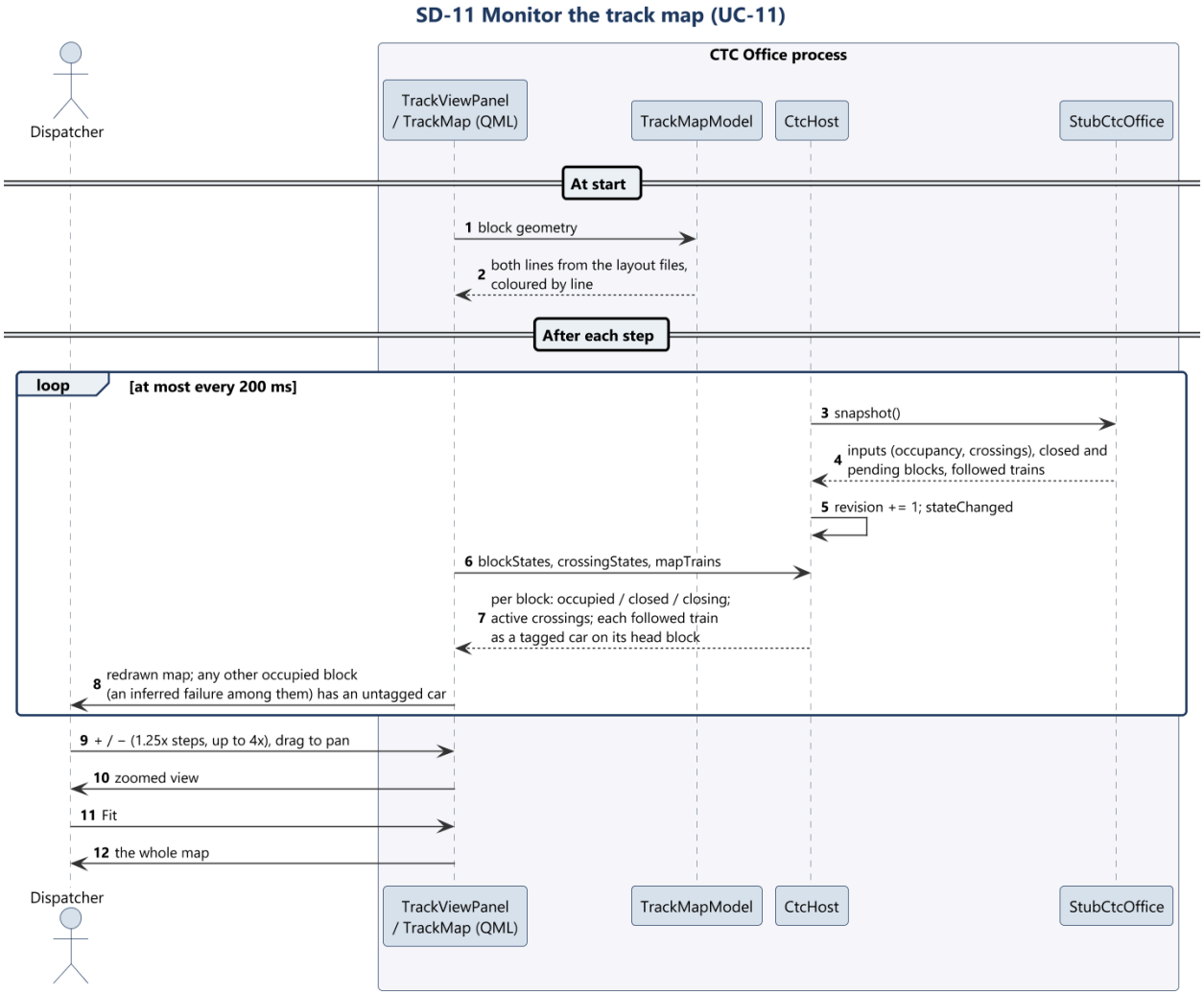

Figure 25. SD-11 Monitor the track map

SD-12 Monitor train occupancy

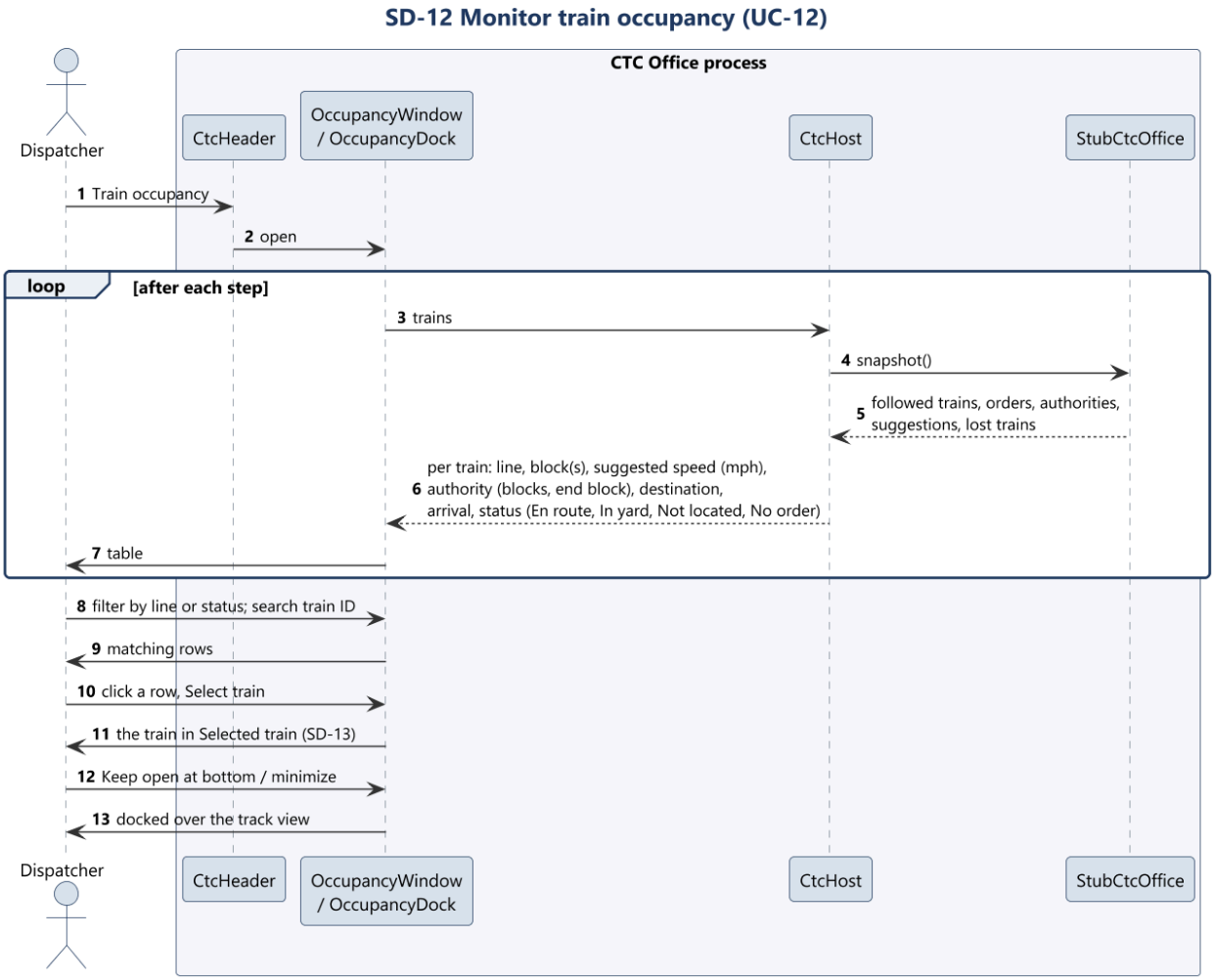

Figure 26. SD-12 Monitor train occupancy

SD-13 View the selected train

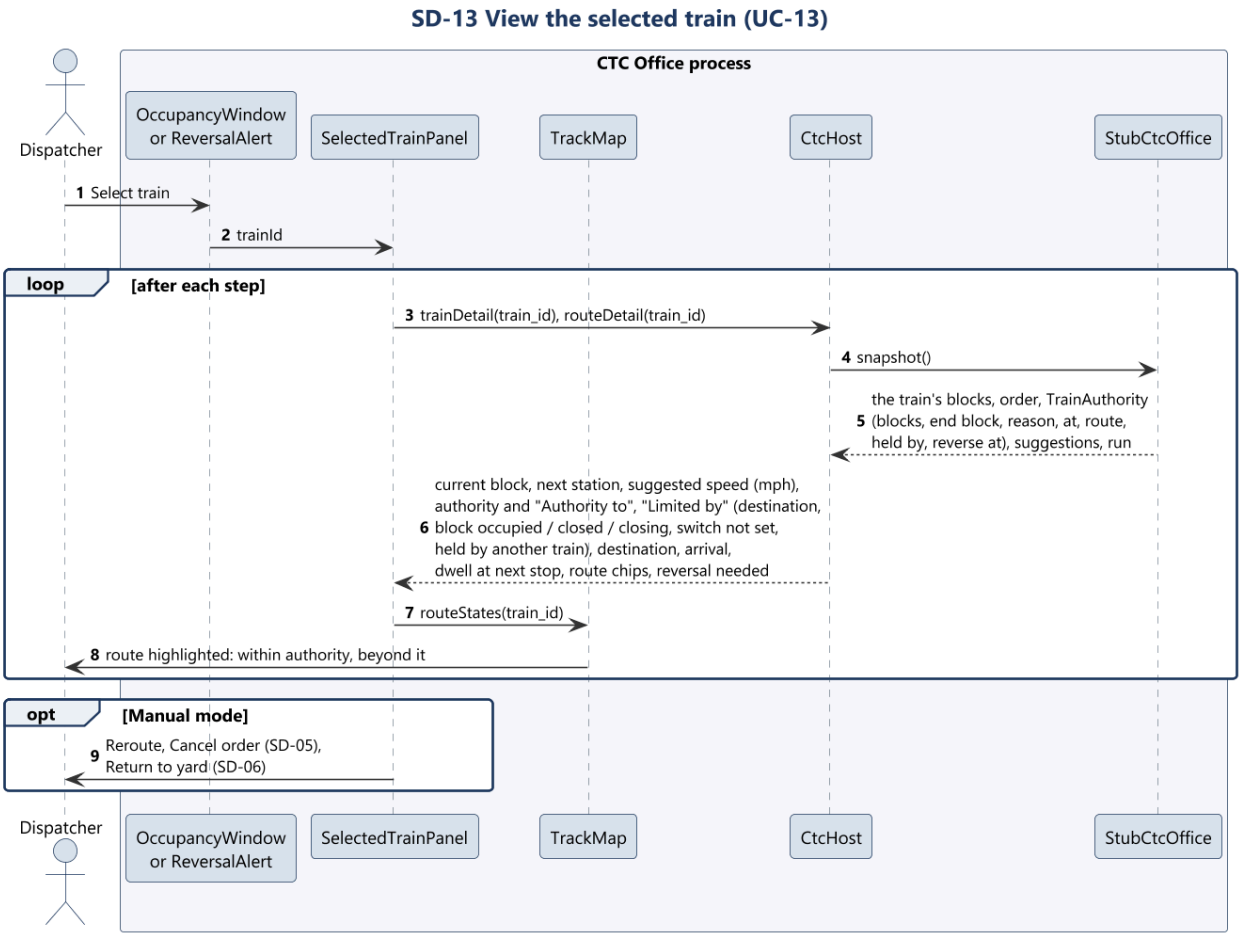

Figure 27. SD-13 View the selected train

SD-14 View throughput metrics

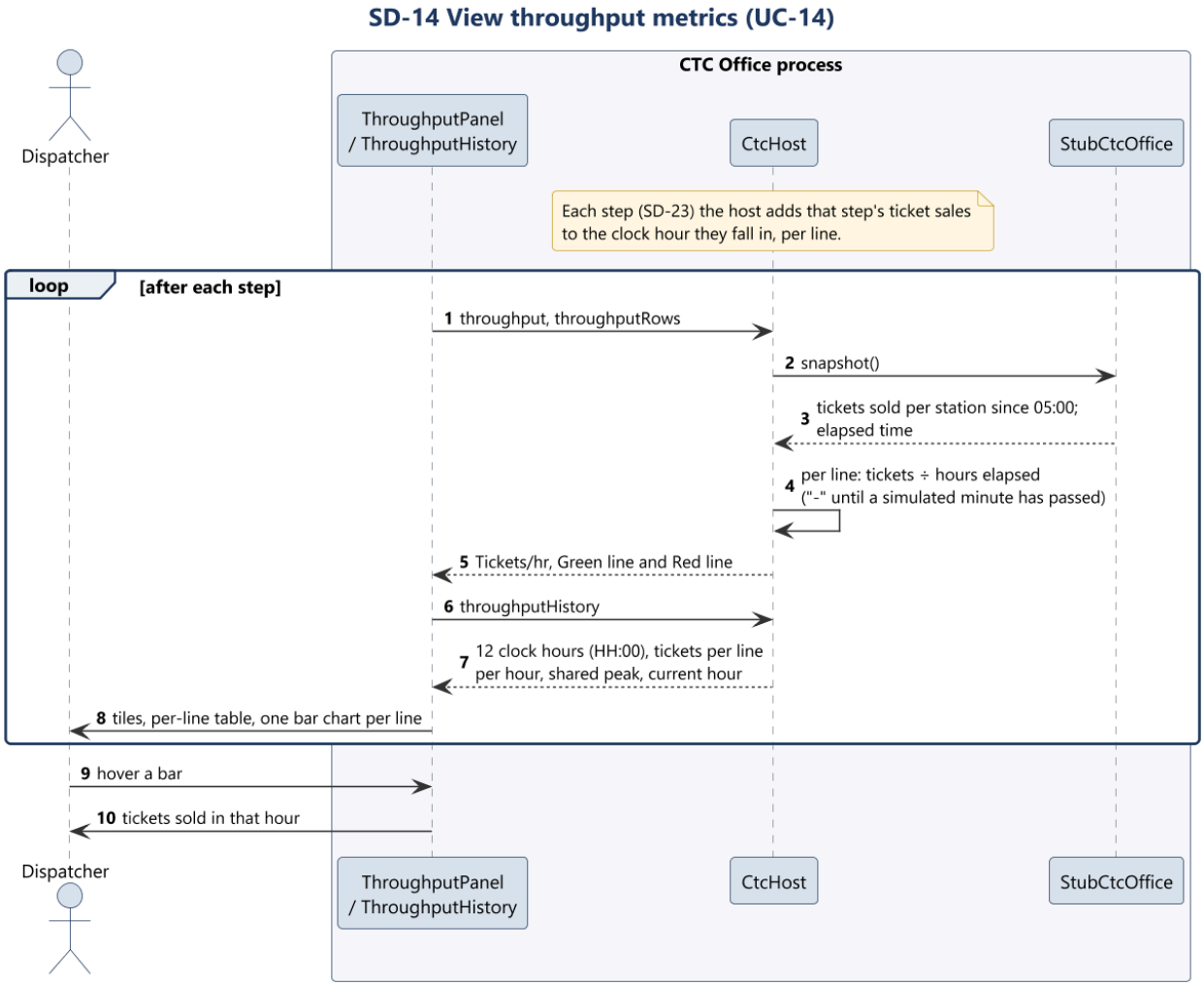

Figure 28. SD-14 View throughput metrics

SD-15 Handle notices and reversal alerts

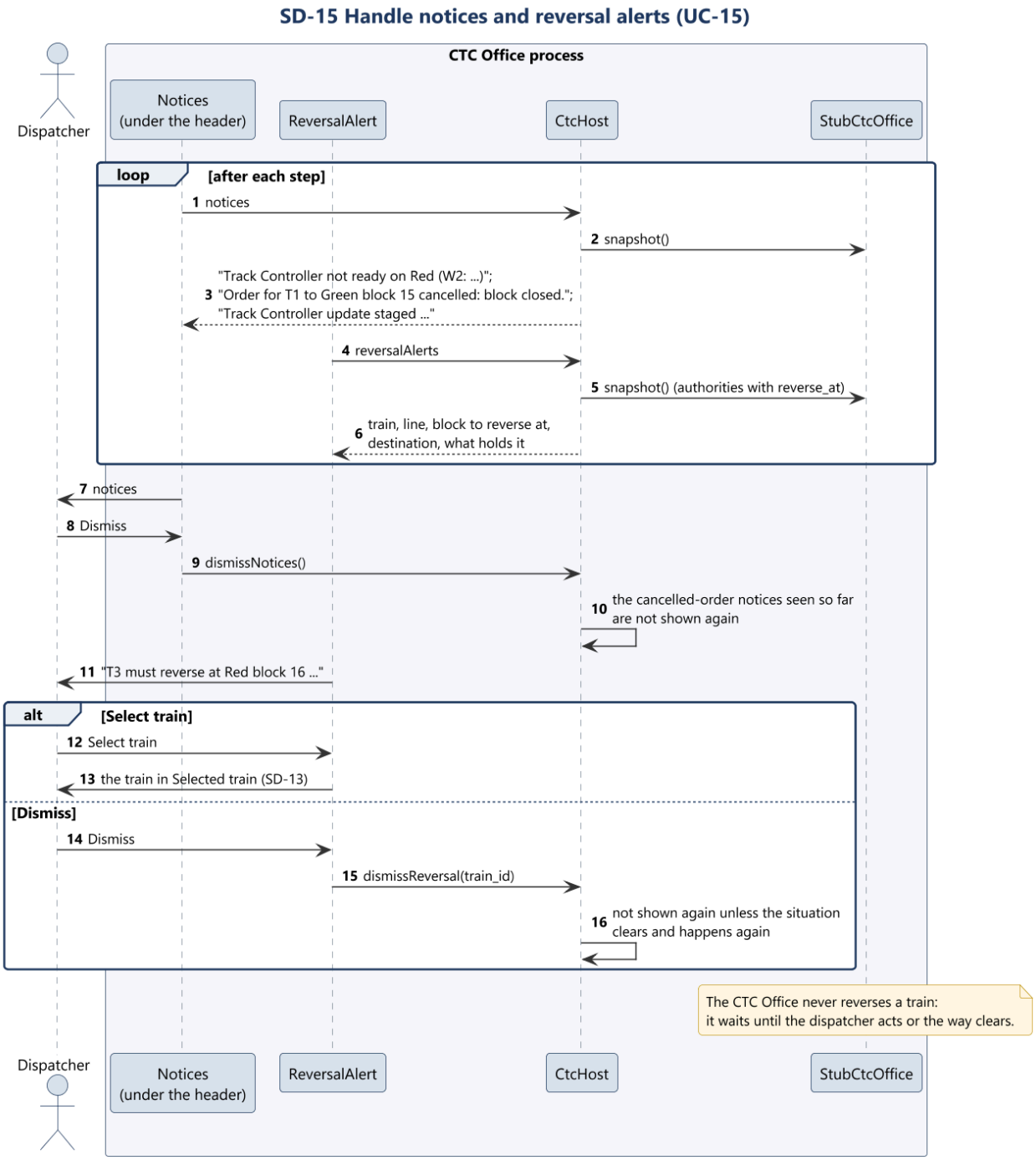

Figure 29. SD-15 Handle notices and reversal alerts

SD-16 Run one step

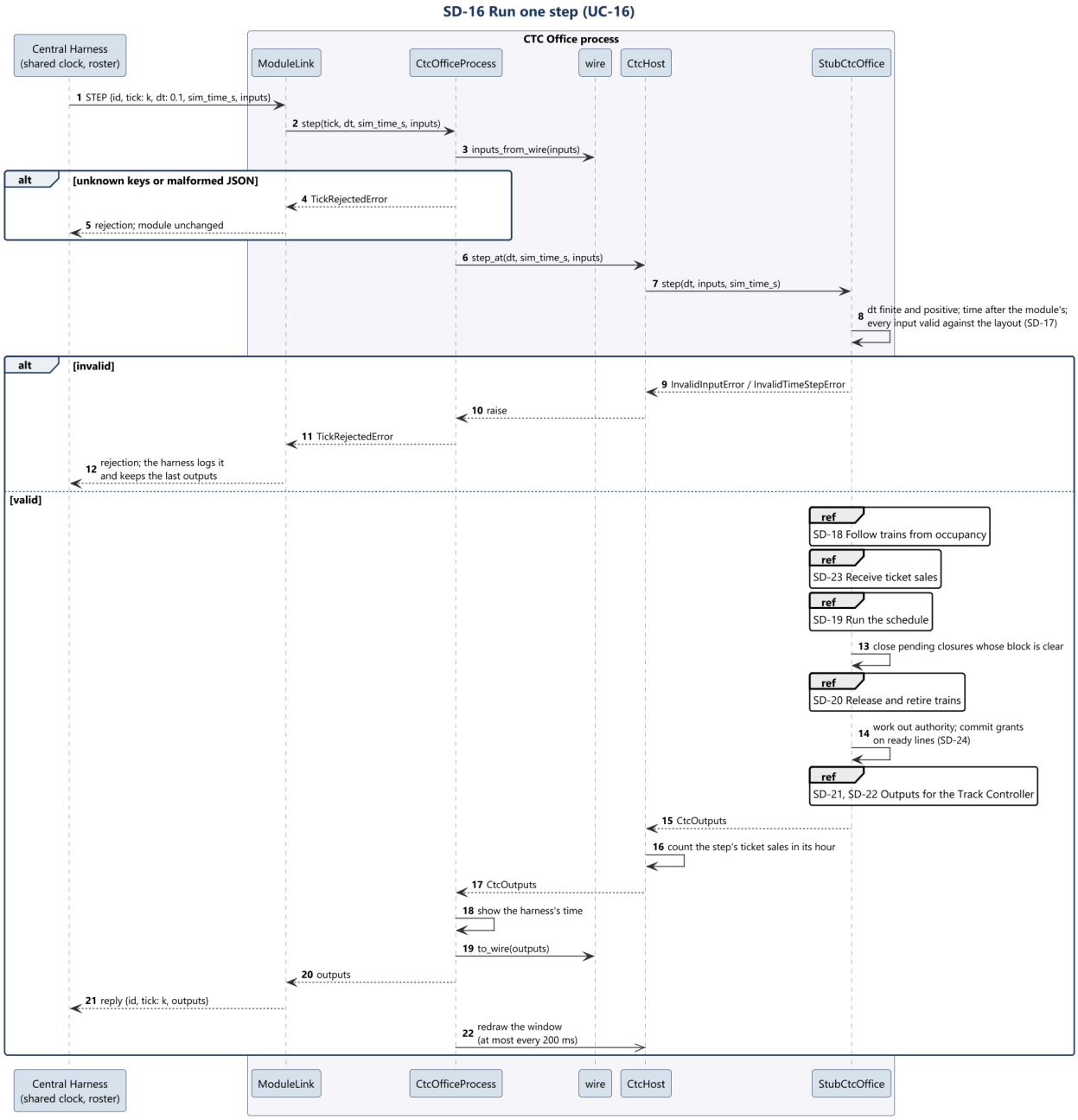

Figure 30. SD-16 Run one step

SD-17 Receive Track Controller reports

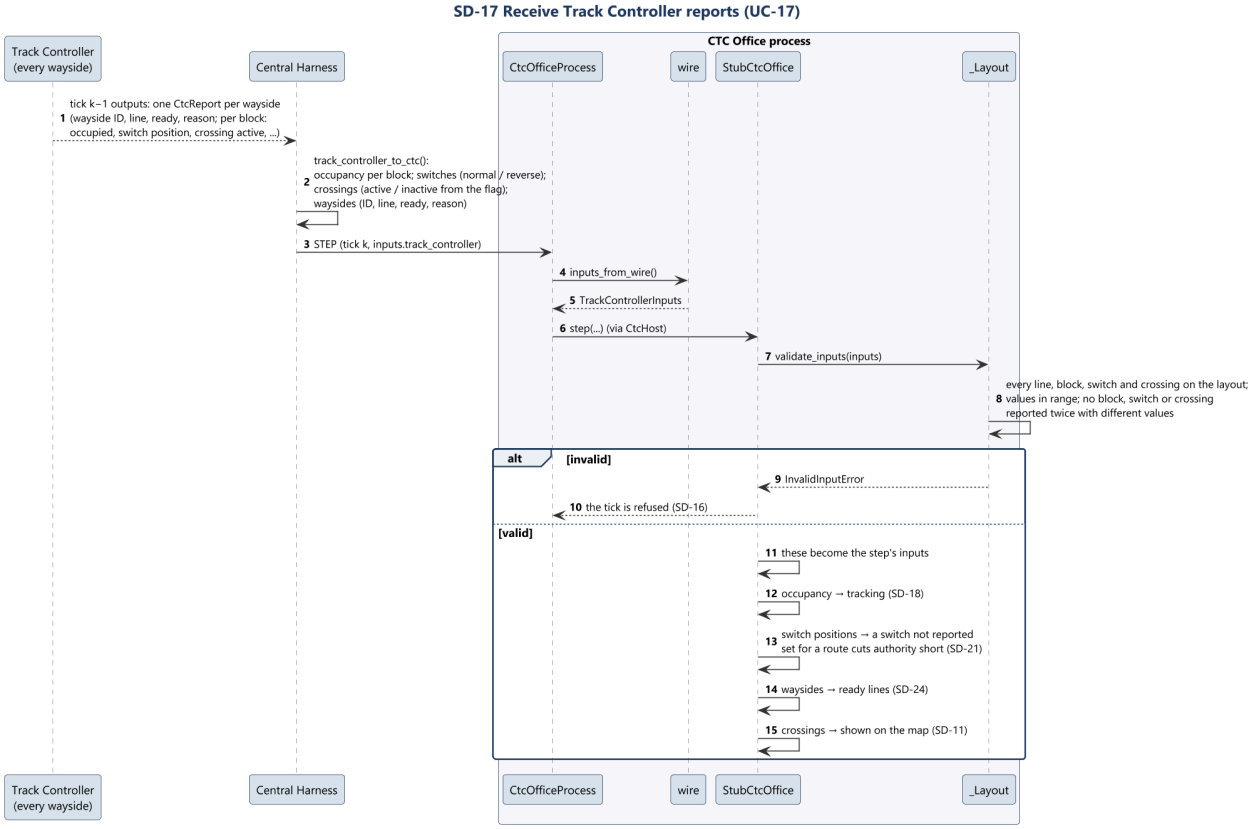

Figure 31. SD-17 Receive Track Controller reports

SD-18 Follow trains from occupancy

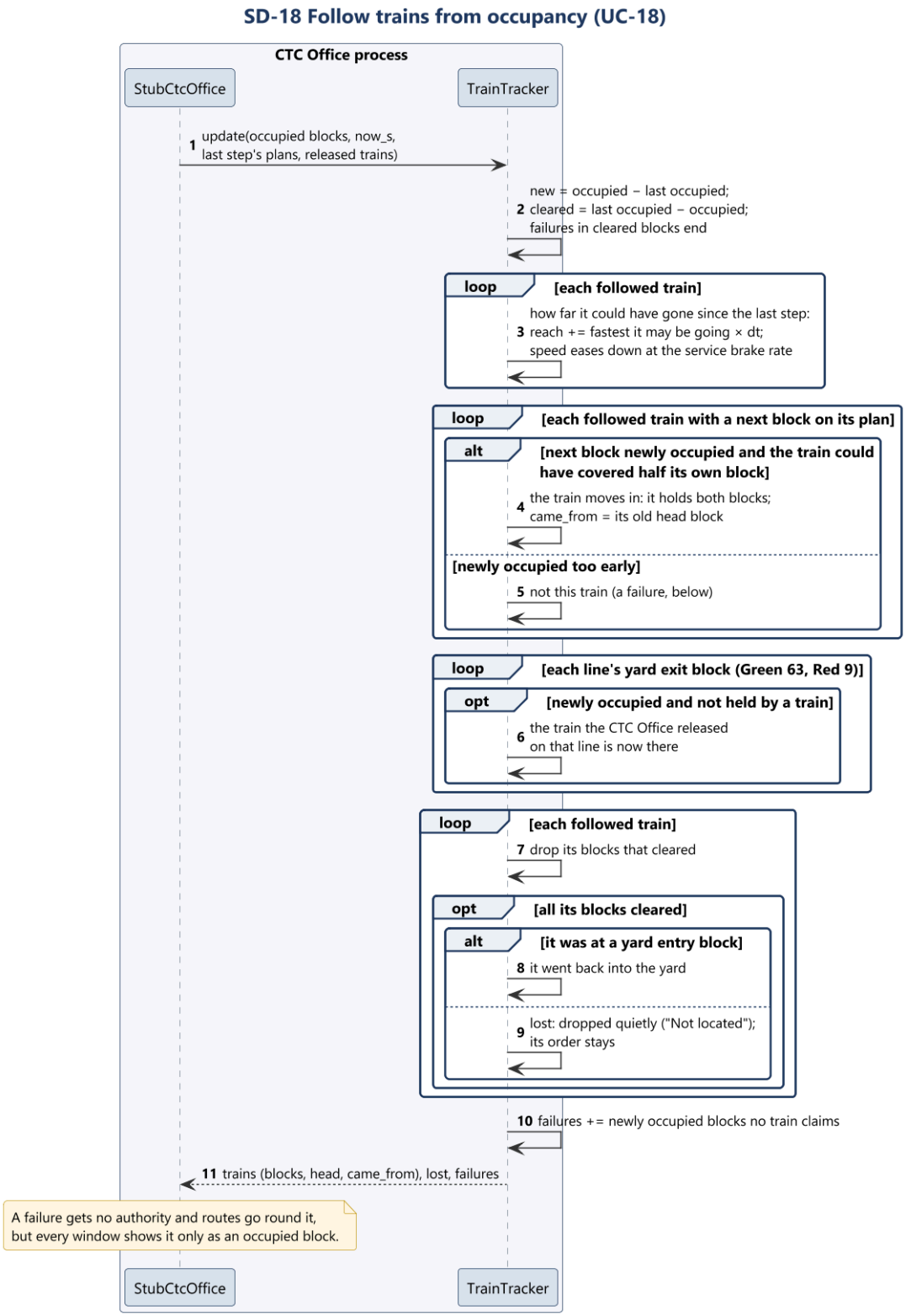

Figure 32. SD-18 Follow trains from occupancy

SD-19 Run the schedule

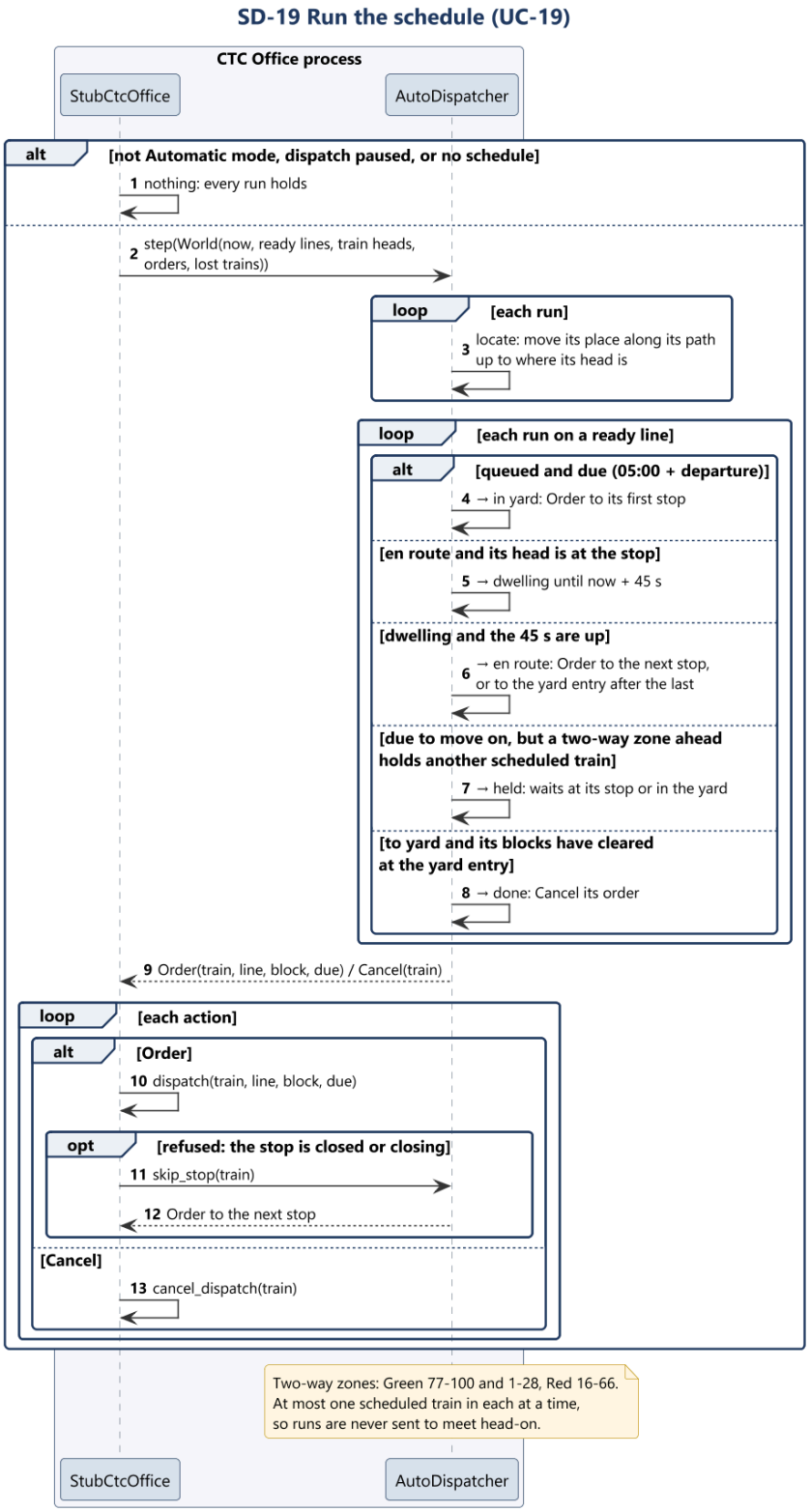

Figure 33. SD-19 Run the schedule

SD-20 Release and retire trains

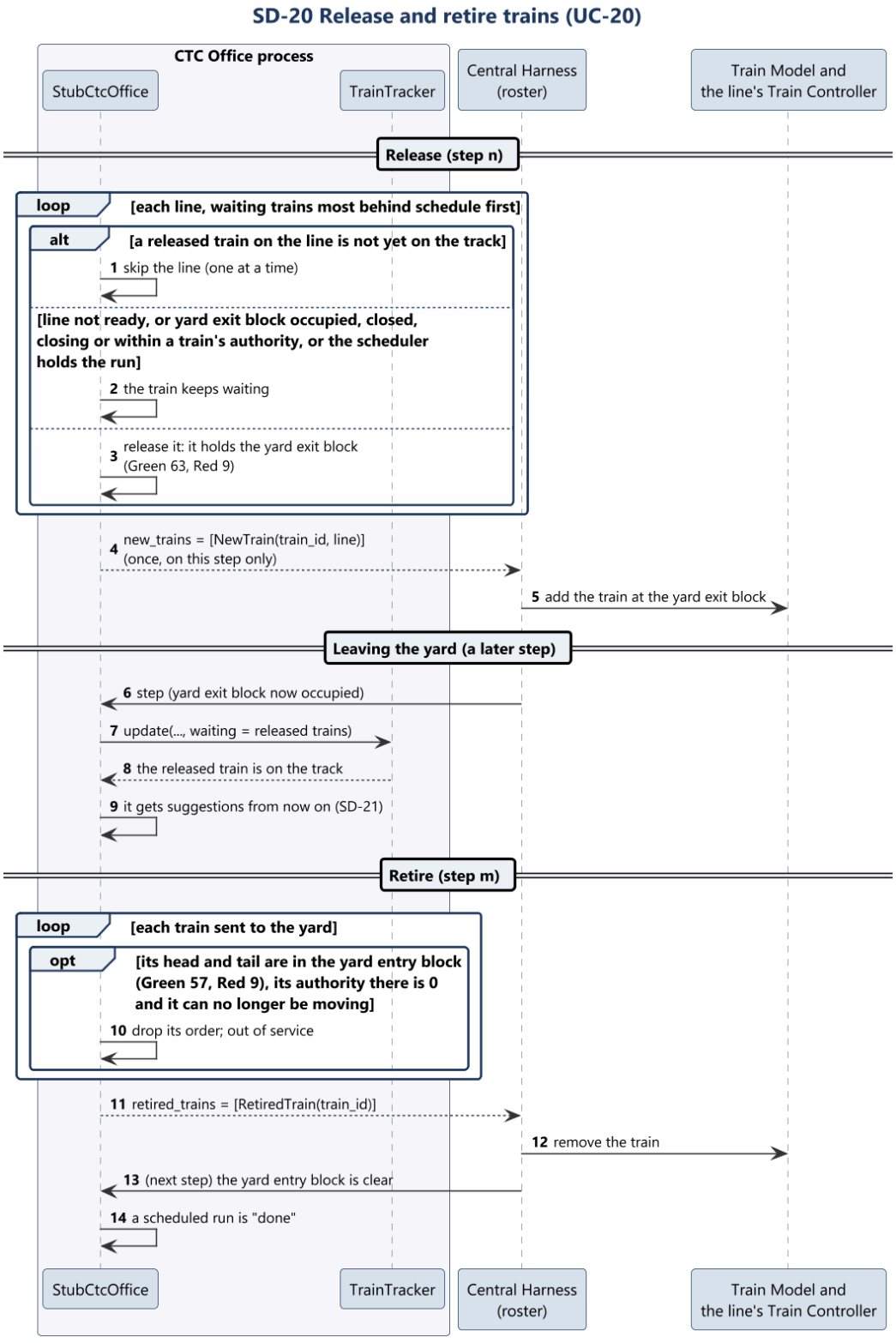

Figure 34. SD-20 Release and retire trains

SD-21 Send speed and authority suggestions

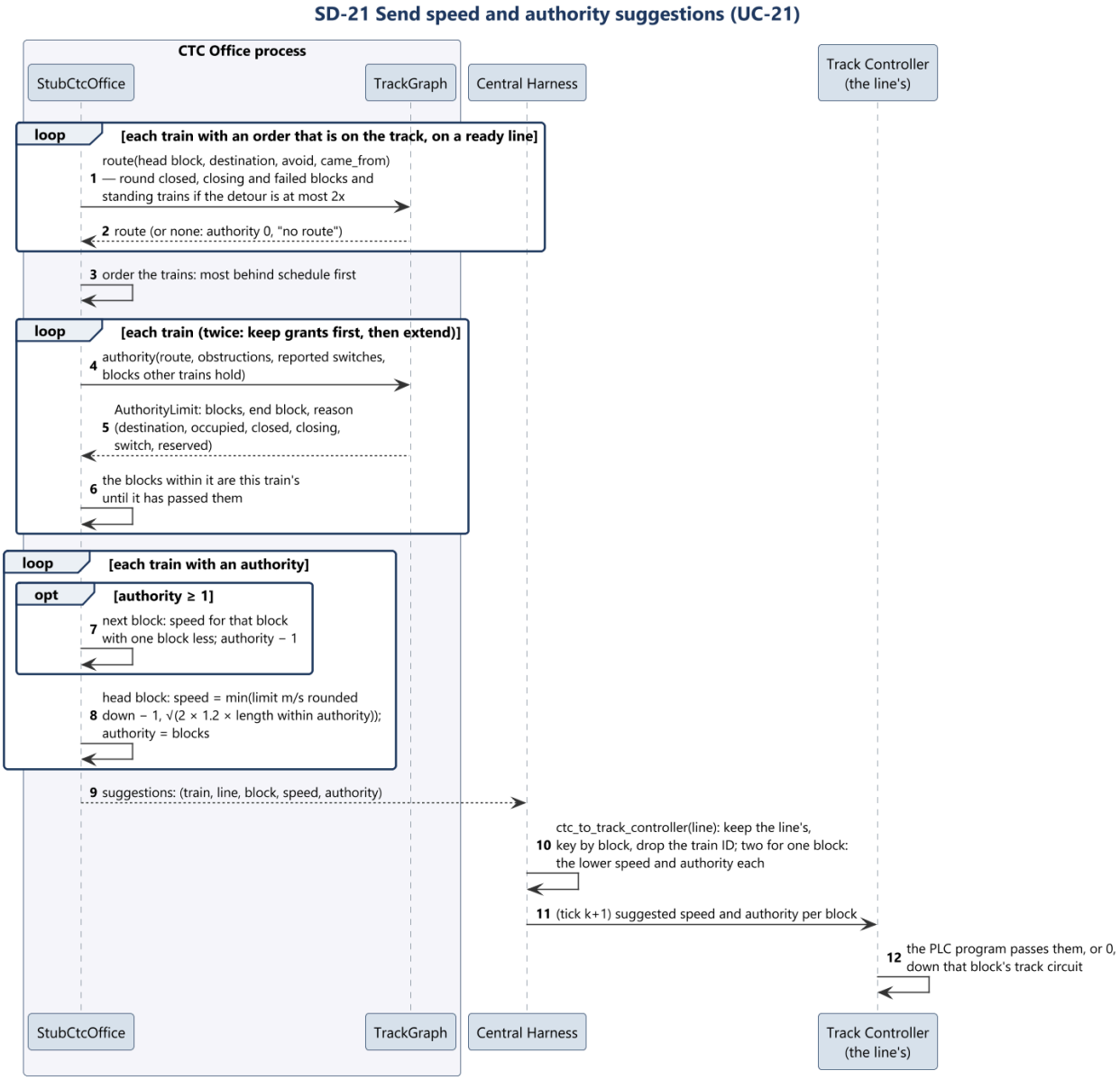

Figure 35. SD-21 Send speed and authority suggestions

SD-22 Send closures and switch commands

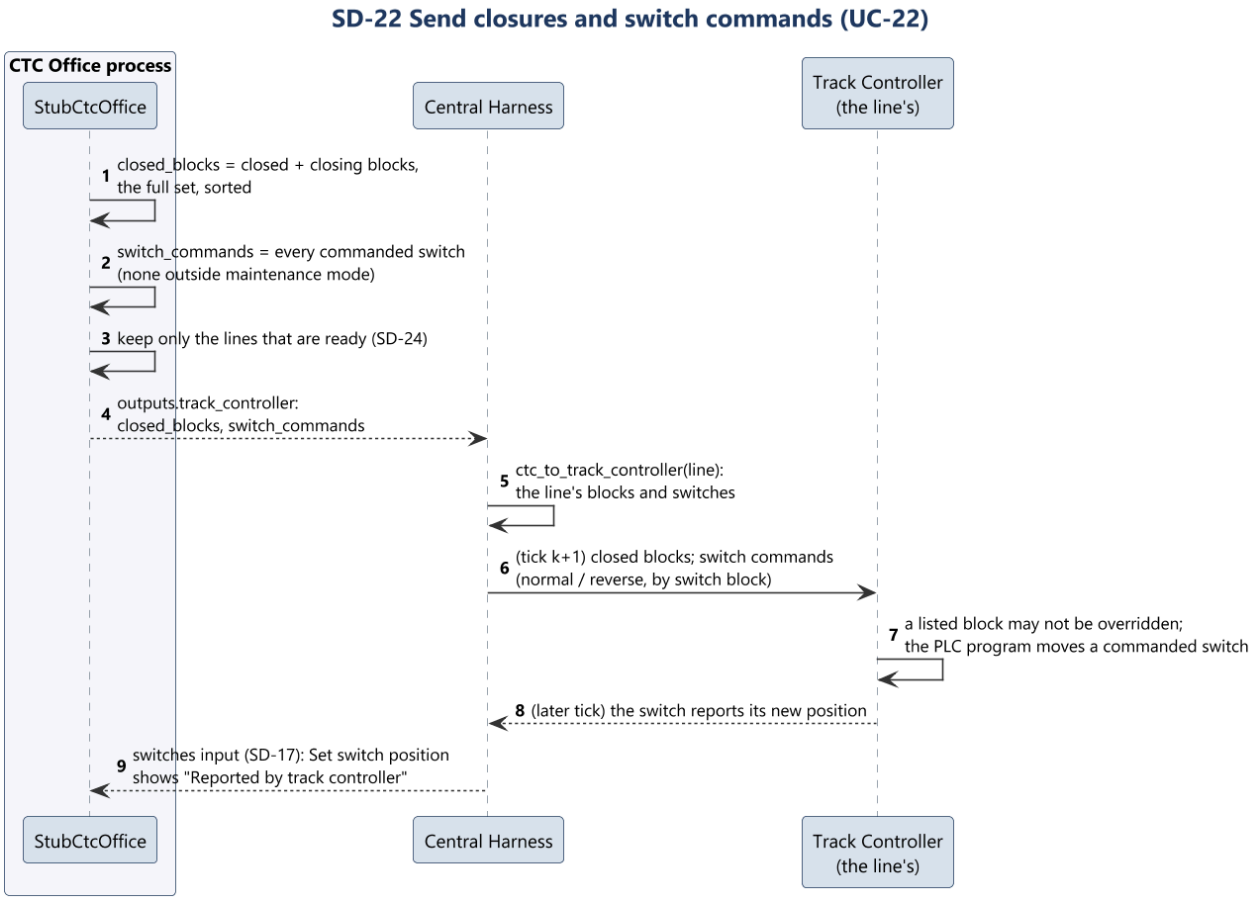

Figure 36. SD-22 Send closures and switch commands

SD-23 Receive ticket sales

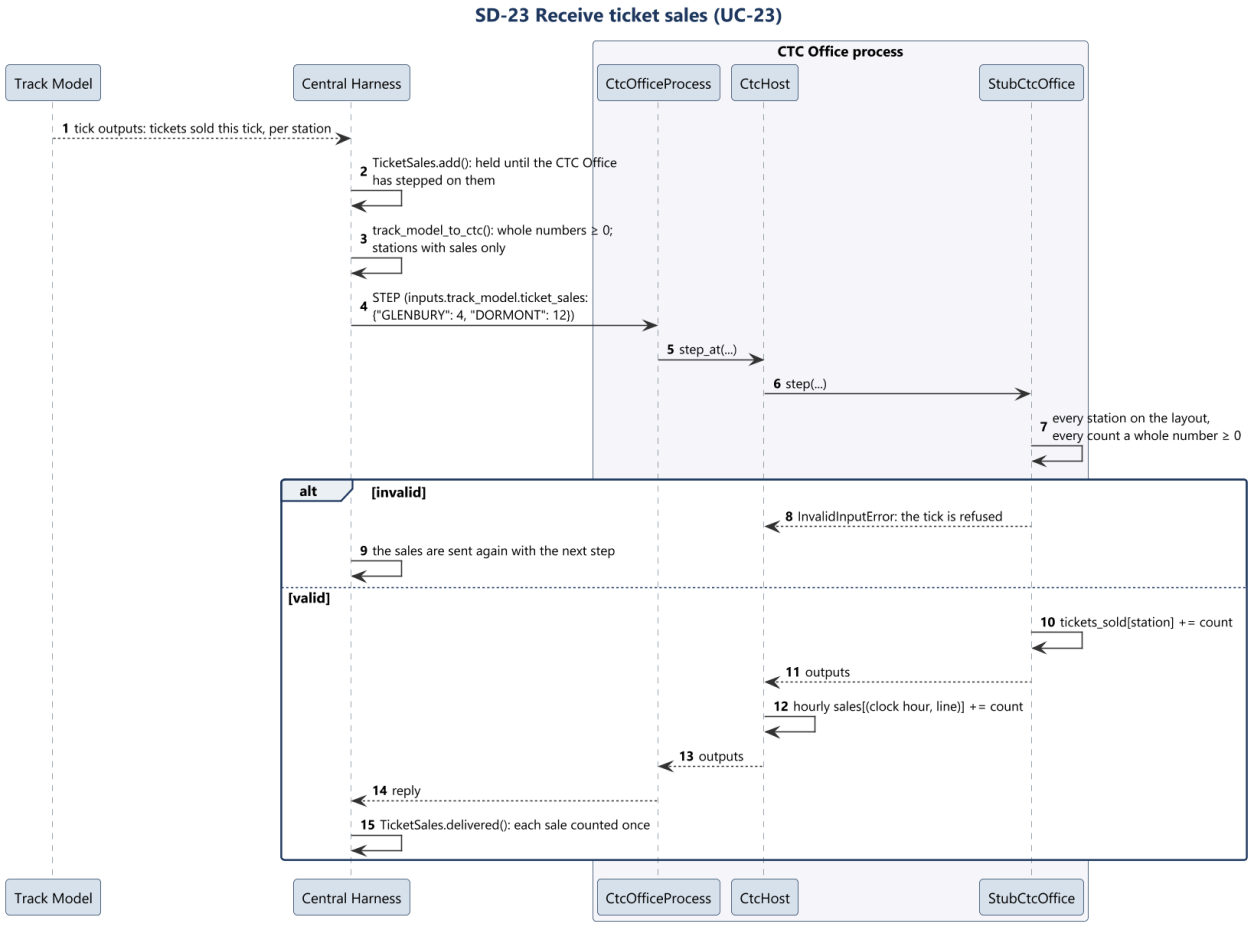

Figure 37. SD-23 Receive ticket sales

SD-24 Hold a line that is not ready

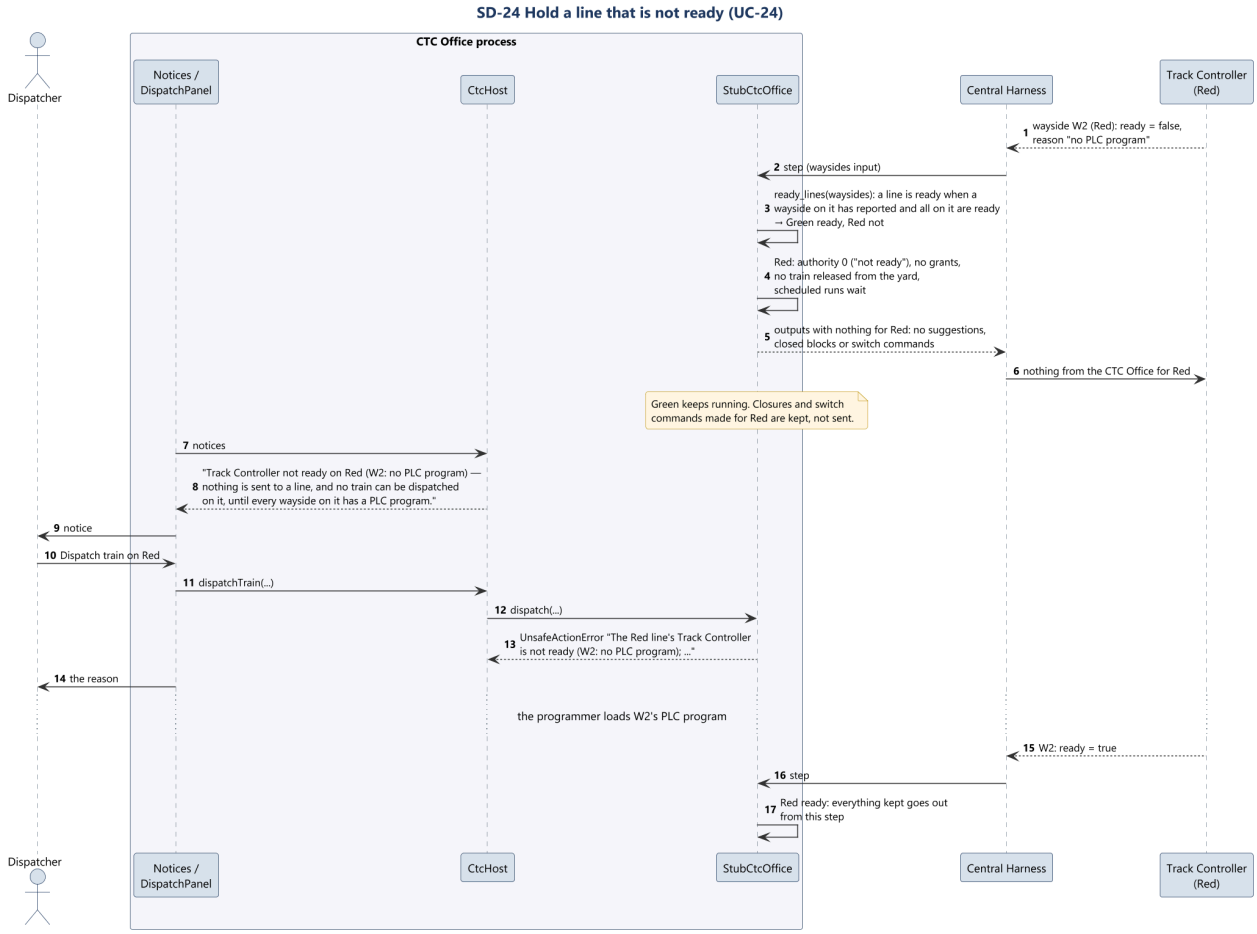

Figure 38. SD-24 Hold a line that is not ready

5.1.8 Open items

#	Item	Owner
1	REQ-INTF-002.3 and 002.4: the map shows blocks, trains and crossings. Switch positions appear in Set switch position and in why a train's authority ends, not on the map. Signal aspects never reach the CTC Office: the waysides report them, but the harness mapping drops them.	Landis
2	Ticket sales: the agreed signal definition is per line, as (count, line) pairs. The CTC Office and the harness use tickets per station. The definition should be updated to match.	Landis

#	Item	Owner
3	The naming convention makes enumerated values all caps (`NORMAL`, `REVERSE`), but the CTC Office and the switch command definition use lower case for switch positions and crossing states. The harness mapping or the module must convert, and the two should agree.	Landis, Kevin
4	D020 says the CTC Office holds a train at its station for the dwell; D007 says the Train Controller holds the dwell. Both hold: the CTC Office keeps the authority at the station for 45 s and the Train Controller holds the doors for 45 s. D020's wording should say so.	Landis
5	Throughput: trains per hour and dwell per line stay empty; nothing reports that data.	Landis
6	The CTC Office's integration changes are on `claude/quirky-fermi-sickg4`, not yet on `development`.	Kevin, Landis

5.2 Track Controller (SW)

Owner: Harry Tye. **Source:** sdd/track_ctrl_sw.md.

Module	Track Controller, software variant (`TrackCtrlSW/`, package `track_ctrl`; test UI `track_ctrl_test`)
Owner	Harry Tye
Status	Draft for WP2 and the design review, 2026-10-21
Basis	Branch `Track-Ctrl-SW` (`f6900f5`); decisions D005, D006, D013, D017, D021 on `truth`

Reading the figures. Orange elements marked «planned» are designed but not built yet. Everything else exists on `Track-Ctrl-SW`. One part of the design is planned:

- **The central harness link** (D005). Today the module is driven by its test UI, over a named pipe. In the design, the central harness spawns the module as a process and steps it with newline JSON. The calls behind the pipe (`step(dt, inputs)`) are the same, so only the endpoint is new.

5.2.1 Scope

The software Track Controller runs every wayside controller of the **Green Line**. The hardware Track Controller runs the Red Line (D021).

- **Territory.** The Green Line is divided into six waysides. The track fixes that number; the user does not choose it. Each wayside governs its blocks, switches, signals and crossings.
- **PLC program.** Each wayside runs its own Boolean PLC program, written in the same language as the hardware variant.
- **Vital code.** Every program has generated vital PLC code attached after it, and only that code writes the outputs.
- **Keyed by block.** The module knows the blocks it governs, never the trains on them. Everything it exchanges is keyed by block (`BlockKey(line, block_id)`).
- **Failures.** A broken rail, a dead track circuit or a power failure reaches it only as an occupied block.

5.2.2 Requirements coverage

At least one use case (section 3) covers each Track Controller item of the functional rubric and the SRS requirements behind it.

Rubric	Points	Requirement	SRS	Use cases
6.0	2	Wayside receives suggested speed and authority from the CTC	REQ-FUNC-049	UC-10, UC-16
6.1	2	Wayside moves switches by PLC program execution	REQ-FUNC-052	UC-09, UC-13
6.2	1	Wayside sets traffic light colour by PLC program execution	REQ-FUNC-053	UC-09, UC-14
6.3	2	Wayside receives train presence from the Track Model	REQ-FUNC-054	UC-12
6.4	2	Wayside sends track occupancy to the CTC	REQ-FUNC-055	UC-17, UC-07
6.5	1	Railway crossing lights and gates activated	REQ-FUNC-056	UC-15
6.6	2	PLC language based only on Boolean variables	REQ-FUNC-051	UC-04, UC-05
6.7	1	Load PLC file	REQ-FUNC-050	UC-03, UC-06
6.8	1	In maintenance mode, manually set a switch position	REQ-FUNC-057, REQ-FUNC-012	UC-08, UC-11, UC-13
6.9	1	Has safety-critical architecture	REQ-FUNC-058, REQ-FUNC-058.1	UC-05, UC-09, UC-16, UC-18
—	—	Failures detected (as occupancy)	REQ-FUNC-059	UC-12
9.0	5	Each sub-system has a user interface	REQ-INTF-001.5	UC-01 to UC-08
—	—	Track layout loaded from JSON	REQ-INTF-012	UC-01
—	—	One shared clock	REQ-INTF-013	UC-09
—	—	Channels to and from the CTC Office, Track Model and train	REQ-INTF-017 to 020	UC-10, UC-11, UC-12, UC-16, UC-17

Rubric 6.8 is met in two ways:

- The CTC Office's dispatcher sets a switch in maintenance mode, and the vital code carries the command out (UC-11, UC-13).

- While maintenance is in effect, the programmer may also set a switch by hand at the wayside (UC-08). The CTC Office's command takes precedence over the hand-set position.

The hardware variant has only the first (section 8, item 4).

5.2.3 Use cases

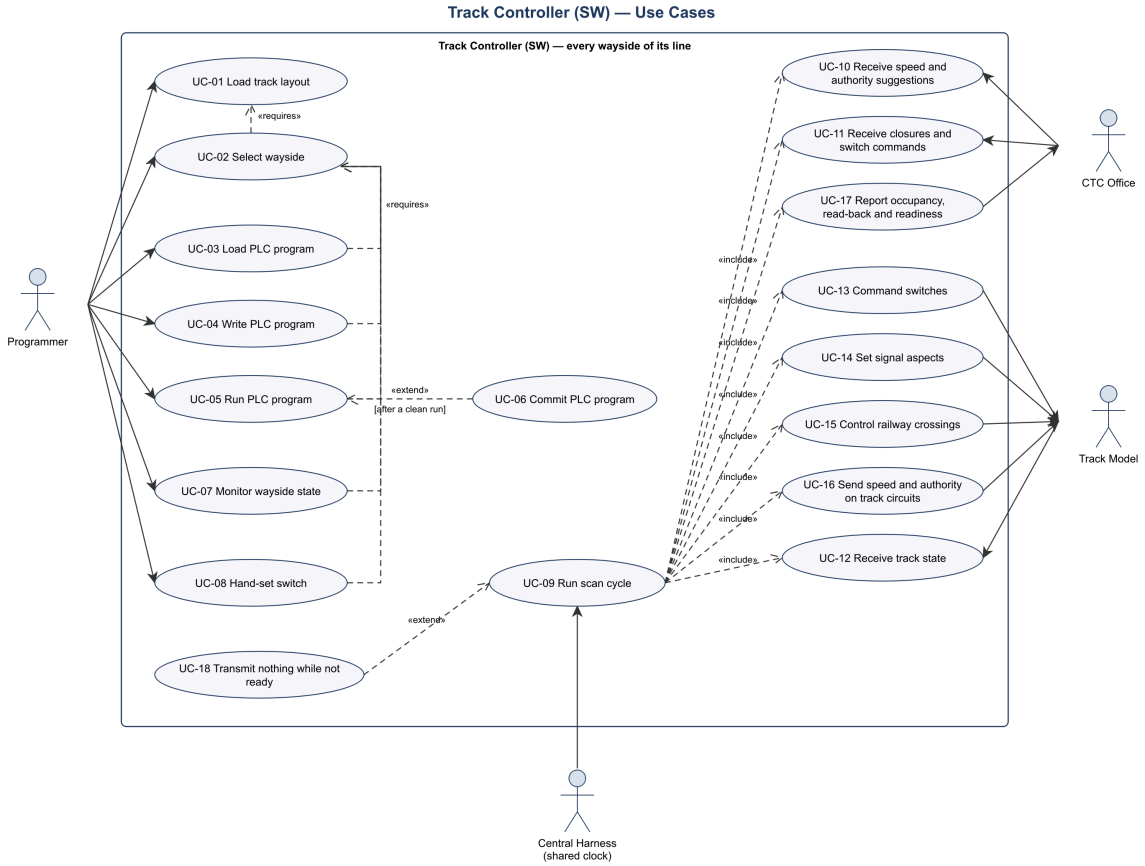

Figure 39. Track Controller (SW): use case diagram

Actors:

- **Programmer** — the person at the Track Controller window. Loads the track and PLC programs, writes and runs programs, watches the waysides and sets switches by hand in maintenance mode. Every interactive control of the window belongs to one of UC-01 to UC-08.
- **Central Harness** — owns the shared clock and steps the module once per tick (D005, D017). It carries everything the CTC Office and the Track Model send and receive.
- **CTC Office** and **Track Model** — the neighbouring modules, reached only through the harness. The diagram gives each of them its own use cases, all included in the scan cycle (UC-09).

«requires» on the diagram marks a precondition. Every programmer use case after UC-02 needs a selected wayside, and UC-02 needs a loaded track.

UC-01 Load track layout

Actors	Programmer
Description	Loads one line of the course layout and divides it among that line's waysides.
Data	Track layout (JSON): the line, and for each block its section, number, length, grade, speed limit, station, switch and railway crossing. A file in the shape of the course layout, for example `TrackModel/green_line.json`.
Stimulus	The programmer clicks **Load track…**, picks a file, checks the summary and clicks **Load**.
Response	The summary lists the line, block, switch and crossing counts and the number of waysides, which the track fixes (`WAYSIDE_COUNTS`, six for Green). On **Load**, the line's blocks are divided among the waysides, each wayside is created with no program, and its vital code is generated and compiled. The line and its first wayside are selected, the editor is empty and the header reads **NOT READY**. Loading a line that is already loaded replaces it and drops its programs.
Alternatives	A file that cannot be read, is not a layout, or is a single wayside's database is refused with the reason, and nothing changes.
Requirements	REQ-INTF-001.5, REQ-INTF-012, REQ-NFR-003, REQ-NFR-004
Sequence	SD-01

UC-02 Select wayside

Actors	Programmer
Description	Chooses which wayside of a loaded line the window shows.

Data	Line name, wayside ID.
Stimulus	The programmer picks a line, then a wayside, in the header.
Response	The editor, file list, block grid and header show that wayside. The editor holds its committed program, or nothing. Uncommitted edits in the editor are discarded.
Alternatives	No track loaded: the pickers are empty. An unknown wayside leaves the selection as it was.
Requirements	REQ-INTF-001.5
Sequence	SD-02

UC-03 Load PLC program

Actors	Programmer
Description	Reads a PLC program file into the editor of the selected wayside.
Data	PLC source text (`.plc`, UTF-8).
Stimulus	The programmer clicks **Load .plc file** and picks a file.
Response	The program appears in the editor marked **NOT COMMITTED**. Nothing reaches the wayside yet: the program must still be run (UC-05) and committed (UC-06).
Alternatives	A file that cannot be read: the terminal says so and the editor is unchanged.
Requirements	Rubric 6.7; REQ-FUNC-050
Sequence	SD-03

UC-04 Write PLC program

Actors	Programmer
Description	Writes or edits a PLC program in the editor.
Data	The language is Boolean only, and the same as the hardware variant's. A program has `VAR_IN` and `VAR_OUT` declarations, then statements `NAME := expression` built from `NOT`, `AND`, `XOR`, `OR`, `0`, `1` and parentheses. **Inputs** per block `n`: `OCC_n`, `CLOSED_n`, `FAIL_n`. Per switch `s`: `SWREV_s`. And `MAINT`. **Outputs:** `AUTH_n`, `SW_s`, `SIG_s_R`, `SIG_s_Y`, `SIG_s_G`, `SIG_s_SG`, and per crossing `c`: `XING_c`.
Stimulus	The programmer clicks **New**, or types in the editor.
Response	The editor shows **Edited**, and **Commit** is disabled until the new text is run cleanly.
Requirements	Rubric 6.6; REQ-FUNC-051
Sequence	SD-04

UC-05 Run PLC program

Actors	Programmer
Description	Compiles the editor's program and runs one scan of it in a sandbox, so that its effect can be checked before it is committed.
Data	The editor's text, and a copy of the selected wayside's current inputs.

Stimulus	The programmer clicks **Run**.
Response	The program is compiled and then checked against this wayside's I/O: every name must be one of its inputs or outputs, and every output must be driven. One scan then runs on the copy of the inputs. The OS decodes the inputs, the program runs, the attached vital code runs, and the OS encodes the outputs. The window shows the sandbox outputs, the log, and every output the vital code overrode. **Commit** is enabled only if the run had no diagnostic. The live outputs and the track are untouched.
Alternatives	No track loaded: refused. A syntax error, an unknown name or an undriven output: the parser pane lists each problem with its line number, and nothing runs.
Requirements	Rubric 6.6, 6.9; REQ-FUNC-051, REQ-FUNC-058
Sequence	SD-05

UC-06 Commit PLC program

Actors	Programmer (extends UC-05, after a clean run)
Description	Installs the program that was just run on the selected wayside.
Data	The verified source and its file name.
Stimulus	The programmer clicks **Commit iteration**.

Response	The program is compiled and checked again, then installed as the next iteration. The wayside keeps its last five iterations, and any of them can be reopened from the file list. It runs from the next scan, and the editor shows **COMMITTED**. Once every wayside on the line has a program, the header's **NOT READY** clears and the line's waysides report ready (UC-17).
Alternatives	No clean run of exactly this text, or any diagnostic: refused, the terminal says why, and the previous program keeps running.
Requirements	Rubric 6.7; REQ-FUNC-050
Sequence	SD-06

UC-07 Monitor wayside state

Actors	Programmer
Description	Shows the live state of the selected wayside, in imperial units.
Data	**Block grid:** occupancy and closure per block. **Outputs:** track-circuit speed and authority, switch, signal and crossing commands. **Also shown:** the CTC suggestions, the vital overrides, maintenance mode, the clock, and READY or NOT READY with the reason.
Stimulus	The programmer opens the **View** tab. The window refreshes after each scan, at most every 200 ms.
Response	The rows update in place. Speeds are converted from m/s to mph at the edge. This view is read-only.
Requirements	REQ-INTF-001.5, REQ-INTF-011. It displays rubric 6.1 to 6.5.
Sequence	SD-07

UC-08 Hand-set switch

Actors	Programmer
Description	In maintenance mode, sets one of the wayside's switches by hand.
Data	Switch ID; normal or reverse.
Stimulus	The programmer clicks **Set switch manually**, picks a switch and a position, or clicks **Release**.
Response	The position is held as `HAND_s` and `HANDREV_s`, which only the vital code can read; no program can declare them. Precedence is the CTC command, then the hand-set position, then the program. The position is sent to the Track Model on the next scan (UC-13). It holds until it is released or maintenance ends.
Alternatives	Not in maintenance: refused, with the reason.
Requirements	Rubric 6.8; REQ-FUNC-057
Sequence	SD-08

UC-09 Run scan cycle

Actors	Central Harness (primary); the CTC Office and Track Model through it
Description	Once per tick, every wayside of a ready line runs its program and the attached vital code against that tick's inputs, and drives the result.
Data	`dt` (fixed, D006), the simulation time, and the tick's inputs (UC-10 to UC-12).

Stimulus	`step(dt, inputs)` from the harness.
Response	The whole tick is validated first. Maintenance is in effect while the CTC Office has a switch command in effect on a switch the module governs. Each wayside then scans: the OS decodes the inputs to bits, the user program runs, and the vital code runs and writes the `OUT_` bits. The OS encodes those bits into commands. The scan produces track-circuit, switch, signal and crossing commands (UC-13 to UC-16) and one report per wayside (UC-17), all returned before the next tick.
Alternatives	An invalid `dt` or input rejects the whole tick before anything changes. A line that is not ready triggers UC-18.
Requirements	Rubric 6.1, 6.2, 6.9; REQ-FUNC-058, REQ-INTF-013, REQ-NFR-002
Sequence	SD-09

UC-10 Receive speed and authority suggestions

Actors	CTC Office (through the harness)
Description	Takes the CTC Office's suggested speed and authority for each block.
Data	Per block: `Suggestion(speed_mps, authority_blocks)`, both whole numbers of 0 or more (D013). The CTC Office sends `TrainSuggestion(train_id, line, block_id, suggested_speed_mps, authority_blocks)`. The harness (`ctc_edges`) keys it by block and drops the train ID. If two suggestions name one block, the more restrictive speed and authority win.
Stimulus	Every tick.

Response	Each suggestion is stored on the wayside that governs the block. The numbers never enter the PLC: the program decides only `AUTH_n`, pass or stop (UC-16). A block with no suggestion gets no track-circuit command.
Alternatives	A negative or fractional value rejects the tick. Blocks no wayside governs are ignored.
Requirements	Rubric 6.0; REQ-FUNC-049, REQ-INTF-019
Sequence	SD-10

UC-11 Receive closures and switch commands

Actors	CTC Office (through the harness)
Description	Takes the blocks the dispatcher has closed and, in maintenance mode, the switch positions the dispatcher commands.
Data	Closed blocks, and switch commands `{switch block: normal or reverse}`. The CTC Office sends switch commands only in maintenance mode, and sends no maintenance flag.
Stimulus	Every tick.
Response	The program receives `CLOSED_n` and `MAINT`. Only the vital code can read `CTCCMD_s` and `CTCREV_s`. A closed block gets no authority, and a commanded switch follows the command. Ending maintenance releases every hand-set switch and CTC command.
Requirements	Rubric 6.8; REQ-FUNC-057, REQ-FUNC-012
Sequence	SD-11

UC-12 Receive track state

Actors	Track Model (through the harness)
Description	Takes occupancy and the actual state of the waysides' equipment.
Data	Occupied blocks (a train and a failure look the same), switch positions, crossing states and signal aspects lit.
Stimulus	Every tick.
Response	The program receives `OCC_n` and `SWREV_s`, and the read-back goes into the report (UC-17). A failed block is protected exactly like an occupied one. `FAIL_n` is always 0, because the Track Model reports failures as occupancy.
Requirements	Rubric 6.3; REQ-FUNC-054, REQ-FUNC-059, REQ-INTF-018
Sequence	SD-12

UC-13 Command switches

Actors	Track Model
Description	Drives each switch machine from `OUT_SW_s`.
Data	`SW_s` from the program; `OUT_SW_s`, where 1 is reverse and 0 is normal.
Stimulus	Each scan (UC-09).
Response	The vital code picks the request: the CTC command, else the hand-set position, else the program. It keeps a switch on an occupied block where it actually is. While a switch moves, its block's `OUT_AUTH_n` is 0. The scan sends a switch command to the Track Model.

Requirements	Rubric 6.1, 6.8; REQ-FUNC-052
Sequence	SD-13

UC-14 Set signal aspects

Actors	Track Model
Description	Lights each signal from the program's four aspect outputs.
Data	`SIG_s_R`, `_Y`, `_G`, `_SG`.
Stimulus	Each scan.
Response	The vital code passes the program's aspect only if exactly one aspect is lit and the block is clear and open. Otherwise the signal shows red. A yellow signal sets `OUT_HALF_n`, which halves its block's speed. The scan sends a signal command to the Track Model.
Requirements	Rubric 6.2; REQ-FUNC-053
Sequence	SD-14

UC-15 Control railway crossings

Actors	Track Model
Description	Activates each crossing from `OUT_XING_c`.
Data	`XING_c` from the program; the occupancy of the approach blocks.
Stimulus	Each scan.

Response	`OUT_XING_c` is `XING_c` OR a train on the approach. When it is 1, the lights are on and the gates down. The scan sends a crossing command to the Track Model.
Requirements	Rubric 6.5; REQ-FUNC-056
Sequence	SD-15

UC-16 Send speed and authority on track circuits

Actors	Track Model (delivers them to the train in the block)
Description	Sends each block that has a suggestion its commanded speed and authority.
Data	`OUT_AUTH_n = AUTH_n AND NOT CLOSED_n AND NOT (its switch moving)`; `OUT_HALF_n`.
Stimulus	Each scan.
Response	Each block with a suggestion gets one track-circuit command: the suggestion when `OUT_AUTH_n` is 1 (half the speed if `OUT_HALF_n` is set), else speed 0 and authority 0. The CTC Office keeps speeds under the posted limit and gives zero speed for zero authority.
Requirements	Rubric 6.0, 6.9; REQ-INTF-017, REQ-FUNC-058.1
Sequence	SD-16

UC-17 Report occupancy, read-back and readiness

Actors	CTC Office (through the harness)
Description	Tells the CTC Office what each wayside sees, every tick.

Data	One `CtcReport` per wayside: wayside, line, time, ready, and the reason if not. Per block: occupied, and the Track Model's own switch, crossing and signal read-back. No failures, no trains.
Stimulus	Each scan.
Response	The harness passes the reports to the CTC Office. The CTC Office dispatches nothing on a line until every wayside on it is ready.
Requirements	Rubric 6.4; REQ-FUNC-055, REQ-INTF-020
Sequence	SD-17

UC-18 Transmit nothing while not ready

Actors	Central Harness (extends UC-09)
Description	A line where any wayside has no program runs nothing and transmits nothing.
Data	Why: no track loaded, or a wayside on the line has no PLC program.
Stimulus	Either condition at a scan.
Response	Every wayside of that line idles. It sends no switch, signal, crossing or track-circuit command. Its outputs are absent, not safe defaults, because a wayside running no program cannot drive anything. The reports say not ready and why, the window shows **NOT READY** with the reason, and the CTC Office holds the line. Another, ready line is not held.
Requirements	Rubric 6.9; REQ-FUNC-058
Sequence	SD-18

5.2.4 Track Controller (SW) architecture (component view)

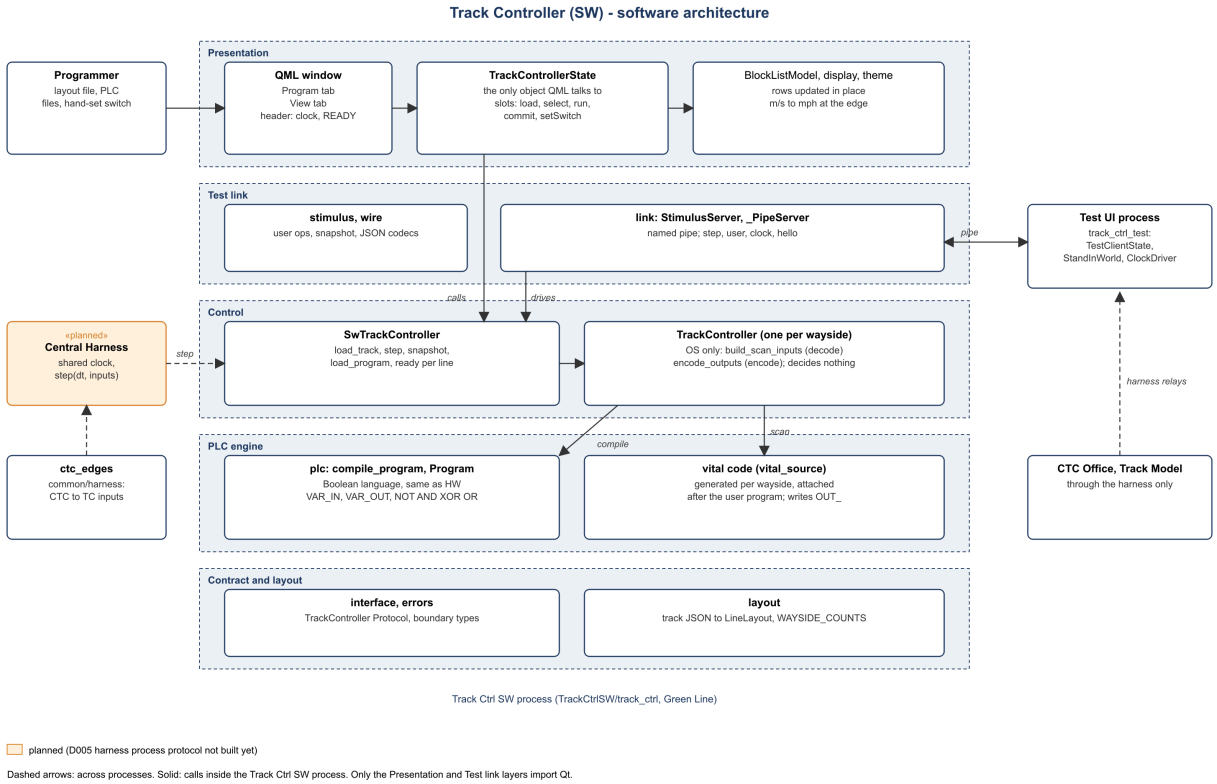

Figure 40. Track Controller (SW) architecture (component view)

The module is layered so that the safety-relevant part has no user interface and no Qt:

Layer	Parts	Responsibility
Presentation	QML window (`ui/`), `TrackControllerState`, `models`, `display`, `theme`	Shows the waysides in imperial units, and turns clicks into calls on the module. Never decides anything.
Test link	`link` (`StimulusServer`), `stimulus`, `wire`	Lets the test UI step the module and drive the window's actions over a named pipe, as newline JSON.
Control	`SwTrackController`, `TrackController`	One line's waysides: input image, scan, outputs, report, readiness per line. In each wayside the OS only decodes inputs to bits and encodes bits to commands.
PLC engine	`plc` (`compile_program`, `Program`), `vital`	Compiles and scans Boolean programs. Generates each wayside's vital code, which is attached after the user program.

Layer	Parts	Responsibility
Contract and layout	`interface`, `errors`, `layout`	The `TrackController` Protocol and the boundary types; the track JSON parser.
Harness edge	`common/harness/ctc_edges`	Maps the CTC Office's `TrainSuggestion` and switch commands to `CtcInputs`.

Design choices:

- **The PLC makes every decision.** The OS (`build_scan_inputs`, `encode_outputs`) decides nothing. The user program runs first. The vital code then runs: it is generated for each wayside from its territory, it is plain PLC code, and it can only make an output more restrictive. Only the vital code's `OUT_` bits leave the wayside. A wrong program therefore stops the railway; it cannot cause a collision.
- **One interface, three users.** The harness, the window and the test UI drive the module only through `step`, `snapshot`, `load_program` and `reset`, plus the window's track load. The window owns the one `SwTrackController`, and the test link steps that same object, so what the window shows is what was sent.
- **The core is Qt-free.** `interface`, `module`, `controller`, `plc`, `vital`, `layout`, `wire`, `stimulus`, `display` and `errors` import no Qt, and are unit-tested headlessly (191 tests).
- **Validate, then change.** `step` rejects a bad tick before any state changes. The window rejects a bad file or program before anything changes.
- **Run before commit.** A program reaches a wayside only after a clean sandbox run of exactly that text, and it is checked again at commit.
- **No safe defaults.** A wayside with no program transmits nothing, and its line reports not ready.

5.2.5 Class diagrams

All four diagrams are in one StarUML project, `track_ctrl_sw_classes.mdj`.

Core classes

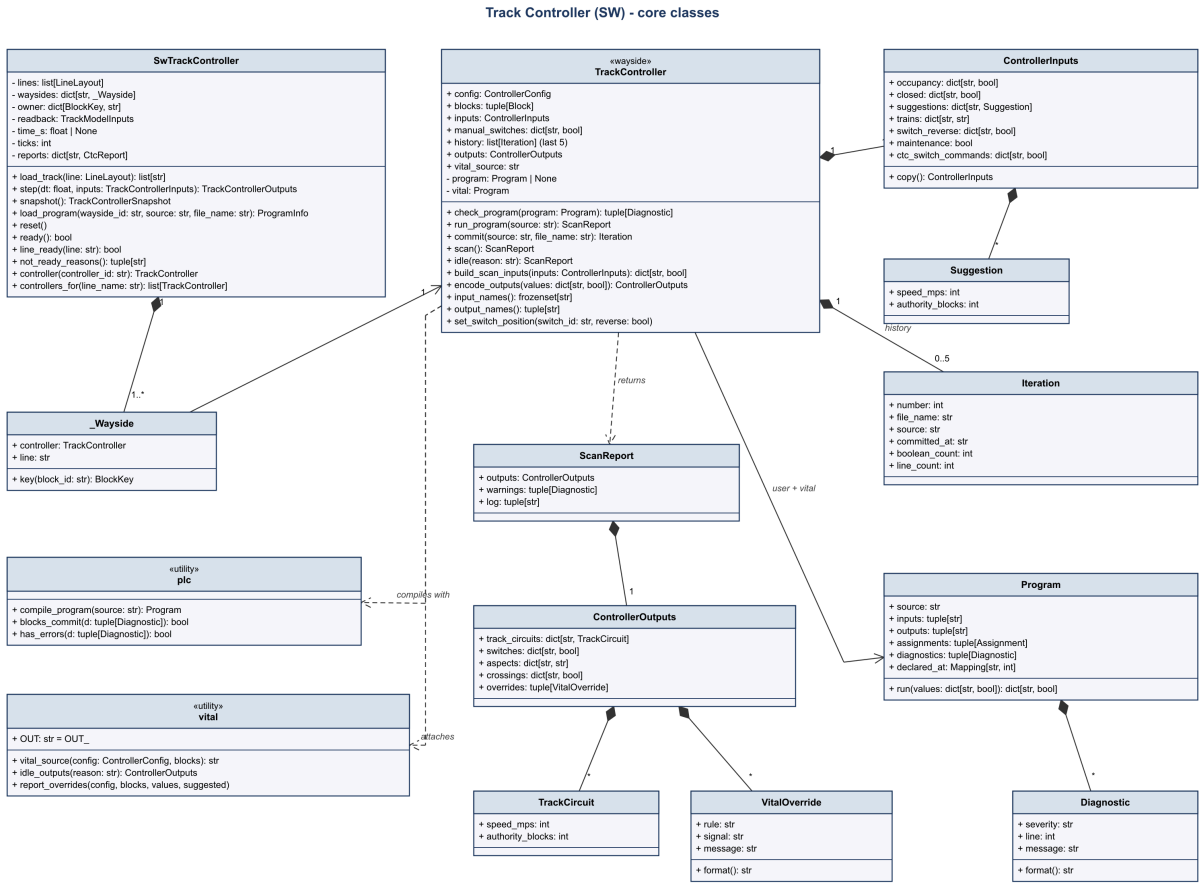

Figure 41. Core classes

- `SwTrackController` holds one `_Wayside` per wayside of every loaded line, and scans each once per `step`. While any wayside of a line has no program, every wayside of that line idles.
- `TrackController` is one wayside.
  - It owns its territory (`ControllerConfig`, `Block`s), its `ControllerInputs`, the user's `Program`, and the compiled vital `Program`.
  - `scan` decodes the inputs, runs both programs and encodes the `OUT_` bits into `ControllerOutputs`.
  - `run_program` does the same on a copy of the inputs (UC-05).
  - `commit` installs a program as a new `Iteration`, keeping the last five.
- `Program` is immutable: a compiled program never changes while it is scanned. `diagnostics` lists any problems; any diagnostic blocks a commit.

Track layout

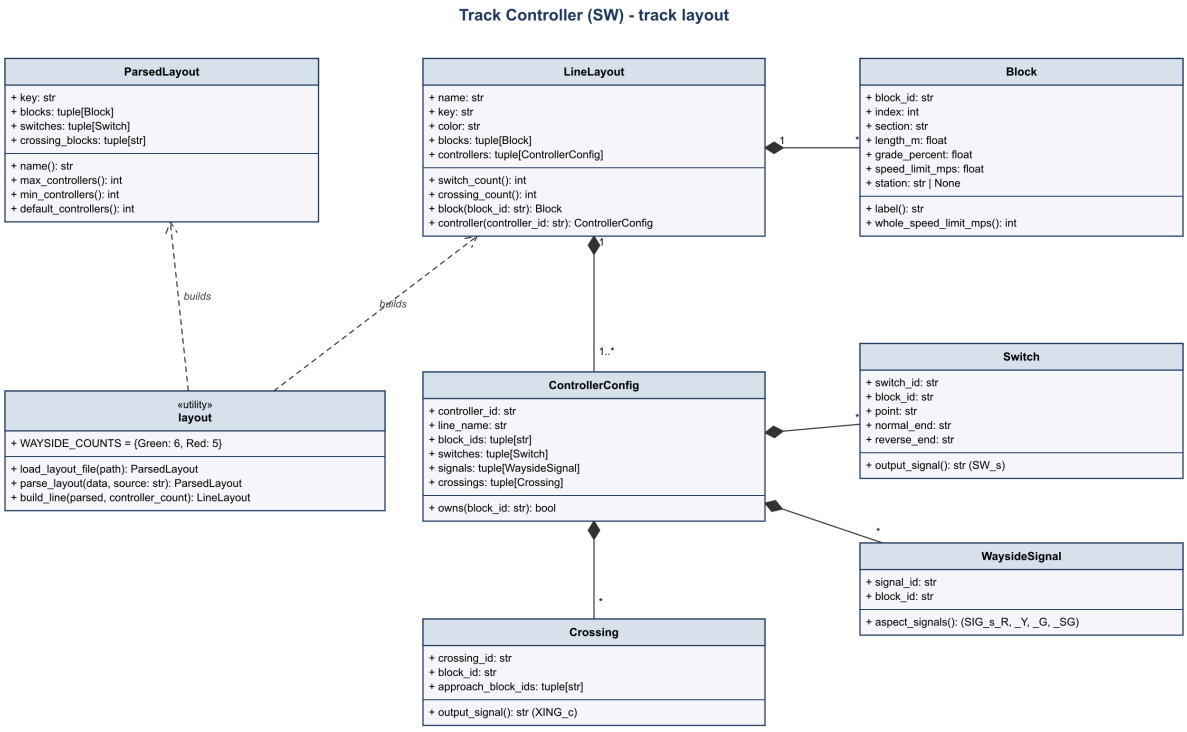

Figure 42. Track layout

- `layout.parse_layout` reads the course layout into a `ParsedLayout`.
- `build_line` divides its blocks among the fixed number of waysides (`WAYSIDE_COUNTS`) into `ControllerConfig`s, so that every switch lies inside one wayside.
- `Switch`, `WaysideSignal` and `Crossing` name their PLC outputs (`SW_s`, `SIG_s_*`, `XING_c`).

Boundary types

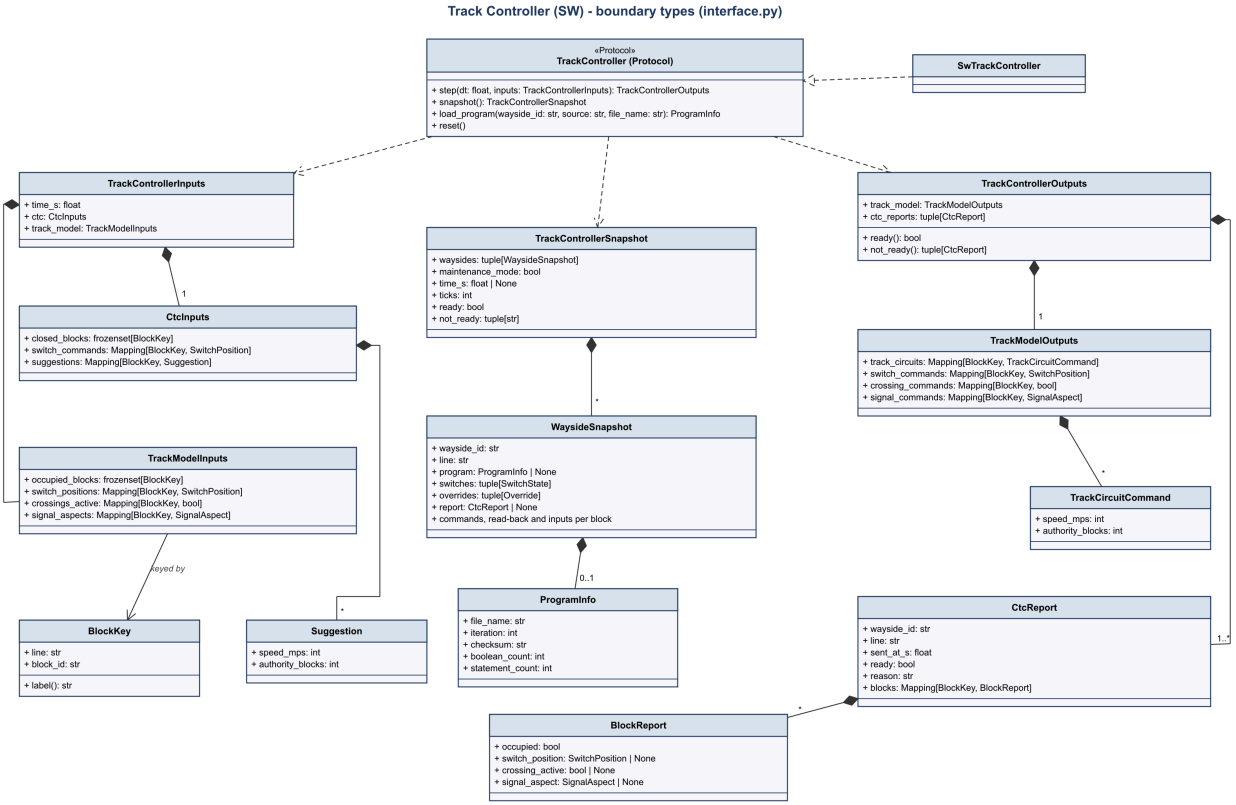

Figure 43. Track Controller (SW): boundary types

- All the boundary types are frozen dataclasses.
- Every block is named by `BlockKey(line, block_id)`. A switch, its signal and a crossing are named by the block they sit on.
- Speeds are whole m/s; authorities are block counts (D013).
- `TrackControllerOutputs.ready` is derived from the per-wayside `CtcReport`s.

Host-side classes

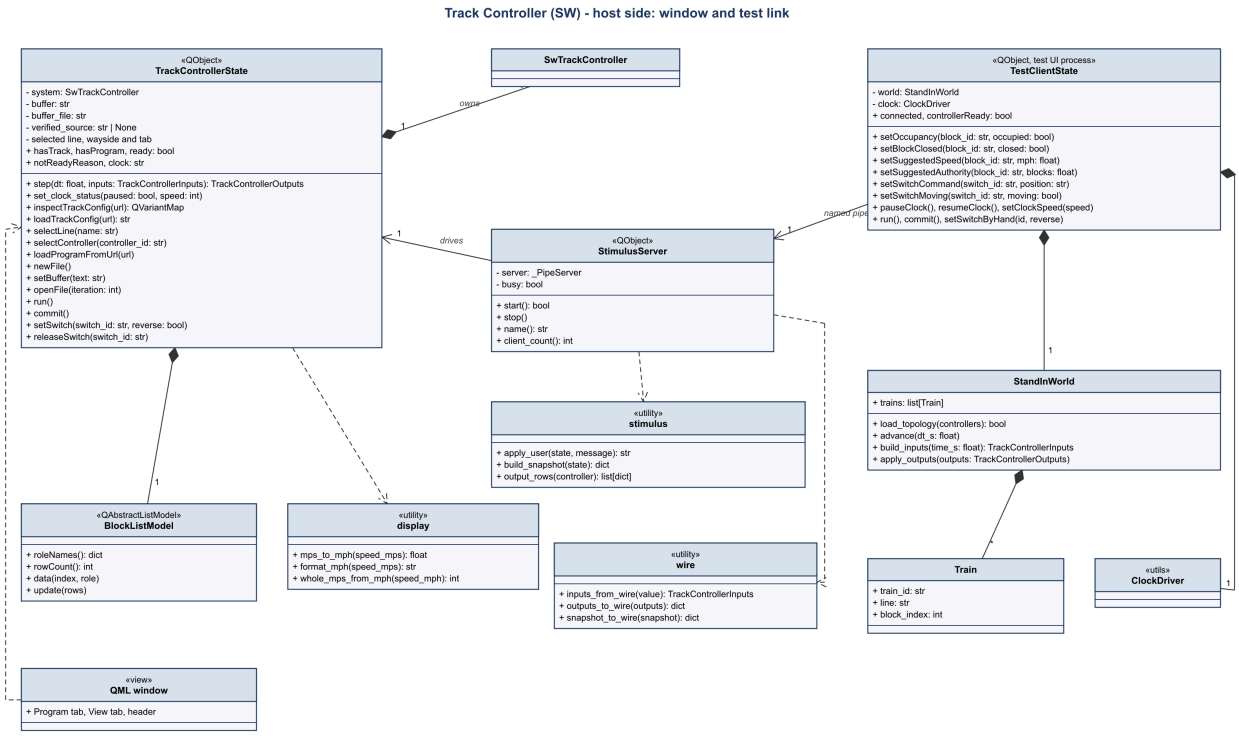

Figure 44. Host-side classes

- `TrackControllerState` is the only object QML talks to. It owns the `SwTrackController`, holds the editor buffer and its last clean run, and pushes rows into `BlockListModel`, which updates in place.
- `StimulusServer` accepts one test client at a time on a named pipe. It runs `step`, `user` (a window action), `clock` and `hello` messages against `TrackControllerState`.
- `TestClientState` runs in the test UI process.
  - Its `StandInWorld` plays the CTC Office and the Track Model, moving simple `Train`s block by block.
  - Its `ClockDriver` plays the shared clock.

5.2.6 Sequence diagrams

There is one sequence diagram per use case.

Diagram	Use case
SD-01 Load track layout	UC-01
SD-02 Select wayside	UC-02

Diagram	Use case
SD-03 Load PLC program	UC-03
SD-04 Write PLC program	UC-04
SD-05 Run PLC program	UC-05
SD-06 Commit PLC program	UC-06
SD-07 Monitor wayside state	UC-07
SD-08 Hand-set switch	UC-08
SD-09 Run scan cycle	UC-09
SD-10 Receive speed and authority suggestions	UC-10
SD-11 Receive closures and switch commands	UC-11
SD-12 Receive track state	UC-12
SD-13 Command switches	UC-13
SD-14 Set signal aspects	UC-14
SD-15 Control railway crossings	UC-15
SD-16 Send speed and authority on track circuits	UC-16
SD-17 Report occupancy, read-back and readiness	UC-17
SD-18 Transmit nothing while not ready	UC-18

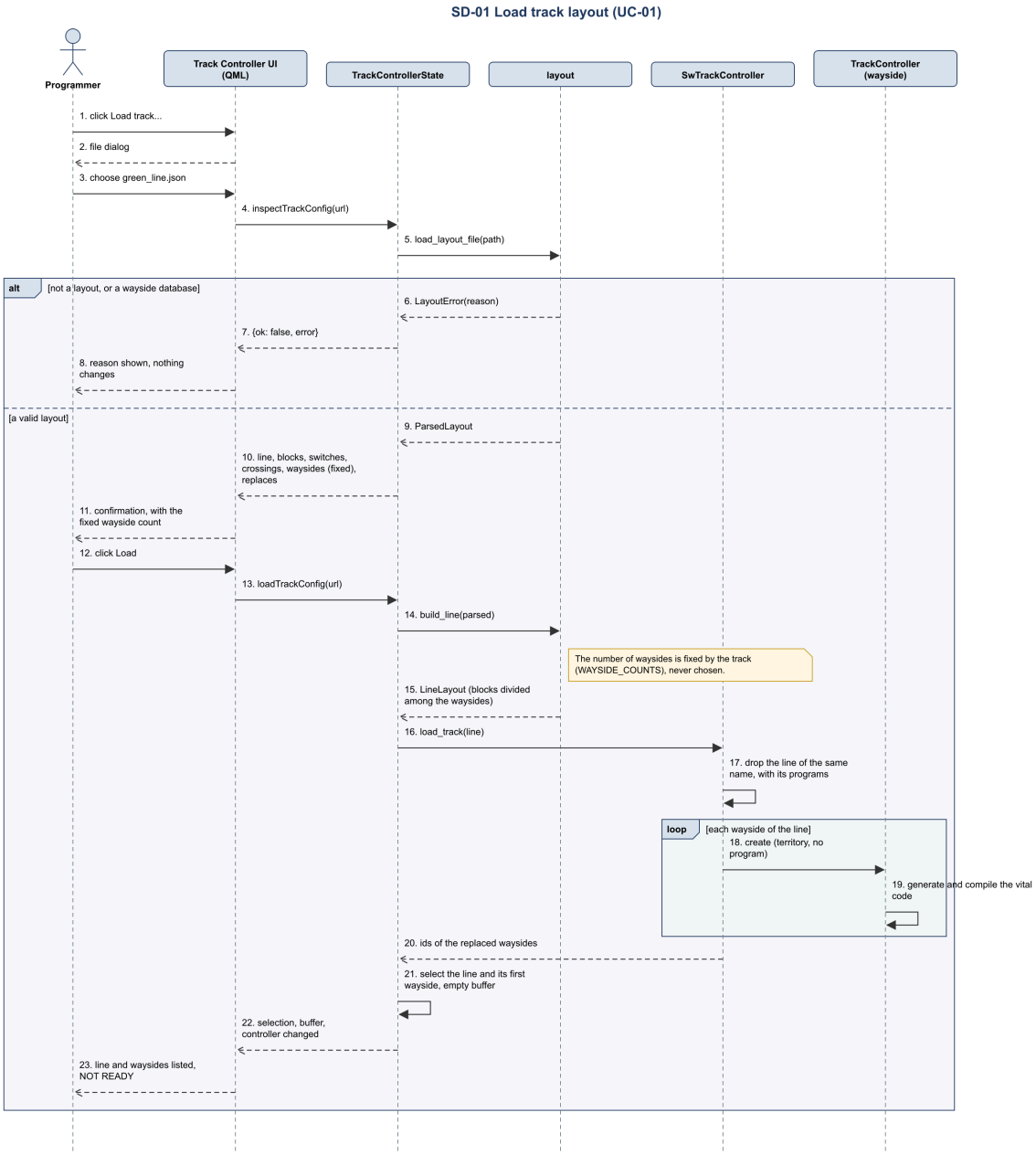

Figure 45. SD-01 Load track layout (UC-01)

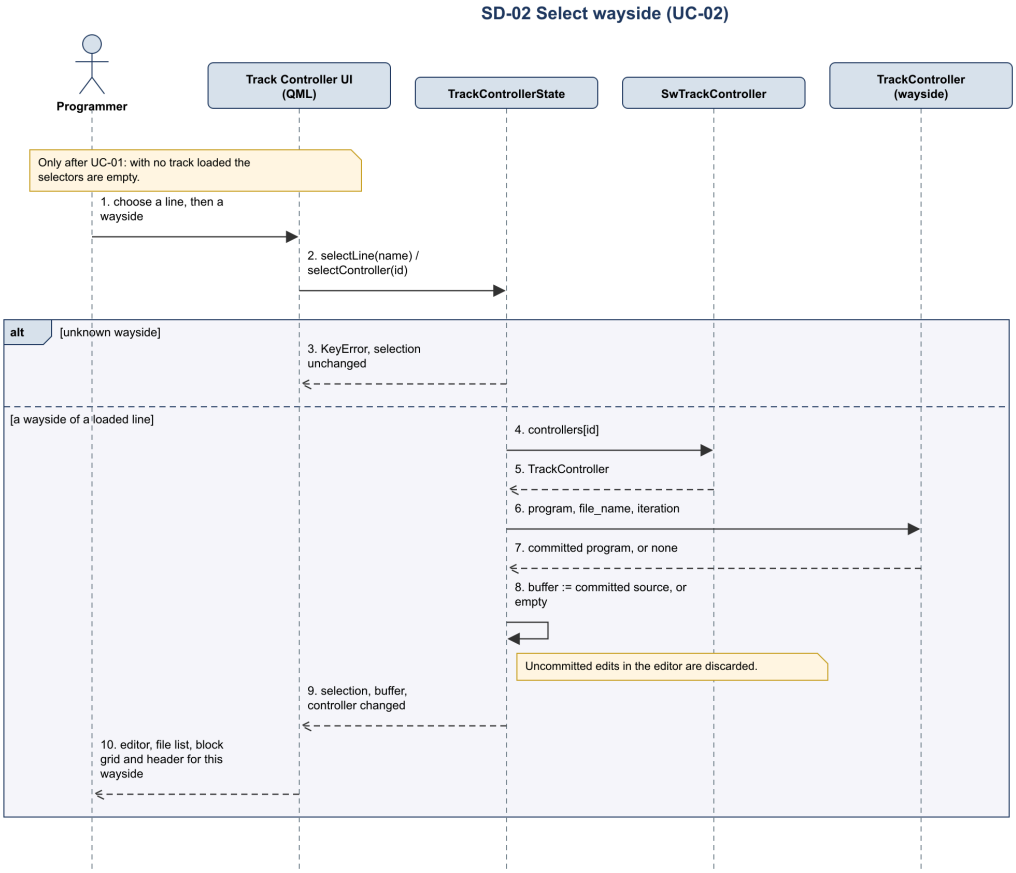

Figure 46. SD-02 Select wayside (UC-02)

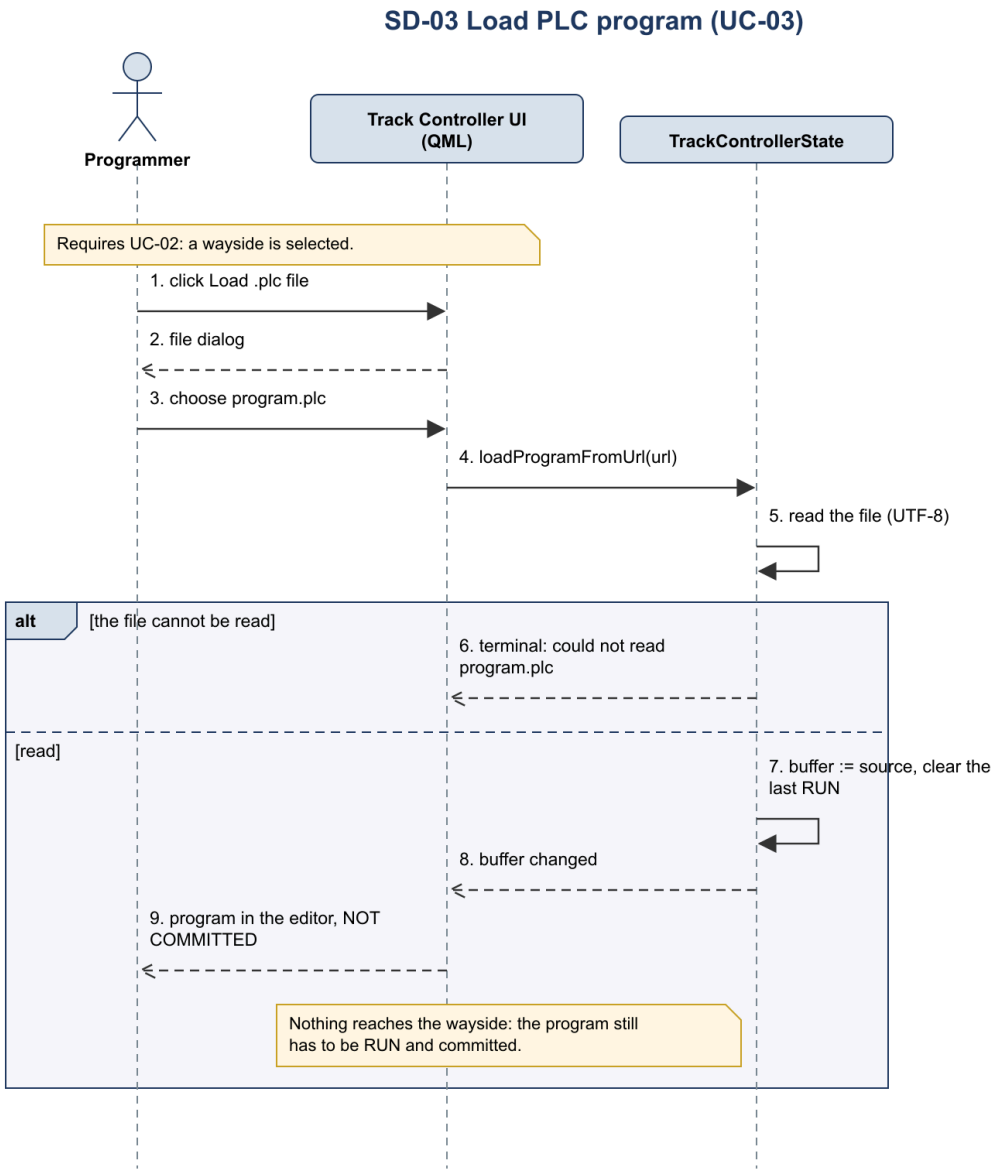

Figure 47. SD-03 Load PLC program (UC-03)

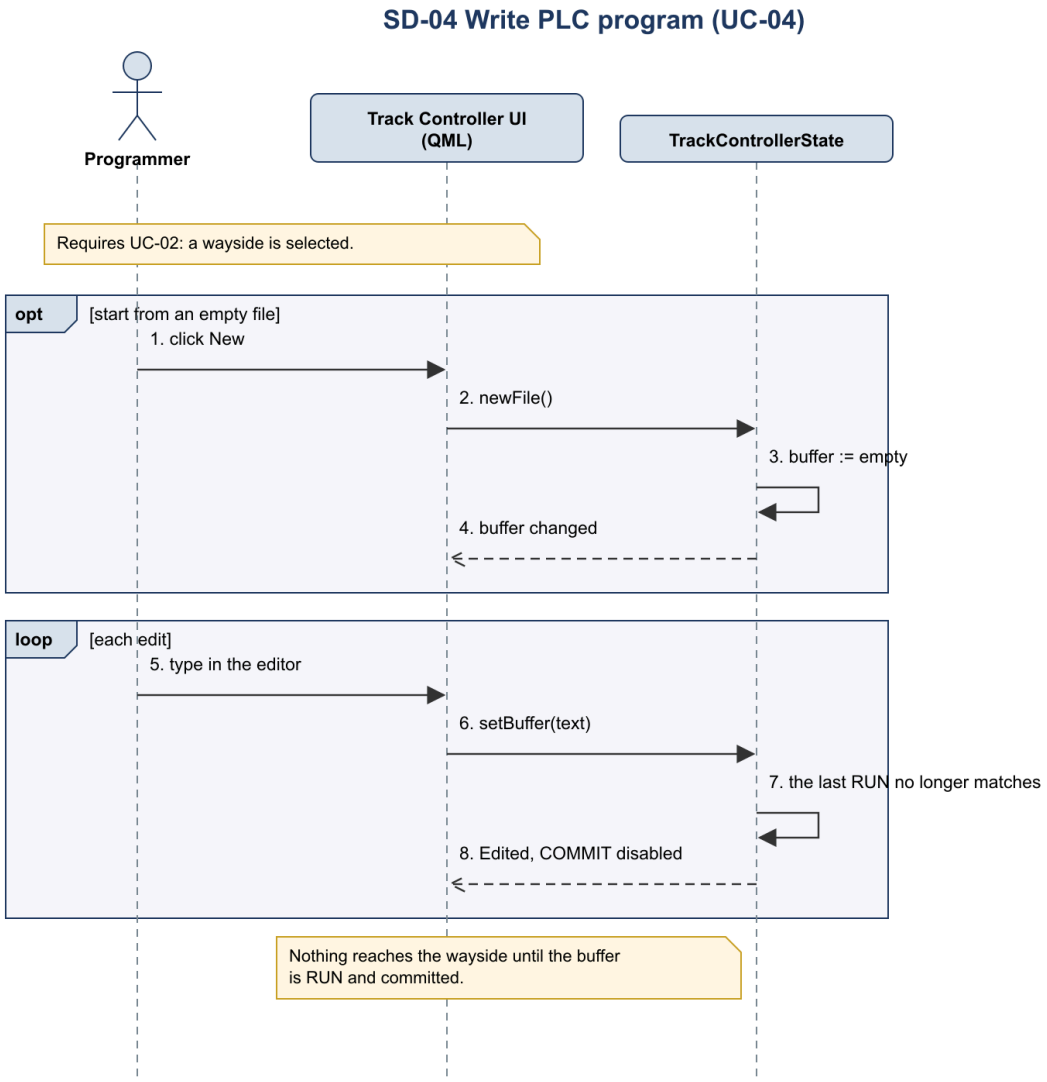

Figure 48. SD-04 Write PLC program (UC-04)

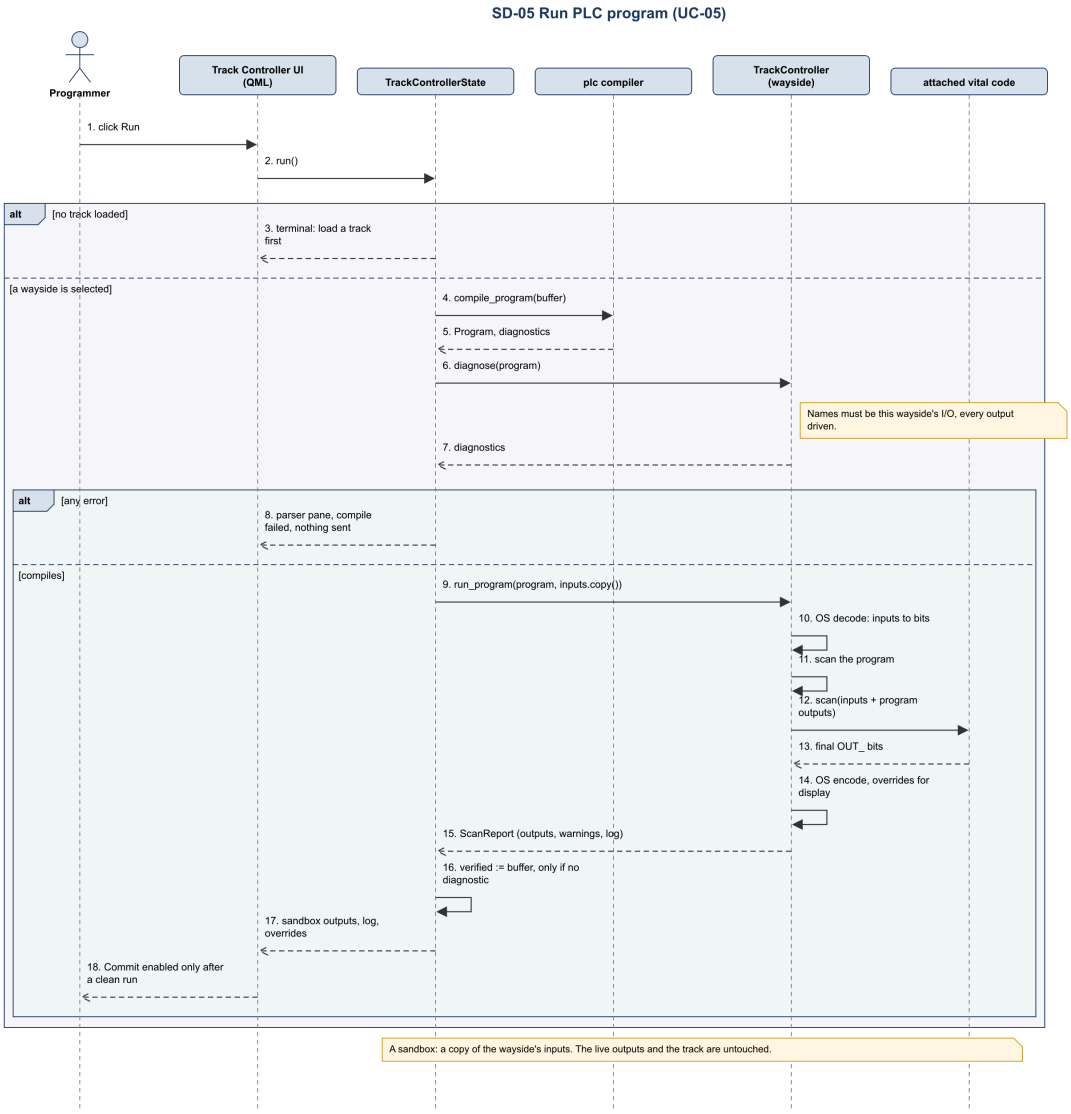

Figure 49. SD-05 Run PLC program (UC-05)

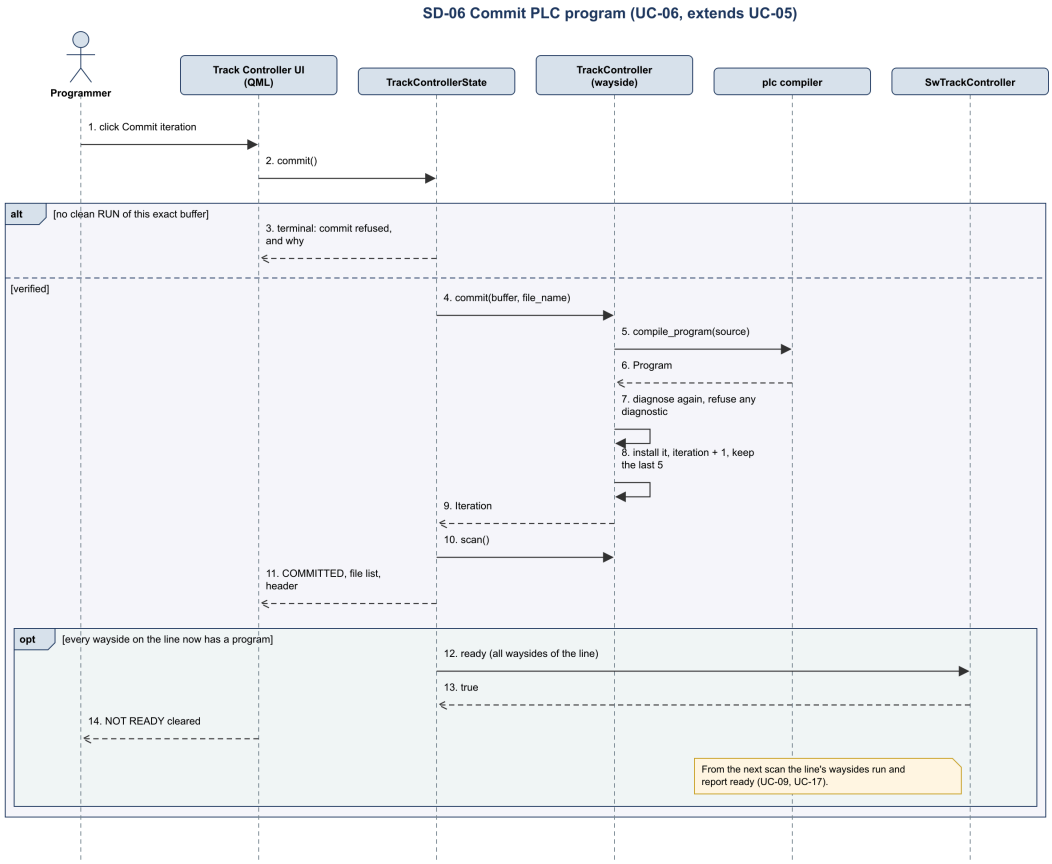

Figure 50. SD-06 Commit PLC program (UC-06)

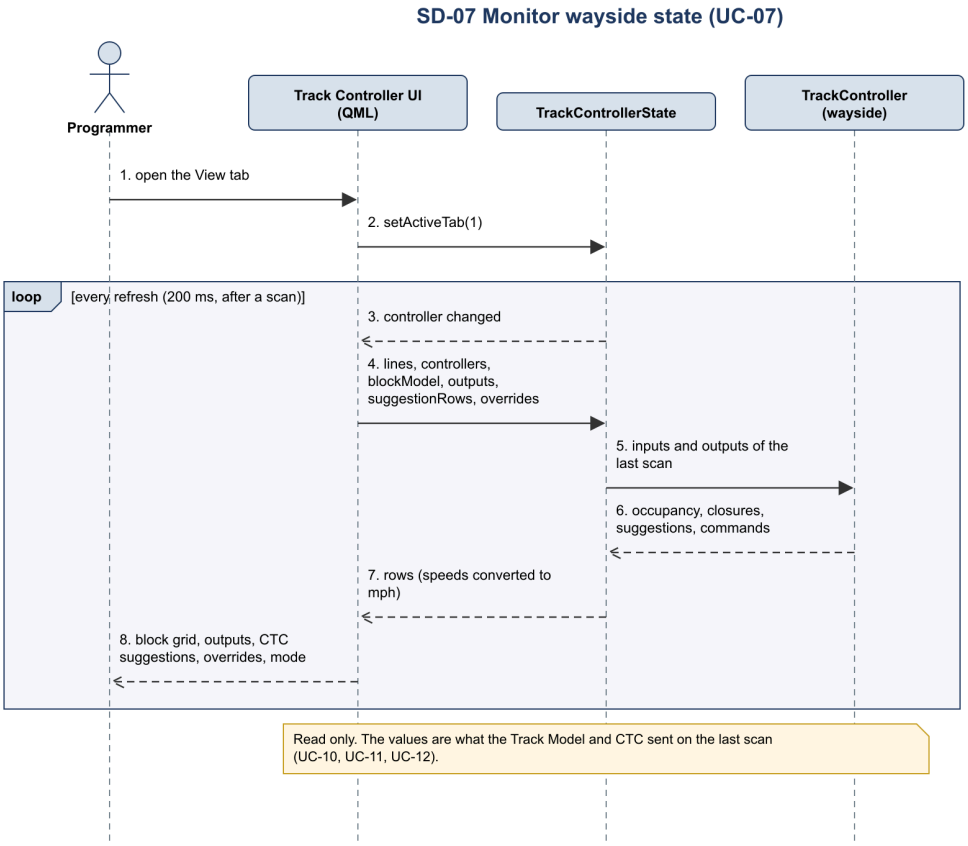

Figure 51. SD-07 Monitor wayside state (UC-07)

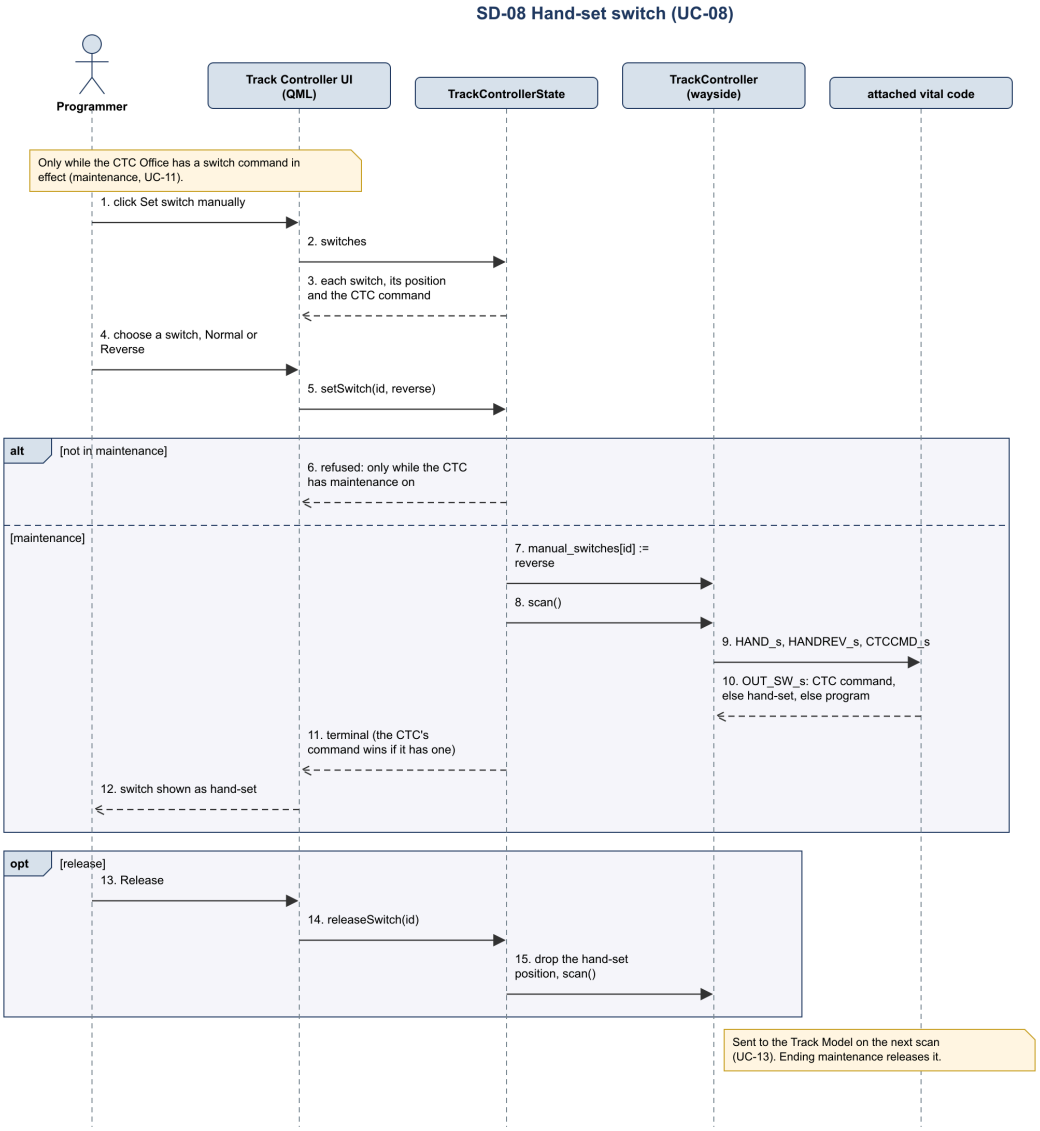

Figure 52. SD-08 Hand-set switch (UC-08)

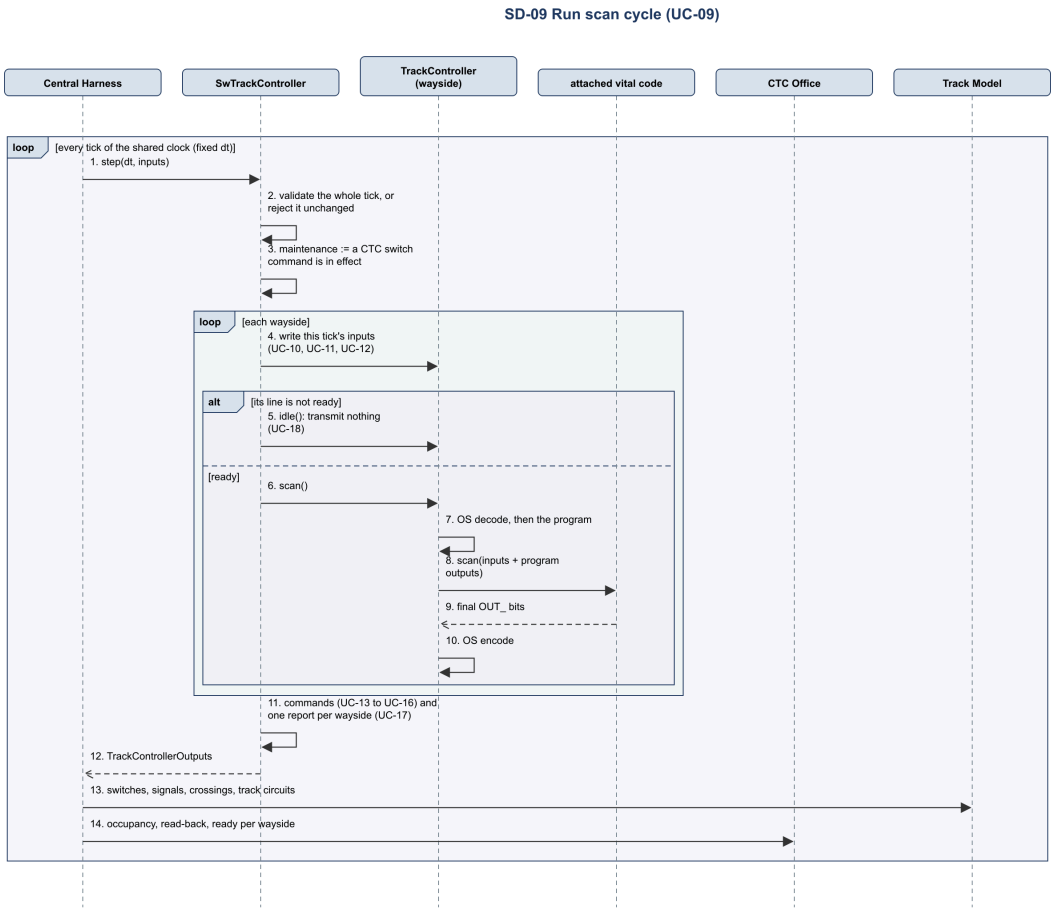

Figure 53. SD-09 Run scan cycle (UC-09)

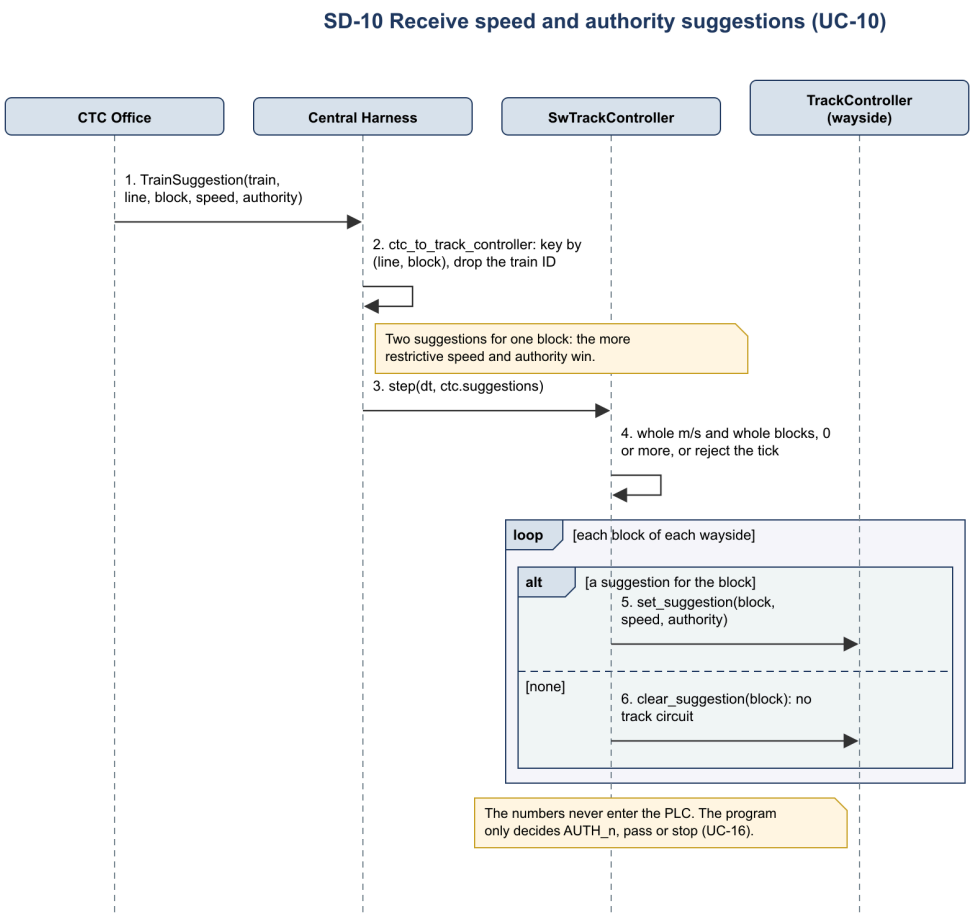

Figure 54. SD-10 Receive speed and authority suggestions (UC-10)

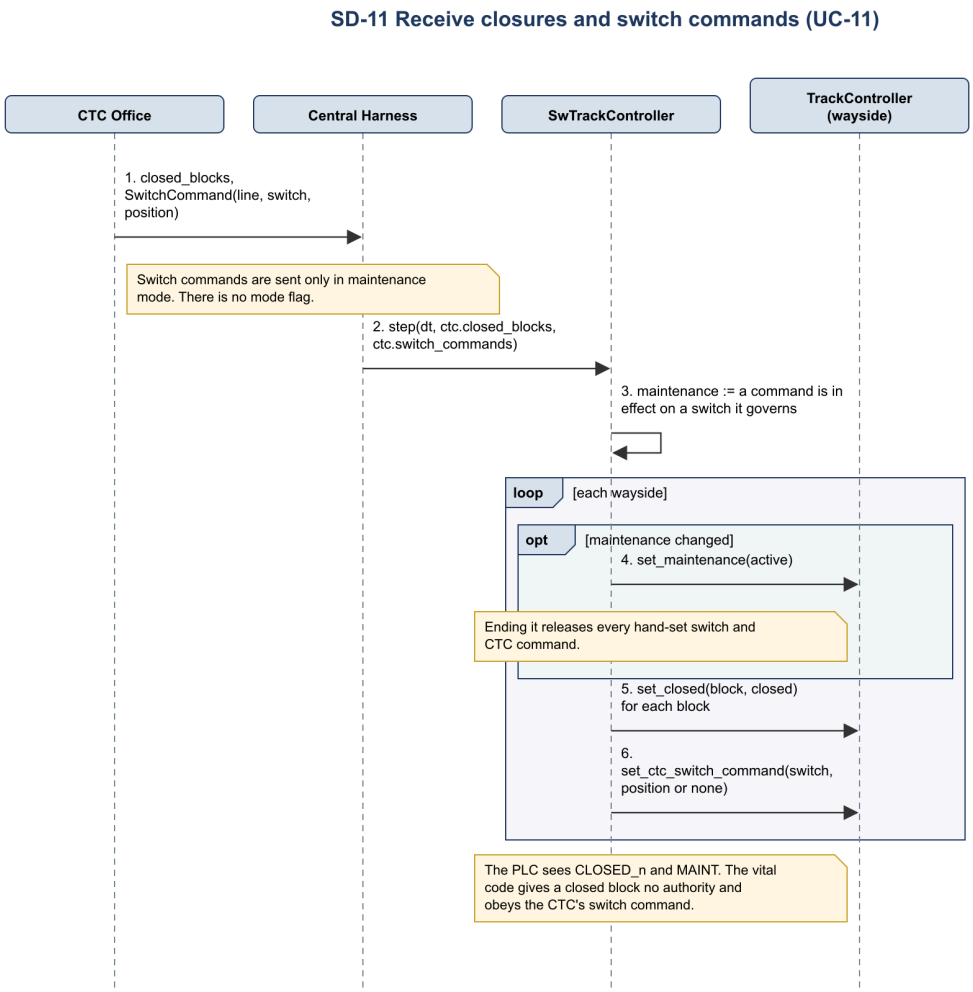

Figure 55. SD-11 Receive closures and switch commands (UC-11)

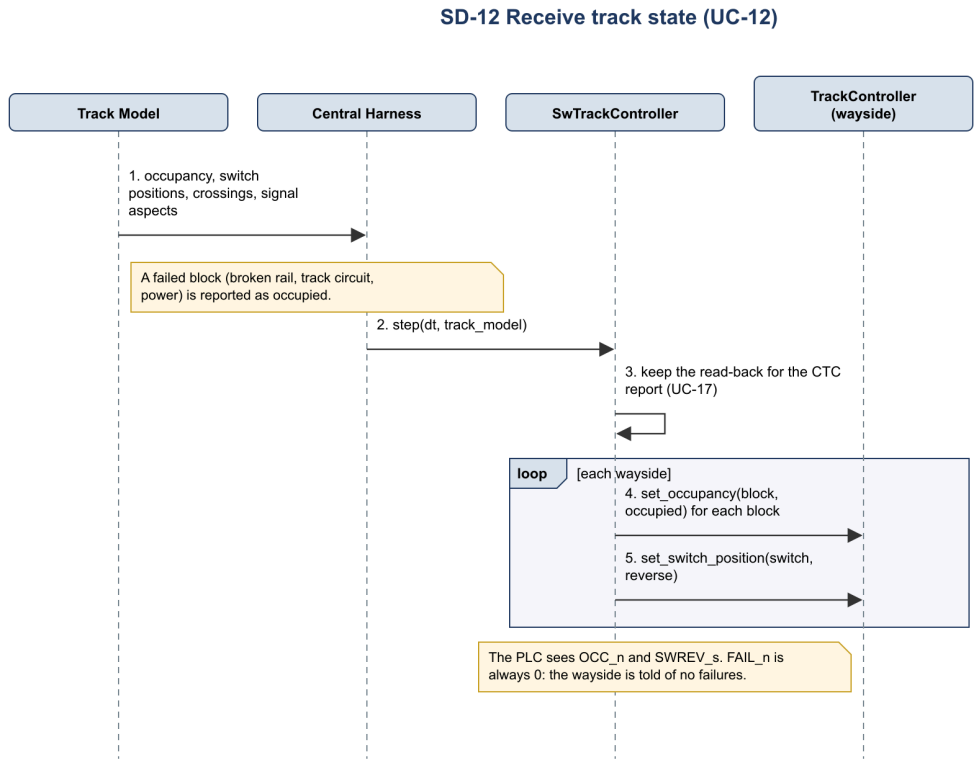

Figure 56. SD-12 Receive track state (UC-12)

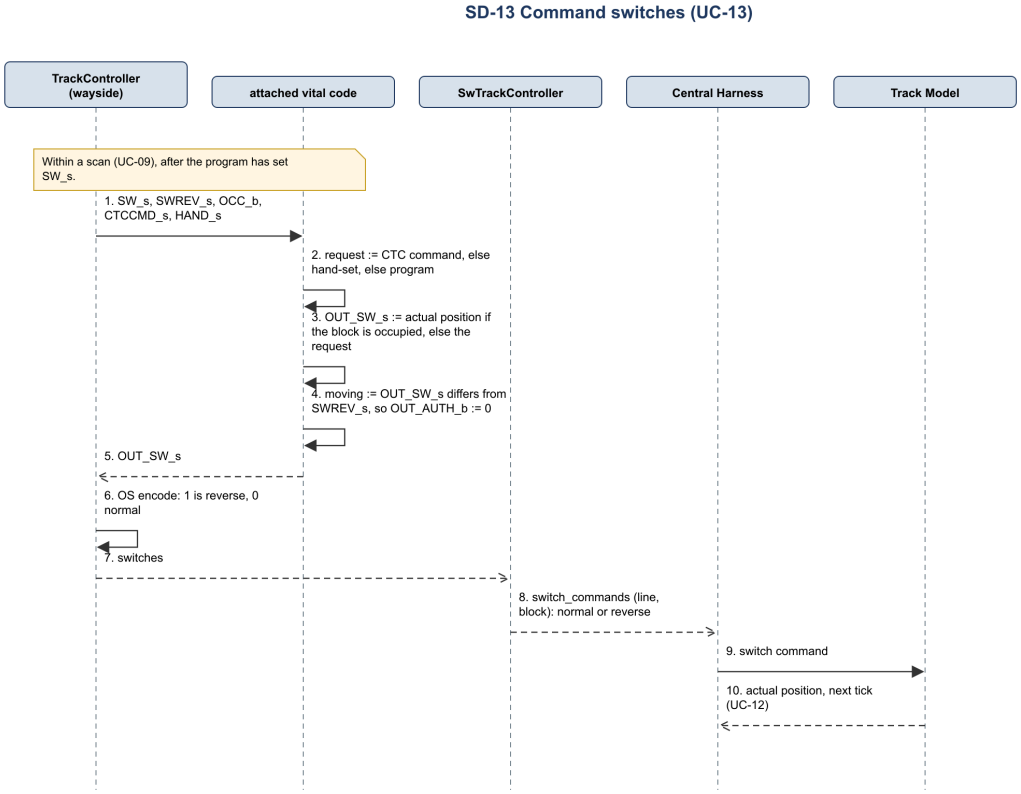

Figure 57. SD-13 Command switches (UC-13)

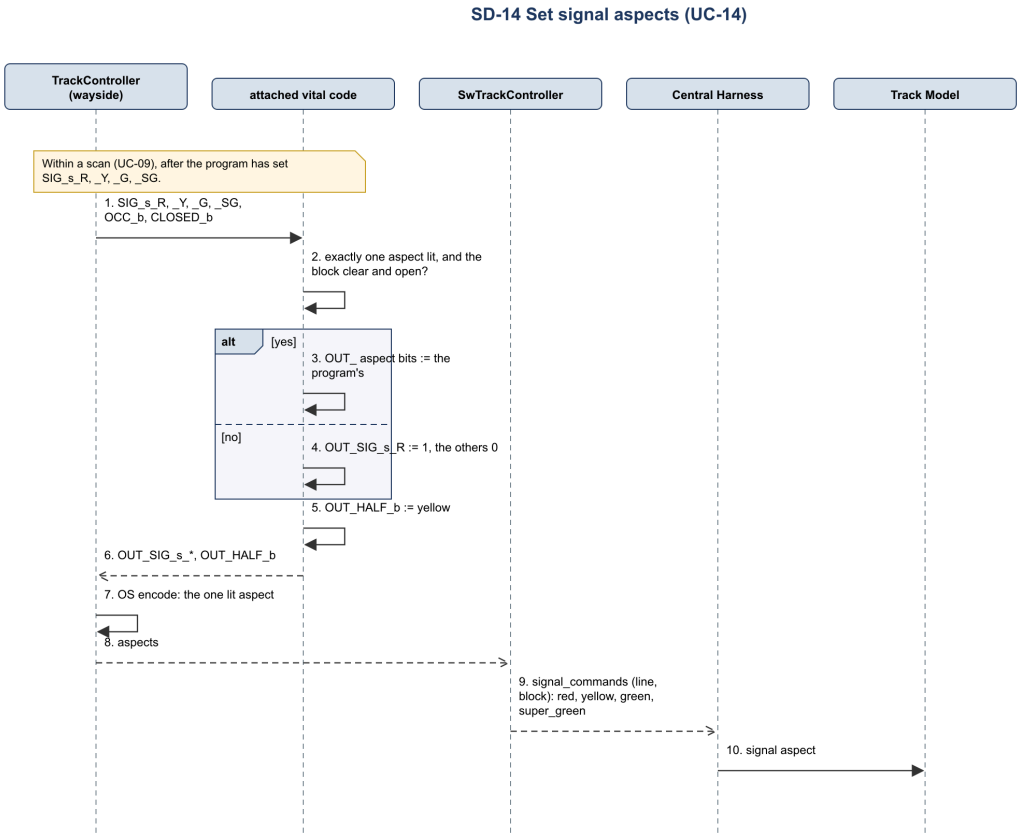

Figure 58. SD-14 Set signal aspects (UC-14)

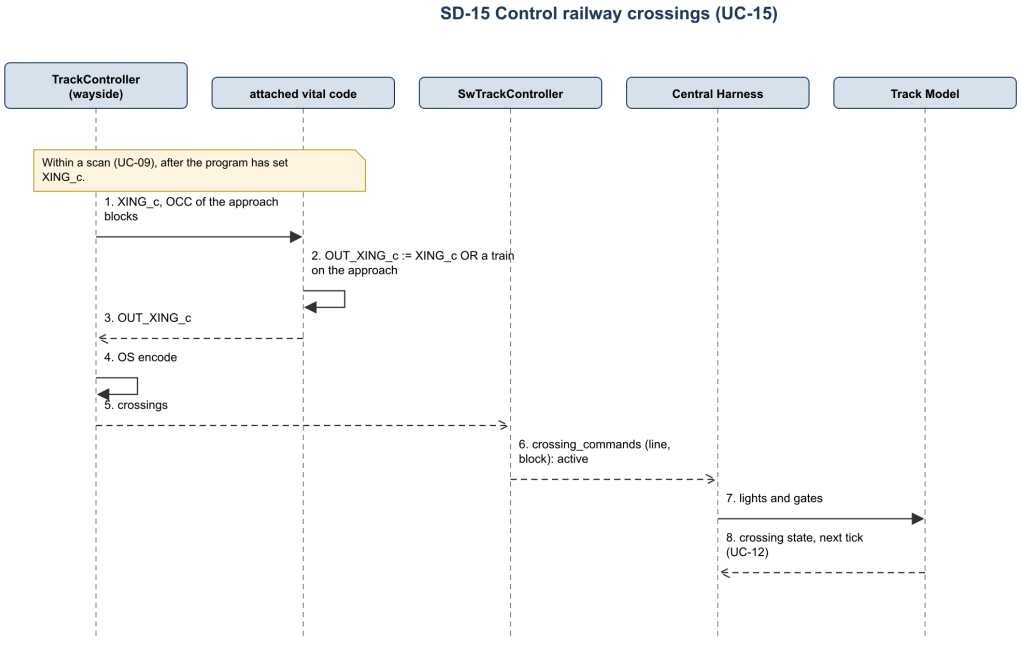

Figure 59. SD-15 Control railway crossings (UC-15)

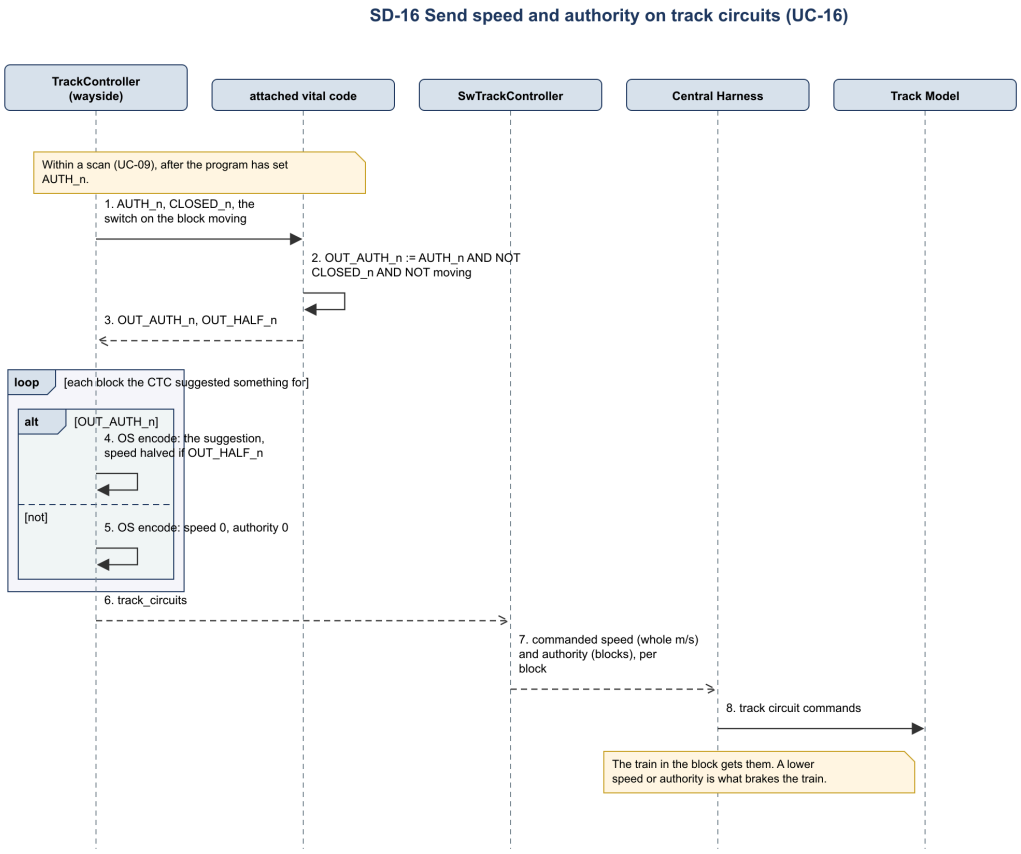

Figure 60. SD-16 Send speed and authority on track circuits (UC-16)

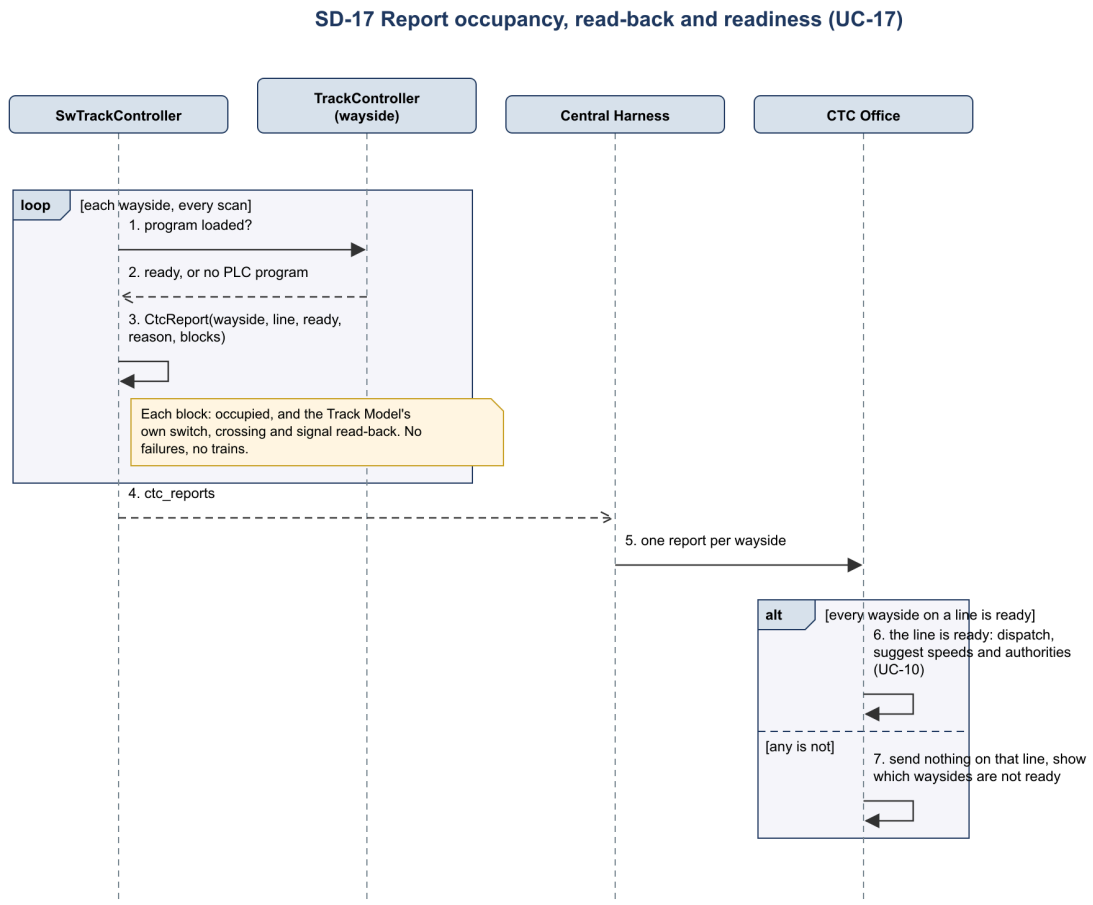

Figure 61. SD-17 Report occupancy, read-back and readiness (UC-17)

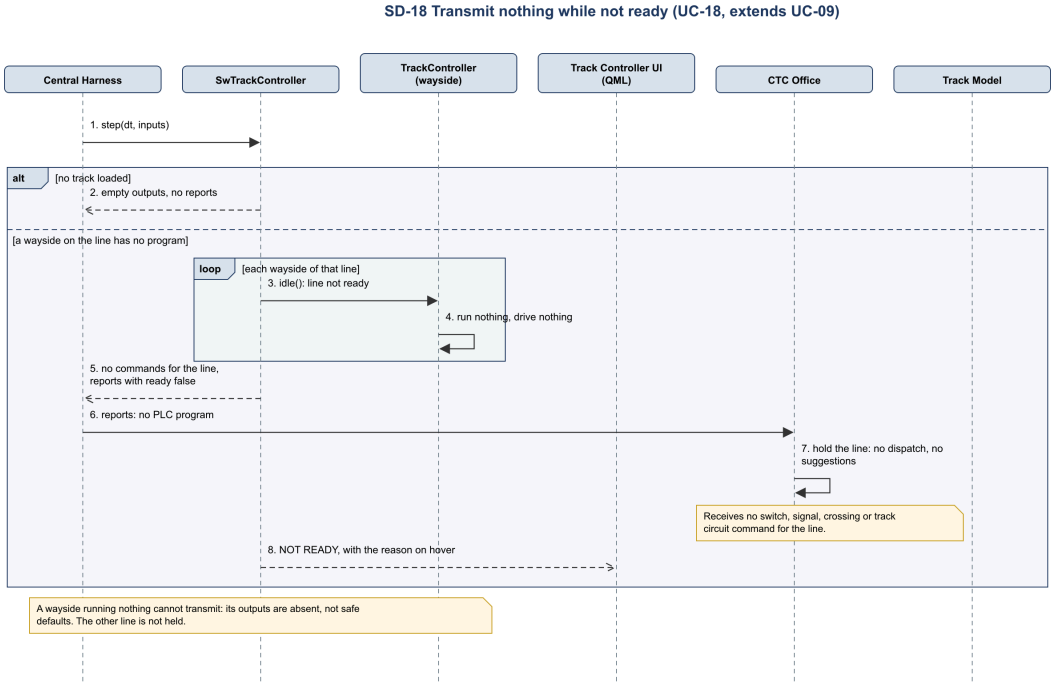

Figure 62. SD-18 Transmit nothing while not ready (UC-18)

5.2.7 Open items

#	Item	Owner
1	D021 gives the software variant the Green Line only, but the module and window still load any line, Red included (`WAYSIDE_COUNTS` has Red: 5). Decide whether to refuse other lines.	Harry
2	The harness endpoint of D005 is not built. It needs the process protocol (newline JSON with a request ID and a tick number), a single driver, and a ready message. Today `StimulusServer` accepts `step` with an ID only.	Harry
3	The PLC names differ between the variants, so one program file does not yet load on both (REQ-INTF-009). HW PR #43 uses `CTCSW_n`, `CTCREV_n`, `SPD_n_b` and `CMD_n_b`. The software variant uses `MAINT` and has no speed bits, and matches the HW code on this branch.	Braden, Harry
4	Hand-set switch (rubric 6.8). The software variant lets the programmer set a switch in maintenance mode (UC-08); the hardware variant assumes only the CTC Office does. Agree one behaviour.	Braden, Harry
5	Vital code. In the software variant every program has generated vital PLC code attached after it, and only that code writes the outputs. This answers HW open item 1 for the software side. The vital rules and `OUT_` names should be agreed for both variants.	Harry, Braden
6	The Protocols differ: HW has `load_territory` and SW loads a whole line through the window (`load_track`). REQ-INTF-009 asks for equivalent external interfaces.	Harry, Braden
7	`FAIL_n` is always 0, because the Track Model reports failures as occupancy. It is kept so that programs match HW. Check that REQ-FUNC-059 is satisfied by protecting failures as occupancy.	Harry
8	REQ-FUNC-057 ("know when the system is in maintenance mode"). The CTC Office sends no flag; maintenance is inferred from a CTC switch command being in effect. The wording is left open.	Harry, CTC owner
9	Keeping a speed under the posted limit, and zero speed for zero authority, belong to the CTC Office. The track controller only passes the suggestion, halves it, or sends 0. The CTC owner to confirm.	CTC owner
10	REQ-INTF-011 asks for 720 x 450, but the window's minimum size is 1100 x 720.	Harry

#	Item	Owner
11	Stale text in the code. `plc.py` cites REQ-FUNC-066 (the Boolean-only requirement is REQ-FUNC-051). The `interface.py` docstrings still say a not-ready wayside drives fail-safe outputs; it transmits nothing.	Harry
12	Two proposals in `truth/_inbox/Track-Ctrl-SW/` are superseded: `20261008-1740-vital-restrictions.md` and `20261014-0950-track-controller-vital-restrictions.md`. They should be denied at promotion.	Harry (human promotion)
13	The old `track_controller_class_diagram*` files in `documents/uml/track_ctrl_sw/` predate the boundary types and are superseded by `class_*.png`. Remove them or keep them as history.	Harry
14	«requires» on the use case diagram is not a standard UML stereotype; it marks a precondition. Keep it or replace it with notes.	Harry

5.3 Track Controller (HW)

Owner: Braden Bosilovich. **Source:** sdd/track_ctrl_hw.md.

Module	Track Controller, hardware variant (`TrackCtrlHw/`, package `track_ctrl_hw`)
Owner	Braden
Status	Draft for WP2 and the design review, 2026-10-22
Basis	PR #43 (merged into `development`); integration branch `claude/quirky-fermi-sickg4` (`common/hw_link/`, built by Kevin Schillinger); decisions D005, D006, D013, D017, D021, D027 on `truth`

Reading the figures. Orange elements marked «planned» are designed but not built yet. One part of the design is planned: **the read-only vital PLC block**, a design assumption until Harry Tye confirms it. Each wayside would run a fixed, generated PLC block after the user's program, which can only make an output more restrictive. The split between the PC and the Raspberry Pi (D021, D027) is built: the window runs on the PC and the core on the Pi, linked by `common/hw_link/`.

5.3.1 Scope

The hardware Track Controller runs every wayside controller of the **Red Line**; the software Track Controller runs the Green Line (D021). Each wayside governs a territory loaded from a database file and runs its own Boolean PLC program. The module knows the blocks it governs, never the trains on them: everything it exchanges is keyed by block. A broken rail, a dead track circuit or a power failure reaches it only as an occupied block.

5.3.2 Requirements coverage

Every Track Controller item of the functional rubric, and the SRS requirements behind it, is covered by at least one use case (section 3).

Rubric	Points	Requirement	SRS	Use cases
6.0	2	Wayside receives suggested speed and authority from the CTC	REQ-FUNC-049	UC-08, UC-14

Rubric	Points	Requirement	SRS	Use cases
6.1	2	Wayside moves switches by PLC program execution	REQ-FUNC-052	UC-07, UC-11
6.2	1	Wayside sets traffic light colour by PLC program execution	REQ-FUNC-053	UC-07, UC-12
6.3	2	Wayside receives train presence from the Track Model	REQ-FUNC-054	UC-10
6.4	2	Wayside sends track occupancy to the CTC	REQ-FUNC-055	UC-15, UC-06
6.5	1	Railway crossing lights and gates activated	REQ-FUNC-056	UC-13
6.6	2	PLC language based only on Boolean variables	REQ-FUNC-051	UC-02, UC-07
6.7	1	Load PLC file	REQ-FUNC-050	UC-02
6.8	1	In maintenance mode, manually set a switch position	REQ-FUNC-057, REQ-FUNC-012	UC-09, UC-11
6.9	1	Has safety-critical architecture	REQ-FUNC-058	UC-07, UC-16
—	—	Failures detected (as occupancy)	REQ-FUNC-059	UC-10
9.0	5	Each sub-system has a user interface	REQ-INTF-001.4	UC-01 to UC-06
—	—	Track layout loaded from JSON	REQ-INTF-012	UC-01
—	—	Channels to and from the CTC Office, Track Model and train	REQ-INTF-017 to 020	UC-08, UC-10, UC-14, UC-15

Rubric 6.8 is met by the CTC Office's dispatcher: only the CTC Office sets a switch in maintenance mode, and the wayside's program carries the command out (UC-09). Whether the programmer may also set a switch by hand at the wayside is still open with the software Track Controller (section 8).

5.3.3 Use cases

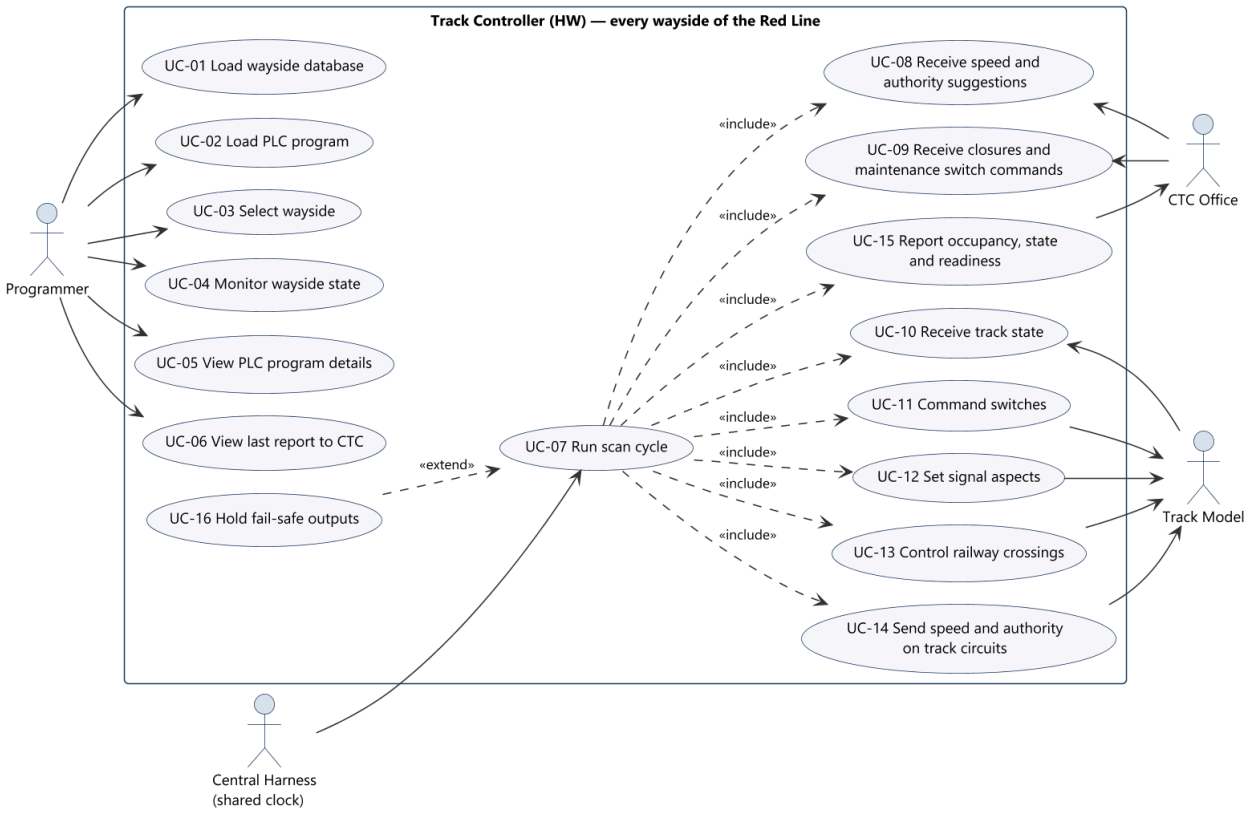

Figure 63. Track Controller (HW): use case diagram

Actors:

- **Programmer** — the person at the Track Controller window. Loads databases and PLC programs and watches the waysides. Every interactive control of the window belongs to one of UC-01 to UC-06.
- **Central Harness** — owns the shared clock and steps the module once per tick (D005, D017). It carries everything the CTC Office and the Track Model send and receive.
- **CTC Office** and **Track Model** — the neighbouring modules, reached only through the harness.

UC-01 Load wayside database

Actors	Programmer
Description	Adds one wayside to the Track Controller, or replaces the wayside with the same ID.

Data	Wayside database (JSON): line, wayside ID, and per block the number, section, length, speed limit, switch and railway crossing. A cut of the course layout file.
Stimulus	The programmer clicks **Load database** and picks a file.
Response	The PC parses the file and sends the territory to the Pi. The wayside appears in the header's picker, selected, and the header reads **Not ready** until it has a PLC program. A replaced wayside keeps its program if the program still fits; otherwise the program is unloaded with a notice.
Alternatives	An unreadable or malformed file, a database for another line, or a block another wayside already governs: refused with a message; nothing changes.
Requirements	REQ-INTF-001.4, REQ-INTF-012, REQ-NFR-003, REQ-NFR-004
Sequence	SD-01

UC-02 Load PLC program

Actors	Programmer
Description	Installs a PLC program on the selected wayside.
Data	PLC source text, Boolean only: `VAR_IN` and `VAR_OUT` declarations, then `NAME := expression` over `NOT`, `AND`, `XOR`, `OR`, `0`, `1` and parentheses.
Stimulus	The programmer clicks **New PLC** (on the Wayside PLC strip or in the PLC details dialog) and picks a file.
Response	The program is compiled twice, once per channel, and checked against the names this wayside provides. It runs from the next scan; until then the wayside holds its outputs fail-safe. The window shows the file name, checksum and any warnings. Once every wayside has a program, the header reads **Ready**.

Alternatives	A syntax error, or a name that is not this wayside's input or output: refused; a dialog lists every problem with its line number, and the previous program keeps running. No wayside loaded: refused.
Requirements	Rubric 6.6, 6.7; REQ-FUNC-050, REQ-FUNC-051
Sequence	SD-02

UC-03 Select wayside

Actors	Programmer
Description	Chooses which loaded wayside the window shows.
Data	Wayside ID.
Stimulus	The programmer picks a wayside in the header.
Response	Every panel and dialog shows that wayside.
Requirements	REQ-INTF-001.4
Sequence	SD-03

UC-04 Monitor wayside state

Actors	Programmer
Description	Shows the live state of the selected wayside, in imperial units.
Data	Territory diagram (blocks, switches with the set leg, signals, crossings); block table (section, block, length, speed limit, occupancy or closure, track-circuit speed and authority); CTC suggestions; switches (commanded, reported, CTC request, agreement); signals and crossings (commanded, reported); header (line, clock, ready, maintenance banner with closures and CTC switch commands).
Stimulus	Any tick; the window refreshes at most every 100 ms. Zoom and **Fit** on the diagram.

Response	Tables update in place; the diagram redraws.
Requirements	REQ-INTF-001.4, REQ-INTF-011; displays rubric 6.1 to 6.5
Sequence	SD-04

UC-05 View PLC program details

Actors	Programmer
Description	Shows the running program and the health of its two channels.
Data	File, load time, checksum, Boolean and statement counts, warnings; channel A/B agreement, channel fault, why outputs are held. Planned: the vital block's checksum, shown read-only.
Stimulus	The programmer clicks **i** on the Wayside PLC strip.
Response	The PLC program details dialog opens; **New PLC** there starts UC-02.
Requirements	Rubric 6.9; REQ-FUNC-050
Sequence	SD-05

UC-06 View last report to the CTC Office

Actors	Programmer
Description	Shows exactly what the selected wayside last sent the CTC Office.

Data	Wayside ready, or why not; per block: occupancy, switch position, signal aspect, crossing state.
Stimulus	The programmer clicks **View** on the CTC uplink strip.
Response	The Last report dialog opens.
Requirements	Rubric 6.4; REQ-FUNC-055
Sequence	SD-06

UC-07 Run scan cycle

Actors	Central Harness (primary); CTC Office and Track Model through it
Description	Once per tick, every wayside of the line runs its PLC program against that tick's inputs and drives the result onto the track.
Data	dt (fixed, 0.1 s), simulation time, the tick's inputs (UC-08 to UC-10).
Stimulus	A `step` message from the harness.
Response	Before the next tick: track-circuit commands, switch, signal and crossing commands, and one report per wayside. Each statement runs on two independently written channels and both must agree; the vital block then runs the same way.
Alternatives	Invalid dt or input: the whole tick is rejected before anything changes, and the harness logs it. Any wayside not ready: UC-16.
Requirements	Rubric 6.1, 6.2, 6.6, 6.9; REQ-FUNC-058, REQ-INTF-013, REQ-NFR-002
Sequence	SD-07

UC-08 Receive speed and authority suggestions

Actors	CTC Office (through the harness)

Description	Takes the CTC Office's suggested speed and authority for each block.
Data	Per block: speed in whole m/s (0 to 31), authority in blocks (D013).
Stimulus	Every tick.
Response	The speed enters the program as five bits, `SPD_n_0..4`; the authority waits for the program's `AUTH_n`. Blocks of other lines or no wayside are ignored.
Alternatives	A negative, fractional or too-large value rejects the tick.
Requirements	Rubric 6.0; REQ-FUNC-049, REQ-INTF-019
Sequence	SD-08

UC-09 Receive closures and maintenance switch commands

Actors	CTC Office (through the harness)
Description	Takes the blocks the dispatcher has closed and, in maintenance mode, the switch positions the dispatcher commands.
Data	Closed blocks; switch commands `{switch block: normal | reverse}`, sent only while the dispatcher is in maintenance mode. There is no maintenance flag.
Stimulus	Every tick.
Response	`CLOSED_n`, `CTCSW_s` and `CTCREV_s` inputs to the program. The window shows each command as "CTC asks" and lists closures and commands in the maintenance banner.
Requirements	Rubric 6.8; REQ-FUNC-057, REQ-FUNC-012
Sequence	SD-09

UC-10 Receive track state

Actors	Track Model (through the harness)

Description	Takes occupancy and the actual state of the wayside's equipment.
Data	Occupied blocks (a train or a failure, indistinguishable); switch positions; crossing states; signal aspects lit.
Stimulus	Every tick.
Response	`OCC_n` and `SWREV_s` inputs to the program; the read-back goes into the report (UC-15). A failed block is protected exactly like an occupied one.
Requirements	Rubric 6.3; REQ-FUNC-054, REQ-FUNC-059, REQ-INTF-018
Sequence	SD-10

UC-11 Command switches

Actors	Track Model
Description	Drives each switch machine from the program's `SW_s` output.
Data	`SW_s`: 1 reverse, 0 normal.
Stimulus	Each scan (UC-07).
Response	A switch command to the Track Model. The vital block holds a switch where it reports while a train is on its blocks.
Requirements	Rubric 6.1, 6.8; REQ-FUNC-052
Sequence	SD-11

UC-12 Set signal aspects

Actors	Track Model
Description	Lights each signal from the program's four aspect outputs.
Data	`SIG_s_R`, `_Y`, `_G`, `_SG`. Red when `_R` is 1 or none is; otherwise the first lit of yellow, green, super green.

Stimulus	Each scan.
Response	A signal command to the Track Model.
Requirements	Rubric 6.2; REQ-FUNC-053
Sequence	SD-12

UC-13 Control railway crossings

Actors	Track Model
Description	Activates each crossing from the program's `XING_c` output.
Data	`XING_c`: 1 lights on and gates down.
Stimulus	Each scan.
Response	A crossing command to the Track Model.
Requirements	Rubric 6.5; REQ-FUNC-056
Sequence	SD-13

UC-14 Send speed and authority on track circuits

Actors	Track Model (delivers them to the train in the block)
Description	Sends each block that has a suggestion its commanded speed and authority.
Data	Speed from `CMD_n_0..4` in whole m/s; authority is the CTC Office's count when `AUTH_n` is 1, else 0.
Stimulus	Each scan.
Response	One track-circuit command per block with a suggestion.
Requirements	Rubric 6.0, 6.9; REQ-INTF-017, REQ-FUNC-058.1
Sequence	SD-14

UC-15 Report occupancy, state and readiness

Actors	CTC Office (through the harness)
Description	Tells the CTC Office what each wayside sees, every tick.
Data	One report per wayside: line, wayside, time, ready and why not; per block: occupied, switch position, signal aspect, crossing state, as the Track Model reports them.
Stimulus	Each scan.
Response	The harness maps the reports into the CTC Office's inputs. The CTC Office dispatches nothing on a line until every wayside of it is ready.
Requirements	Rubric 6.4; REQ-FUNC-055, REQ-INTF-020
Sequence	SD-15

UC-16 Hold fail-safe outputs

Actors	Central Harness (extends UC-07)
Description	Puts a wayside's outputs in their safe state whenever no trustworthy program runs.
Data	Why: no program; a new program that has not scanned yet; channels A and B disagreed (latched until a new program is loaded); or another wayside of the line is not ready.
Stimulus	Any of those conditions at a scan.
Response	Every signal red, every crossing active, speed 0 and authority 0 on every track circuit, switches left where they are. The report says not ready and why.
Requirements	Rubric 6.9; REQ-FUNC-058
Sequence	SD-16

5.3.4 Track Controller (HW) architecture (component view)

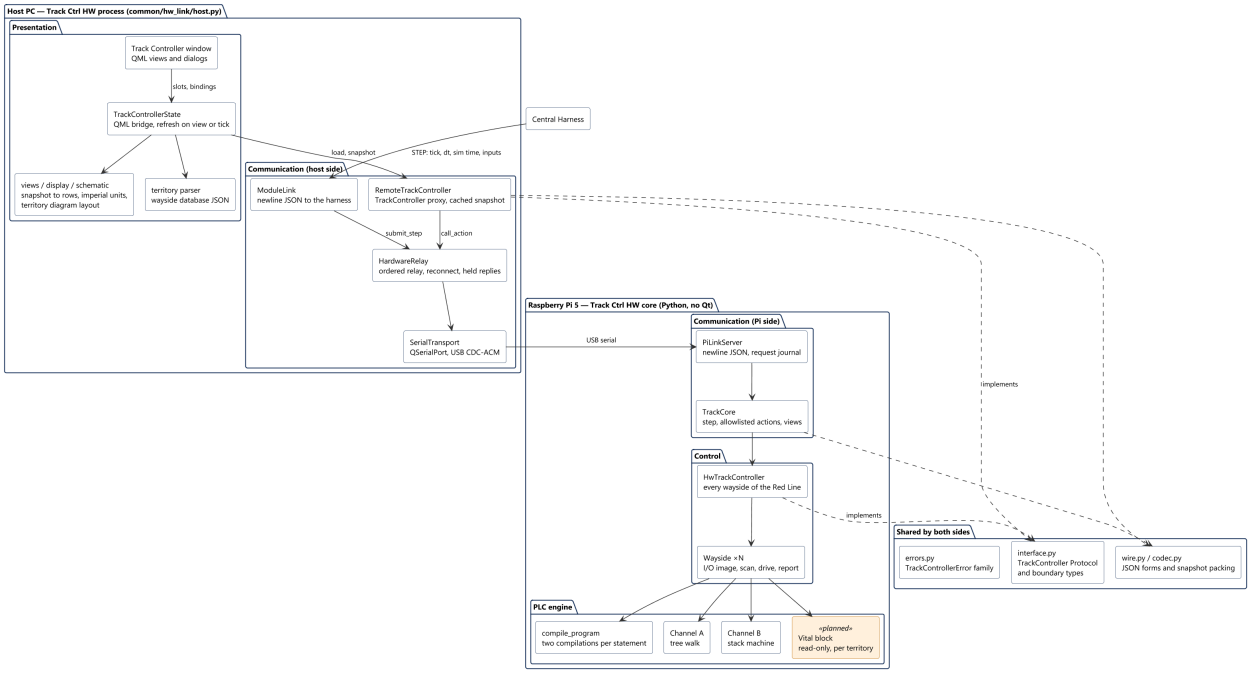

Figure 64. Track Controller (HW) architecture (component view)

The module is layered so that the safety-relevant part has no user interface and no Qt:

Layer	Runs on	Parts	Responsibility
Presentation	PC	QML window and dialogs, `TrackControllerState`, `views`, `display`, `schematic`	Shows the waysides in imperial units; turns clicks into calls on the `TrackController` interface. Never decides anything.
Communication	PC and Pi	`ModuleLink`, `RemoteTrackController`, `HardwareRelay`, `SerialTransport` (PC); `PiLinkServer`, `TrackCore` (Pi); `wire`, `codec` (both)	Carries harness ticks and window requests to the Pi and the replies back, as newline JSON over USB serial (D027). Built in `common/hw_link/`.
Control	Pi	`HwTrackController`, `Wayside`	One line's waysides: input image, scan, outputs, report, ready, fail-safe hold.
PLC engine	Pi	`compile_program`, channel A, channel B, vital block	Compiles each statement twice, by different algorithms, and runs both every scan.

Layer	Runs on	Parts	Responsibility
Contract	both	`interface.py`, `errors.py`	The `TrackController` Protocol and the boundary types. The software variant implements the same Protocol.

Design choices:

- **One interface, three users.** The harness, the window and the test UI drive the module only through `TrackController` (`step`, `snapshot`, `load_territory`, `load_program`, `reset`). `RemoteTrackController` implements the same Protocol on the PC and forwards each window action to the Pi, so the window did not change when the core moved. Harness ticks go straight from `ModuleLink` to the Pi through `HardwareRelay`.
- **The core is Qt-free.** `interface`, `controller`, `wayside`, `plc`, `territory`, `wire` and `errors` import no Qt, so they run on the Pi with plain Python and are unit-tested headlessly (74 tests).
- **Validate, then change.** `step` rejects a bad tick before any state changes; the window rejects a bad file before anything is sent.
- **Two channels.** Channel A parses by recursive descent and walks the tree; channel B parses with the shunting-yard algorithm and runs postfix code on a stack. A disagreement is a latched fault.

5.3.5 Class diagrams

Core classes (Raspberry Pi)

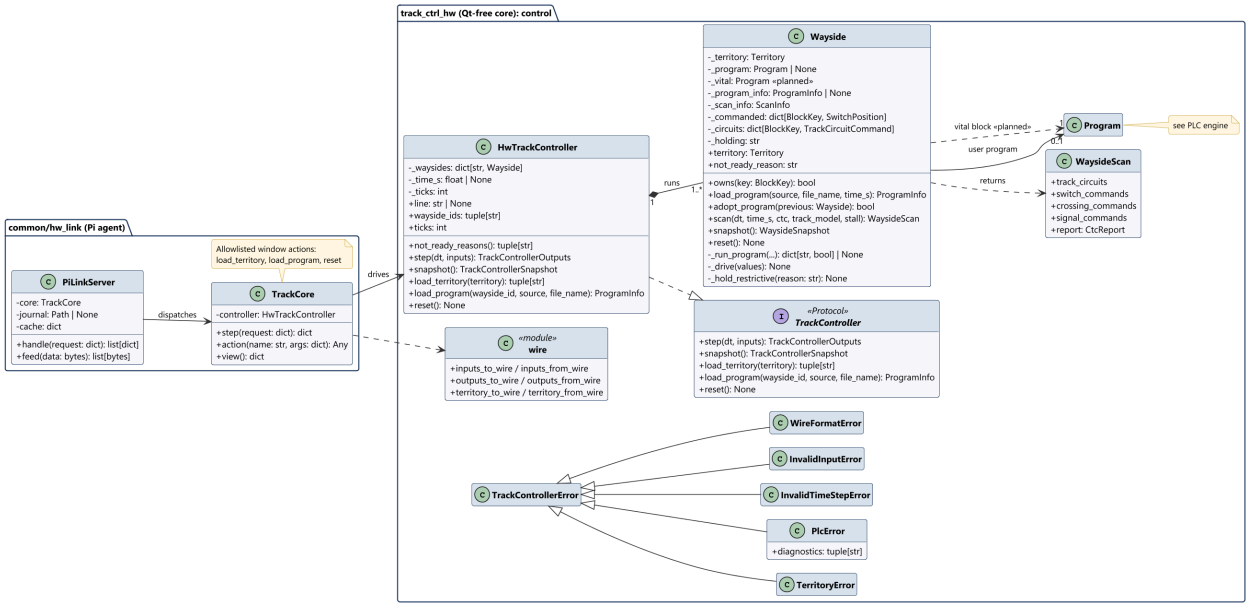

Figure 65. Core classes: control

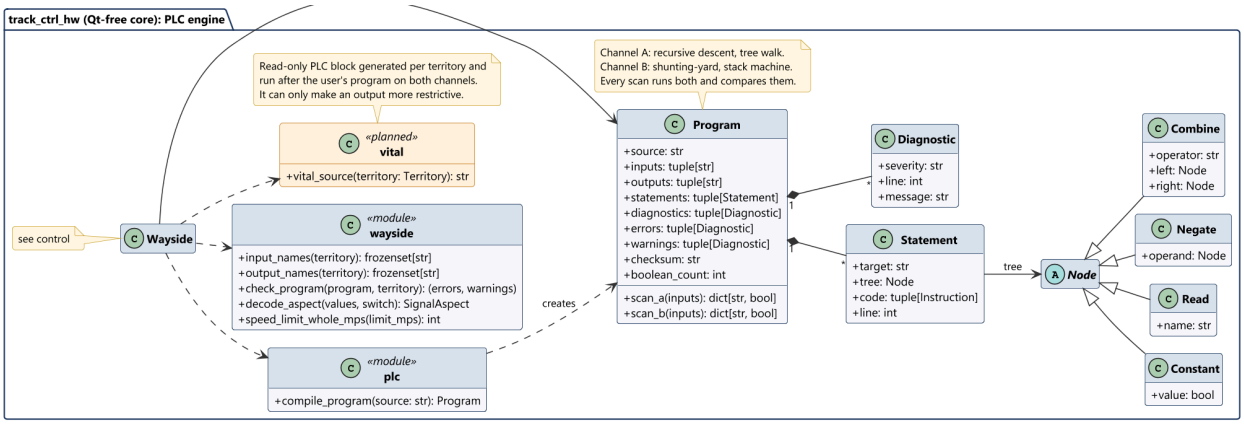

Figure 66. Core classes: PLC engine

- `HwTrackController` holds one `Wayside` per loaded database, all on one line, and scans each once per `step`. While any wayside is not ready, every wayside holds fail-safe.
- `Wayside` owns its territory, the user's `Program`, and (planned) the generated vital `Program`. `scan` builds the input image, runs the programs on both channels, compares them, drives the outputs and builds the `CtcReport`.
- `Program` is immutable: a compiled program never changes while it is scanned. Each `Statement` carries a tree (channel A) and postfix code (channel B).
- On the Pi, `PiLinkServer` (`common/hw_link/agent.py`) reads requests from the USB gadget serial port, journals them, and passes them to `TrackCore`, which steps the controller, runs the allowlisted window actions (`load_territory`, `load_program`, `reset`) and packs snapshots for the window.

Boundary types

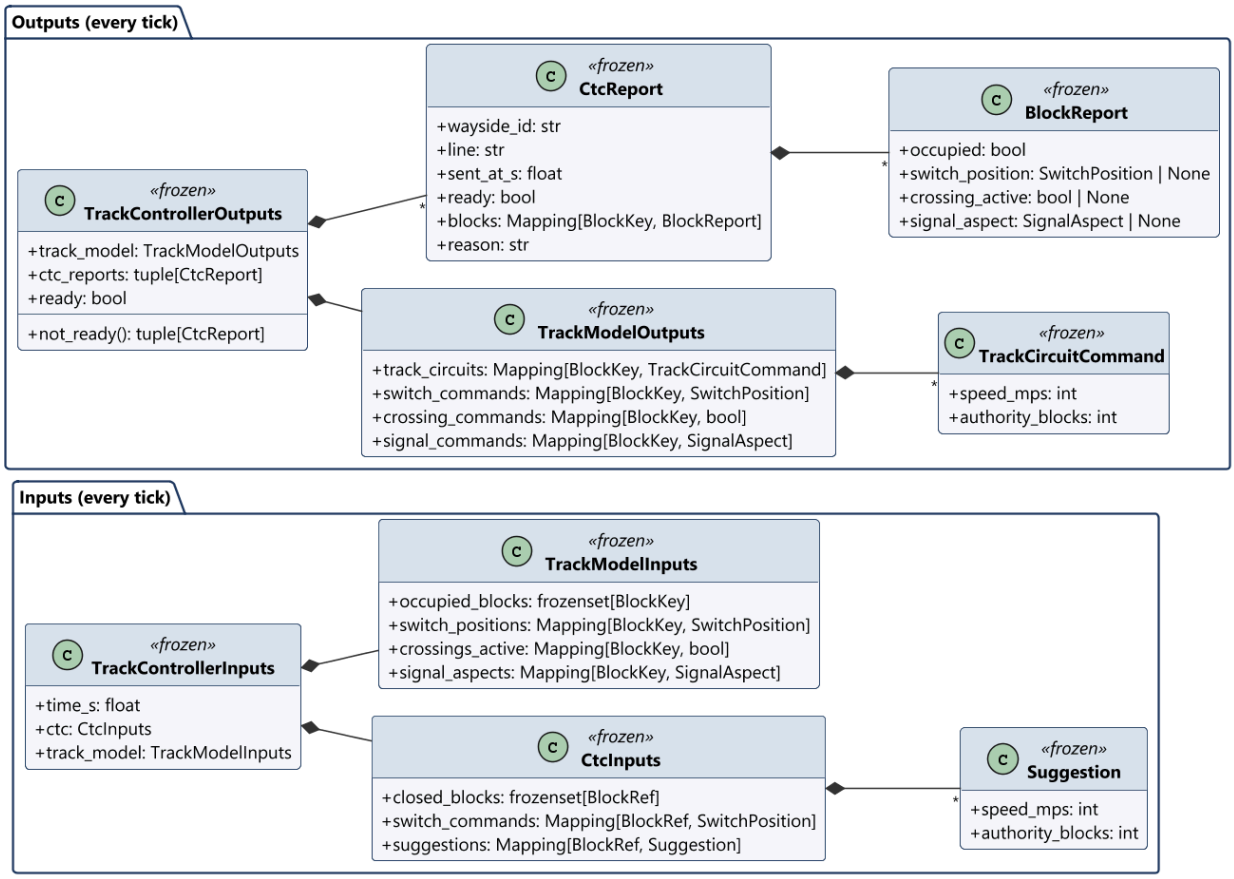

Figure 67. Boundary types: inputs and outputs

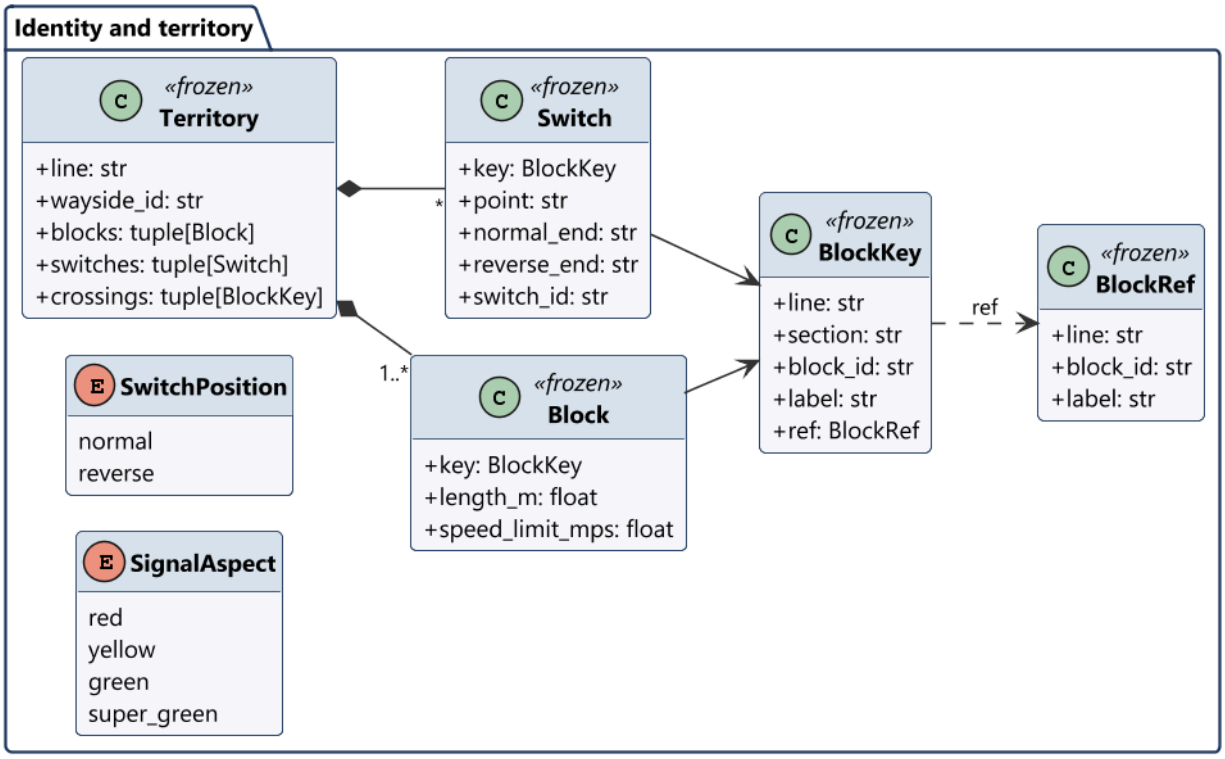

Figure 68. Boundary types: territory

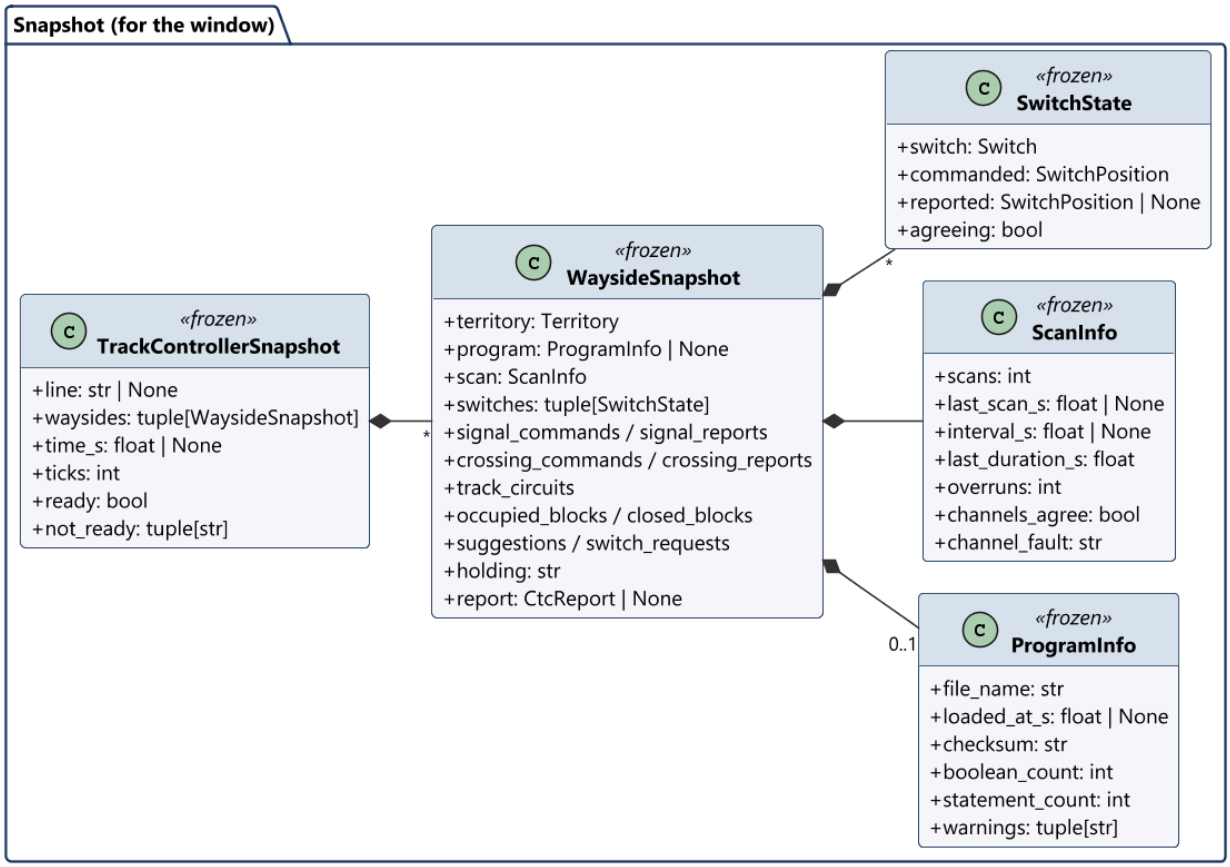

Figure 69. Boundary types: snapshot

All are frozen dataclasses. The CTC Office names a block by `BlockRef` (line and number, `Red:15`); the Track Model by `BlockKey` (line, section, number). A switch, its signal and a crossing are named by the block they are listed on. Speeds are whole m/s; authorities are block counts (D013).

Host-side classes (PC)

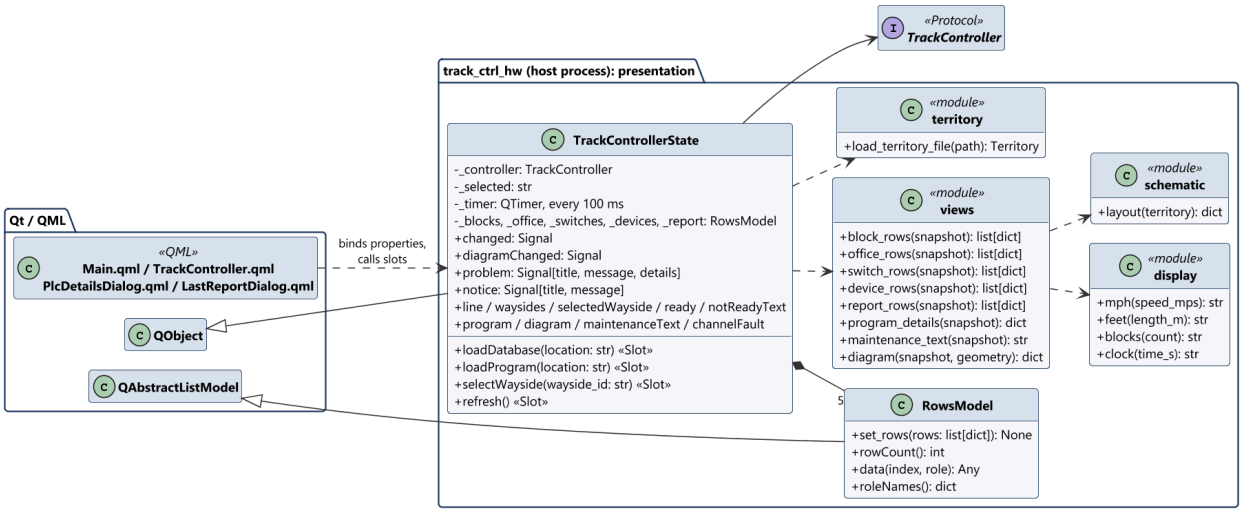

Figure 70. Host-side classes: presentation

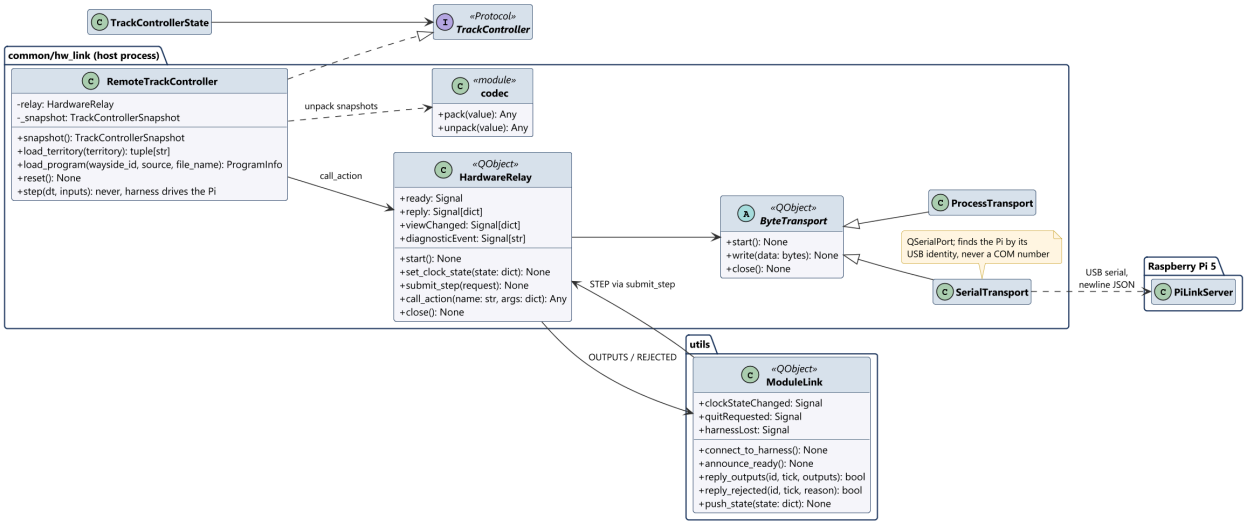

Figure 71. Host-side classes: link to the Pi

- `TrackControllerState` is the only object QML talks to. It reads a snapshot, turns it into rows through `views`, and pushes them into five `RowsModel`s that update in place.
- `RemoteTrackController` (`common/hw_link/remote_track.py`) implements the same Protocol as `HwTrackController`, so the window is unchanged. It caches the snapshot the Pi pushes and turns each window action into one request through `HardwareRelay`; errors come back typed.

- `ModuleLink` answers the harness: each STEP goes to `HardwareRelay.submit_step`, and the Pi's reply comes back as OUTPUTS or REJECTED. `HardwareRelay` keeps requests in order, holds replies while the USB link is lost, and `SerialTransport` finds the Pi by its USB identity.

5.3.6 Sequence diagrams

One sequence diagram per use case.

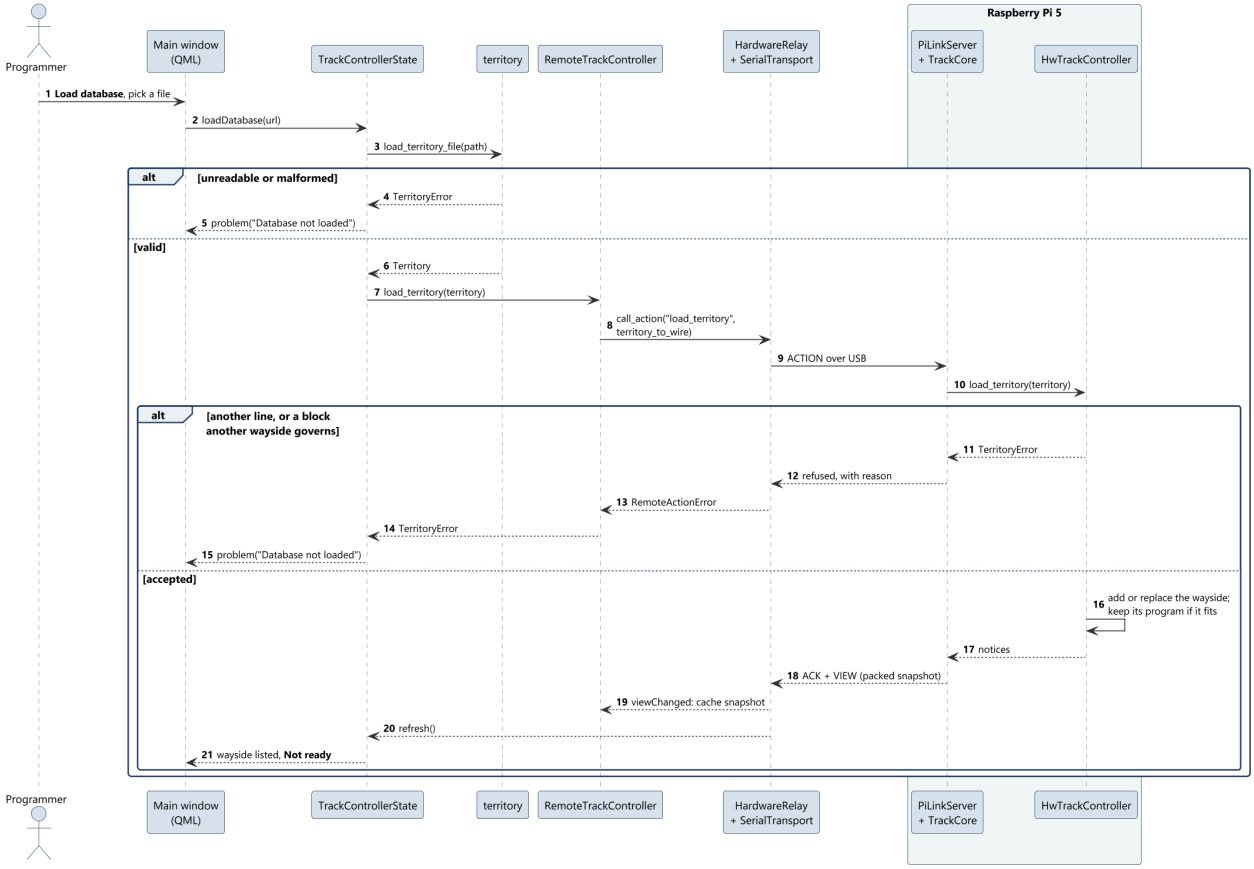

Figure 72. SD-01 Load wayside database (UC-01)

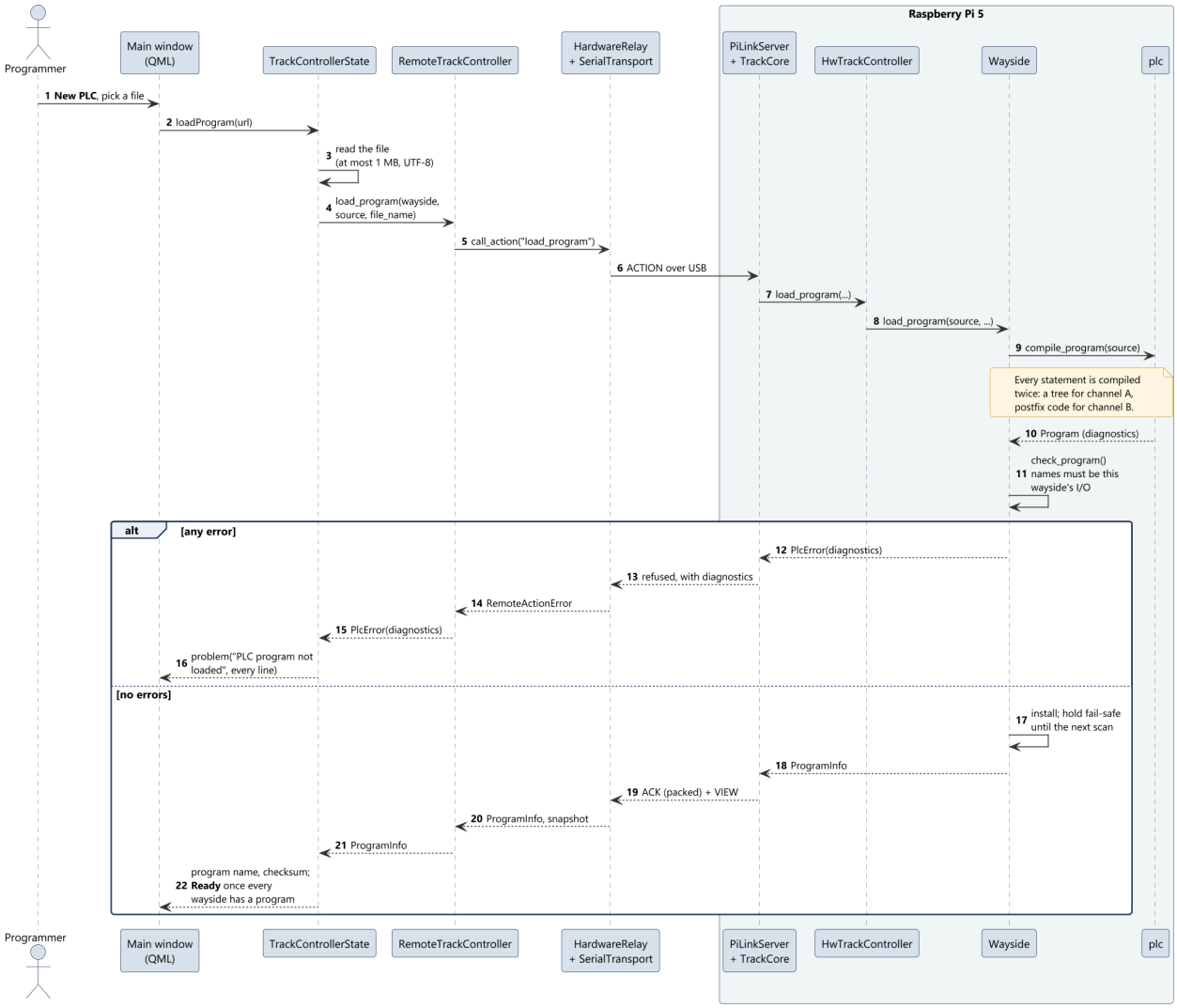

Figure 73. SD-02 Load PLC program (UC-02)

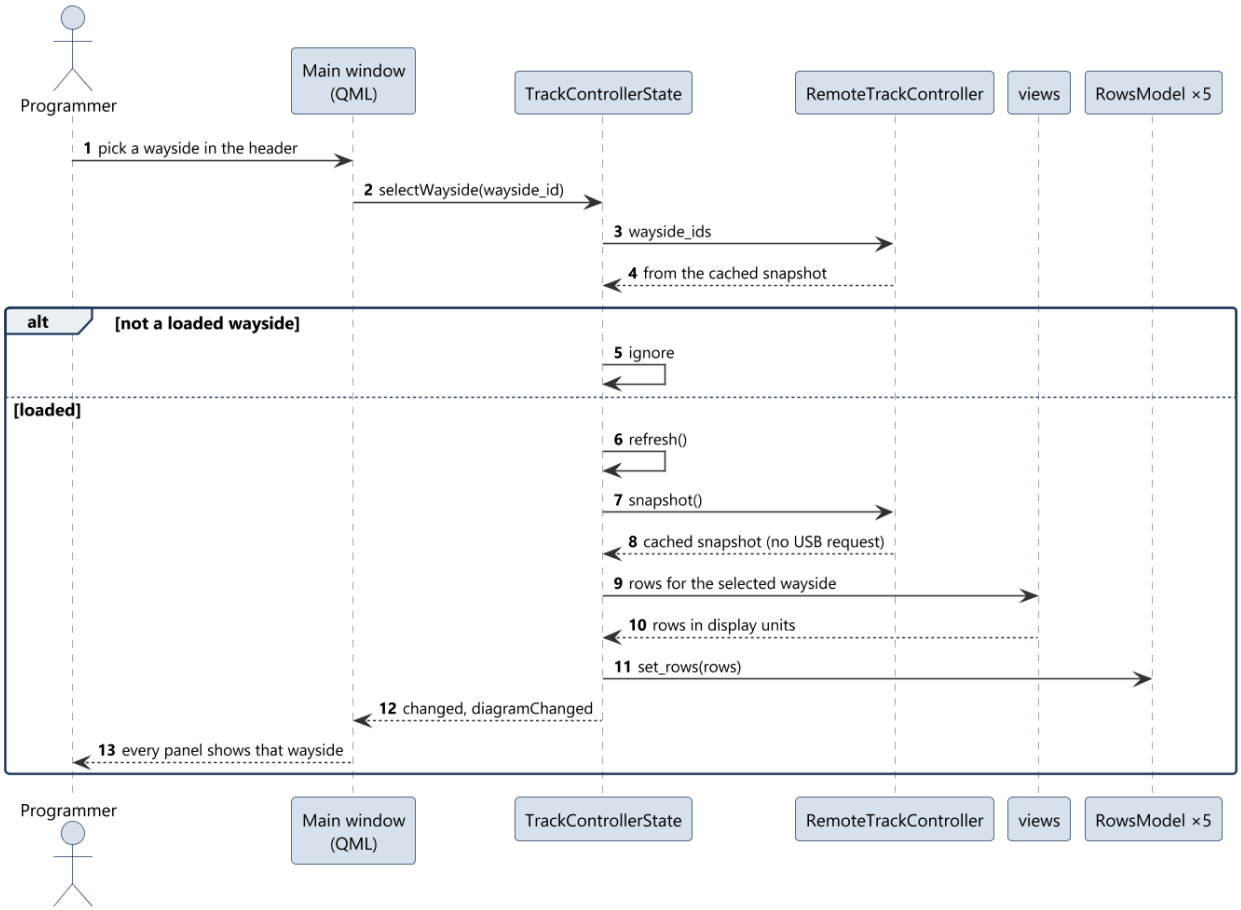

Figure 74. SD-03 Select wayside (UC-03)

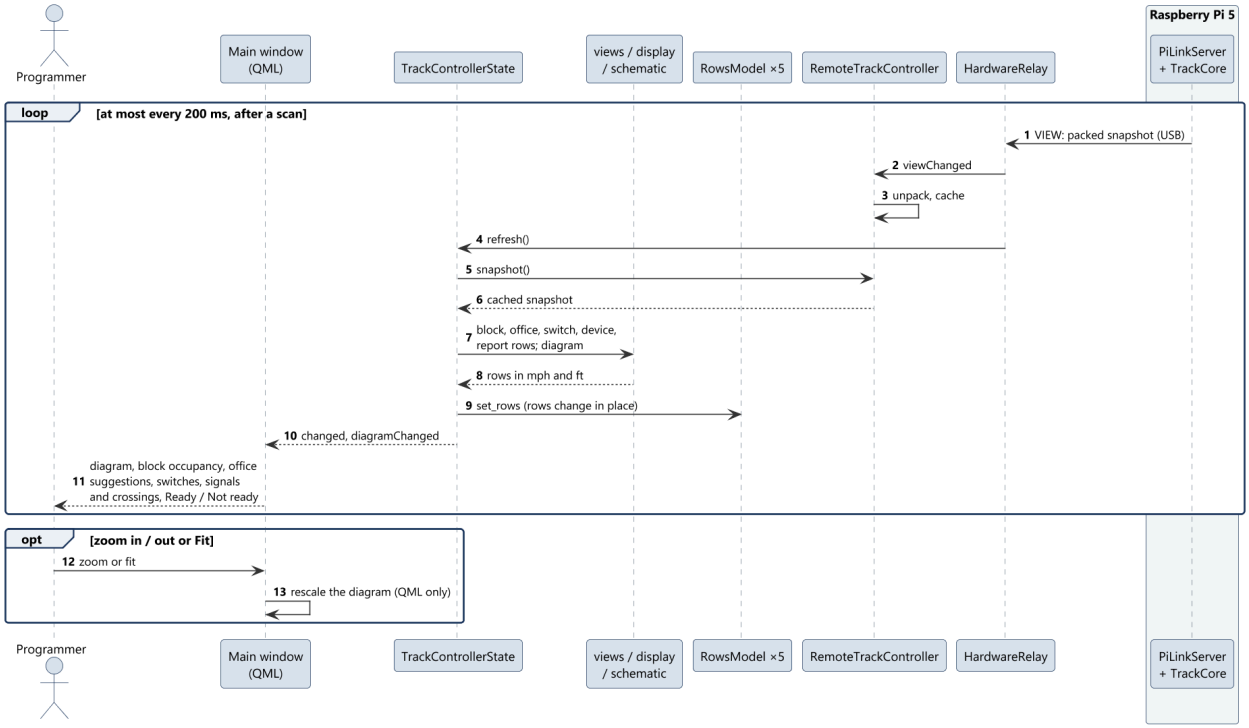

Figure 75. SD-04 Monitor wayside state (UC-04)

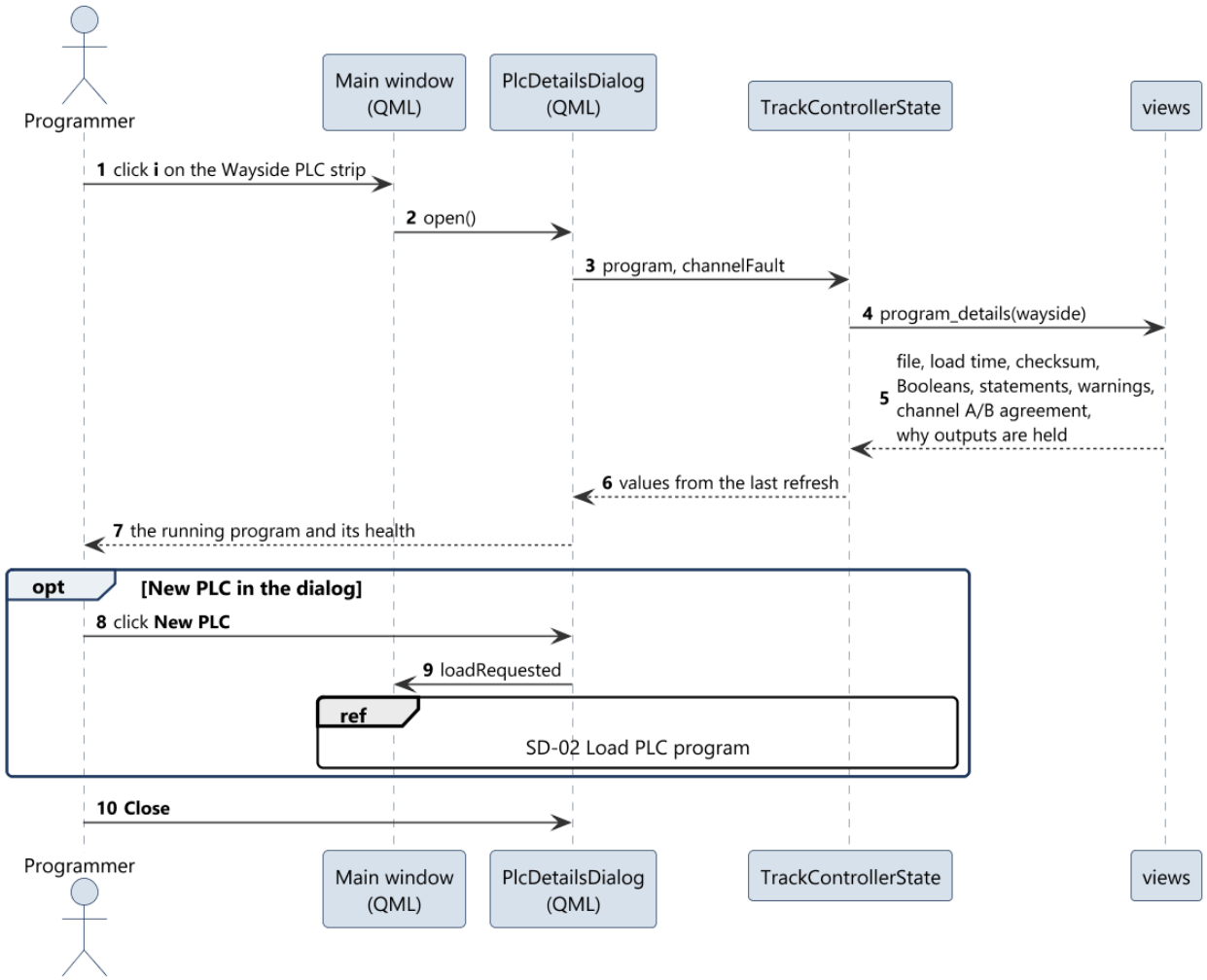

Figure 76. SD-05 View PLC program details (UC-05)

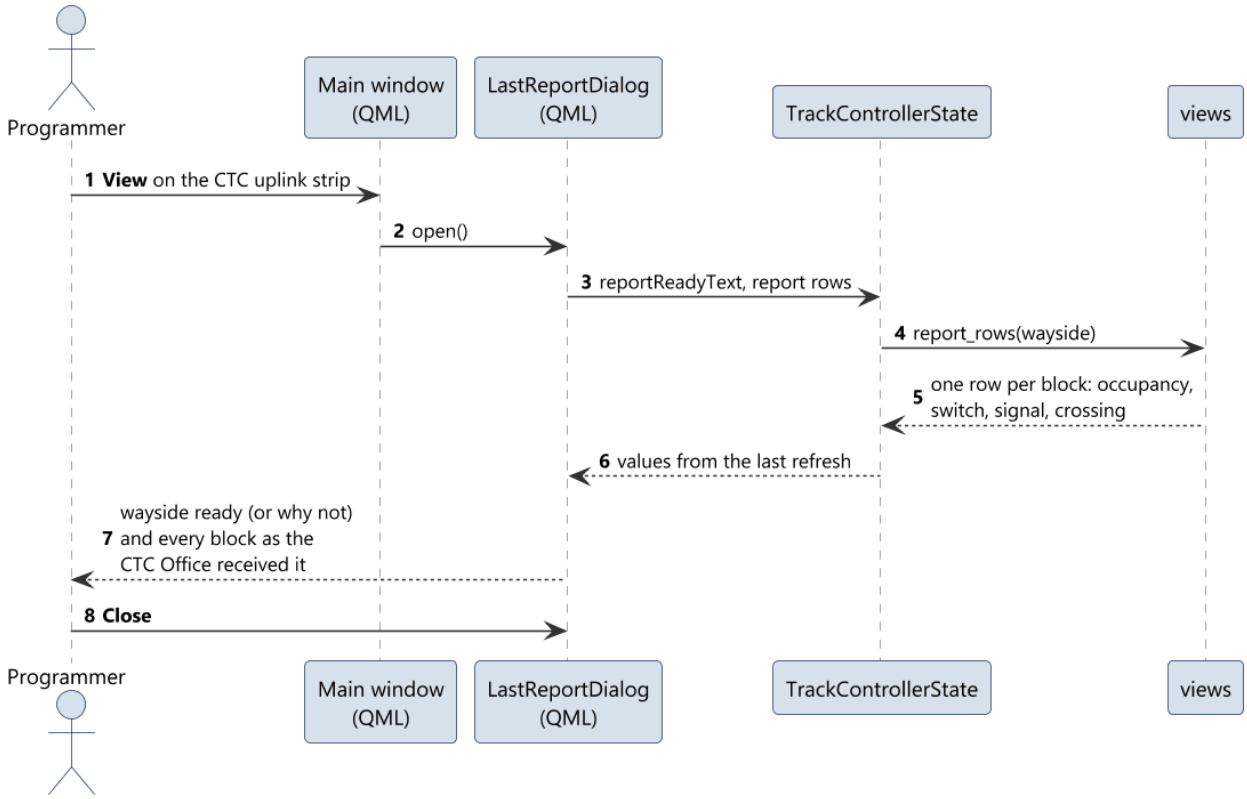

Figure 77. SD-06 View last report to the CTC Office (UC-06)

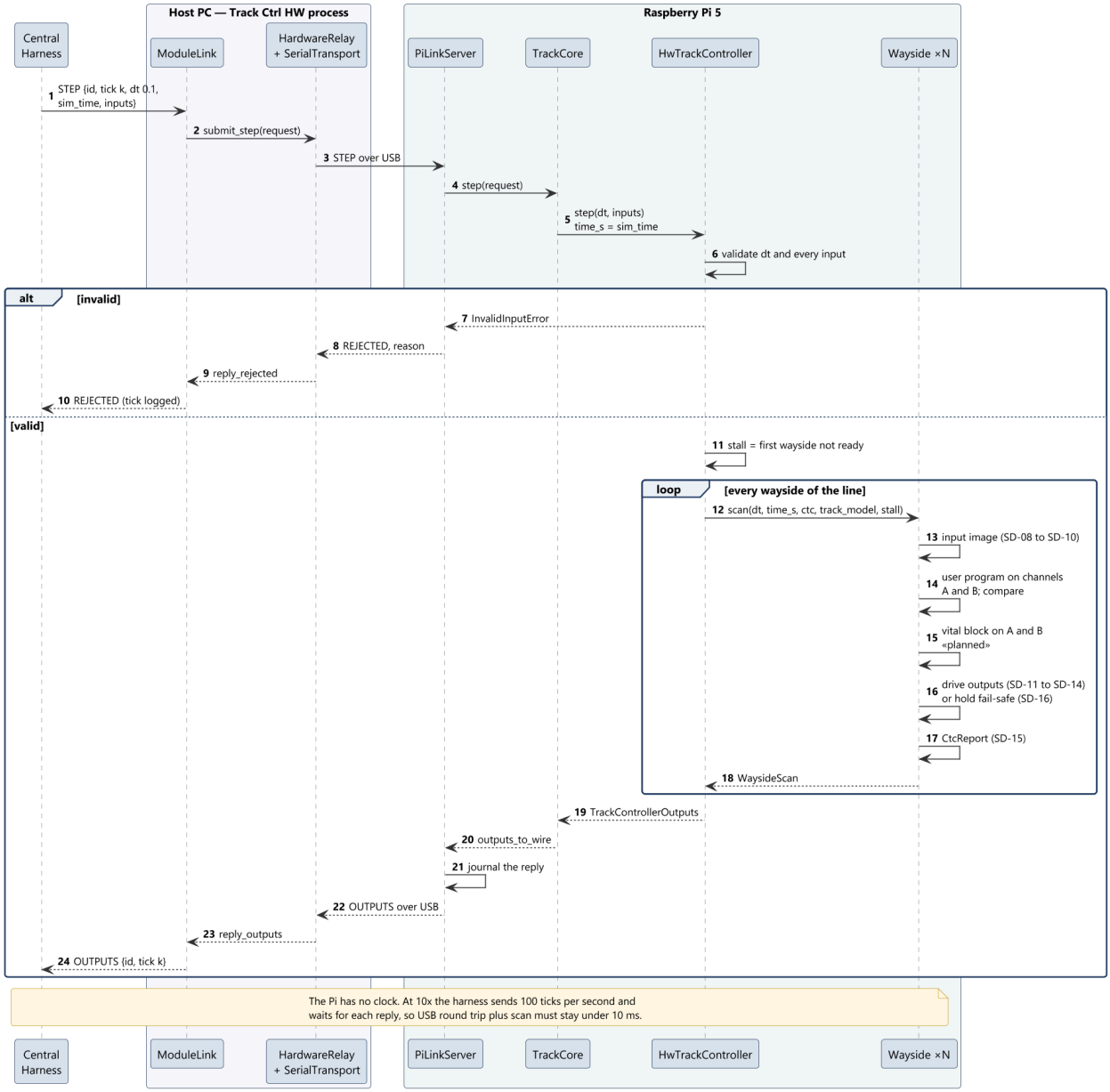

Figure 78. SD-07 Run scan cycle (UC-07)

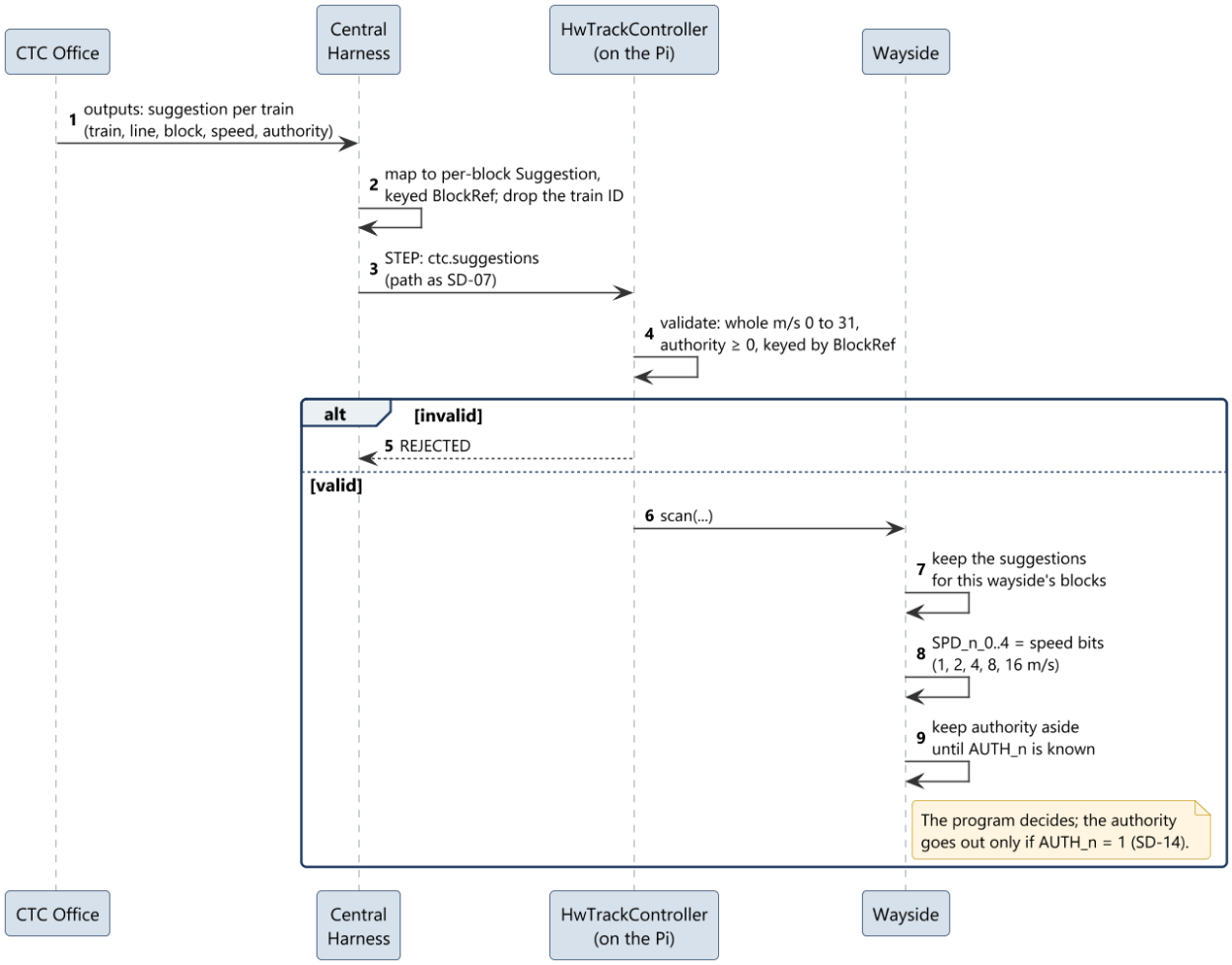

Figure 79. SD-08 Receive speed and authority suggestions (UC-08)

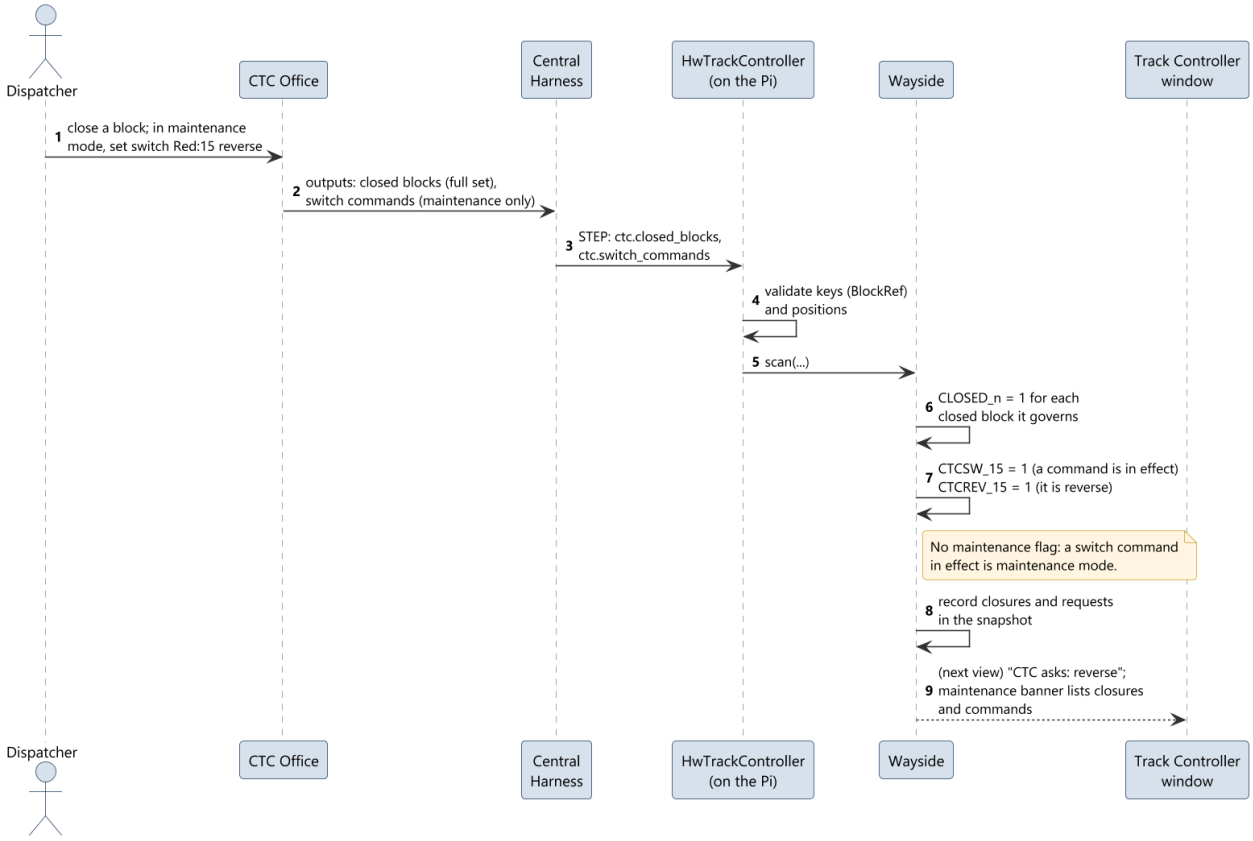

Figure 80. SD-09 Receive closures and maintenance switch commands (UC-09)

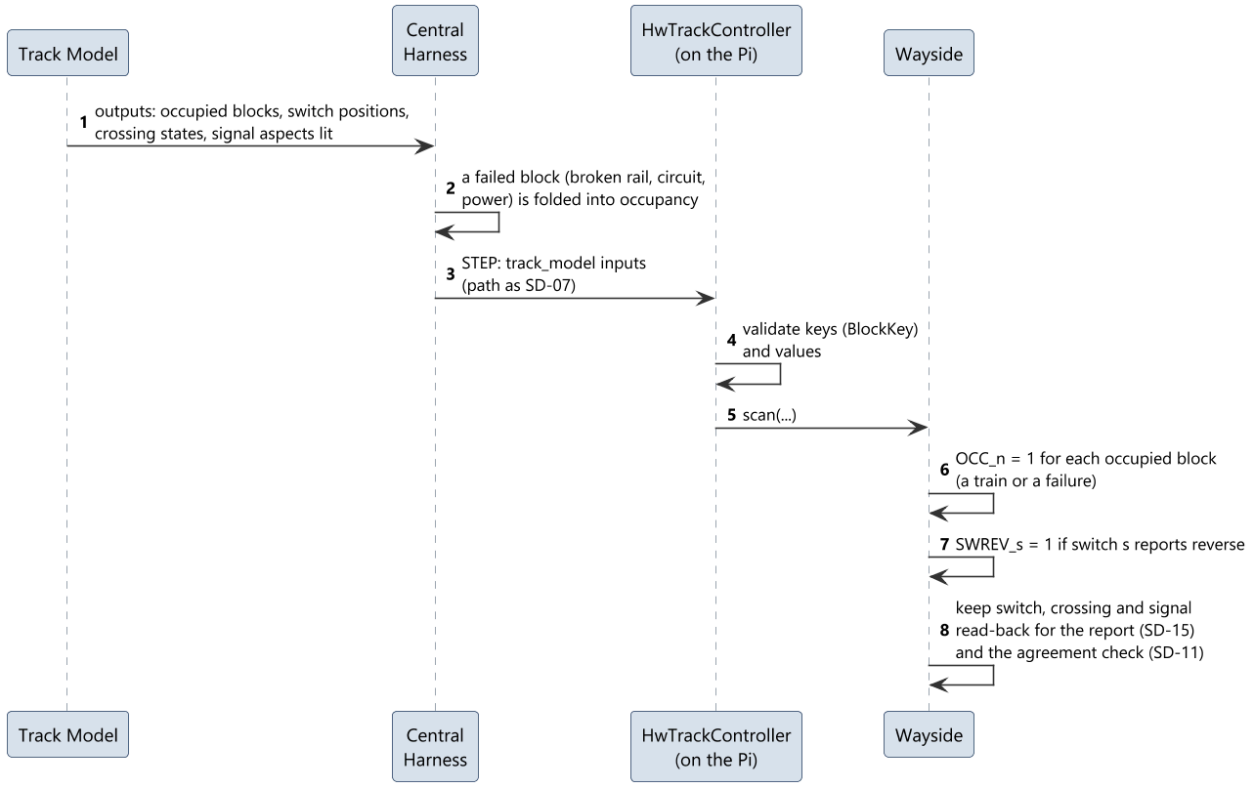

Figure 81. SD-10 Receive track state (UC-10)

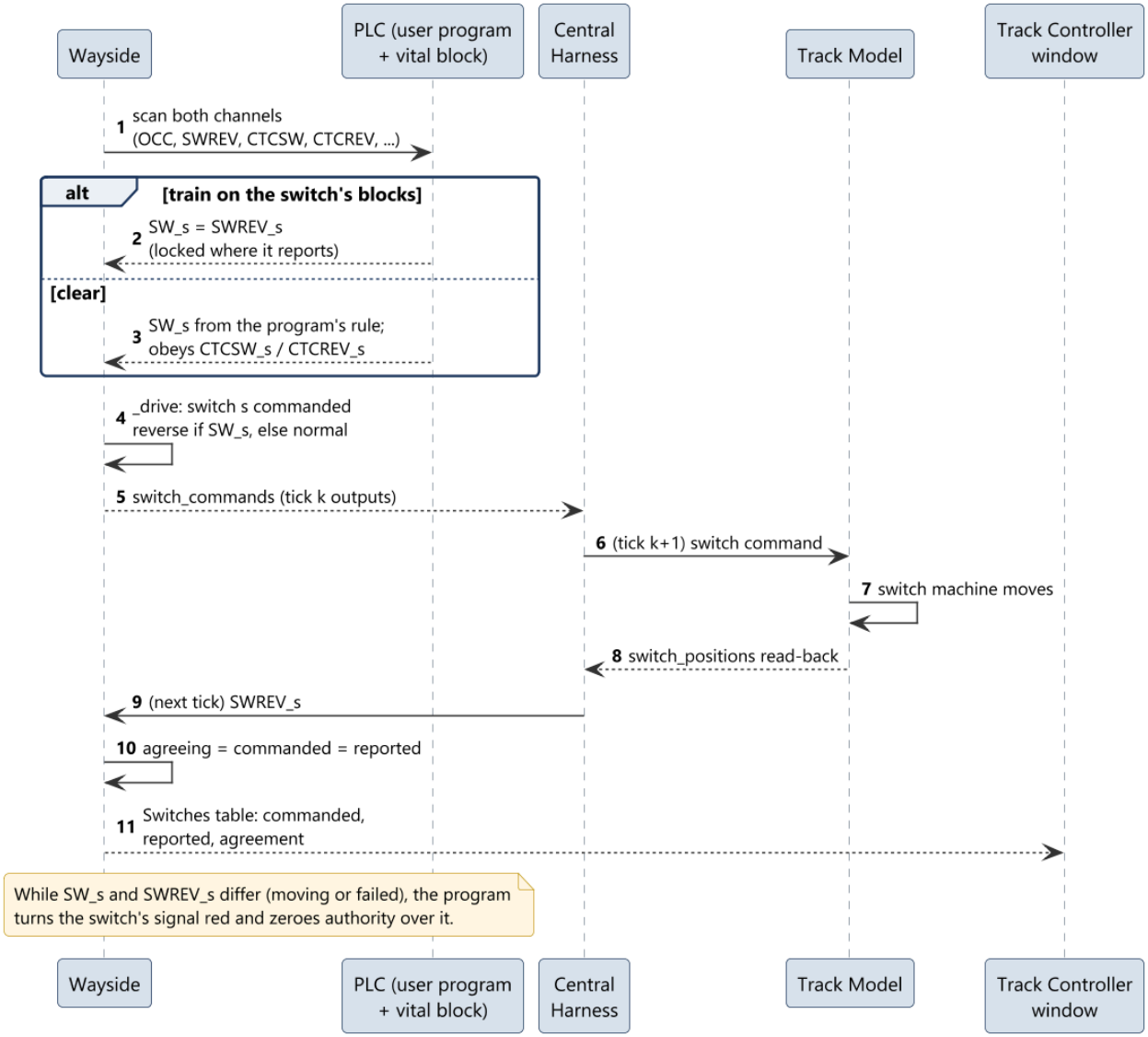

Figure 82. SD-11 Command switches (UC-11)

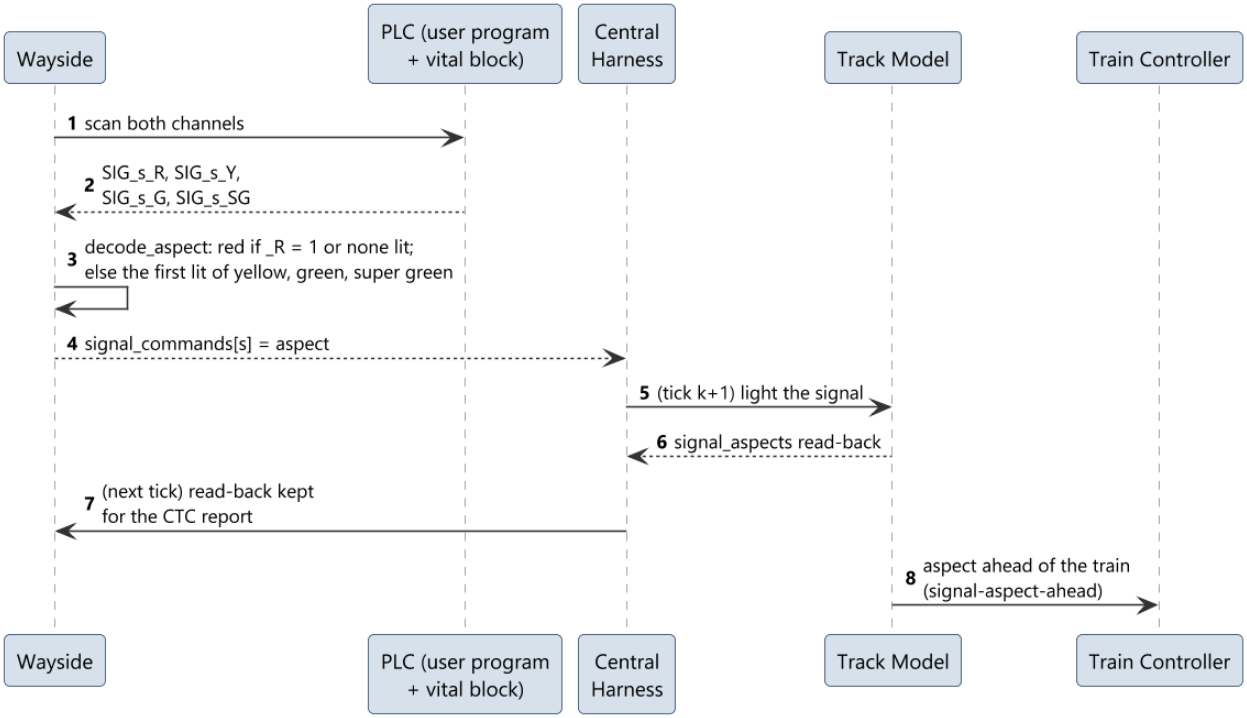

Figure 83. SD-12 Set signal aspects (UC-12)

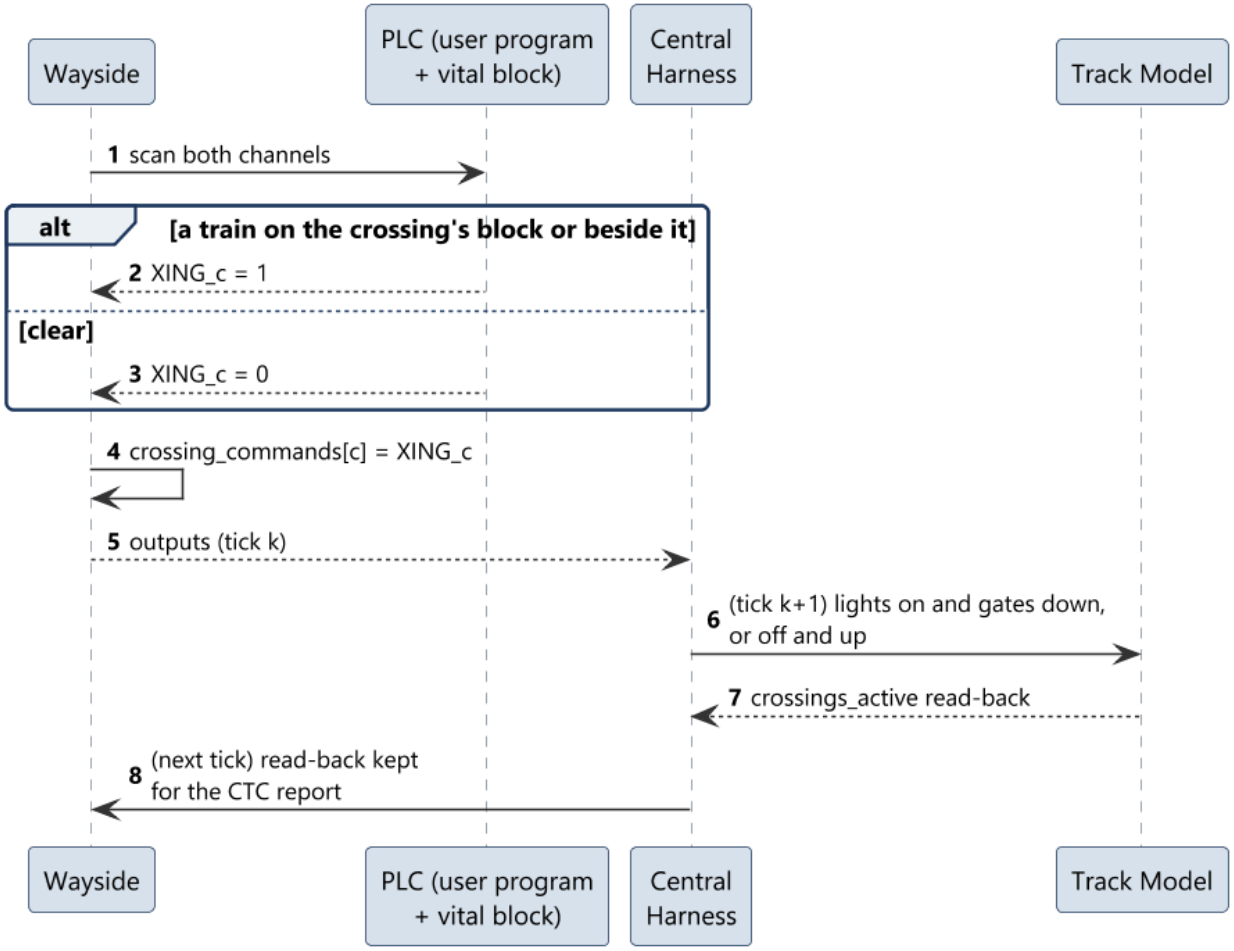

Figure 84. SD-13 Control railway crossings (UC-13)

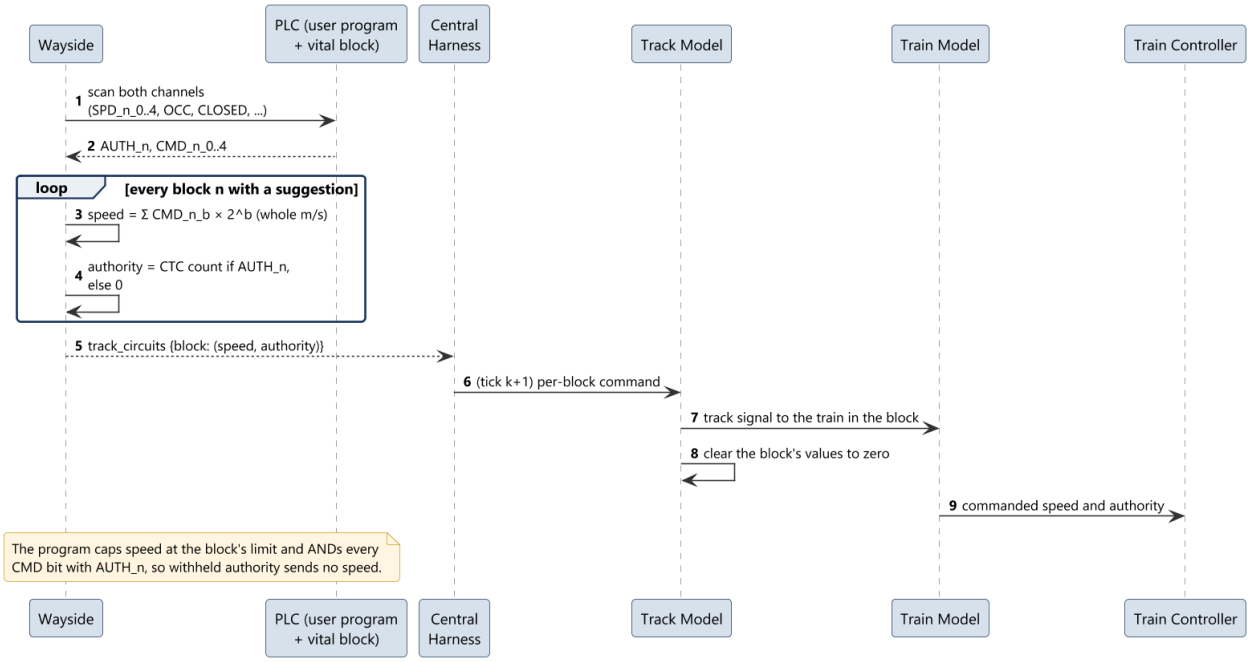

Figure 85. SD-14 Send speed and authority on track circuits (UC-14)

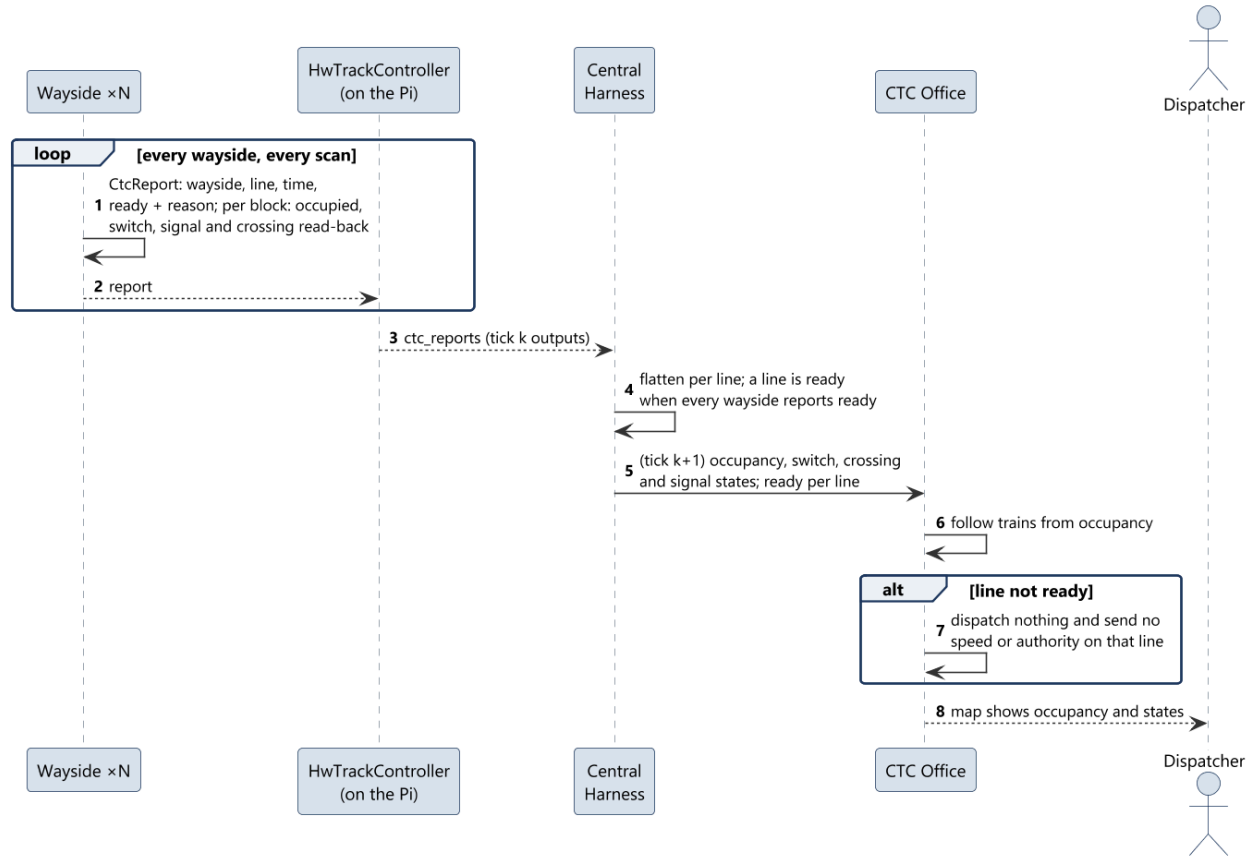

Figure 86. SD-15 Report occupancy, state and readiness (UC-15)

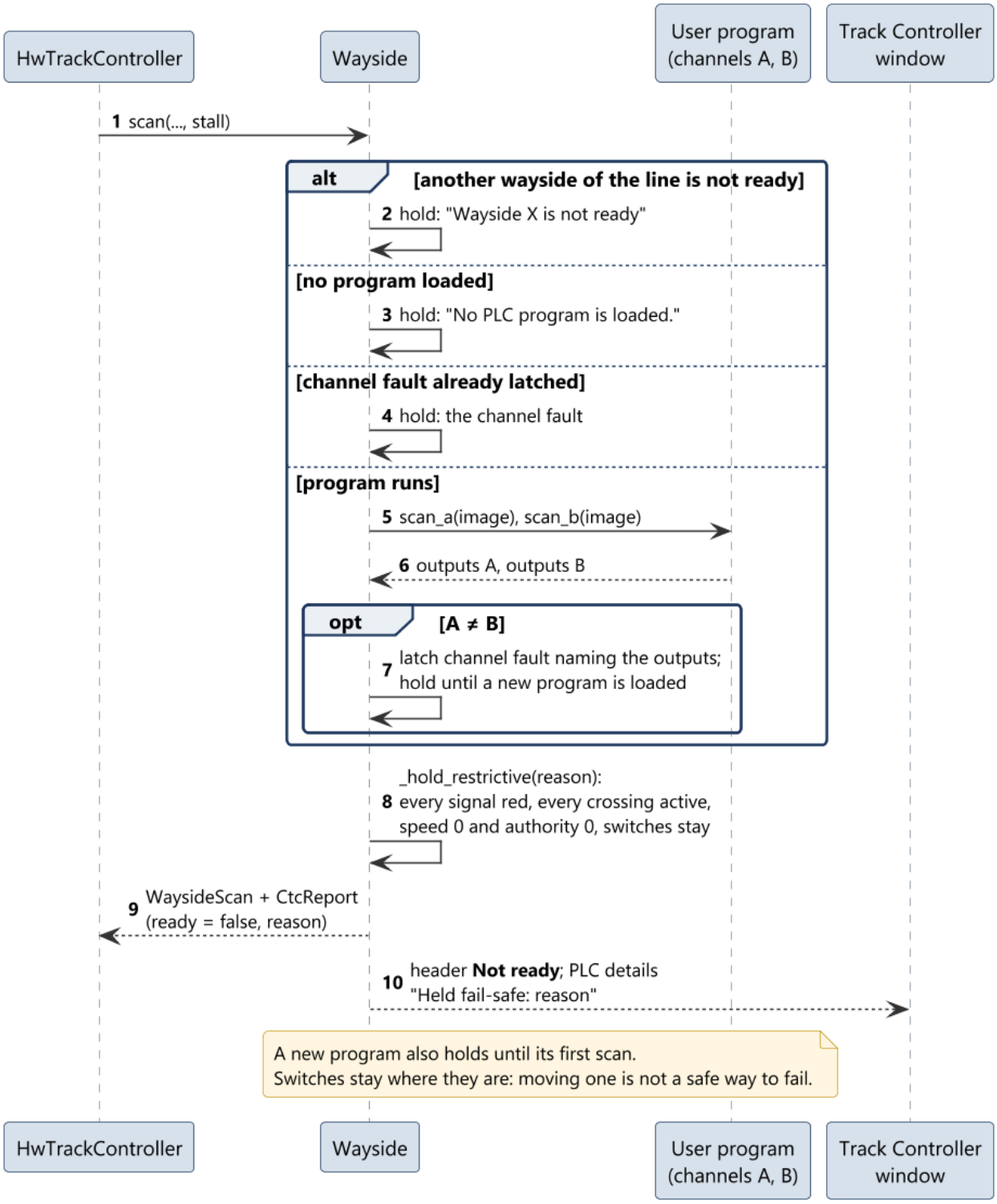

Figure 87. SD-16 Hold fail-safe outputs (UC-16)

5.3.7 Open items

#	Item	Owner
1	The read-only vital PLC block is assumed. Harry Tye to confirm for both variants; implementation steps are recorded.	Braden, Harry
2	Whether the programmer may set a switch by hand in maintenance mode (rubric 6.8). Assumed: only the CTC Office sets a switch. The software variant has a manual switch.	Braden, Harry
3	The Red Line's wayside boundaries. The integration branch carries test copies (Red-1: blocks 1–24 and 73–76; Red-2: 25–45 and 67–72; Red-3: 46–66) that keep every switch inside one wayside; they await Braden's approval.	Braden
4	Power for the Pi 5: its USB-C port is both data link and power input.	Braden, Kevin
5	At 10x the harness sends 100 ticks per second and waits for every reply, so the USB round trip and the scan must stay under 10 ms. To be measured on the Pi.	Braden
6	The PLC names still differ between the variants (the software variant keeps `MAINT`), so one program file does not yet load on both (REQ-INTF-009).	Braden, Harry
7	REQ-FUNC-057 ("know when the system is in maintenance mode"): the CTC Office sends no flag, so the wayside knows only through switch commands. Wording left open.	Braden

5.4 Track Model

Owner: Deni Ephraim. **Source:** sdd/track_model.md.

5.4.1 Stakeholder concerns

Stakeholder	Concern	Addressed By
Track Model developer	Build and test the module alone, before the other modules are integrated.	Section 5.4.6; D005; Section 5.4.11
Owners of CTC, Track controller, Train model, Train controller	Stable boundary types, with one place (the harness edge function) where types and units are matched.	Section 5.4.4; Section 5.4.6; D005; D002
Dispatcher	Correct block occupancy and device state, and ticket sales, reaching the CTC Office.	005-interaction; Section 5.4.2 (UC-4, UC-11)
Track Builder	Load a line and inspect its properties.	Section 5.4.2 (UC-1, UC-2)
Murphy	Inject failures and set the temperature to test the system.	Section 5.4.2 (UC-12, UC-14); Section 5.4.13
Passenger	Wait for and board a train at a station.	Section 5.4.2 (UC-10)

5.4.2 Use cases

Human actors are the Track Builder and Murphy. System actors are the Track controller, Train model, Train controller and CTC office.

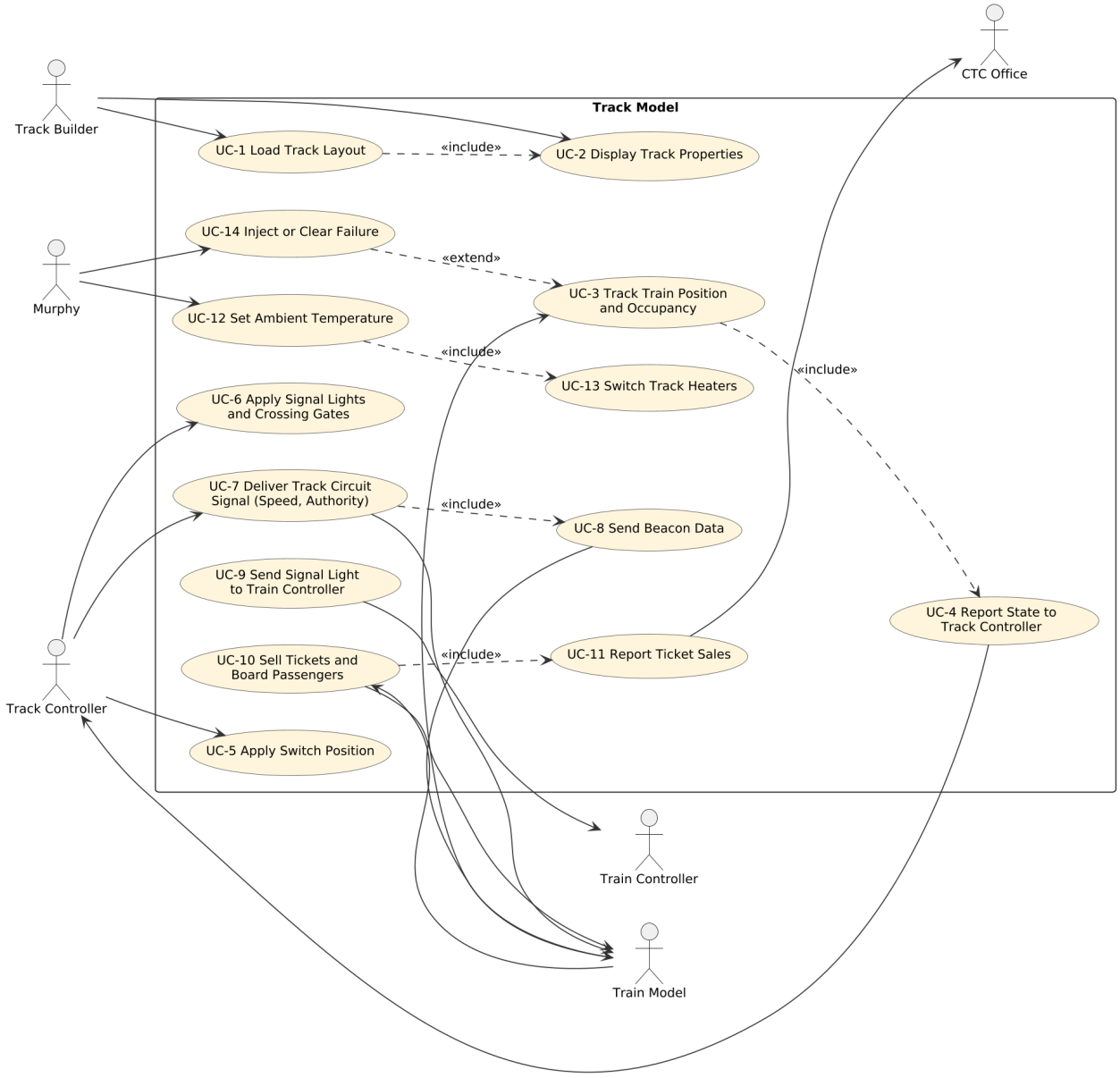

Figure 88. Use case diagram: Track Model, all actors

The same use cases, one diagram for each actor:

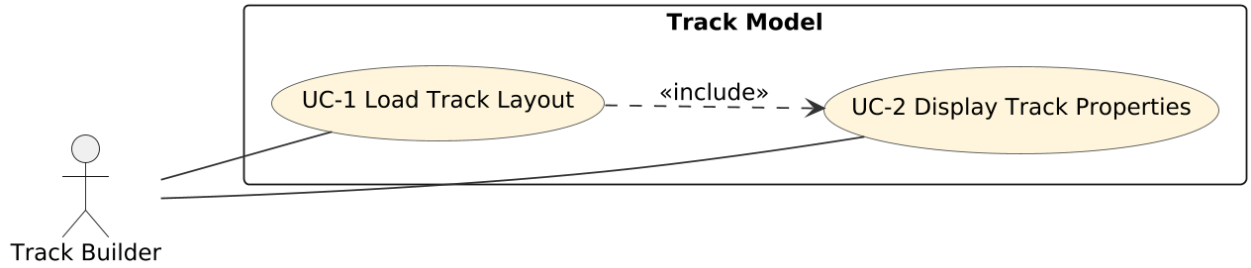

Figure 89. Use cases of the Track Builder

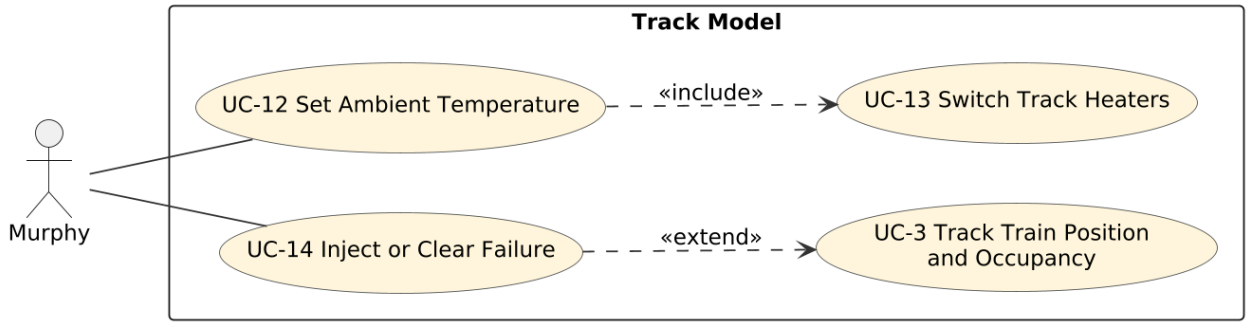

Figure 90. Use cases of Murphy

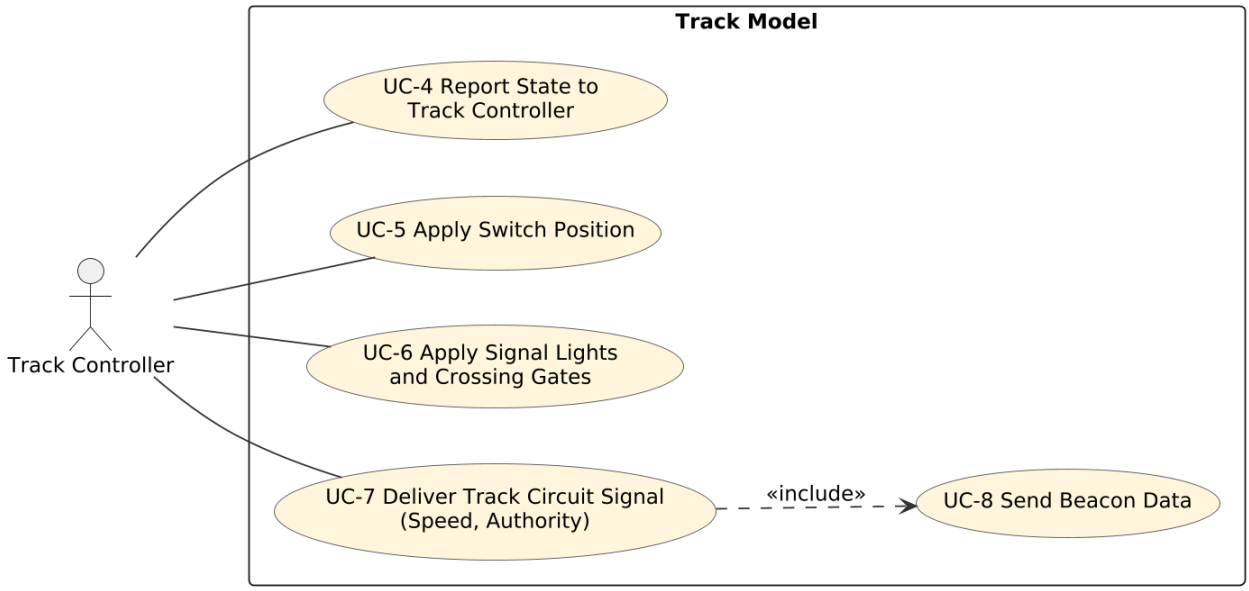

Figure 91. Use cases of the Track Controller

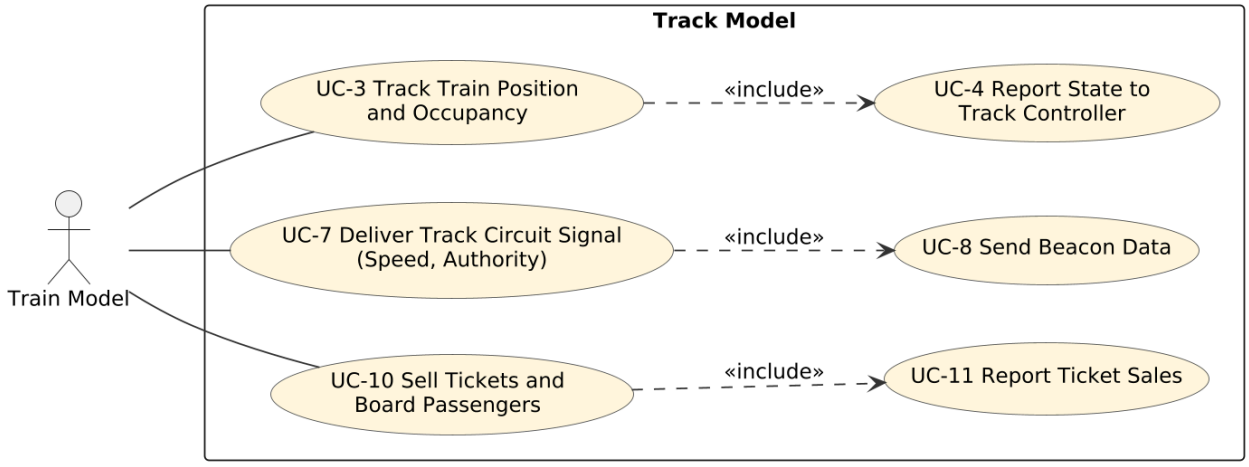

Figure 92. Use cases of the Train Model

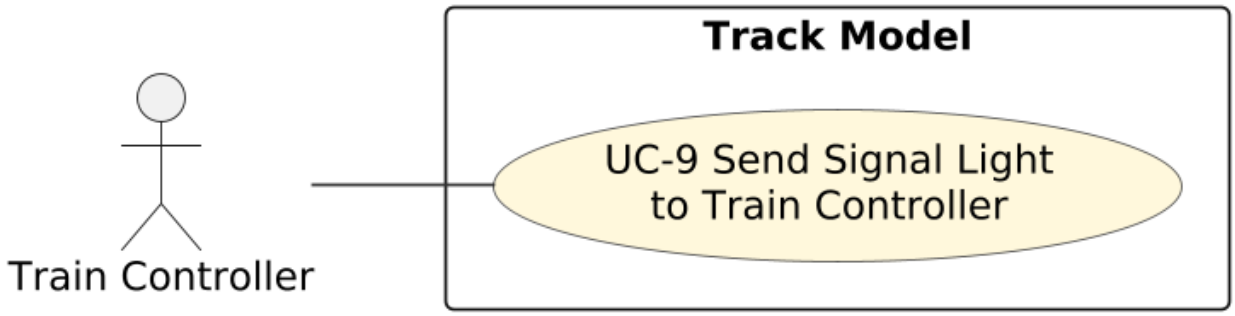

Figure 93. Use cases of the Train Controller

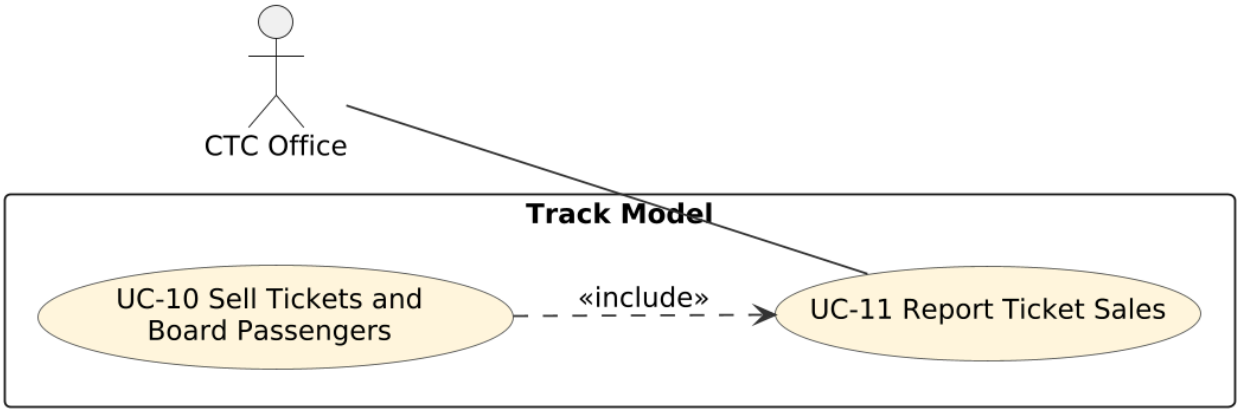

Figure 94. Use cases of the CTC Office

ID	Use case	Actors	Description	Data	Stimulus and response	Sequence
UC-1	Load Track Layout	Track Builder	(1) The builder chooses a layout JSON file with Load track... (2) The module reads each block and its infrastructure, converts grade and speed to backend units, finds switches, signals and beacons and builds the track graph. (3) The file replaces the loaded line of the same name or adds a new line, and the module restarts. At startup the module loads each `*_line.json` file beside it.	In: layout file. Out: loaded line, or an error message.	File chosen. A bad file changes nothing and shows why.	SD-01

ID	Use case	Actors	Description	Data	Stimulus and response	Sequence
UC-2	Display Track Properties	Track Builder	(1) The user selects a block. (2) The window shows its length, grade, speed limit, elevation, commanded speed, authority, station, platform, direction of travel, beacon, heater, track temperature, underground flag, crossing, signal, switch and occupancy, in imperial units.	In: selected block. Out: block table and detail panel.	Block selected. All properties are shown.	SD-02
UC-3	Track Train Position and Occupancy	Train Model	(1) Each tick the Train Model reports each train's block and offset. (2) The Track Model adds the offset to the distance it kept from earlier blocks and moves the train to the next block when that passes the block length, or back when it goes below 0, following the switches as set. (3) A block is occupied when a train is in it or its track circuit has failed.	In: TrainReport per train. Out: occupancy.	Tick. Occupancy matches the train positions.	SD-03
UC-4	Report State to Track Controller	Track Controller	(1) Every tick the Track Model builds the actual state of every device: block occupancy, switch positions, crossing gates, signal aspects, failure status, heater states and track temperature. (2) It returns it for the Track controller. The CTC office reads it through the Track controller.	Out: TrackControllerOutputs.	Tick or failure injected. The read-back shows the actual state, which a failure can make differ from the command.	SD-04

ID	Use case	Actors	Description	Data	Stimulus and response	Sequence
UC-5	Apply Switch Position	Track Controller	(1) The Track controller commands a switch position. (2) The switch moves to it unless its block has lost power. (3) The actual position is reported.	In: switch commands. Out: switch states.	Command received. The switch shows its actual position.	SD-05
UC-6	Apply Signal Lights and Crossing Gates	Track Controller	(1) The Track controller commands signal aspects and crossing gates. (2) A light shows its commanded aspect, or RED without power. A gate is down when commanded, or without power.	In: signal and crossing commands. Out: signal and gate states.	Command received. Lights and gates show the new state.	SD-06
UC-7	Deliver Track Circuit Signal	Track Controller, Train Model	(1) The Track controller sends a speed (whole m/s) and an authority (blocks) for each block. (2) The Track Model stores them as the signal of that block's track circuit. (3) Each train receives the signal of the block it is in. A failed circuit, or a block never commanded, carries speed 0 and authority 0.	In: speed and authority per block. Out: TrackSignal in the train feed.	Tick. The train gets the signal of its block.	SD-07
UC-8	Send Beacon Data	Train Model	(1) Beacons stand on the blocks next to a station block. (2) When a train is in a beacon block, its feed carries the station name, platform side and underground flag.	Out: Beacon in the train feed.	Train in a beacon block. The feed carries the beacon.	SD-08

ID	Use case	Actors	Description	Data	Stimulus and response	Sequence
UC-9	Send Signal Light to Train Controller	Train Controller	(1) A signal stands on the block before each switch. (2) On the tick a train enters such a block, the Track Model reports the light's colour for that train; on every other tick it reports none. (3) The harness sends it to the Train controller, which keeps it.	Out: `signal_seen` per train.	Train enters a signal block. The Train controller receives the colour once.	SD-09
UC-10	Sell Tickets and Board Passengers	Train Model	(1) Each station may sell one ticket per tick, at a configured probability, and keeps a queue of waiting passengers. (2) A train that is stopped in a station block with a door open takes min(waiting, capacity) passengers. (3) The count is returned in the train feed.	In: speed, doors, capacity. Out: passengers boarded.	Train stopped with a door open. The waiting count falls and the train is told how many boarded.	SD-10
UC-11	Report Ticket Sales	CTC Office	(1) Every tick the Track Model reports the tickets sold on that tick, by station. (2) It keeps no running total and sends none. (3) The harness sends the sales to the CTC office as (count, station) pairs.	Out: `tickets_sold_now`.	Tick. The CTC office receives the sales of that tick.	SD-11
UC-12	Set Ambient Temperature	Murphy	(1) Murphy enters a temperature in °F in the Track Model window. (2) It replaces the temperature the harness or test UI sends, until the module restarts.	In: temperature. Out: displayed ambient temperature.	Value entered. Track temperature follows it.	SD-12

ID	Use case	Actors	Description	Data	Stimulus and response	Sequence
UC-13	Switch Track Heaters	Murphy (indirect)	(1) The Track Model switches each section's heaters itself (D026). (2) Each section's track temperature eases toward ambient, or toward ambient plus 10 °C while its heaters run. (3) A power failure in any block of the section stops its heaters.	In: ambient temperature. Out: heater state and track temperature per section.	Temperature change. Heater states and track temperatures change.	SD-13
UC-14	Inject or Clear Failure	Murphy	(1) Murphy selects a block and a failure: broken rail, track circuit or power. (2) The block reads occupied and its track circuit carries nothing, so a train in it gets speed 0 and authority 0. (3) A power failure also holds the block's switch, shows its light RED, drops its crossing gates and stops its section's heaters.	In: block and failure. Out: failure shown in the window and in the read-back.	Failure selected. The Track controller sees it at once, between ticks.	SD-14

5.4.3 Track Model architecture (component view)

The Track Model runs as one process with a window. The window (Qt Quick, PySide6) shows state through `TrackModelState`, which converts to display units. The module core (`model.py`, `layout.py`) has no Qt code. The boundary types and the `TrackModel` Protocol are in `interface.py`. The test scaffolding (`test_link/`, `test_ui/`) is not part of the module and is removed at integration. At integration the harness calls `step` through the edge functions in `common/harness/`.

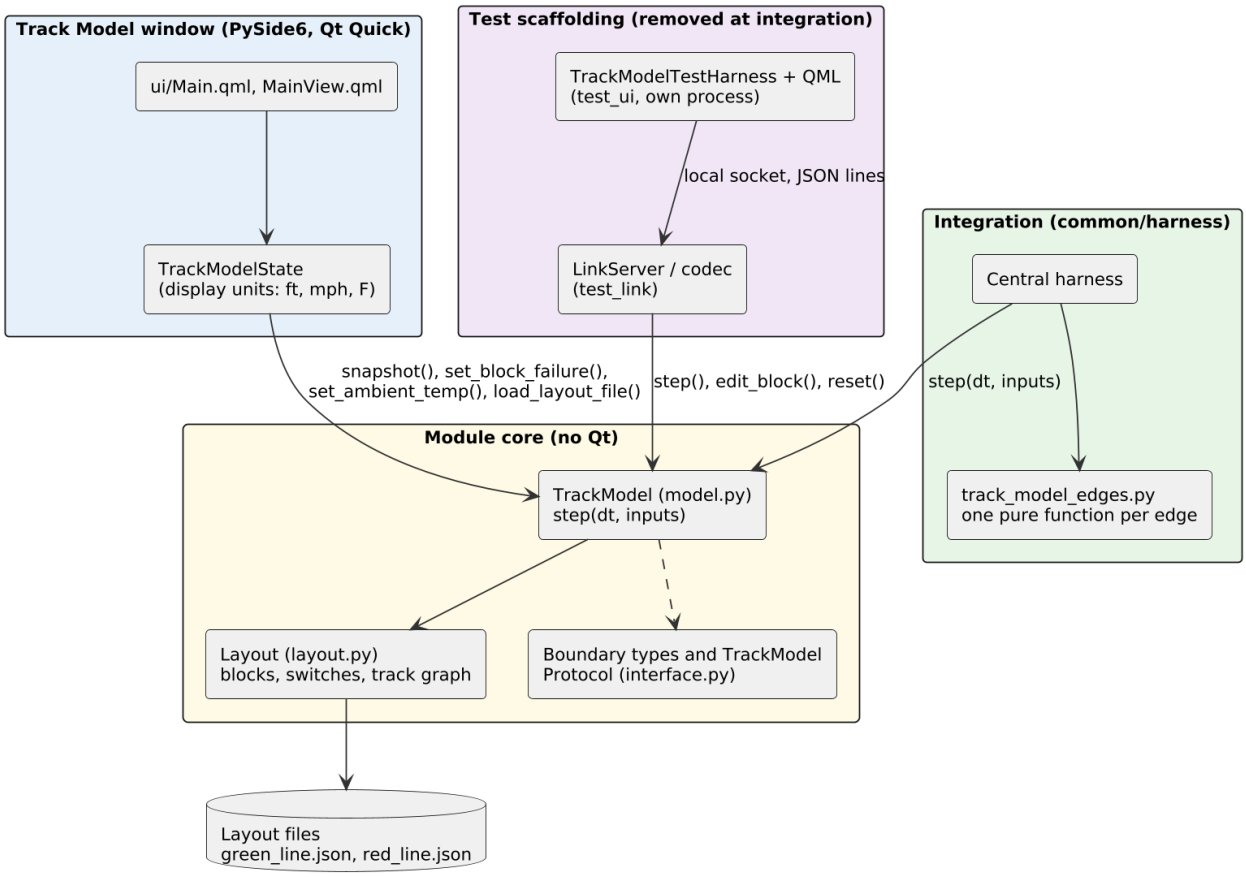

Figure 95. Track Model architecture (component view)

5.4.4 Boundary types

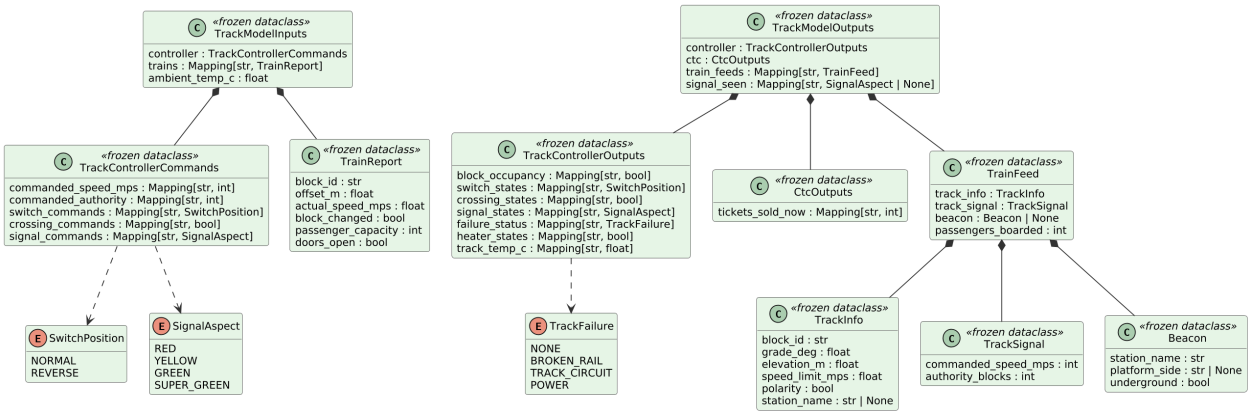

Figure 96. Boundary types of the Track Model (`track_model/interface.py`)

- **More Information:**

  Every type is a frozen dataclass or an enumeration, so a value cannot change after it is sent. Every value is in its backend unit: metres, m/s, degrees and degrees Celsius (D002). Every ID is a string: a block is `GREEN D-13`, a section is `GREEN D`. A switch is keyed by the block of its

point, a signal by the block it stands on, a crossing by its block. Commanded speed and authority are per block, never per train, because the Track Controller knows which blocks are occupied, not which train is in them. Speed is a whole number of m/s and authority is a whole number of blocks (D013). The fields are described in Section 5.4.7.

5.4.5 Class diagrams

Figure 97 shows the module core and Figure 98 the window and the test link.

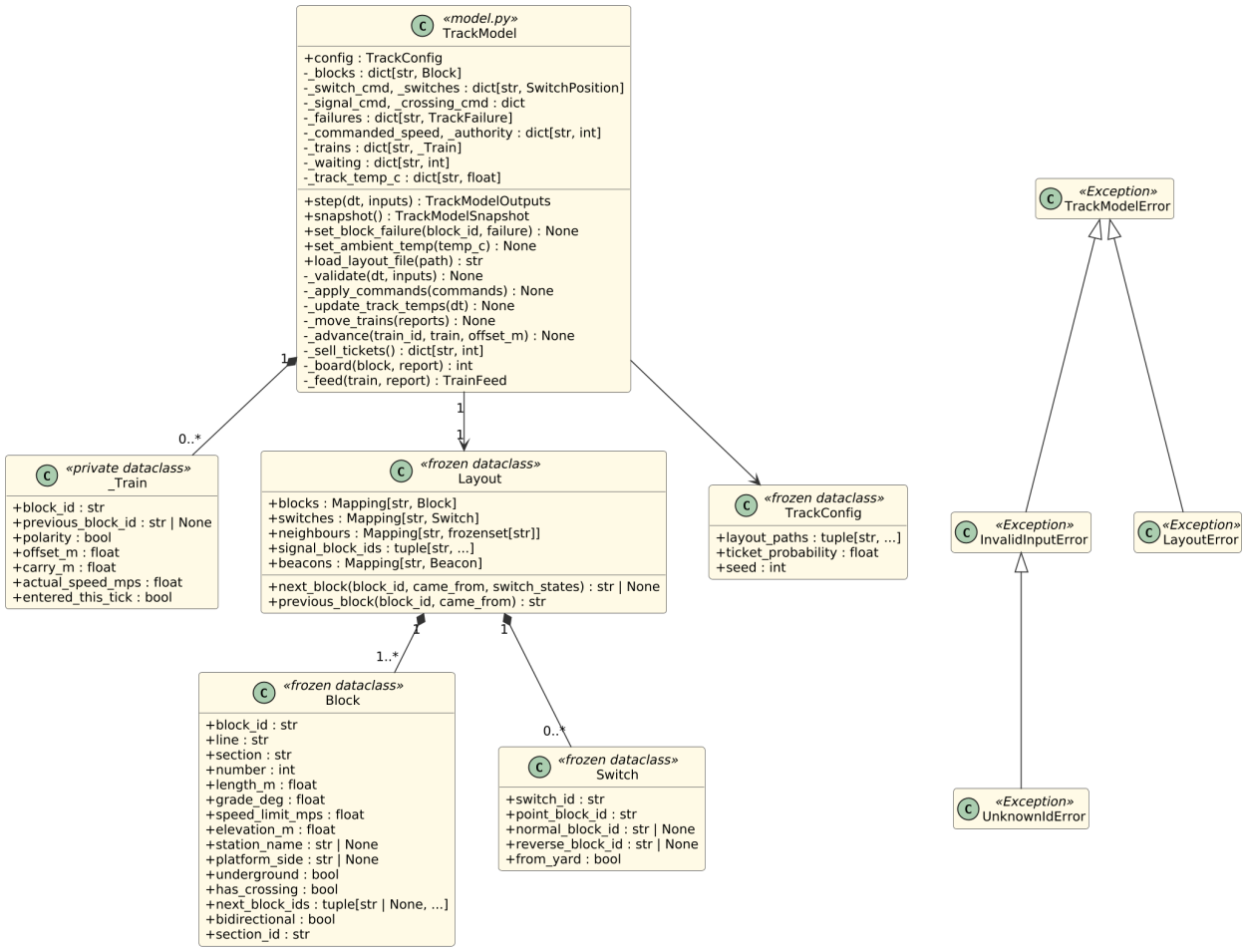

Figure 97. Class diagram 1: module core

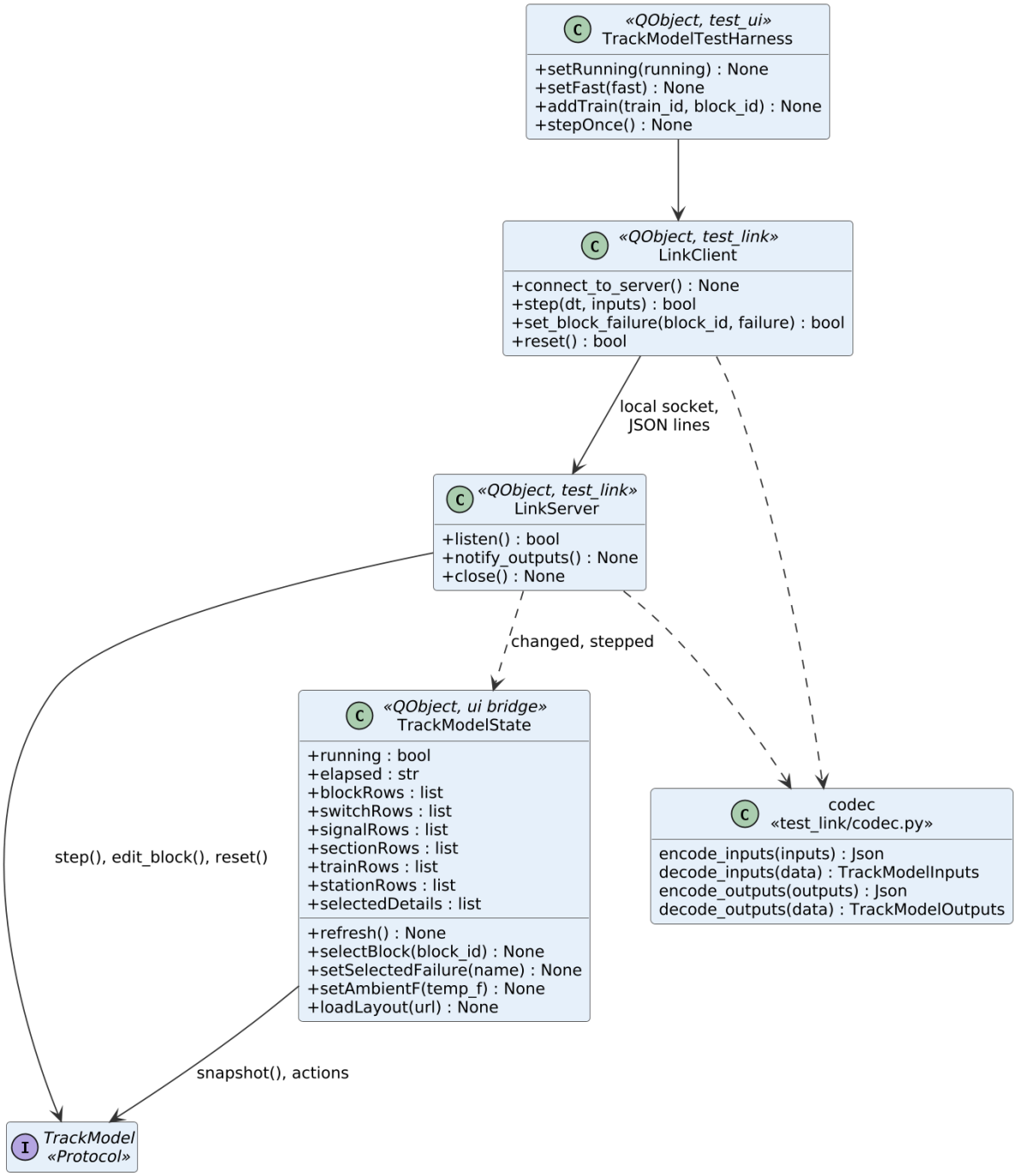

Figure 98. Class diagram 2: window and test link

5.4.6 Interface

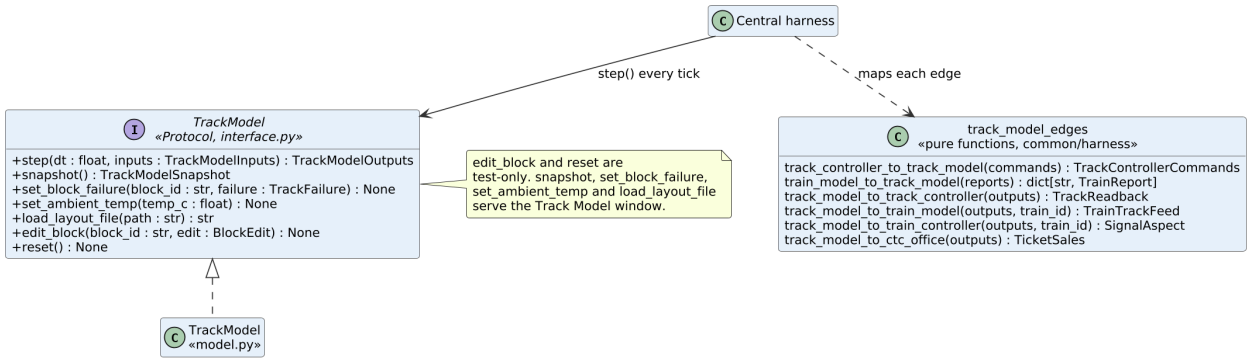

Figure 99. The `TrackModel` Protocol and the harness edge functions

- **More Information:**

  The whole interface for the other modules is one call, `step(dt, inputs)`, which returns the outputs. The harness calls it once per tick. The other methods serve the Track Model window (`snapshot`, `set_block_failure`, `set_ambient_temp`, `load_layout_file`) or the test UI (`edit_block`, `reset`). The module does not know that a test UI exists.

  The harness owns one pure function for each connection:

Edge	Function (`common/harness/track_model_edges.py`)
Track Controller → Track Model	`track_controller_to_track_model`
Train Model → Track Model	`train_model_to_track_model`
Track Model → Track Controller	`track_model_to_track_controller`
Track Model → Train Model	`track_model_to_train_model`
Track Model → Train Controller	`track_model_to_train_controller`
Track Model → CTC Office	`track_model_to_ctc_office`

The CTC office reads occupancy and device state through the Track controller. Ticket sales go to the CTC office directly.

5.4.7 Interface dictionary

Every value is in its backend unit (D002). Per-block data is keyed by block ID, per-train data by train ID, per-station data by station name and per-section data by section ID.

ID	Direction	Signal	Type	Units	Description
IF-01	Track Controller → Track Model	commanded_speed_mps	dict[block, int]	m/s	Speed for the track circuit of each block, a whole number.
IF-02	Track Controller → Track Model	commanded_authority	dict[block, int]	blocks	Blocks a train in that block may still enter. 0 = stop.
IF-03	Track Controller → Track Model	switch_commands	dict[switch, SwitchPosition]	-	Keyed by the block of the switch point.
IF-04	Track Controller → Track Model	crossing_commands	dict[block, bool]	-	True = gate closed.
IF-05	Track Controller → Track Model	signal_commands	dict[block, SignalAspect]	-	Aspect of the light on the block.
IF-06	Train Model → Track Model	block_id	str	-	Block the train is in. Empty = not on the track yet.
IF-07	Train Model → Track Model	offset_m	float	m	Distance since entering the block. Restarts at 0 in each block; negative while rolling back.
IF-08	Train Model → Track Model	actual_speed_mps	float	m/s	Signed speed.
IF-09	Train Model → Track Model	doors_open	bool	-	A door is open. Boarding needs one.

ID	Direction	Signal	Type	Units	Description
IF-10	Train Model → Track Model	passenger_capacity	int	persons	Passengers the train can take.
IF-11	Train Model → Track Model	block_changed	bool	-	Recorded, not used.
IF-12	Harness → Track Model	dt	float	s	Tick length, fixed at 0.1 s.
IF-13	Harness → Track Model	ambient_temp_c	float	deg C	Ambient temperature. A value set in the Track Model window replaces it.
IF-14	Murphy (window) → Track Model	set_block_failure	(block, TrackFailure)	-	Inject or clear NONE, BROKEN_RAIL, TRACK_CIRCUIT or POWER.
IF-15	Murphy (window) → Track Model	set_ambient_temp	float	deg C	Entered in deg F and converted by the window.
IF-16	Track Builder (window) → Track Model	load_layout_file	path to .json	-	Replaces or adds a line and restarts the module.
IF-17	Track Model → Track Controller	block_occupancy	dict[block, bool]	-	True if a train is in the block or its circuit has failed.
IF-18	Track Model → Track Controller	switch_states	dict[switch, SwitchPosition]	-	Actual positions.

ID	Direction	Signal	Type	Units	Description
IF-19	Track Model → Track Controller	crossing_states	dict[block, bool]	-	True = gate closed (also closed without power).
IF-20	Track Model → Track Controller	signal_states	dict[block, SignalAspect]	-	Actual aspects (RED without power).
IF-21	Track Model → Track Controller	failure_status	dict[block, TrackFailure]	-	Active failure on each block.
IF-22	Track Model → Track Controller	heater_states	dict[section, bool]	-	Heaters actually running.
IF-23	Track Model → Track Controller	track_temp_c	dict[section, float]	degC	Track temperature of each section.
IF-24	Track Model → CTC Office	tickets_sold_now	dict[station, int]	tickets	Tickets sold this tick. No running total is kept or sent. The edge sends (count, station) pairs.
IF-25	Track Model → Train Model	track_info	block_id, grade_deg, elevation_m, speed_limit_mps, polarity, station_name	deg, m, m/s	Terrain of the occupied block. Polarity reverses at every block change.
IF-26	Track Model → Train Model	track_signal	commanded_speed_mps: int, authority_blocks: int	m/s, blocks	Track circuit signal of the occupied block. 0 and 0 on a dead circuit.
IF-27	Track Model → Train Model	beacon	station_name, platform_side, underground	-	Platform side is L, R or LR. None when not over a beacon.

ID	Direction	Signal	Type	Units	Description
IF-28	Track Model → Train Model	passengers_boarded	int	persons	Passengers who boarded this tick.
IF-29	Track Model → Train Controller	signal_seen	dict[train, SignalAspect or None]	-	Colour of the light as the train enters a signal block, on that tick only. Sent directly, not through the Train Model.

5.4.8 Data design

Layout files. Each line is one JSON file with a line name and an array of blocks. A block has a block number, section letter, length (m), grade (%), speed limit (km/h), elevation and cumulative elevation (m), an optional station side, an optional `infrastructure` object (station, switch, railway crossing, beacon, transponder, underground) and `next_blocks`. The loader converts grade to degrees and speed to m/s on read, and checks `next_blocks` against the switches. A line without `next_blocks` is run both ways. Only the Green and Red lines are used at integration (D023).

Identifiers. A block is `<LINE> <section>-<number>`, such as `GREEN D-13`. A section is `<LINE> <section>`, such as `GREEN D`. A switch is named after the block of its point.

Derived from the layout. Switch legs come from the switch text in the layout. The first leg listed is NORMAL. A signal stands on each block next to a switch point on its fixed side. A beacon stands on each neighbour of a station block.

Runtime state. Each device keeps what the Track controller commanded and what it is actually doing: `_switch_cmd` and `_switches`, `_signal_cmd`, `_crossing_cmd`. Also kept: `_failures` by block, `_commanded_speed` and `_authority` by block, `_trains`, `_waiting` passengers by station, and `_track_temp_c` by section. Devices start safe: switches NORMAL, gates up, lights RED, heaters off. `TrackConfig` holds the layout paths, the ticket probability per station per tick (default 0.02) and a random seed.

5.4.9 Algorithms

One tick (`step`).

1. Check the whole input. If anything is wrong, raise and change nothing.
2. Store the Track controller's commands. A switch moves to its command unless its block has lost power.
3. Work out each device's actual state: what was commanded, unless a failure overrides it.

4. Move each section's track temperature toward ambient, or toward ambient plus 10 °C while the section's heaters run. The time constant is 300 s. Both numbers are placeholders.
5. Move trains across block boundaries from their reported offsets.
6. Sell tickets, board passengers and build every output.

Train position. The Train Model restarts its offset at 0 in each new block. The Track Model keeps what that reset throws away, in `carry_m`: the overshoot when a train moves forward, or the new block's length when it rolls back into it. The true distance into the block is the reported offset plus the carry. When it passes the block length, the train moves to `Layout.next_block`, which follows the switches as set. When it goes below 0, the train rolls back into the block it came from. A report that still names the block the train just left is stale and only corrects the carry. A train that reports no block is not on the track yet and is skipped. A train that runs into the yard leaves the track.

Occupancy. A block is occupied if a train is in it or its track circuit has failed.

Failures. All three failures cut the block's track circuit and make it read occupied:

Failure	Occupancy	Track circuit	Other effects
BROKEN_RAIL	occupied	speed 0, authority 0	—
TRACK_CIRCUIT	occupied	speed 0, authority 0	—
POWER	occupied	speed 0, authority 0	switch held where it is, light RED, crossing gates down, section heaters stop; commands sent meanwhile take effect when power returns

Signal seen. A train sees a light once, on the tick it enters a block with a light. The Train controller keeps it.

Stations. Each station rolls for one new ticket per tick. Passengers board only when a train is stopped (speed exactly 0) in a station block with a door open, up to the capacity the train reports.

5.4.10 User interface

The Track Model window is a Qt Quick window (`ui/Main.qml`, `MainView.qml`) fed by `TrackModelState`. It shows imperial units: ft, mph and °F. Its areas are:

- **Header:** Running or Paused, elapsed time, ambient temperature entry (°F), line selector, block filter and **Load track…**.

- **Block table:** block, direction of travel, length, grade, speed limit, commanded speed (**Cmd mph**), authority (**Auth**), station, beacon, heater, occupied, signal and failure.
- **Switch, signal and section tables:** switch legs and position, signal aspects, and heater state and track temperature per section.
- **Train and station tables:** each train's block, offset and speed, and the passengers waiting at each station.
- **Selected block:** all properties of the block in focus, and Murphy's failure selector.

5.4.11 Test interface

The test UI is a separate program (`test_ui/`). It stands in for the Track controller, the Train model and the clock, and reaches the module only through the test link: newline-delimited JSON over a local socket (a named pipe on Windows). Its requests are `step`, `set_block_failure`, `edit_block` and `reset`. It builds the module's inputs, ticks the clock at 1x or 10x with a fixed dt, and shows the outputs. With **Act as the Train Model** ticked it moves its trains: add a train, give it a speed, press Run. At integration the test UI and the link are deleted and the harness calls `step` instead.

5.4.12 Sequence diagrams

Each use case has one sequence diagram, numbered to match: SD-01 is the diagram of UC-1, and so on. SD-15 is an overview of one tick and SD-16 is the system-level dispatch scenario. Figure 5 (Section 3) is the movement authority request.

Diagram	Title	Use case
SD-01	Load a track layout	UC-1
SD-02	Display track properties	UC-2
SD-03	Track train position and occupancy	UC-3
SD-04	Report state to the Track Controller	UC-4
SD-05	Apply a switch position	UC-5
SD-06	Apply signal lights and crossing gates	UC-6
SD-07	Deliver the track circuit signal	UC-7
SD-08	Send beacon data	UC-8
SD-09	Send the signal light to the Train Controller	UC-9
SD-10	Sell tickets and board passengers	UC-10
SD-11	Report ticket sales	UC-11

Diagram	Title	Use case
SD-12	Set the ambient temperature	UC-12
SD-13	Switch the track heaters	UC-13
SD-14	Inject or clear a failure	UC-14
SD-15	One tick in the Track Model (overview)	UC-3 to UC-9, UC-13
SD-16	Dispatch a train (system level)	all

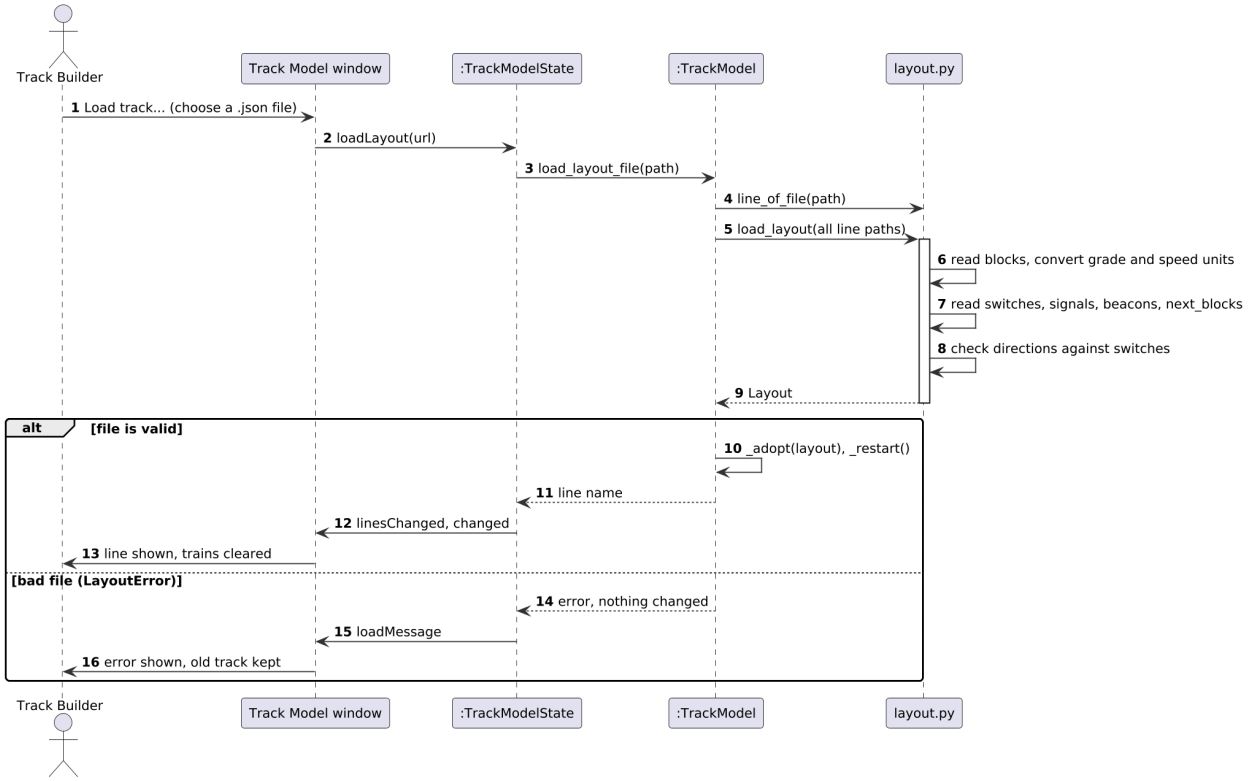

Figure 100. SD-01 Load a track layout (UC-1)

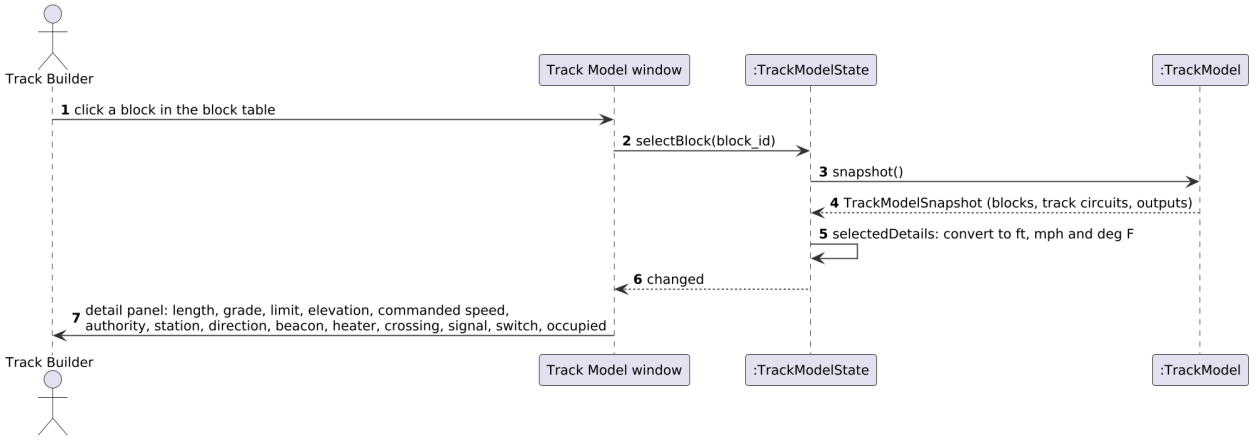

Figure 101. SD-02 Display track properties (UC-2)

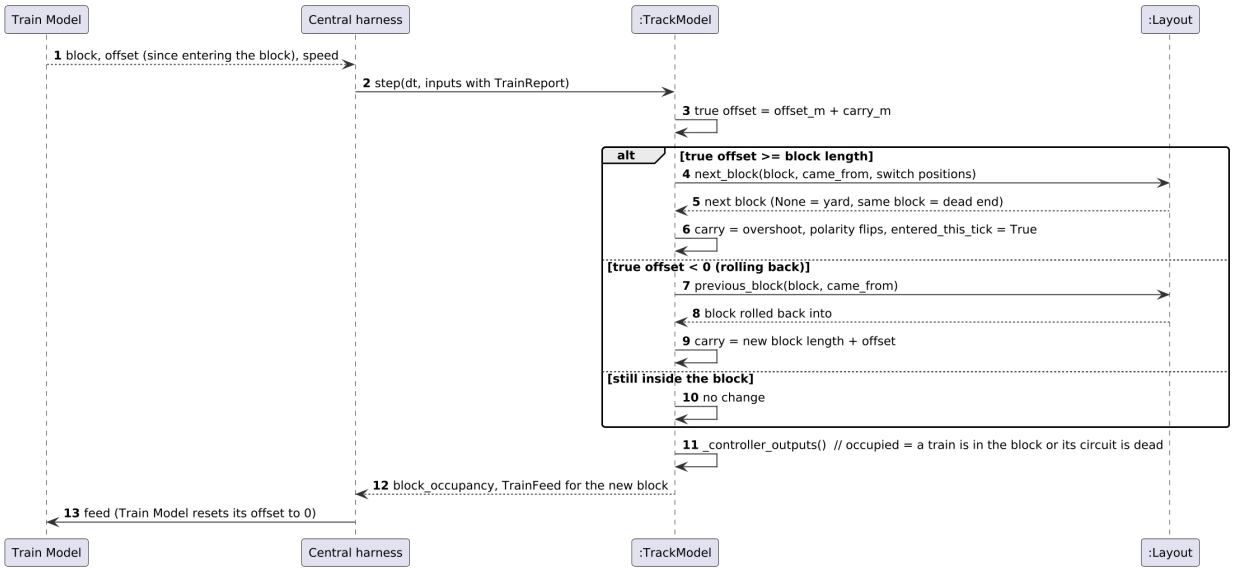

Figure 102. SD-03 Track train position and occupancy (UC-3)

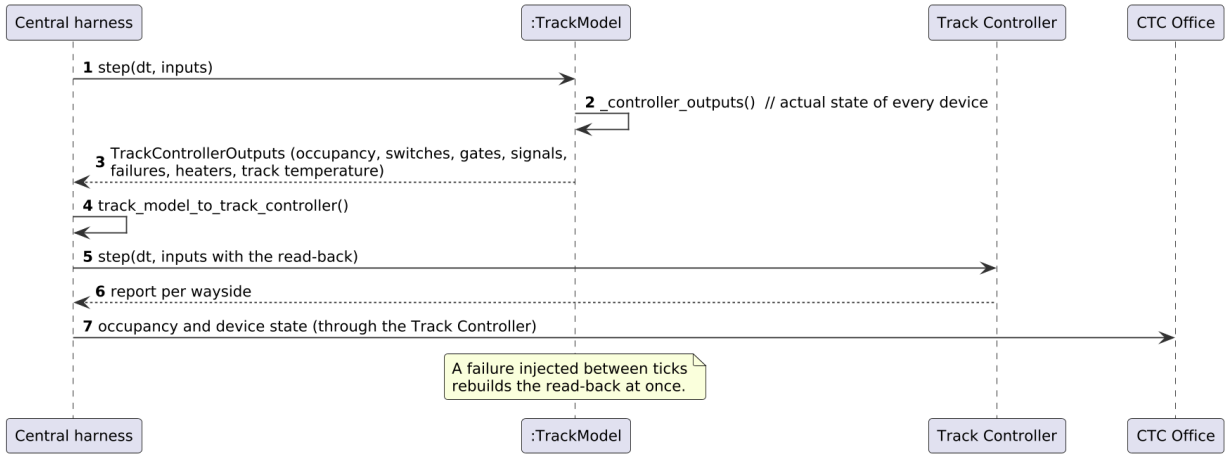

Figure 103. SD-04 Report state to the Track Controller (UC-4)

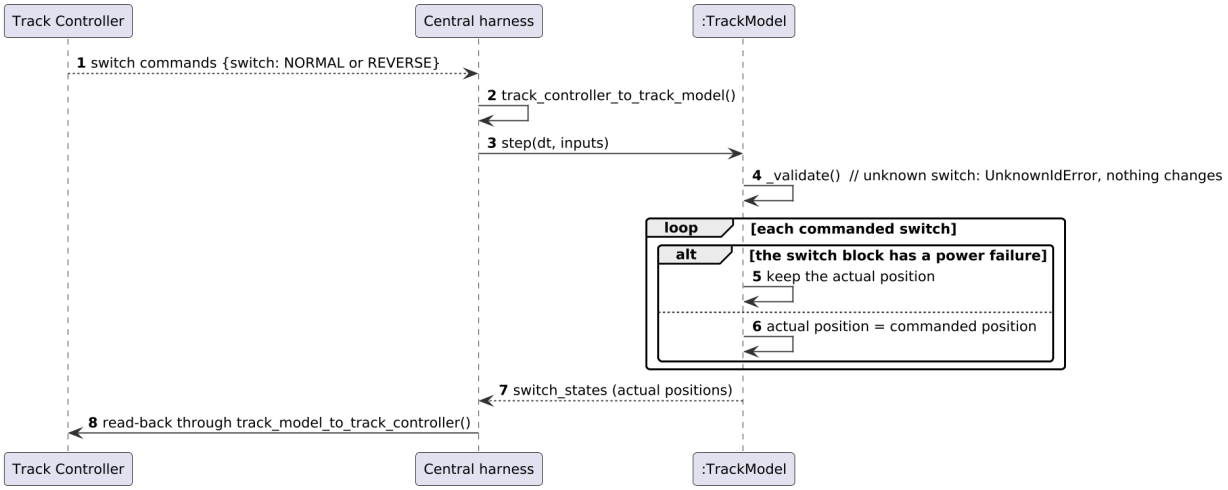

Figure 104. SD-05 Apply a switch position (UC-5)

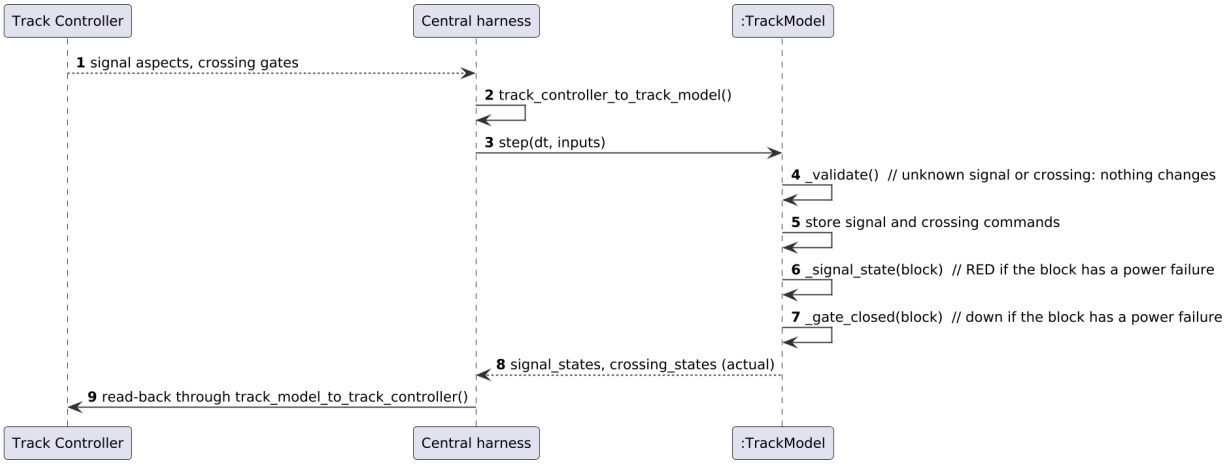

Figure 105. SD-06 Apply signal lights and crossing gates (UC-6)

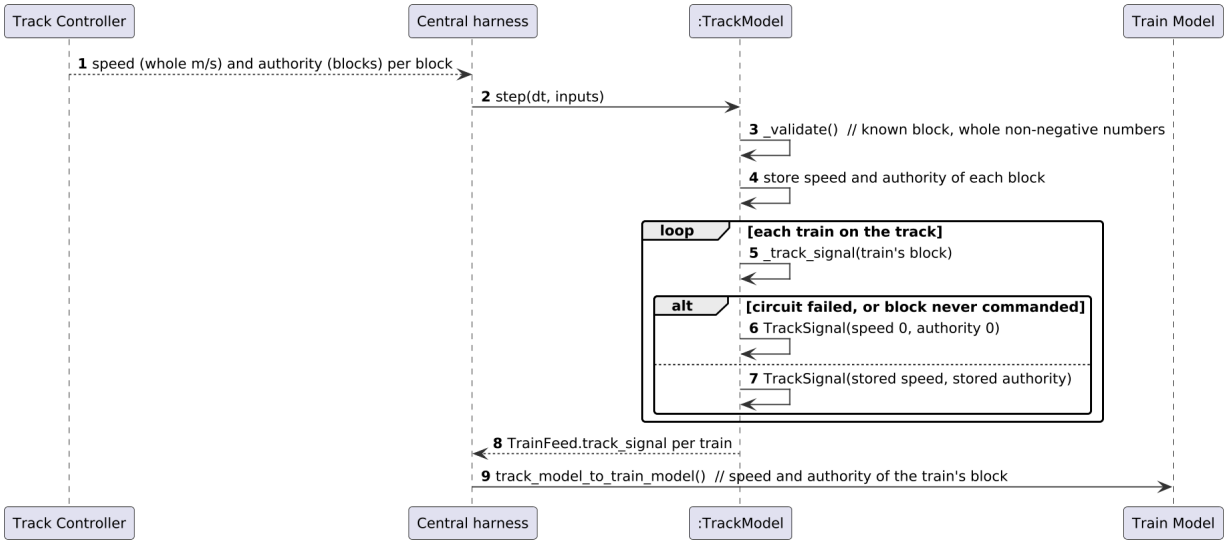

Figure 106. SD-07 Deliver the track circuit signal (UC-7)

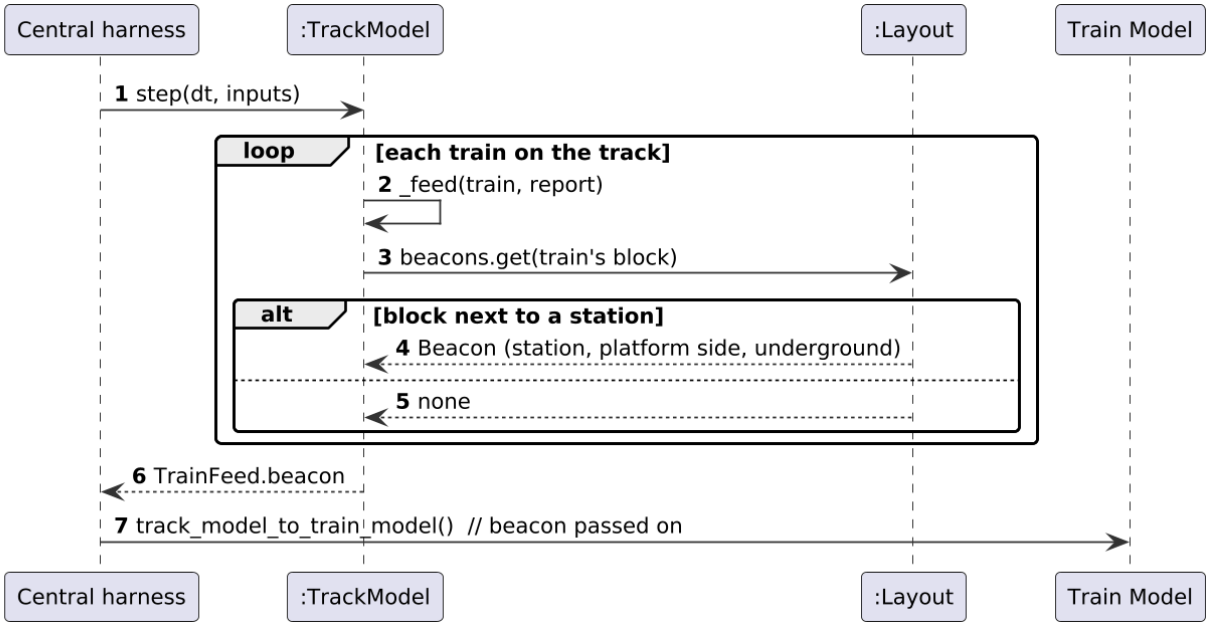

Figure 107. SD-08 Send beacon data (UC-8)

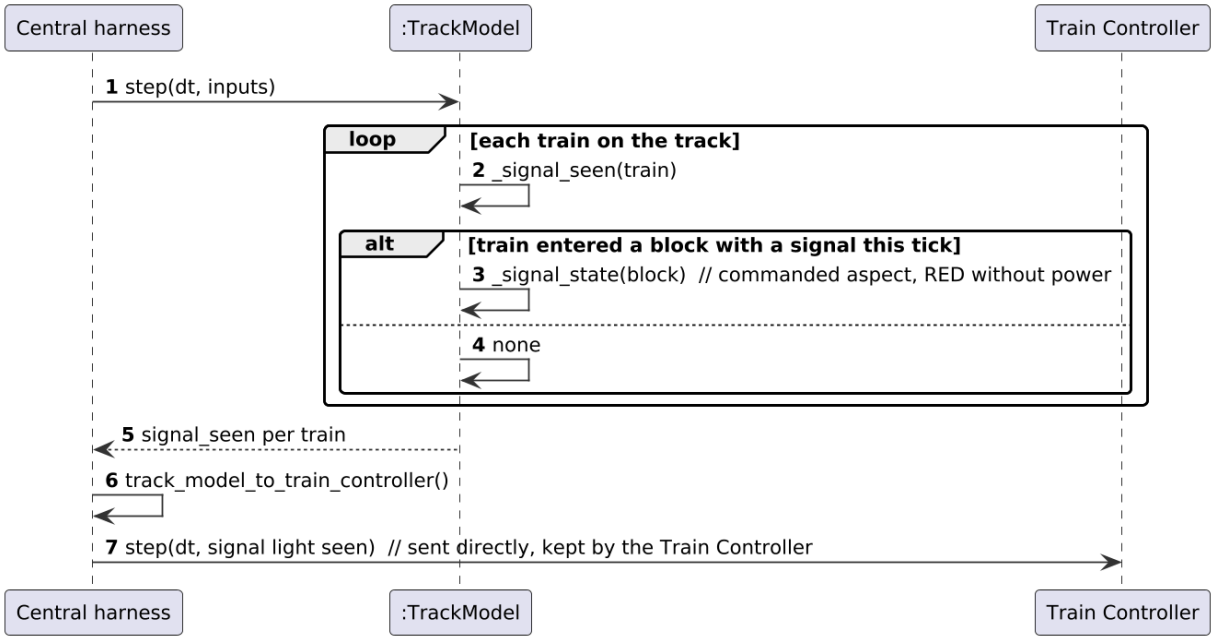

Figure 108. SD-09 Send the signal light to the Train Controller (UC-9)

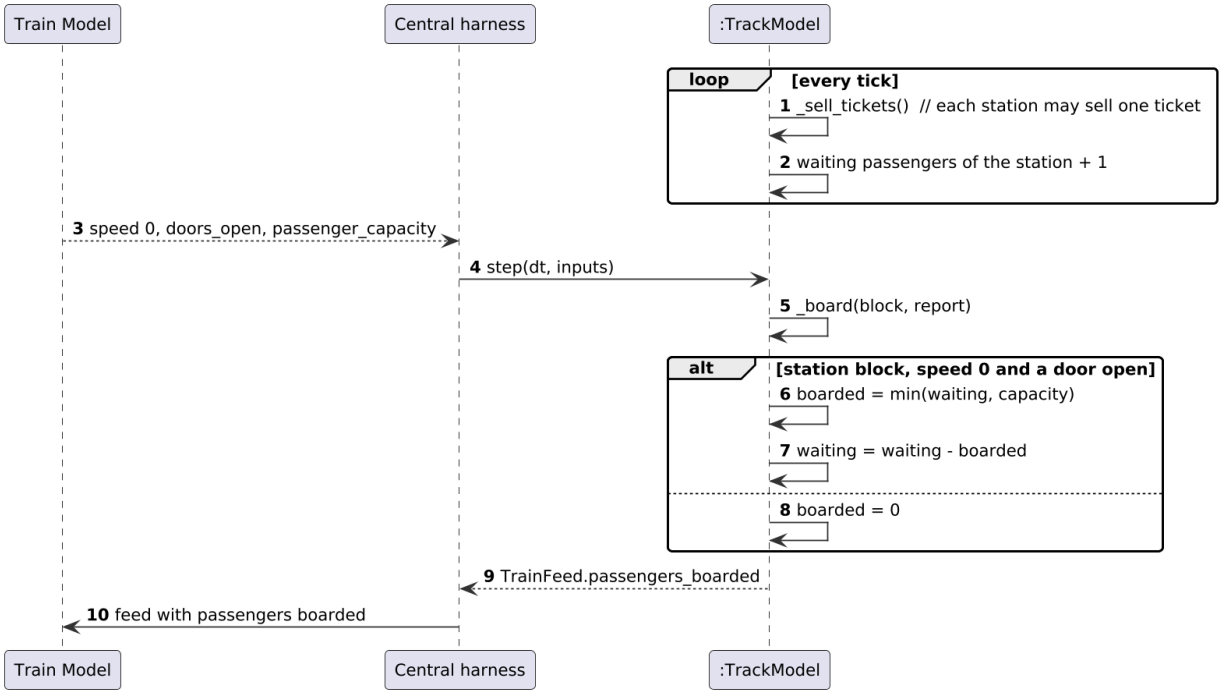

Figure 109. SD-10 Sell tickets and board passengers (UC-10)

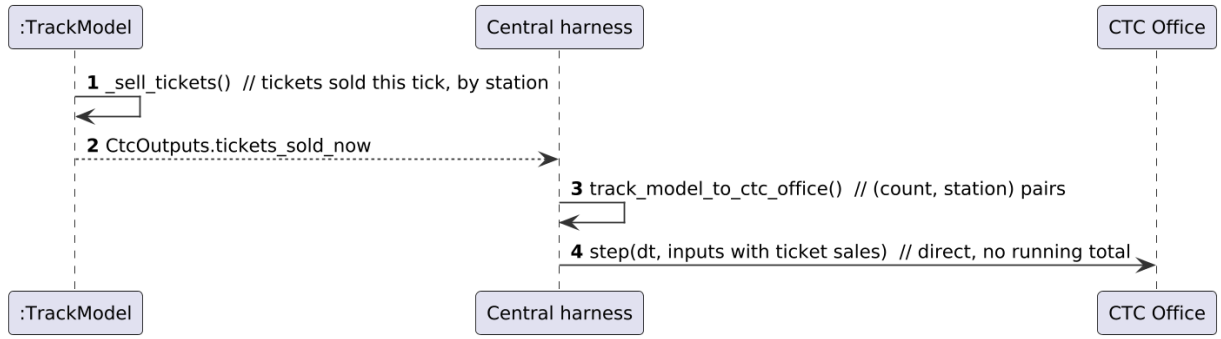

Figure 110. SD-11 Report ticket sales (UC-11)

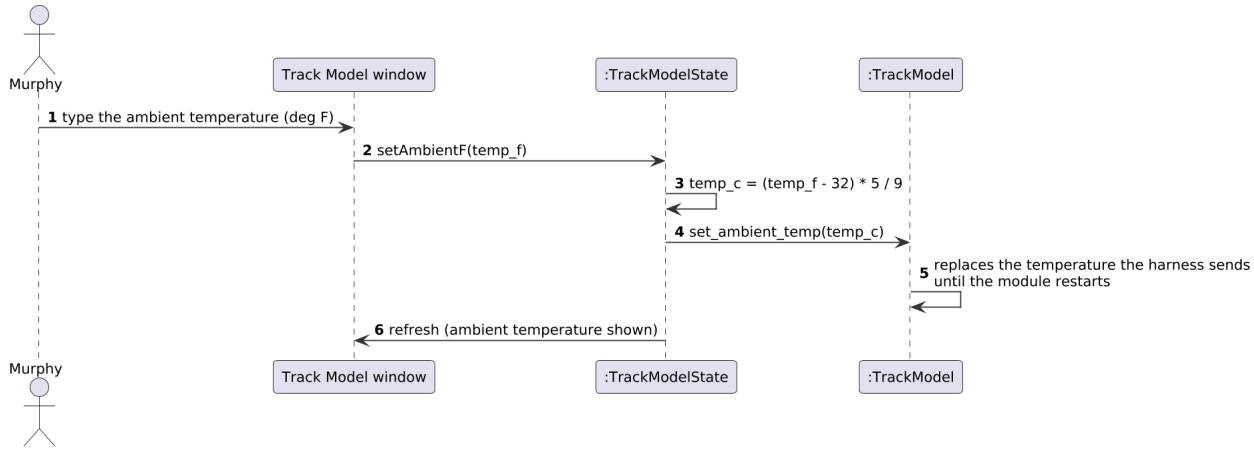

Figure 111. SD-12 Set the ambient temperature (UC-12)

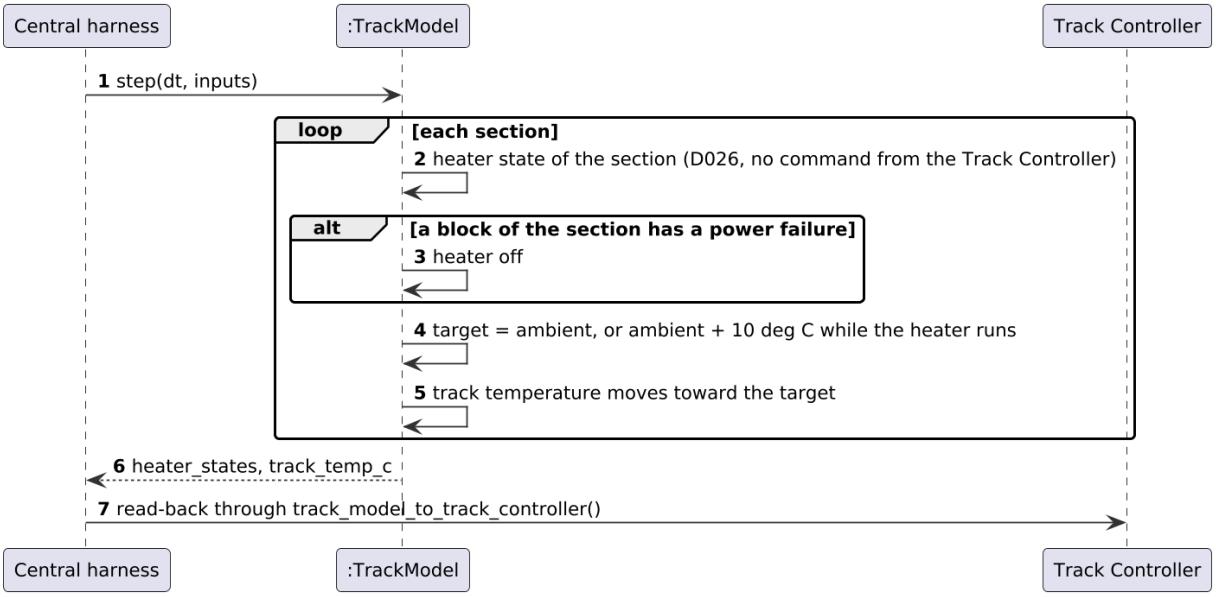

Figure 112. SD-13 Switch the track heaters (UC-13)

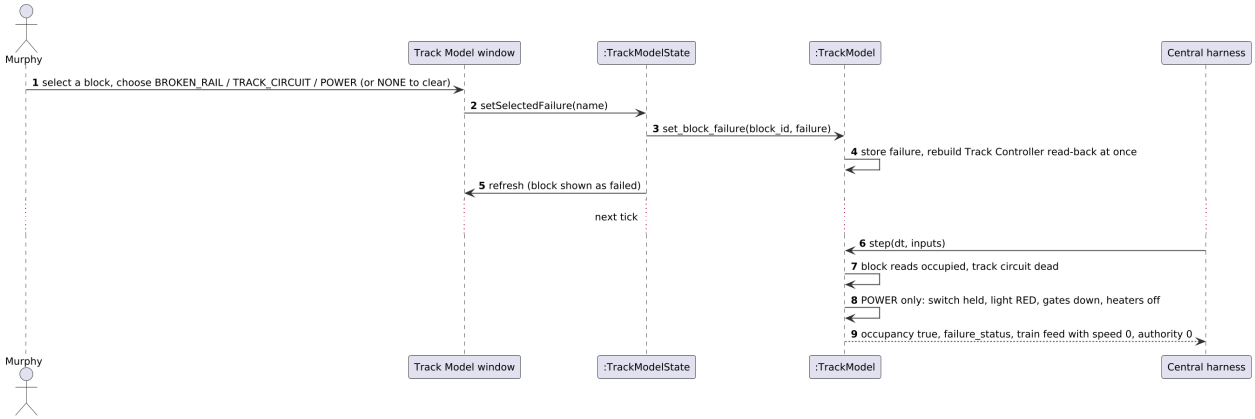

Figure 113. SD-14 Inject or clear a failure (UC-14)

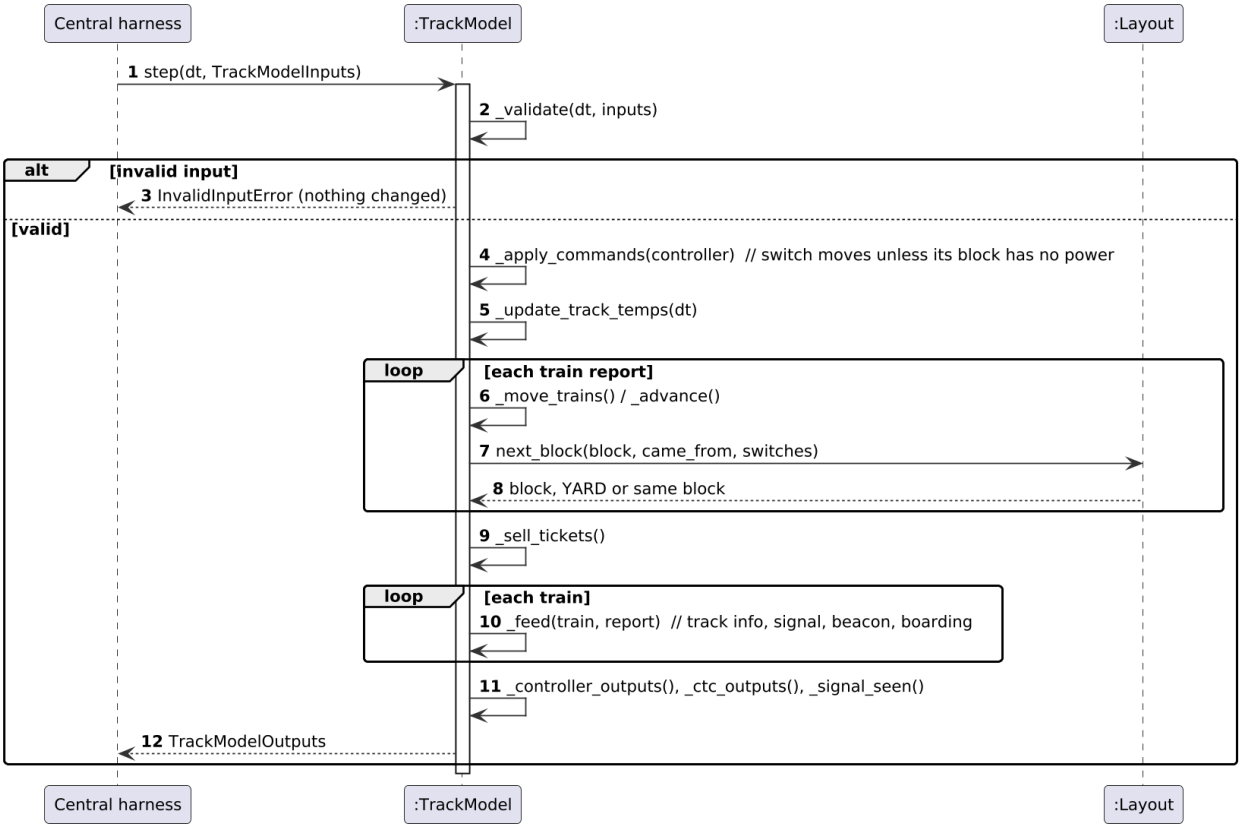

Figure 114. SD-15 One tick in the Track Model (overview)

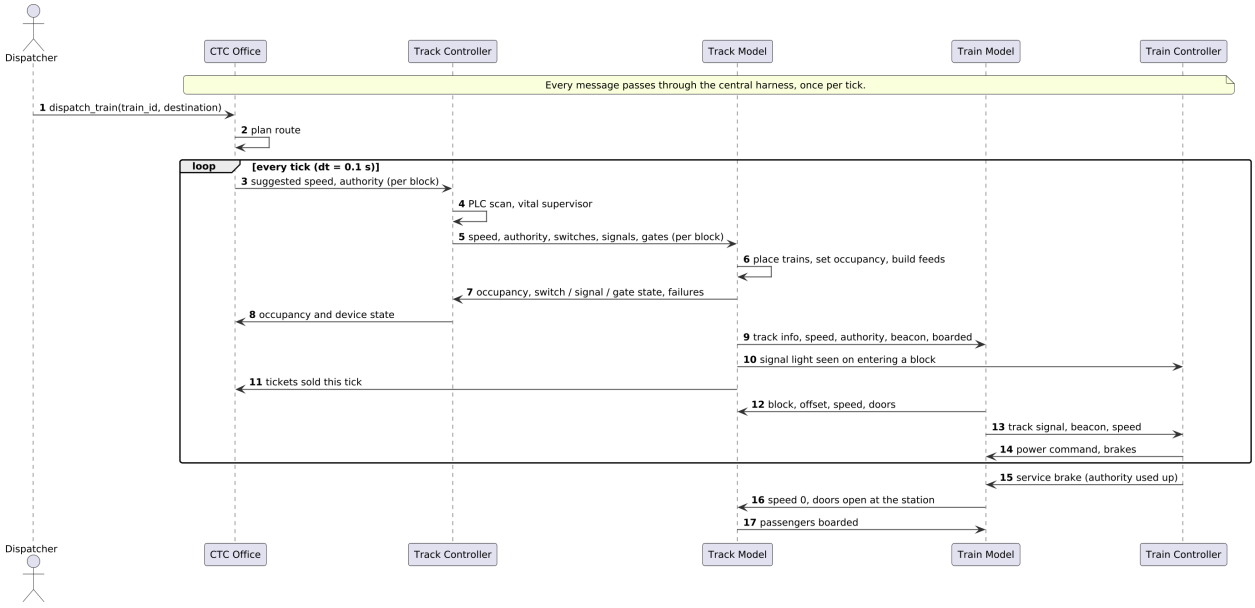

Figure 115. SD-16 Dispatch a train (system level)

5.4.13 State machine

Each block is always in one of three states. A failure overrides the occupancy states and reads as occupied.

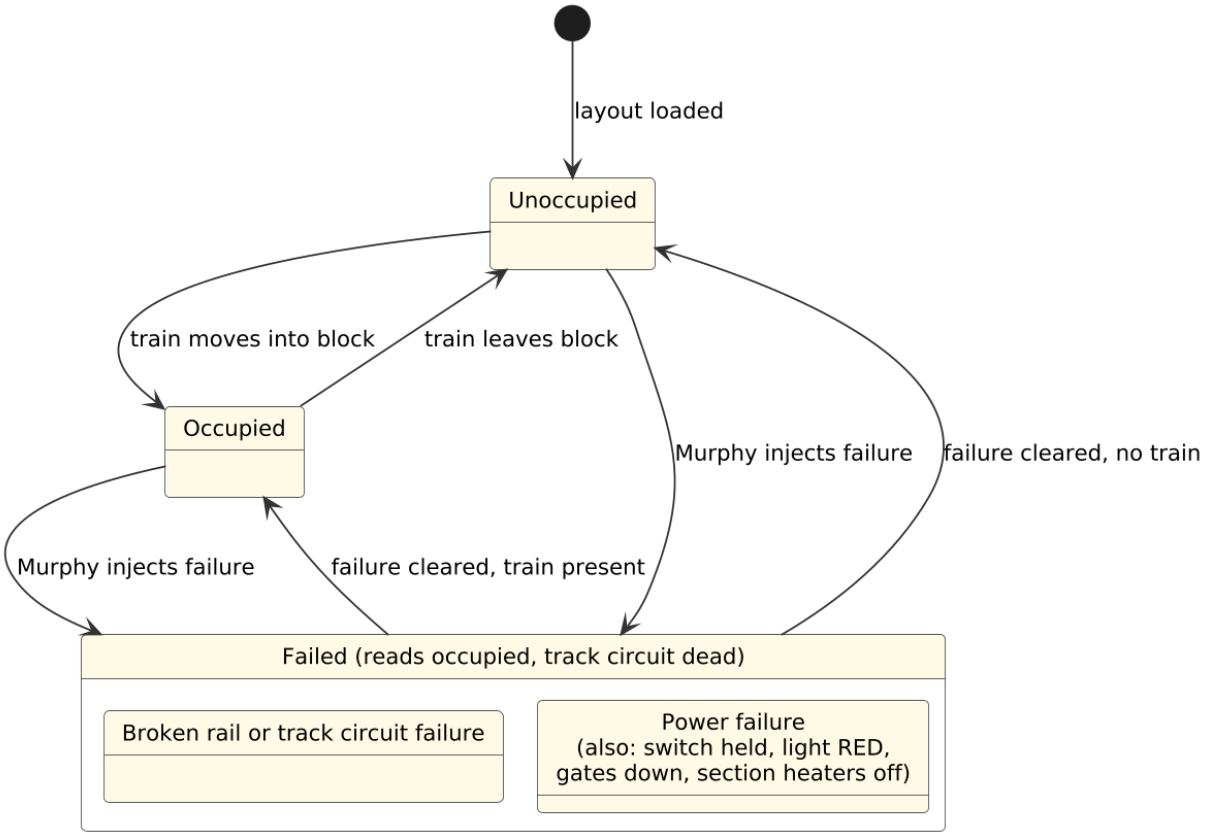

Figure 116. Block state machine

5.4.14 Design decisions

Interface Contract Mechanism

- **ID:** D005 — A central harness sits between the modules
- **Context:** Each module needs types for the data it exchanges with others. If each module imported the others' structures, every pair of modules would be coupled and one field change would break the neighbours.
- **Options:**
  - Each module defines only its own boundary types; the harness owns one pure mapping function per producer-to-consumer edge.
  - A shared catalog of types (`common/interfaces.py`) that every module imports. It is now stale and not treated as true (D008).
  - Modules call each other directly.

- **Outcome:** The first option. The harness is one process that starts a process for each module and is every module's single driver. They exchange newline-delimited JSON messages, each carrying a request ID and the tick number. The Track Model's contract is the `TrackModel` Protocol in `track_model/interface.py`.

Shared Boundary Data Types

- **ID:** D002, D013 — Backend units are SI; authority is a block count
- **Context:** Values that cross a boundary must mean the same thing to every module, including their units.
- **Options:**
  - SI units for everything stored or sent, with conversion to imperial units only in the display layer.
  - Mixed units chosen by each module.
- **Outcome:** The first option. Backend: m, m/s, degrees, degrees Celsius, whole blocks. Display: ft, mph, degrees Fahrenheit, converted in `track_model/state.py` only. Authority is a non-negative integer count of blocks ahead of the train's current block (D013). Enumerated values are all caps.

Contract Enforcement in CI

- **ID:** none on record — practice of the Track Model (`TrackModel/README.md`)
- **Context:** A contract that is not checked drifts, and a broken interface must be found when it changes.
- **Options:**
  - A static type check plus tests that the module matches its Protocol and that the edge functions carry values unchanged.
  - Runtime checks only.
  - Manual review only.
- **Outcome:** The first option: `mypy --strict`, `flake8 --max-line-length=79` and `pytest`, including `tests/test_contract.py` (the module matches its Protocol), `tests/test_harness_edges.py` (the edge functions carry values unchanged) and `tests/test_end_to_end.py`.

Stub-First Development

- **ID:** none on record — practice of the Track Model (`TrackModel/README.md`)
- **Context:** The Track Model needs a Track controller, a Train model and a clock to run against before the other modules are finished.
- **Options:**

    - A separate test UI process that stands in for the Track controller, the Train model and the clock.
    - Mocks inside each test only.
    - Wait for the real modules.
- **Outcome:** The first option. The test UI reaches the module only through a local socket link (JSON lines), so it can be removed at integration with no change to `track_model/`. Its option "Act as the Train Model" moves its fake trains.

Contracts Package Ownership

- **ID:** D005 — Each module owns its boundary types
- **Context:** Boundary types are the one thing other modules depend on, so an uncoordinated change breaks them.
- **Options:**
    - Each module owns its own boundary types; only the harness edge functions know two modules' types.
    - One shared types package that every module imports.
- **Outcome:** The first option. A change to the Track Model's boundary types changes only the Track Model and its edge functions in `common/harness/`.

External Input Validation

- **ID:** none on record — practice of the Track Model; REQ-NFR-003, REQ-NFR-004
- **Context:** Inputs come from other modules, from the window (Murphy, Track Builder) and from layout files. A wrong value must not corrupt the track state.
- **Options:**
    - Check the whole tick first and reject it as a whole, so that nothing changes.
    - Check each value as it is used and keep the valid part.
    - Trust the caller.
- **Outcome:** The first option. `step` raises `InvalidInputError` or `UnknownIdError` for a non-finite number, a speed or authority that is not a whole non-negative number, or an ID that is not in the layout. A bad layout file raises `LayoutError` and the loaded track is kept.

Knowledge Capture and Decision Logging

- **ID:** AGENTS.md — The `truth` branch is the record of decisions
- **Context:** Reasons for design choices get lost between work packages.
- **Options:**
    - A `truth` branch holding interfaces, signals, conventions and decisions as one file per entry, changed by proposals that a human promotes.

    - Decisions only in Jira comments.

- **Outcome:** The first option. Facts live on the `truth` branch; status is read from the repository.
- **More Information:**

  Decisions on record that shape the Track Model:

ID	Decision	Effect on the Track Model
D002	Two unit systems, split at the display layer	Backend values are SI; the window shows ft, mph and °F.
D005	A central harness sits between the modules	One `step(dt, inputs)` call per tick; edge functions map the types.
D006	Fixed time step	dt is constant; 10x changes the tick rate, never dt. The Track Model does not sub-step.
D013	Authority is a count of blocks ahead of the train	Authority is an integer in every signal the Track Model handles.
D017	One shared simulation clock, run by the harness at 1x or 10x	The Track Model keeps no clock; it is stepped by the harness.
D023	The Blue Line is out of scope	Only the Green and Red lines are loaded at integration.
D026	The Track Model switches the track heaters	No heater command comes from the Track controller.

5.4.15 Requirements traceability

SRS requirements that the Track Model covers, and where each is designed.

Requirement	Summary	Use cases	Design elements	Figures
REQ-FUNC-015, REQ-INTF-012	Load the track layout from JSON at startup	UC-1	`layout.py` (`load_layout`), `Layout`	SD-01

Requirement	Summary	Use cases	Design elements	Figures
REQ-FUNC-016	Display each segment's grade, elevation, length, speed limit, direction, crossings, heaters and beacons	UC-2	`Block`, `TrackModelState`	SD-02, Figure 15
REQ-FUNC-017	Track ticket sales and send them to the CTC office	UC-10, UC-11	`_sell_tickets`, `CtcOutputs`, `track_model_to_ctc_office`	SD-10, SD-11
REQ-FUNC-018	Show passengers boarding. Disembarking is reported by the Train Model and is not an input to the Track Model.	UC-10	`_board`, `TrainFeed.passengers_boarded`	SD-10
REQ-FUNC-019	Show train occupancy on the display	UC-3	`_move_trains`, `TrackModelState`	SD-03
REQ-FUNC-020, REQ-INTF-018	Send and receive track circuit signals; occupancy channel with the Track controller	UC-4, UC-7	`TrackSignal`, `TrackControllerOutputs`	SD-04, SD-07, Figure 5
REQ-FUNC-021	Set the environmental temperature	UC-12	`set_ambient_temp`	SD-12
REQ-FUNC-022	Simulate track heaters	UC-13	`_heater_on`, `_update_track_temps`	SD-13
REQ-FUNC-016.8	Beacons: display them and send them with the train feed	UC-8	`Layout.beacons`, `Beacon`	SD-08
REQ-FUNC-023	Display switch positions and light states	UC-5, UC-6	`TrackModelState` switch and signal rows	SD-05, SD-06
REQ-FUNC-024	Simulate broken rail, track circuit and power failures	UC-14	`TrackFailure`, `set_block_failure`	SD-14, Figure 32
REQ-INTF-001.2	A GUI for the Track Model	UC-2	`TrackModelState`, `ui/*.qml`	Section 5.4.10

Requirement	Summary	Use cases	Design elements	Figures
REQ-INTF-013, REQ-FUNC-001 to 003	One shared clock; real time, 10x and pause	-	`SystemClock`, harness	Section 3.7.4
REQ-NFR-003, REQ-NFR-004	Validate input; reject bad input without changing state	-	`_validate`, `InvalidInputError`	Section 4.6
REQ-DSN-004	Modules interact through defined interfaces	-	`TrackModel` Protocol, edge functions	Figure 4

— End of document —

5.5 Train Model

Owner: Kevin Schillinger. **Source:** Railway_Sim_SDD_Train_Model.md.

5.5.1 Scope

The Railway Simulation has five functional modules: CTC Office, Track Controller, Track Model, Train Model, and Train Controller. The Track Controller and Train Controller have hardware and software variants. The Train Model is software, owned by Kevin Schillinger. Moving Block Overlay and the operational Blue Line are outside system scope under D001 and D023. The Blue Line remains a standalone Train Model test fixture under D010.

The Train Model contribution covers:

- Longitudinal motion under power, grade, rolling resistance, and braking.
- A fixed five-module Bombardier FLEXITY 2 consist, with one physical state per train.
- Passenger boarding, disembarking, capacity, and their effect on operating mass.
- Door and light states, cabin temperature, and announcement presentation.
- Passenger emergency brake activation and Train Controller release.
- Engine, signal pickup, and service brake failure injection.
- The module window, fleet selection, boundary types, process adapter, and removable test UI.

Routing, authority allocation, dispatch, occupancy calculation, speed control, station stopping, and control interlocks are responsibilities of the other modules. The Train Model applies commands and reports physical consequences. Its configured maximum speed is vehicle data; it is not a speed governor (D009).

5.5.2 Stakeholder concerns

Stakeholder	Concern	Addressed By
Passenger	Emergency brake response, doors, announcements, and cabin conditions.	UC-TM-03 through UC-TM-06; failure behavior.
Murphy	Independent, composable fault injection and visible effects on the selected train.	UC-TM-07; Train Model window.
Observer / reviewer	Understandable motion, dimensions, mass, occupancy of the cabin, and equipment readouts.	UC-TM-02; UI snapshot and display units.

Stakeholder	Concern	Addressed By
Train Controller developer	Typed commands, predictable physical response, brake feedback, and speed/signal data.	Interface dictionaries and tick ordering.
Track Model developer	Grade and circuit inputs; position, block changes, and available capacity outputs.	Track dictionaries and UC-TM-04/05.
Integration developer	Fixed timing, train identity, process lifecycle, and isolated train rejection.	Composition, interaction, and deployment views.
Module developer	Repeatable inputs, removable stand-ins, and rejection without partial physical updates. This is a development concern, not a product actor.	Developer Test Utility; validation; Section 5.5.12.

5.5.3 Train Model in the system

The CTC Office dispatches trains and suggests routes, speed, and authority to the Track Controller. Trackside logic supplies the resulting commanded speed and authority through the Track Model's circuit data. The Train Model receives those values and passes them to the Train Controller. The Train Controller computes power and equipment commands; the Train Model returns actual motion and equipment state. All integrated exchanges pass through harness mappings (D005).

Architecture topic from rubric	Train Model contribution
Route	Receives current block and terrain; integrates signed offset. Route selection, switch traversal, and occupancy belong outside the Train Model. A polarity transition identifies a block change.
Authority	Passes `authority_blocks` to the Train Controller. It does not allocate authority or brake merely because authority is zero. CTC suggested speed and downstream commanded speed are distinct.
Wayside distribution	Has no wayside database or territory logic. The Track Model supplies the terrain and circuit input for each train, independent of the upstream hardware/software implementation.

Architecture topic from rubric	Train Model contribution
Timing	The harness supplies fixed dt and simulation time on each rising edge. Default dt is 0.1 s; 1x/10x changes tick frequency, and pause stops ticks (D006, D017).
Vitality	Train Controller and Track Controller contain the vital control responsibilities. The Train Model simulates the plant and failures; this document makes no safety certification claim for it. Communications channels are non-vital under REQ-INTF-021.

The integrated clock starts at 05:00:00 and sends unwrapped simulation seconds. At 1x, the default is 10 ticks per real second; at 10x, it is 100. The harness waits for module replies before issuing the next tick. Consumers receive the previous tick's producer outputs, so module execution order does not change the input snapshot for a tick. The Train Model does not own a timer that advances physics.

The window displays the harness-supplied time of day during integrated operation. The module also keeps an independent estimate derived from each train's accepted elapsed steps. Every 15 harness ticks it compares that estimate with the supplied unwrapped simulation time. A train's initial epoch accounts for its creation time, so a late-created train does not appear to have drifted simply because its elapsed time starts at zero. Display reconciliation never overwrites elapsed physics time or silently clears a drift discrepancy. The backend/UI work is deferred.

5.5.4 Use cases

Actor	Role
Passenger	Pulls the emergency brake and receives announcements; participates in simulated boarding/alighting.
Murphy	Injects and clears selected-train failures through the module window.
Observer	Views state and selects a train.
Train Controller	Supplies onboard commands, receives feedback, and releases a passenger pull.
Track Model	Supplies terrain, circuit, beacon and boarding data; consumes position and capacity.
Harness / simulation clock	Supplies fixed-duration ticks, routes boundary data, and manages the integrated roster.

The train driver acts through the Train Controller, rather than through direct speed or brake-command controls on the Train Model window. The developer test UI is a utility outside this product actor model; it is not a use case actor, and its operations are not product use cases.

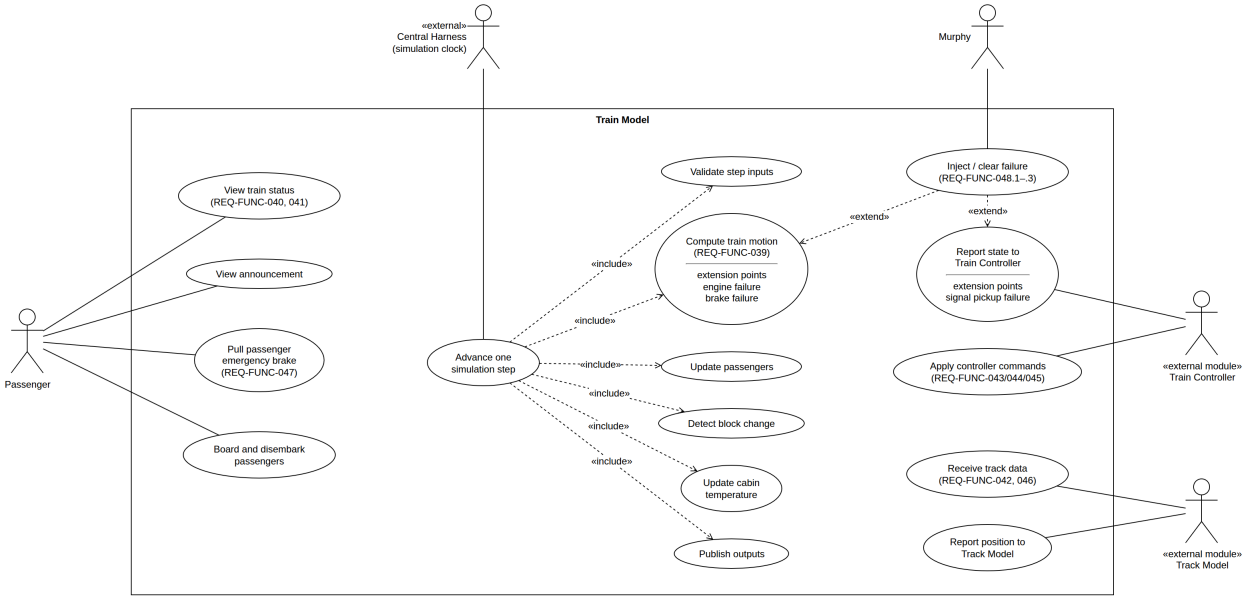

Figure 117. Train Model use case diagram

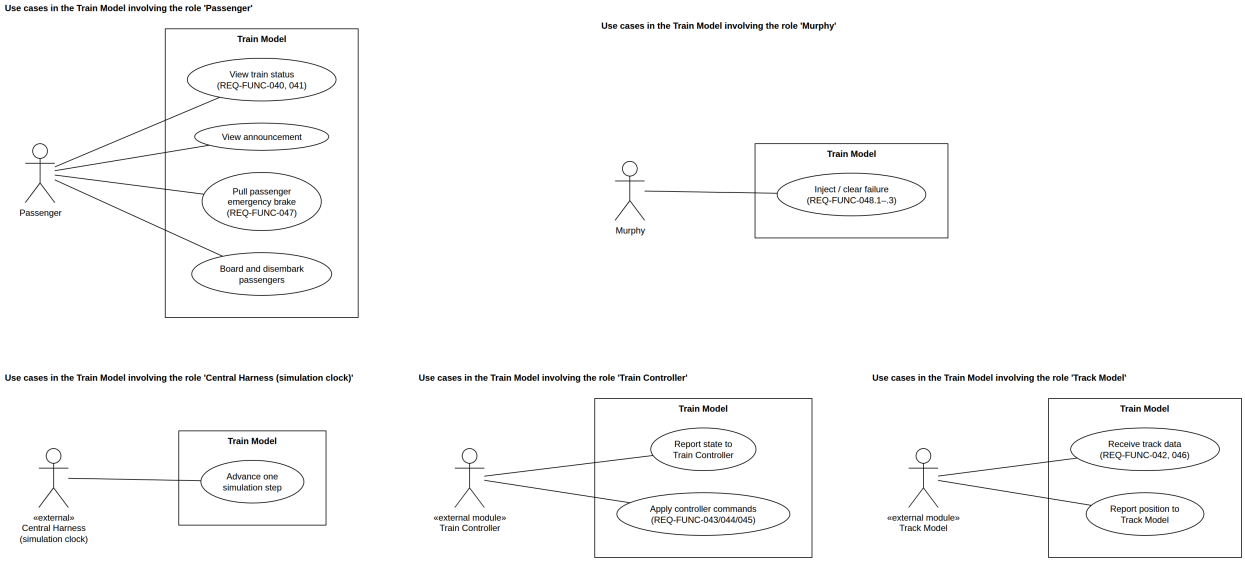

Figure 118. Train Model: use cases by role

Sea-level use cases (one per actor goal)

Train Model: View train status (REQ-FUNC-040, 041)	
Actors	Passenger
Description	The Train Model window shows the train's current state after every step: mass, acceleration, velocity, crew and passenger counts, door states, whether the passenger emergency brake is pulled, active failures, elapsed time and whether the simulation is running.
Data	TrainModelSnapshot; TrainModelState properties snapshot, doors, failures, activeFailureCount and running.
Stimulus	A simulation step completes (snapshotChanged).
Response	The window shows the new values.
Comments	Read only. Backend values are SI, with grade in degrees and temperature in °C.

Train Model: View announcement	
Actors	Passenger
Description	The passenger sees the announcement the Train Controller sent with its commands.
Data	ControllerCommands.announcement
Stimulus	A step carries an announcement from the Train Controller.
Response	The announcement is shown to passengers.
Comments	The Train Controller decides what to announce; the Train Model only shows it.

Train Model: Pull passenger emergency brake (REQ-FUNC-047)	
Actors	Passenger
Description	A passenger pulls the emergency brake from the Train Model window. The pull latches, so the emergency brake stays applied on every following step.
Data	passenger_ebrake_pulled; emergency_brake_active (ControllerOutputs); emergency deceleration 2.73 m/s².
Stimulus	The passenger pulls the brake (applyEmergencyBrake, pull_passenger_emergency_brake).
Response	The latch is set and the train brakes at the emergency rate.
Comments	How the latch is released is undecided (D011); releaseEmergencyBrake() does nothing.

Train Model: Board and disembark passengers	
Actors	Passenger
Description	While the train is stopped with a door open, passengers get off first, then the boarding passengers get on.
Data	passengers_boarded (from the Track Model, see Receive track data); n_passengers; capacity 222; passenger mass 77.1107 kg; passenger_capacity (TrackOutputs).
Stimulus	A step with the train stopped and a door open.
Response	The passenger count and the train's mass change; passenger_capacity is reported to the Track Model.
Comments	Included stage: Update passengers. The number getting off is drawn at random, seeded by TrainConfig.seed.

Train Model: Advance one simulation step	
Actors	Central Harness (simulation clock)
Description	On each clock tick the central harness calls step(dt, inputs). The Train Model validates the inputs, moves the doors, updates passengers, detects a block change, computes motion, updates the cabin temperature and elapsed time, and returns its outputs.
Data	dt; TrainModelInputs (ControllerCommands, TrackInputs); TrainModelOutputs (ControllerOutputs, TrackOutputs).
Stimulus	A clock tick: every 0.1 s at 1x, ten times as often at 10x.
Response	TrainModelOutputs is returned and the window refreshes.
Comments	dt is fixed; speed changes how often ticks come, never dt. Invalid inputs raise InvalidInputError or InvalidTimeStepError.

Train Model: Inject / clear failure (REQ-FUNC-048.1–.3)	
Actors	Murphy
Description	Murphy turns the engine, signal pickup and brake failures on or off from the Train Model window.
Data	FailureState (engine, signal_pickup, brake); TrainModelState failures and activeFailureCount.
Stimulus	Murphy toggles a failure (setFailure(name, active)).
Response	From the next step: engine zeroes traction; brake blocks the service brake (the emergency brake still works); signal pickup zeroes the commanded speed and authority passed to the Train Controller.
Comments	Failures are not reported to the Train Controller. Extends Compute train motion and Report state to Train Controller.

Train Model: Report state to Train Controller	
Actors	Train Controller
Description	After each step the Train Model passes the train's state to the Train Controller.
Data	ControllerOutputs: actual speed, emergency and service brake states, doors, lights, cabin temperature, commanded speed, authority, speed limit, beacon.
Stimulus	A step completes.
Response	The Train Controller receives ControllerOutputs.
Comments	Extension point: signal pickup failure zeroes commanded speed and authority.

Train Model: Apply controller commands (REQ-FUNC-043/044/045)	
Actors	Train Controller
Description	The Train Model applies the Train Controller's commands on the next step.
Data	ControllerCommands: power, service brake, emergency brake, interior and exterior lights, left and right doors, temperature setpoint, announcement.
Stimulus	A step arrives with new commands.
Response	Traction, brakes, doors, lights, cabin setpoint and announcement follow the commands.
Comments	The Train Model does not police commands: no speed governing, no 70 km/h cap, no traction cut under braking, no door interlock (D009).

Train Model: Receive track data (REQ-FUNC-042, 046)	
Actors	Track Model
Description	Each step the Track Model supplies the block the train is on, the track signal, any beacon, and the passengers boarding.
Data	TrackInfo (block, grade, elevation, speed limit, polarity, station); TrackSignal (commanded speed, authority); Beacon [0..1]; passengers_boarded.
Stimulus	A step arrives with track inputs.
Response	Grade affects motion, polarity drives block detection, and the signal and beacon pass through to the Train Controller.
Comments	A beacon is present only at some blocks.

Train Model: Report position to Track Model	
Actors	Track Model
Description	After each step the Train Model tells the Track Model where the train is.
Data	TrackOutputs: block_id, offset_m, actual_speed_mps, block_changed, passenger_capacity.
Stimulus	A step completes.
Response	The Track Model receives TrackOutputs.
Comments	block_changed is set on the step where a polarity flip is detected.

Included use cases (fish level: the stages of every step)

Train Model: Validate step inputs	
Actors	None (included by Advance one simulation step)
Description	Checks dt and every input field before the step does anything else.
Data	dt; TrainModelInputs; MAX_DT_S = 60; MAX_COUNT = 2^31 − 1; MAX_BEACON_CHARS = 128.
Stimulus	The start of every step.
Response	Passes, or raises InvalidTimeStepError (bad dt) or InvalidInputError (bad input).
Comments	validate_inputs() is static, so a caller can check inputs without a model. InvalidInputError also derives from ValueError.

Train Model: Compute train motion (REQ-FUNC-039)	
Actors	None (included by Advance one simulation step)
Description	Integrates the train's motion over dt with substepped trapezoidal integration, from traction, brake force, grade and rolling resistance at the current mass.
Data	power_cmd_w, service_brake, emergency_brake, grade_deg, mass; TrainConfig: max power 480 kW, service 1.2 m/s², emergency 2.73 m/s², c_rr 0.002, g 9.81 m/s².
Stimulus	Every valid step.
Response	New velocity, acceleration and offset.
Comments	Extension points: engine failure (traction zeroed), brake failure (service brake blocked). No speed cap (D009).

Train Model: Update passengers	
Actors	None (included by Advance one simulation step)
Description	When the train is stopped with a door open, draws the passengers getting off, then adds the passengers boarding.
Data	door_left_open, door_right_open, passengers_boarded, n_passengers.
Stimulus	Every valid step.
Response	Updated passenger count.
Comments	The draw uses the random generator seeded by TrainConfig.seed.

Train Model: Detect block change	
Actors	None (included by Advance one simulation step)
Description	Compares the track polarity with the last step. A flip means the train entered a new block, which resets the offset within the block.
Data	polarity (TrackInfo); last polarity; offset_m; block_changed (TrackOutputs).
Stimulus	Every valid step.
Response	block_changed is true and the offset is reset when the polarity flips.
Comments	The first step has no previous polarity to compare with.

Train Model: Update cabin temperature	
Actors	None (included by Advance one simulation step)
Description	Moves the cabin temperature toward the commanded setpoint.
Data	temp_setpoint_c; CABIN_TEMP_RATE_PER_S = 1/300; INITIAL_CABIN_TEMP_C = 20; cabin_temp_c (ControllerOutputs).
Stimulus	Every valid step.
Response	Updated cabin temperature.
Comments	Temperatures are in °C.

Train Model: Publish outputs	
Actors	None (included by Advance one simulation step)
Description	Builds TrainModelOutputs for the Train Controller and the Track Model at the end of the step.
Data	ControllerOutputs; TrackOutputs.
Stimulus	The end of every valid step.
Response	step() returns the outputs; outputs() returns them until the next step.
Comments	Feeds Report state to Train Controller and Report position to Track Model.

Figure 119. Train Model: use case descriptions

UC-TM-01: Advance physical motion

Primary actor: Harness / simulation clock. **Supporting actors:** Train Controller and Track Model. **Requirements:** REQ-FUNC-039, REQ-INTF-013.

Preconditions: A train exists and has valid controller and track inputs. **Trigger:** A fixed-duration tick arrives.

Main success scenario: Validate inputs; determine operating mass and active forces; integrate signed motion; update temperature and elapsed time; return controller and track outputs. The window refreshes the train's readouts.

Alternate flows: Invalid input rejects that train's step without changing its physical state. Engine failure removes traction. A grade may hold the train or cause rollback. Pause produces no new physical step.

Postcondition: Accepted outputs describe the end of the tick.

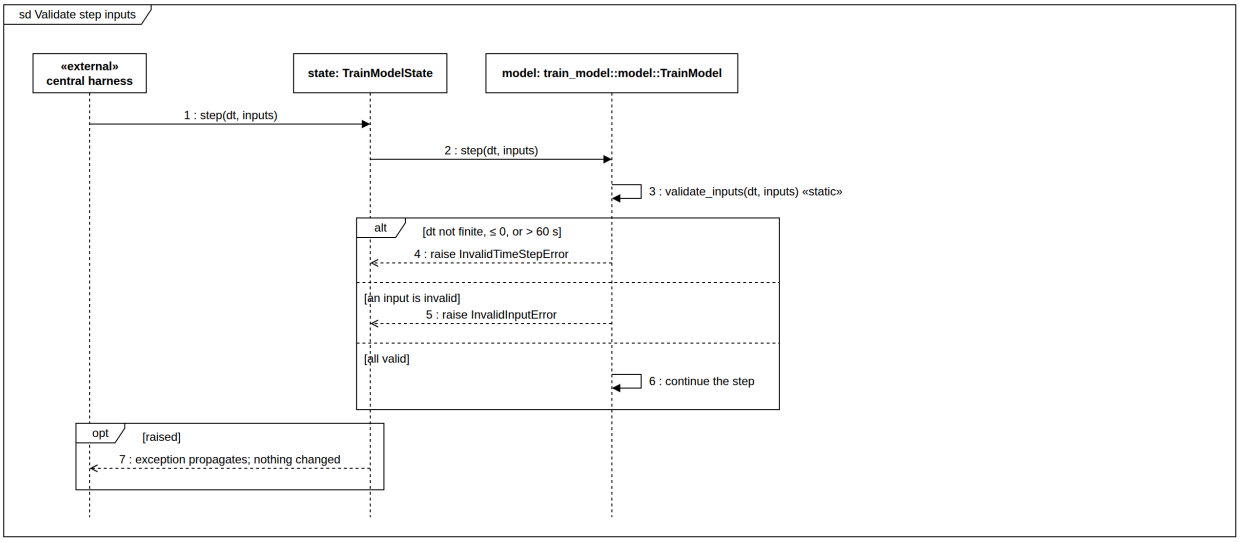

Figure 120. Sequence diagram: validate step inputs

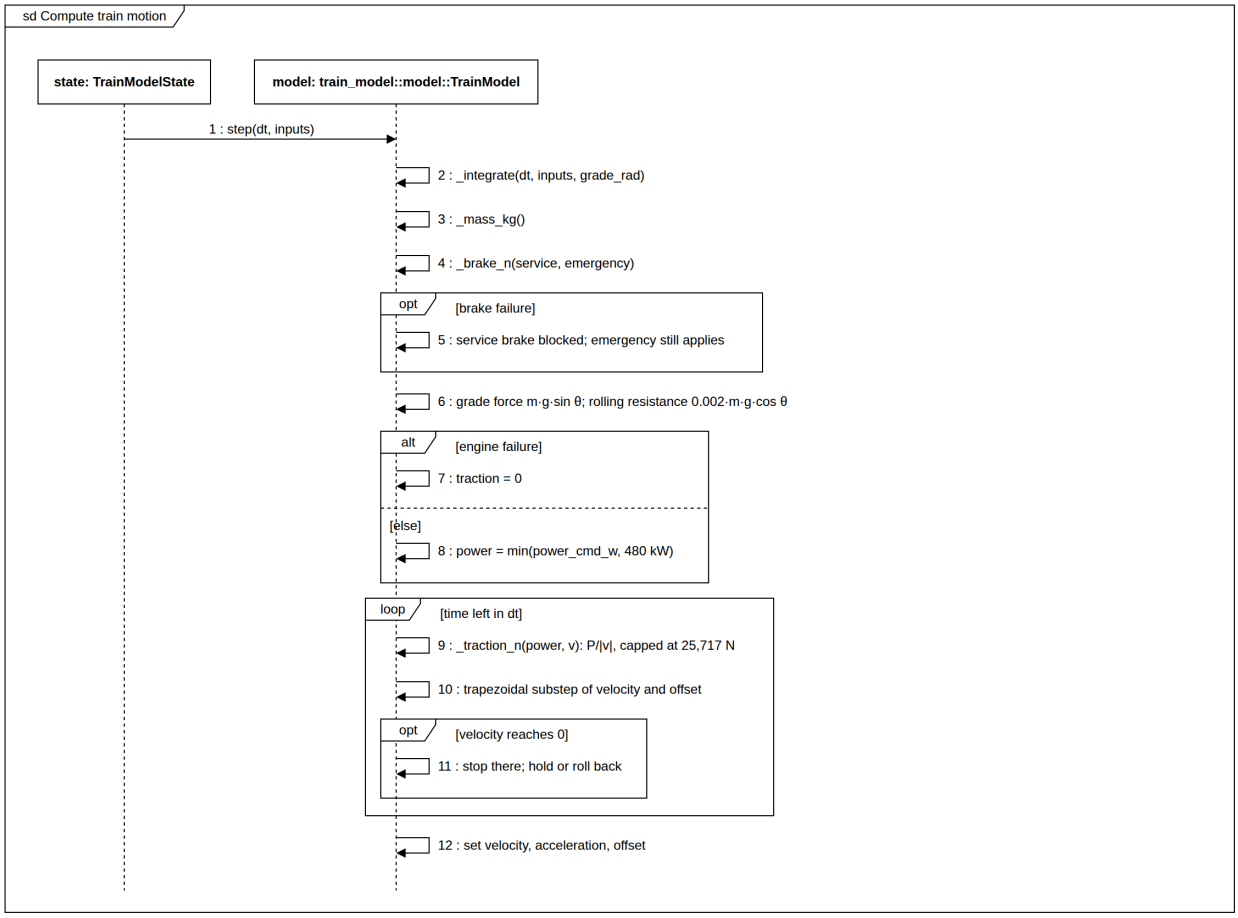

Figure 121. Sequence diagram: compute train motion

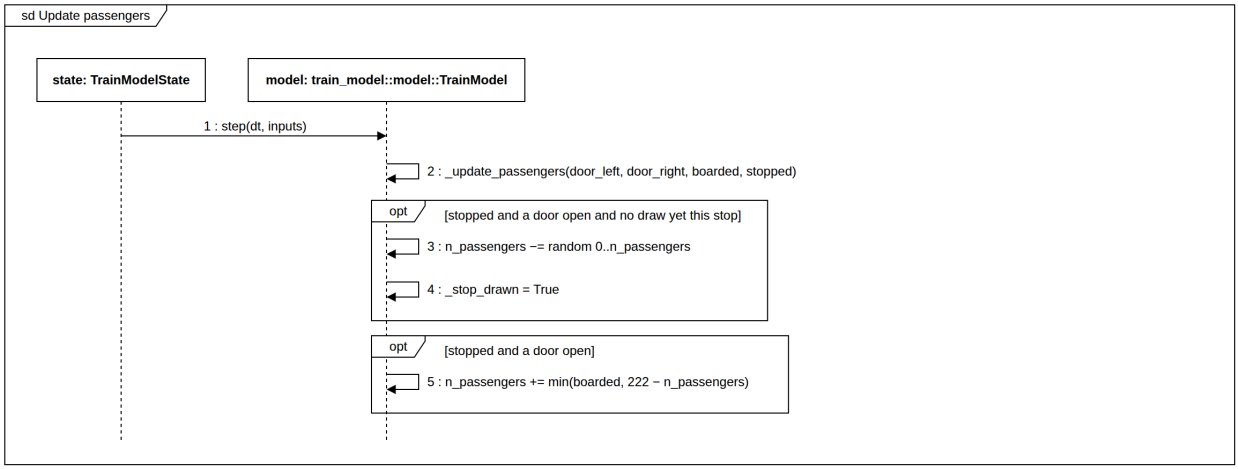

Figure 122. Sequence diagram: update passengers

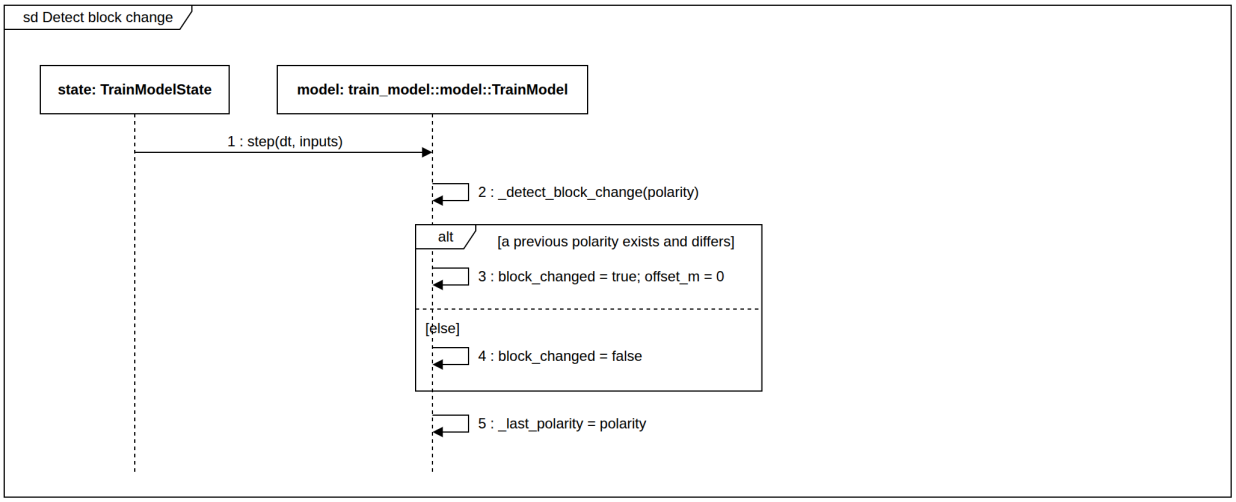

Figure 123. Sequence diagram: detect block change

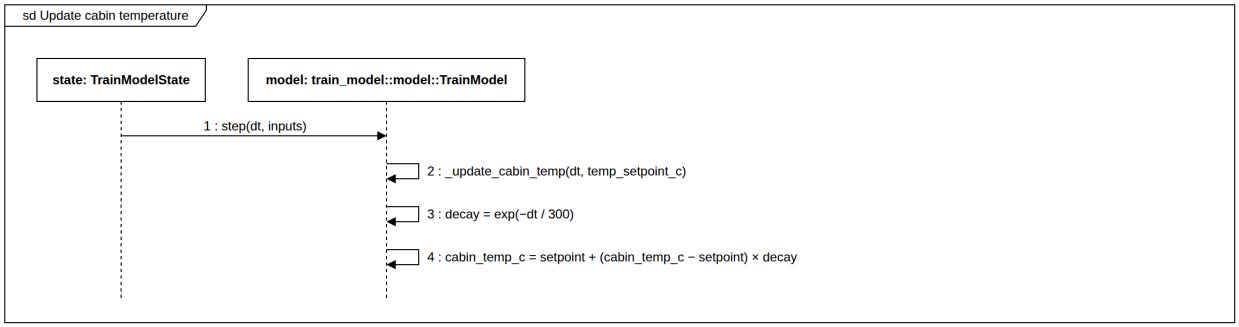

Figure 124. Sequence diagram: update cabin temperature

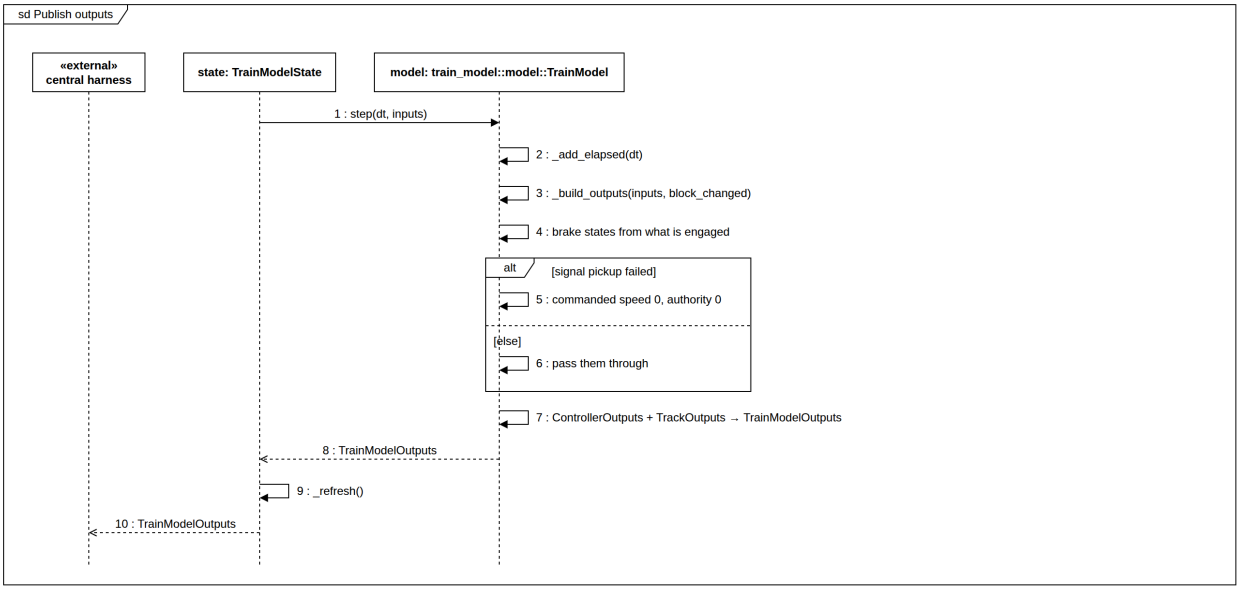

Figure 125. Sequence diagram: publish outputs

UC-TM-02: View and select a train

Primary actor: Observer. **Requirements:** REQ-FUNC-040.1 through 040.6, REQ-FUNC-041.1/041.2, REQ-INTF-001.3.

Preconditions: The module window is open. **Trigger:** A train is selected or its state changes.

Main success scenario: Select a fleet entry; bind to its state; display dimensions, operating mass, acceleration, speed, crew/passenger counts, power command, doors, brakes, lights, cabin temperature, position, and station data. Convert values only for display.

Alternate flows: An empty fleet displays the idle state. Removing the selected train selects another remaining entry. Arrival time is omitted from the intended window; station name and platform-side information remain. Selecting an unknown ID is ignored.

Postcondition: The window observes the selected train; every active train continues to be stepped independently.

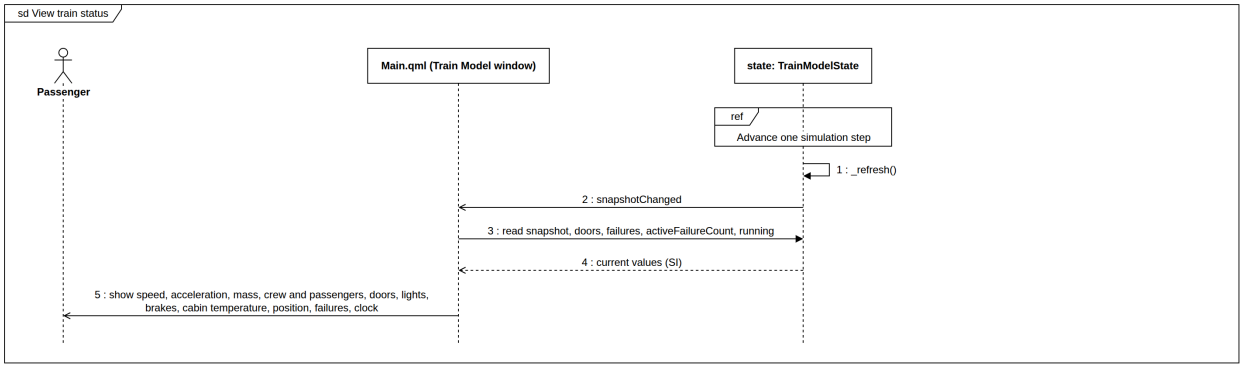

Figure 126. Sequence diagram: view train status

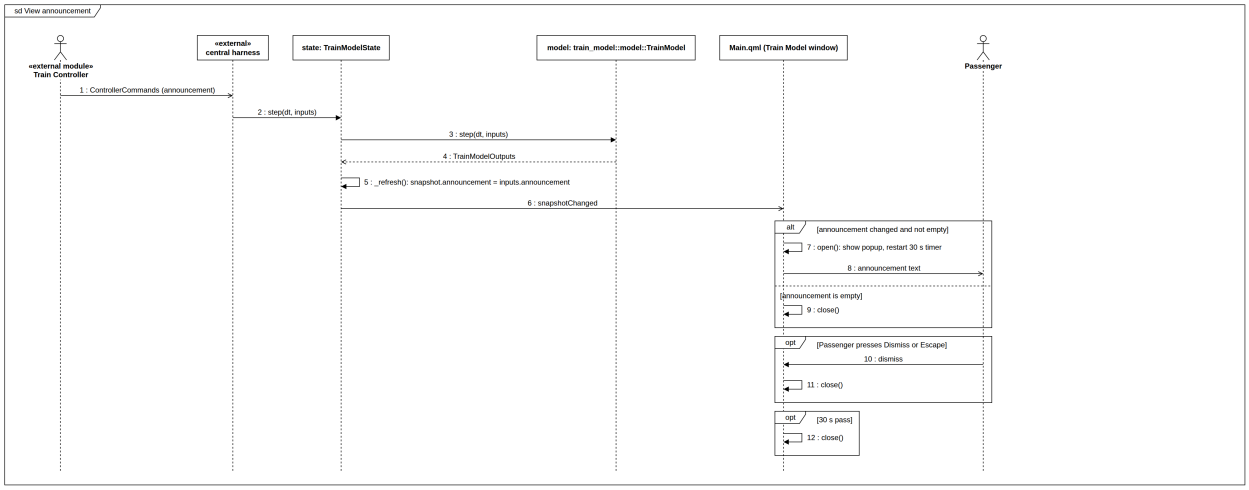

Figure 127. Sequence diagram: view announcement

UC-TM-03: Apply onboard equipment commands

Primary actor: Train Controller. **Supporting actor:** Passenger. **Requirements:** REQ-FUNC-043, 044, 045.

Preconditions: Valid equipment commands are available. **Trigger:** A tick contains new or held door, light, temperature, or announcement commands.

Main success scenario: Apply door and light states; move cabin temperature toward the target; report resulting states; show each new announcement in a passenger popup.

Alternate flows: Invalid flags or values reject the step. The Train Model follows door commands at any speed; the Train Controller owns the door interlock.

Postcondition: Feedback represents equipment state and temperature; the UI presents the announcement without advancing physics independently.

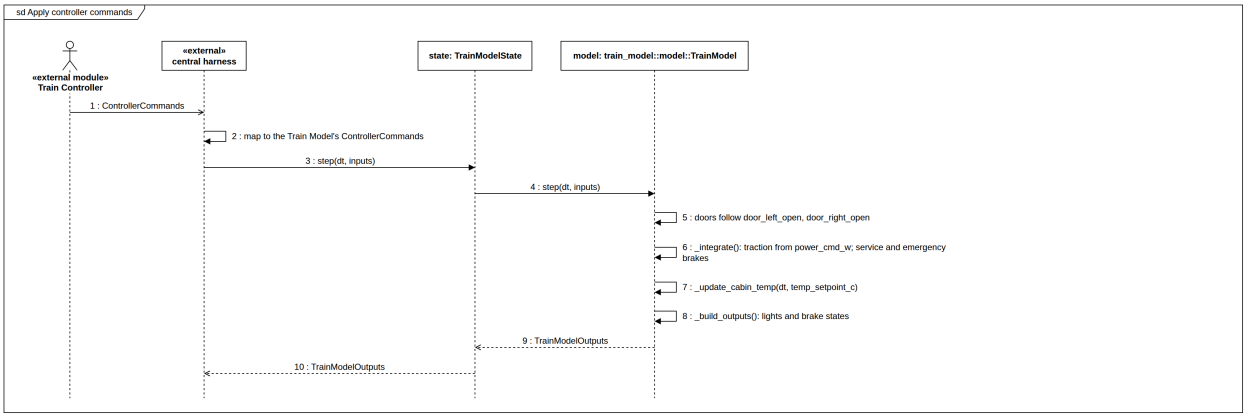

Figure 128. Sequence diagram: apply controller commands

UC-TM-04: Receive track circuit, terrain, and beacon data

Primary actor: Track Model. **Supporting actor:** Train Controller. **Requirements:** REQ-FUNC-042, 046.

Preconditions: Valid block data and any current beacon are supplied. **Trigger:** A track input accompanies a tick.

Main success scenario: Use grade in the force balance; detect a block transition from polarity; pass commanded speed, authority, speed limit, and the current beacon to controller outputs; publish block, offset, and transition flag to the Track Model.

Alternate flows: The first polarity sample establishes a baseline. Under pickup failure, commanded speed and authority become zero; terrain and beacon pass-through remain available. A tick without a beacon reports `None`.

Postcondition: The controller receives the modeled pickup information, and the Track Model receives current position feedback.

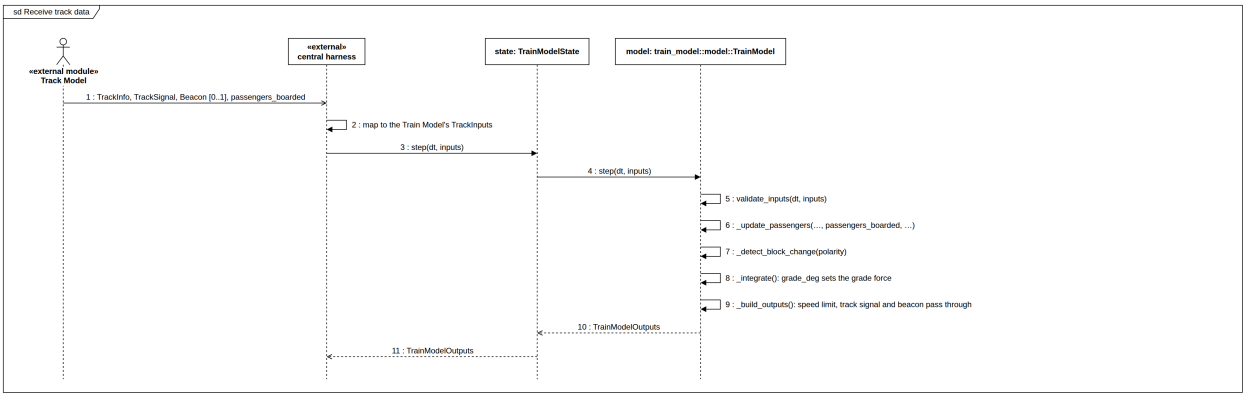

Figure 129. Sequence diagram: receive track data

UC-TM-05: Board and disembark passengers

Primary actors: Passenger and Track Model. **Requirement:** REQ-FUNC-041; passenger exchange supports the displayed count and physical mass.

Preconditions: The train is stopped with at least one door open. **Trigger:** An eligible stopped tick, optionally containing a boarding event.

Main success scenario: Draw the disembark count once for the stop; subtract those passengers; add the received boarding count up to remaining capacity; recompute operating mass; return remaining capacity and update the window count.

Alternate flows: A boarding event while moving or with both doors shut is ignored and not deferred. Negative or out-of-range counts reject the step. Door reopening during the same stop does not cause another disembark draw.

Postcondition: Passenger count stays within 0 to 222, with mass consistent with that count. Station scheduling and the 45 s dwell remain control/test duties.

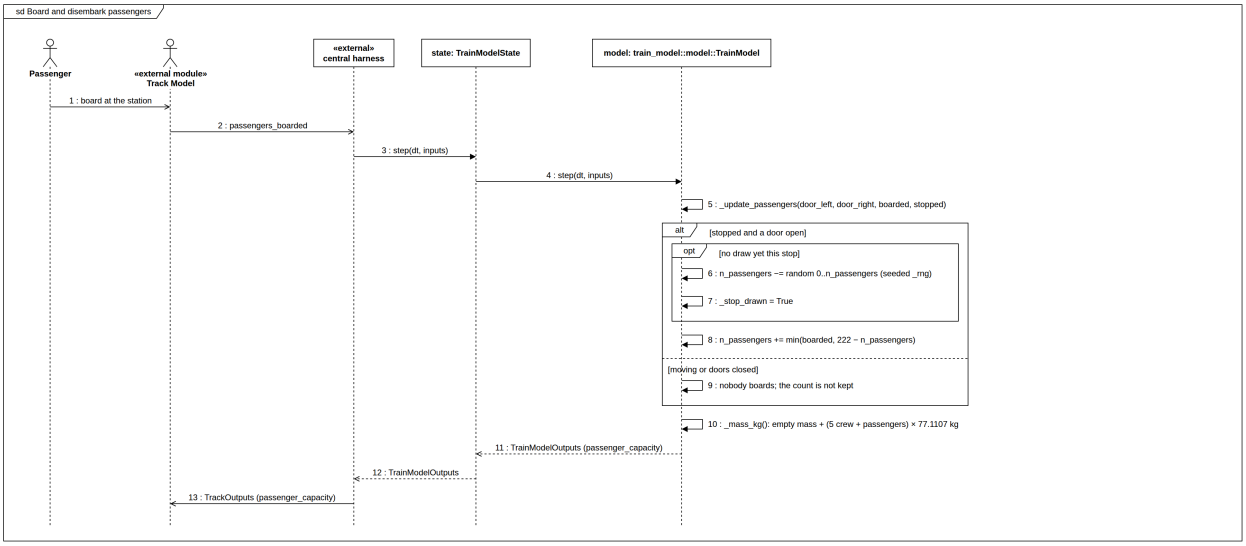

Figure 130. Sequence diagram: board and disembark passengers

UC-TM-06: Pull and release the passenger emergency brake

Primary actor: Passenger. **Supporting actor:** Train Controller. **Requirement:** REQ-FUNC-047; promoted `passenger-brake-release` signal.

Preconditions: The selected train exists. **Trigger:** The passenger activates the emergency-brake button in the Train Model window.

Main success scenario: Latch the passenger pull; report emergency brake active immediately; apply its physical braking force on subsequent steps. The Train Controller later sends a one-shot `passenger_brake_release` to clear the pull.

Alternate flows: A brake failure does not disable the emergency brake. A held controller emergency command keeps emergency braking active after passenger release. The module window provides no passenger-release action.

Postcondition: The pull remains latched until the release command is accepted.

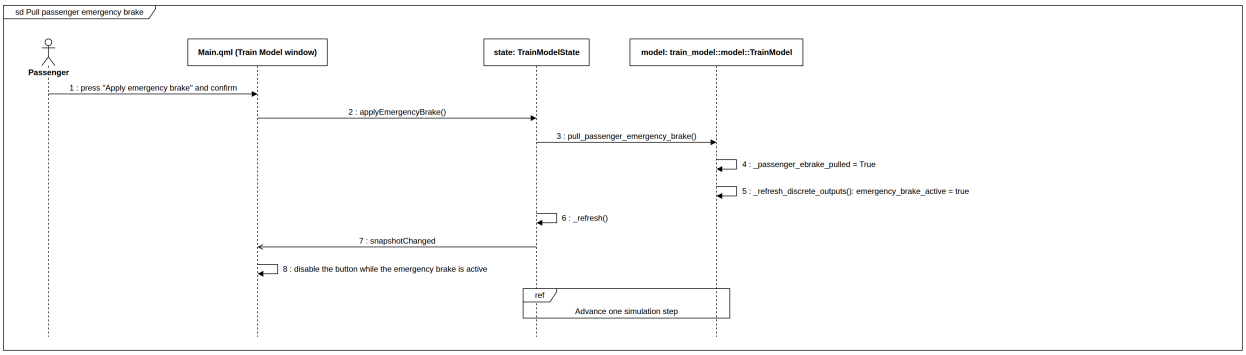

Figure 131. Sequence diagram: pull passenger emergency brake

UC-TM-07: Inject or clear a failure

Primary actor: Murphy. **Requirements:** REQ-FUNC-048.1, 048.2, 048.3.

Preconditions: A train is selected in the module window. **Trigger:** Murphy toggles an engine, signal pickup, or brake failure button.

Main success scenario: Change only that train's failure flag; update its window indicator; recompute discrete outputs; apply the physical effect on the next step. Multiple independent flags may be active together.

Failure	Simulated effect
Engine	Zero traction despite a nonzero power command.
Signal pickup	Zero commanded speed and authority in controller feedback.
Brake	Block service braking; emergency braking remains available.

Alternate flows: Clearing a failure restores the relevant command path. The test UI has no independent failure injection controls. The Train Controller receives effects and equipment states, not the failure flags.

Postcondition: The selected train reflects the fault combination; other trains retain their own independent failure states.

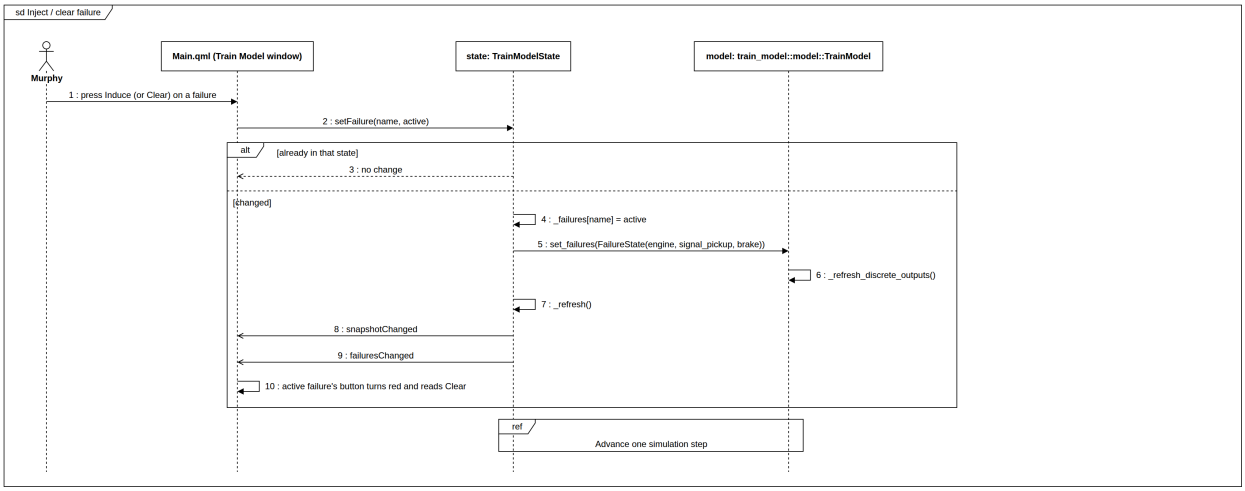

Figure 132. Sequence diagram: inject clear failure

Developer Test Utility

The separate test UI exists only to help developers exercise the public boundary. It is not part of the functional actor/use case model. Developers can stage input values, run or pause the standalone

clock, select 1x/10x, advance one tick, and inspect cross-module outputs. Invalid values are rejected before model advancement. The local link permits one driver; test-only reset and latch-clear hooks support repeatable experiments. Disconnection holds the utility clock.

Track traversal, speed limiting, and the 45 s dwell are stand-ins used by this utility and are removed at integration under D010. The integrated clock check specified here runs every 15 harness ticks. The utility's existing 30-tick drift check belongs to its older standalone path; updating that separate cadence is not required by this review.

5.5.5 Train Model architecture (component view)

The five system modules retain separate ownership. The CTC Office owns dispatch and routing suggestions; the Track Controller owns wayside control; the Track Model owns railway state; the Train Controller owns onboard control; the Train Model owns physical response. This section details only the Train Model.

Part	Responsibility	Source
Boundary contract	Frozen input, output, snapshot, and configuration dataclasses; structural model protocol.	`train_model/interface.py`
Physics core	Input validation, force balance, integration, passengers, block-change detection, cabin temperature, and output construction. Has no Qt dependency.	`train_model/model.py`
Qt state wrapper	Owns one physical model; exposes snapshot properties and change signals; receives selected-train UI actions.	`train_model/state.py`
Fleet	Owns one state per train ID; adds/removes trains and exposes the selected train to QML.	`train_model/fleet.py`
Process adapter	Connects to the harness, follows its roster, decodes each train's inputs, steps valid trains, and returns outputs/rejections.	`train_model/module_process.py`
Wire conversion	Converts nested JSON objects to module types and outputs back to plain values.	`train_model/wire.py`

Part	Responsibility	Source
Main window	Readouts, train selector, passenger brake, failure buttons, and announcement popup.	`ui/Main.qml`, `ui/MainView.qml`
Bootstrap	Creates the Qt application, QML engine, theme, and context objects.	`train_model/app.py`, `main.py`
Test setup	Separate test UI, local socket link, track stand-in, speed limiter, and test clock driver; removed at integration under D010.	`test_ui.py`, `train_model/harness.py`, `link.py`, `track_stub.py`, `speed_limiter.py`, `ui/TestMain.qml`, `ui/TestView.qml`

Paths in this table are relative to `TrainModel/`. The production harness transport uses shared `utils/module_link.py`; it is distinct from the removable test link.

One Train Model process holds the fleet; there is no process per train. Each train has independent motion, passengers, cabin state, failures, and random generator. The fleet derives a repeatable seed from its train ID unless a configuration is provided. Selecting a train changes the window binding, not which trains step. With no trains the window uses an idle state that is never stepped.

The process adapter validates and steps trains separately. A bad input for one train holds that train's physical state and returns its last outputs while the other trains advance. The `TrainModelFleet.step_all` helper instead validates the whole fleet before advancing any train; the process adapter does not use that helper for integrated stepping.

5.5.6 Class diagrams

Boundary types are shared between the Train Model's own layers. Other modules define their own types; the harness maps between them. The module does not import the stale `common/interfaces.py` catalog (D008).

Type / class	Main relationships and purpose
`TrainConfig`	Frozen primitives and derived force/mass properties; referenced by the physical model.
`TrainModelInputs`	Contains one `ControllerCommands` and one `TrackInputs`.
`TrackInputs`	Contains `TrackInfo`, `TrackSignal`, optional `Beacon`, and a boarding count.
`TrainModelOutputs`	Contains one `ControllerOutputs` and one `TrackOutputs`.

Type / class	Main relationships and purpose
`TrainModelSnapshot`	Full UI observation, including failures, passengers, mass, acceleration, elapsed time, last inputs, and outputs.
`interface.TrainModel`	Protocol defining `step`, `snapshot`, failure injection, and passenger-brake operations, including a test-only clear hook.
`model.TrainModel`	Implements the physical behavior; owns mutable physical state and a per-train random generator.
`TrainModelState`	QObject owning one model and publishing `snapshotChanged`, `failuresChanged`, and `runningChanged`.
`TrainModelFleet`	QObject holding zero or more states by ID, plus selected/idle state for the window.
`TrainModelProcess`	Holds the fleet and a harness `ModuleLink`; orchestrates roster and individual train steps.

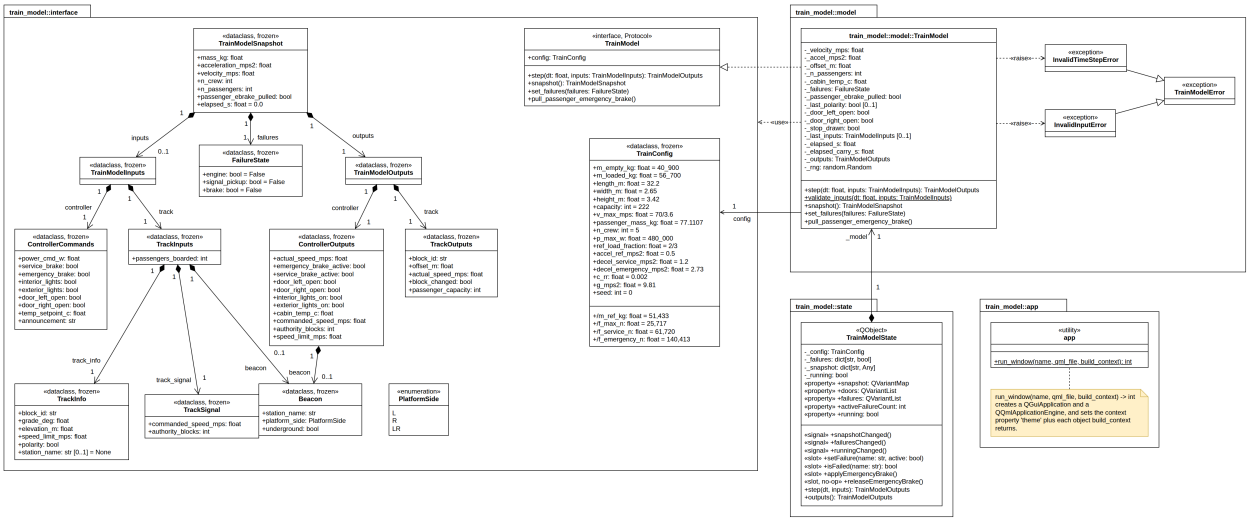

Figure 133. Train Model class diagram

Vehicle configuration and mass

Parameter	Default value / derivation	Qualification
Vehicle	Bombardier FLEXITY 2, fixed five-module consist	Train Model contract, R2.
Length / width / height	32.2 m / 2.65 m / 3.42 m	Vehicle data recorded in R2.

Parameter	Default value / derivation	Qualification
Empty / datasheet loaded mass	40,900 kg / 56,700 kg	Loaded figure is the datasheet reference load.
Crew / passenger capacity	5 / 222	Passenger capacity excludes crew.
Mass per person	77.1107 kg	Applies to crew and passengers.
Maximum power	480,000 W	Confirmed by Kevin Schillinger in R10.
Vehicle maximum speed	70/3.6 m/s	Recorded, not enforced by this module.
Reference mass	`40,900 + (2/3)*(56,700 - 40,900)` = approximately 51,433 kg	Used to derive the fixed force limits.
Traction limit	`m_ref * 0.5` = approximately 25,717 N	Derived from rated reference-load acceleration.
Service brake force	`m_ref * 1.2` = 61,720 N	Fixed force; actual deceleration varies with operating mass.
Emergency brake force	`m_ref * 2.73` = 140,413 N	Same reference-load basis.
Rolling coefficient / gravity	0.002 / 9.81 m/s²	Current configured values; rolling resistance is the only passive resistance term in scope.

Operating mass is `m_empty + (n_crew + n_passengers) * passenger_mass`. It ranges from approximately 41,286 kg with crew only to 58,404 kg at passenger capacity. The datasheet's loaded mass sizes the reference forces; it is not a clamp on operating mass.

Motion and equipment behavior

Positive velocity is forward and positive grade is uphill in that direction. For moving trains, the force balance is:

```
P_effective = min(power_cmd_w, p_max_w), or 0 under engine failure
F_traction = min(P_effective / abs(v), F_max) when v != 0
F_traction = F_max when v == 0 and P_effective > 0, otherwise 0
F_grade = m * g * sin(grade_deg * pi / 180)
F_roll = C_rr * m * g * cos(grade_deg * pi / 180)
F_net = F_traction - F_grade - sign(v) * (F_brake + F_roll)
a = F_net / m
```

Traction acts forward during rollback as well. At rest, brakes and rolling resistance act as holding forces. A sufficiently steep grade can produce rollback; a brake stops motion without itself reversing the train. Applying power while braking remains physically meaningful: the Train Controller owns the traction cut decision.

The implementation uses exact constant-force segments and midpoint evaluation in the power-limited regime, with internal event/substep boundaries at stops and traction-regime changes. Position uses the average of segment endpoint velocities. Kevin has approved keeping this algorithm. Internal segments allow accurate handling of stopping and traction-regime transitions, while evaluating power-limited traction at midpoint velocity preserves the commanded work over each segment and avoids artificial energy gain. The external step still advances exactly dt at either simulation speed. D006 must be updated through the follow-up documentation work to explain these reasons and remove its no-substep statement.

Cabin temperature begins at 20 °C and approaches the commanded setpoint using `T_next = T_set + (T - T_set) * exp(-dt/300)`. Doors and lights follow controller commands. The Train Controller is responsible for keeping doors closed when necessary; the Train Model does not impose a door-speed interlock. Rolling resistance is the only passive resistance term modeled. Grade remains a gravitational force and braking remains a commanded force; aerodynamic drag, wheel slip, and brake build-up are outside the selected model.

Service and emergency brakes may be engaged simultaneously. The intended brake states are independent: emergency follows the controller command or passenger latch, and service follows its command unless service brake failure is active. Both engaged brakes contribute their configured forces to the total brake force. This force-combination rule is the physical implementation selected for the simultaneous-application decision. Current code still suppresses service when emergency is active; the backend prompt specifies the required change.

Passengers board only when stopped with a door open, capped by remaining capacity. Once per stop, the first eligible open-door tick draws a uniform disembark count from zero through the number aboard. Reopening a door does not draw again until the train moves and begins another stop. The Track Model owns the station restriction and sends boarding events only at stations. The Train Model checks rest, open doors, valid counts, and remaining capacity; it does not add a station-presence gate. Boarding changes operating mass before that tick's motion is integrated.

5.5.7 Interface

The physical entry point is `step(dt, TrainModelInputs) -> TrainModelOutputs`. `snapshot()` observes full module UI state without advancing time. Failure injection and a passenger pull are local UI actions, not fields sent by another module. The normal passenger-release command is a controller input.

Train Controller to Train Model: `ControllerCommands`

Field	Type / backend unit	Meaning
`power_cmd_w`	finite float, W, >= 0	Requested traction power, saturated at the configured physical power limit.
`service_brake`	bool	Apply service brake unless service brake failure is active; emergency braking does not suppress it in the intended design.
`emergency_brake`	bool	Controller emergency-brake command, separate from the passenger latch.
`interior_lights`, `exterior_lights`	bool each	Requested equipment states.
`door_left_open`, `door_right_open`	bool each	Requested door states.
`temp_setpoint_c`	finite float, °C	Cabin temperature target.
`announcement`	str	Passenger announcement text.
`passenger_brake_release`	bool, default false	One-shot release of the passenger latch; does not cancel the controller emergency command.

Track Model to Train Model: `TrackInputs`

Field	Type / backend unit	Meaning
`track_info.block_id`	str	Current block identifier.

Field	Type / backend unit	Meaning
`track_info.grade_deg`	finite float, degrees, strictly between -90 and 90	Positive uphill; source layout grade percent is converted upstream.
`track_info.elevation_m`	finite float, m	Terrain metadata.
`track_info.speed_limit_mps`	finite float, m/s, >= 0	Block limit passed through in model outputs.
`track_info.polarity`	bool	Flip on block entry; a change resets offset and raises the block-change event.
`track_info.station_name`	str or `None`	Station in the current block; implementation uses `None` off-station.
`track_signal.commanded_speed_mps`	finite float, m/s, >= 0	Track-circuit commanded speed passed to the controller.
`track_signal.authority_blocks`	int, 0 through $2^{31} - 1$	Remaining authority in blocks.
`beacon`	`Beacon` or `None`	`station_name: str`, `platform_side: L/R/LR`, `underground: bool`. The entire canonical beacon payload must fit 128 characters; the encoding below replaces the current station-name-only check.
`passengers_boarded`	int, 0 through $2^{31} - 1$	Event count; zero otherwise. Replaying a nonzero count can board more passengers on later ticks.

Train Model to Train Controller: `ControllerOutputs`

Field	Type / backend unit	Meaning
`actual_speed_mps`	float, m/s	Signed physical velocity, including rollback.
`emergency_brake_active`, `service_brake_active`	bool each	Independent engaged states; both may be true. Current precedence behavior requires the deferred backend change.
`door_left_open`, `door_right_open`	bool each	Actual simulated door states.
`interior_lights_on`, `exterior_lights_on`	bool each	Actual simulated light states.
`cabin_temp_c`	float, °C	Simulated cabin temperature.
`commanded_speed_mps`, `authority_blocks`	float m/s; int blocks	Track-signal pass-through; both become zero under signal pickup failure, as confirmed in R10.
`speed_limit_mps`	float, m/s	Block speed limit, unaffected by pickup failure. The current controller edge drops this field because that consumer has no separate speed-limit input.
`beacon`	`Beacon` or `None`	Current tick's beacon; the consumer retains station information if needed.

Failure flags are available to the Train Model window through its snapshot. They are not sent to the Train Controller. The controller must observe physical response and feedback rather than receive Murphy's fault flags.

Train Model to Track Model: `TrackOutputs`

Field	Type / backend unit	Meaning
`block_id`	str	Current block, supplied by Track Info.

Field	Type / backend unit	Meaning
`offset_m`	float, m	Signed integrated offset of the train's front since the last polarity transition.
`actual_speed_mps`	float, m/s	Same physical speed reported to the controller.
`block_changed`	bool	True only on the polarity-transition tick; the first polarity sample is not a change.
`passenger_capacity`	int, persons	Free passenger spaces after this tick's exchange.

The UI snapshot additionally contains mass, acceleration, crew and passenger counts, passenger latch, failures, last inputs, and elapsed model time. These are not automatically cross-module signals.

Beacon payload encoding

The chosen logical beacon payload is a compact JSON array in fixed order: `[station_name, platform_side, underground]`, for example `["Central","LR",false]`. Canonical serialization uses ASCII escapes for non-ASCII text (`ensure_ascii=True`) and no optional whitespace (`separators=(",", ":")`). All JSON syntax, quotes, brackets, commas, and escape characters count toward the 128-character limit. ASCII output makes character count equal to encoded byte count. Invalid or overlength beacons are rejected before state mutation; names are never silently truncated.

Existing typed `Beacon` objects and named fields in harness JSON envelopes remain the readable module boundary. Canonical array encoding defines the bounded beacon payload, not a replacement for the entire harness protocol. Producers and the Train Model must enforce the same rule. `None` remains the output between beacon ticks; the Train Controller retains the last beacon if its control logic needs it. This encoding is specified here for follow-up implementation and documentation.

Validation and units

Validation occurs before any physical state, elapsed time, or random-generator state changes. Invalid dt raises `InvalidTimeStepError`; invalid inputs raise `InvalidInputError`, both within the `TrainModelError` hierarchy. dt must be finite, positive, and at most 60 s; accepting that range defensively does not authorize varying dt during a run. Flags must be actual booleans; counts must be integers and cannot be booleans. Numbers must be finite, and text fields must be strings. Malformed nested objects are rejected during wire decoding.

State and interface payloads use backend units. QML converts speed to mph, distance/dimensions to ft, acceleration to ft/s^2, mass to short tons, and cabin temperature to °F. Power displays in kW.

Authority remains a count of blocks. The Power command readout is a command, not measured energy consumption.

5.5.8 Interaction: one simulation tick

Preconditions: The module is ready, the harness has a roster, and previous producer outputs are available. A newly introduced train receives inert commands until its Train Controller supplies commands.

1. The harness maps the preceding producer outputs into each train's controller and track inputs and sends a `STEP` with request ID, tick, dt, and simulation time.
2. `TrainModelProcess` follows the roster: it adds newly named trains and removes trains no longer named. Train ID and line are fleet metadata.
3. For each train, the adapter decodes its nested objects and validates the step.
4. For an accepted train, the model clears the passenger latch if the one-shot release command is true, then applies doors and passenger exchange using velocity at the start of the tick.
5. It detects a polarity transition, resetting block offset if necessary, integrates forces over dt, ends the passenger stop once moving, and updates cabin temperature.
6. It saves the accepted inputs, advances elapsed time using compensated summation, constructs outputs, and refreshes the Qt snapshot.
7. The adapter encodes each train's outputs in a `trains` object. Rejected trains retain their previous physical outputs and are named in a `rejected` object.
8. The module reconciles the displayed clock with the supplied harness time, independently checks per-train elapsed time for drift every 15 harness ticks, and publishes clock diagnostics without modifying physics time. This is deferred backend work specified by this design.
9. The harness receives the reply, logs rejections, and supplies the outputs to consumers on the following tick. The window observes Qt change signals.

If the top-level `trains` object is missing, the adapter rejects the module tick. If one train's decoded inputs are invalid, the other trains still advance. The roster update occurs before per-train validation; rejection preserves a train's physical step, not an all-or-nothing roster transaction.

An action performed between ticks updates local model outputs and its window immediately. In the integrated system other modules receive that change through the next harness exchange. Physics advances only on an accepted step. The standalone test link can publish changed outputs immediately even while paused.

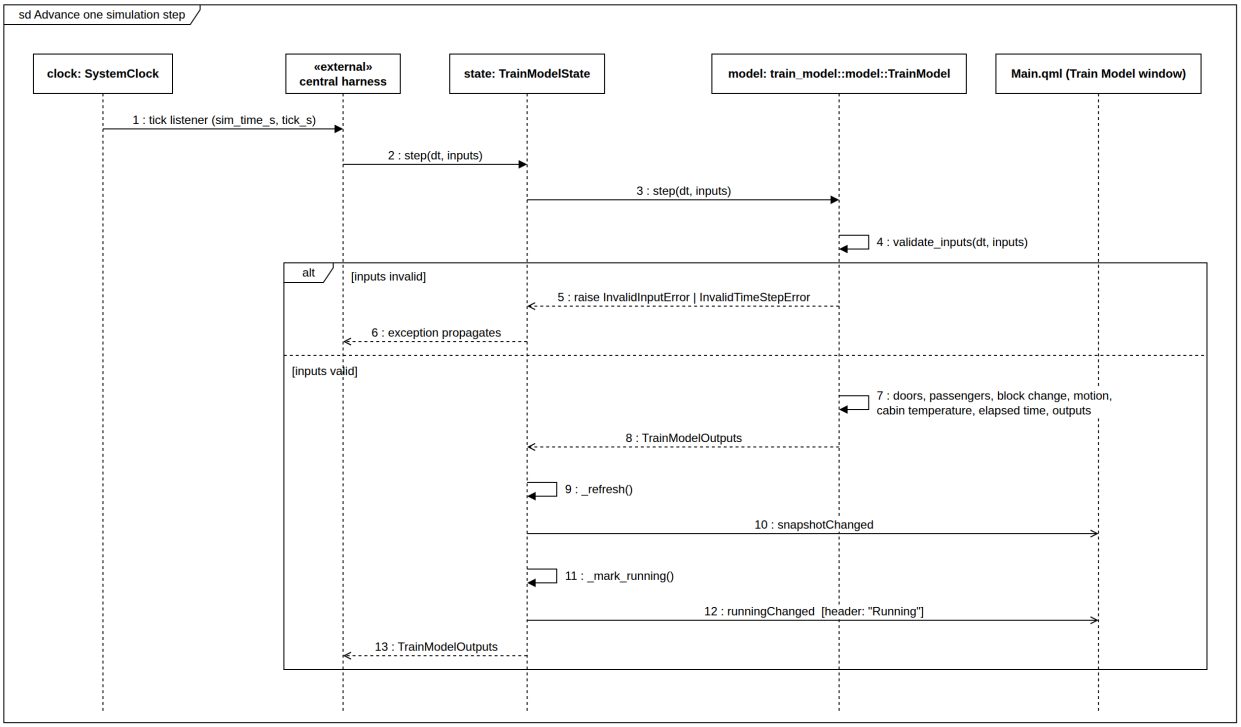

Figure 134. One accepted or rejected Train Model tick

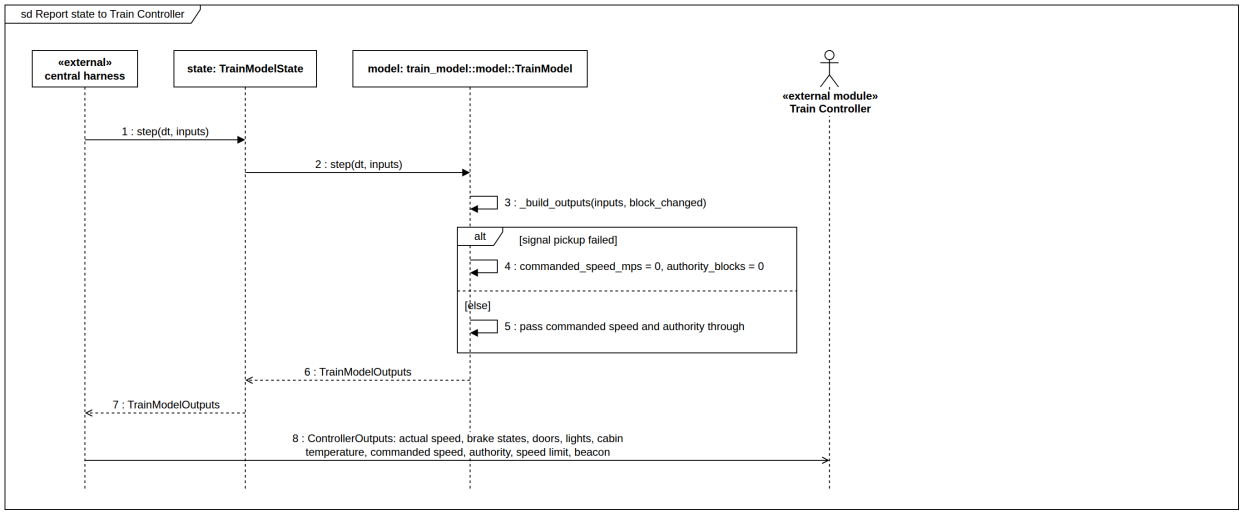

Figure 135. Sequence diagram: report state to train controller

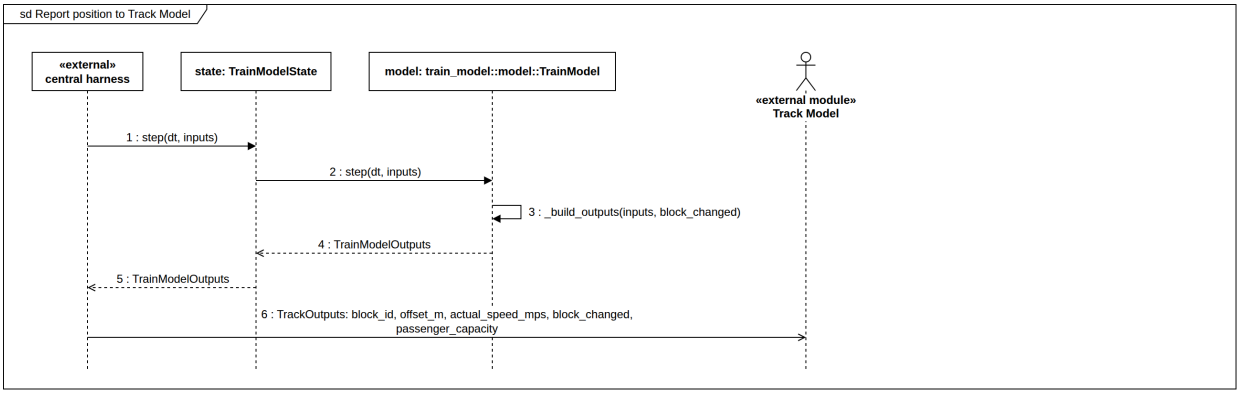

Figure 136. Sequence diagram: report position to track model

5.5.9 Design decisions

The headings below follow the template. D005-style identifiers refer to promoted repository decisions; the template's D-001-style example IDs are not reused as new decisions. R10 supplies the latest review resolutions; Section 5.5.10 identifies which accepted decisions require later code or documentation changes.

Interface Contract Mechanism

Basis: D005; Train Model contract shard.
Context: The physics must be callable from a test setup or the integrated system without depending on another module's internal state.
Outcome: A typed `step` boundary and read-only snapshot separate simulation from presentation. Local failure/brake actions belong to the module window. The harness is the integrated single driver.

Shared Boundary Data Types

Basis: D005, D008, unit conventions.
Context: A universal shared struct catalog would couple module implementations.
Outcome: Frozen Train Model dataclasses describe its own boundary. Pure harness edge mappings translate to peer types without changing backend units. `common/interfaces.py` is stale and is not an intermediate mapping layer.

Contract Verification

Evidence: `TrainModel/tests/test_contract.py` checks protocol members, signatures, output shape, and snapshot behavior. Wire and process tests check malformed payloads, roster following, and rejection isolation. **Outcome:** Developers run focused local checks when changing the relevant behavior. Existing tests are local verification references; this SDD does not report new execution results.

Stub-First Development

Basis: D010.
Context: The Train Model needs external track/controller inputs before the full system is connected.
Outcome: A separate test UI supplies stand-in producers through the public boundary. Speed limiting and dwell in that test setup are not physics-core responsibilities. The current physics implementation is real code; the template's example `STUB = True` policy is not asserted as a Train Model requirement.

Contracts Package Ownership

Basis: D005 and the Train Model shard's named owner.
Context: Boundary ownership must remain clear during integration.
Outcome: Kevin Schillinger owns the Train Model contribution and module-owned types. Integration mappings belong to the harness. There is no newly introduced shared contracts package in this design.

External Input Validation

Basis: Promoted Train Model validation rules; REQ-NFR-003/004.
Context: Invalid values must not partially advance a physical model or consume its random sequence.
Outcome: Validate before mutation, reject with a typed error, and return a named per-train rejection through the process adapter. Preserve the distinction between adapter rejection isolation and the atomic `step_all` helper.

Knowledge Capture and Decision Logging

Basis: Repository `AGENTS.md` and promoted truth entries.
Context: Implementation evidence, pending proposals, and accepted contracts have different authority.
Outcome: Read facts from the fetched truth branch; proposed normative changes go through its inbox and human promotion. This SDD records observed differences without promoting them. For this SDD-only revision, Kevin explicitly deferred all other documentation and proposal work to the follow-up prompts. Implementation status and test results belong in regular project documents, not the truth store.

Rejected Tooling - Summary Table

No formal Train Model tooling-rejection record was found in the reviewed sources. The following are architectural exclusions supported by existing decisions; they are not newly invented tool evaluations.

Tool / approach excluded from this design	Recorded reason
Shared `common/interfaces.py` catalog	Stale and superseded as the mapping catalog under D008.
Direct imports of another module's boundary layouts	D005 assigns translation to the harness so each module owns its types.
Permanent test UI control path in the delivered Train Model	D010 makes the test UI removable and the integrated harness the driver.

5.5.10 Review items and deferred work

Kevin Schillinger resolved the review questions on October 20, 2026 (R10). The table distinguishes the intended design from work deferred out of this SDD-only session. No item below is a request to implement product changes now.

Item	Resolved design	Deferred work
Numerical integration	Keep `model.py` and its internal segments/substeps. Explain transition handling and midpoint power/work accounting while retaining fixed external dt.	Documentation: propose corrected D006 and align module/physics explanations. No algorithm rewrite.
Simultaneous brakes	Both brakes can apply simultaneously; service failure disables service only. The selected physical realization adds the two engaged brake forces.	Backend: remove service suppression, align force and state reporting, and update focused tests.
Station restriction on boarding	Track Model sends boarding events only at stations. Train Model enforces rest, doors, counts, and capacity without a station gate.	Documentation: clarify responsibility in the boarding signal and module descriptions.
Beacon encoding and between-beacon behavior	Canonical compact ASCII-escaped JSON array `[station_name, platform_side, underground]`, maximum 128 characters for the complete payload. Keep `None` between beacon ticks.	Backend: enforce complete payload length consistently. Documentation: record encoding and retention responsibility.
Pickup failure output	Keep commanded speed zero and authority zero.	Documentation: record the settled choice; preserve existing implementation.

Item	Resolved design	Deferred work
Door interlock	Train Controller ensures doors remain closed when required. Train Model follows door commands.	Documentation: remove unresolved ownership language; preserve existing Train Model behavior.
Vehicle parameters	480 kW is confirmed. Rolling resistance is the only passive resistance term in scope; grade and brake forces still apply.	Documentation: remove provisional-power and extra-resistance calibration requests.
Position reference	Use the front of the train for offset.	Documentation: align interface comments, position signal, physics, and integration text.
Integrated clock	Display harness time and independently compare module time against supplied simulation time every 15 harness ticks, accounting for train creation epochs.	Backend: reconcile time and detect drift without resetting elapsed physics time. UI: present current time and selected-train drift diagnostics.
Arrival metadata	Remove arrival time from the Train Model window for now; retain station/platform information.	UI: remove the arrival portion of the row. Documentation: remove the requirement for an arrival data source.
Legacy documentation	Update outdated release, identity, harness/catalog, diagrams' descriptions, and resolved review items.	Documentation: follow the separate prompt, excluding this SDD. Existing diagrams remain author-owned insertion material.

The three implementation prompts are provided inline in the accompanying chat: UI changes, backend changes, and other documentation changes. They are not stored in the repository.

For the integrated clock check, the supplied simulation timestamp represents the end of the tick. Establish each train's epoch at its first roster appearance as `sim_time_s - dt`, and compare `epoch + accepted_model_elapsed_s` with the current supplied timestamp on harness ticks divisible by 15. Check after accepted steps, and also check retained elapsed time for rejected trains. A 1 microsecond absolute tolerance filters floating-point noise. Never realign the comparison epoch on each tick, since that would hide missed physical steps. Window time follows the harness independently; a rejected train can therefore show current global time and a real drift diagnostic simultaneously.

D017 already records the 0.1 s tick as a deviation from the SRS's 1 ms preference. The SRS's distance-based authority definition and REQ-FUNC-042 wording "suggested speed" are interpreted through

promoted D013 and the track-signal contract: blocks and downstream commanded speed, respectively.

5.5.11 Requirements traceability

Requirement	Train Model design / use case	Evidence / verification reference
REQ-FUNC-039	Force balance, mass, motion integration; UC-TM-01.	Physics and integration tests; retained algorithm and deferred D006 correction in Section 5.5.10.
REQ-FUNC-040.1-040.6	Length, height, width, mass, acceleration, and velocity readouts; UC-TM-02.	`state.py`, `MainView.qml`; demonstration of displayed values.
REQ-FUNC-041.1/041.2	Crew and passenger count; UC-TM-02/05.	Passenger, fleet, and mass tests; UI demonstration.
REQ-FUNC-042	Circuit commanded speed and authority pass-through; UC-TM-04.	Contract/integration tests, pickup-failure tests.
REQ-FUNC-043	Exact first-order cabin temperature response; UC-TM-03.	Temperature-response tests.
REQ-FUNC-044	Interior/exterior light commands and feedback; UC-TM-03.	Light-output tests and UI demonstration.
REQ-FUNC-045	Left/right door commands and feedback; UC-TM-03.	Door tests; Train Controller owns the interlock.
REQ-FUNC-046	Beacon input and current-tick pass-through; UC-TM-04.	Beacon/platform-side tests.
REQ-FUNC-047	Passenger latch and emergency response; UC-TM-06.	Brake tests and `test_release_command_clears_a_passenger_pull`.
REQ-FUNC-048.1-048.3	Engine, signal pickup, and brake failures; UC-TM-07.	Failure-effect tests and selected-train UI demonstration.
REQ-INTF-001.3	Dedicated module window; UC-TM-02.	`Main.qml`, `MainView.qml`.
REQ-INTF-013/014	Shared clock contract and PySide6/QML presentation.	Existing process/clock tests; deferred time reconciliation and 15-tick drift work in Section 5.5.10.

Requirement	Train Model design / use case	Evidence / verification reference
REQ-NFR-002	Same dt and input sequence at 1x/10x.	Clock tests; no change in physics dt with wall-clock rate.
REQ-NFR-003/004	Validation before physical mutation; isolated rejection.	Invalid-input, snapshot-invariance, and process tests.
REQ-DSN-001/002/003	Prebuilt executable and local host deployment.	Integrated packaging/delivery verification remains a group activity.
REQ-DSN-004/005	Public boundary, harness mappings, shared clock/logging infrastructure.	Interface, process, and harness mapping inspection.

The functional rubric separately includes **5.9, Train Controller activates Service Brake**. This is covered by `service_brake`, the physical brake response, and `service_brake_active` feedback in UC-TM-01/03 and brake tests, even though the SRS mapping groups Train Model behavior mainly under REQ-FUNC-039 through 048.

5.5.12 Verification references

Existing test source under `TrainModel/tests/`	Design behavior it exercises
`test_contract.py`	Protocol signatures, returned boundary types, and snapshot observation.
`test_physics.py`, `test_physics_extended.py`	Forces, load effects, grades, braking, rollback, block changes, temperature, passengers, and failures.
`test_integration.py`	Kinematics, energy accounting, stops, invalid input preservation, and absence of a physics speed governor.
`test_model_invariants.py`	Physical bounds and invariants over changing inputs.
`test_fleet.py`	Independent trains, identity/selection, deterministic seeds, atomic fleet helper, and multi-train stepping.
`test_module_process.py`	Wire shape, roster, per-train rejection, and controller passenger-brake release.
`test_elapsed_clock.py`, `test_harness_clock.py`	Elapsed-step accounting and standalone clock behavior.

Existing test source under `TrainModel/tests/`	Design behavior it exercises
`test_link.py`, `test_link_malformed.py`, `test_input_submission.py`	Standalone transport, malformed messages, and staged-input submission.

These are verification references, not a new execution report or a claim of complete requirement conformance. Existing tests that encode emergency-over-service precedence require follow-up updates for simultaneous braking. New clock/beacon checks belong to the deferred backend work. Existing modified tests and diagram sources were preserved during this SDD-only revision.

5.6 Train Controller (SW and HW)

Owner: Jonathan Tsang, Ivan Zheng. **Source:** Railway_Sim_SDD_Train_Controller.md.

5.6.1 Scope

One module and one code base for the SW and HW variants. The SW variant keeps this code and runs the Green Line; the HW variant will run the same code on a Raspberry Pi later, for the Red Line (D021, D027).

- *In scope:*
  - Speed regulation: a PI law from velocity error to traction power, 0–480 kW, in Manual (driver setpoint, capped at the commanded speed) and Automatic (commanded speed) modes.
  - The safety path: service and emergency brake commands, traction cut under braking, stopping at authority 0.
  - Inferring engine, brake and signal pickup failures from the signals returned by the Train Model, and latching the emergency brake.
  - Doors, cabin lights, cabin temperature setpoint and station announcement commands.
  - One controller per train, held in a fleet the central harness steps once per 0.1 s tick.
  - The driver and engineer console (PySide6 / QML) and a test bench used only when the harness is not running.
- *Out of scope:*
  - Train physics, mass, grade and brake rates: the Train Model's (D009). The controller assumes perfect movement (M19).
  - Counting authority down: the count is used as given (M16).
  - Moving Block (MBO): out of scope for the whole system (D001).
  - The event log: written by the harness (D005).
  - Running the HW variant on the Raspberry Pi over USB (D021, D027): later; see Section 5.6.13.

5.6.2 Stakeholder concerns

Stakeholder	Concern	Addressed By
Customer: Port Authority of Allegheny County	The train never exceeds its commanded speed, and at authority 0 targets 0 and brakes; stopping in time is the wayside's, through the commanded speed	M1, M17; Section 5.6.7; Section 5.6.9
Customer	A failure stops the train safely	M3–M5; Section 5.6.10

Stakeholder	Concern	Addressed By
Train driver	Clear Manual control of speed, brakes, doors, lights, temperature and announcements	Section 5.6.5 (console); M2, M6, M14
Engineer	Sets Kp and Ki once per train, before it moves	M9, D024
Passengers	Emergency brake reachable; stations announced; doors open on the platform side	M2, M6, M7
Central harness integrator	One call per tick, SI units, all-or-nothing validation, clear errors	Section 5.6.7; Section 5.6.12
Train Model owner (Kevin Schillinger)	Commands map one-to-one onto `ControllerCommands`; brake states read as engaged, not commanded	Section 5.6.7; M18
Track Model and Track Controller teams	Authority and commanded speed used as sent; signal light taken directly	M16, M20; Section 5.6.8
Course staff	Requirements traced; 1× and 10× give the same behaviour	REQUIREMENTS.md §2; M11
Module developers (Jonathan Tsang, Ivan Zheng)	One code base serves the SW variant and, later, the HW variant on the Raspberry Pi	Section 5.6.5 (Qt-free core)

5.6.3 Train Controller in the system

In the simulation the harness maps every edge (D005). The signal aspect ahead comes straight from the Track Model and bypasses the Train Model (M20). It receives the Track Signal through the Train Model, which passes it through from the Track Model. The same logic drives Green and Red Line trains.

5.6.4 Use cases

Senders on the left, receivers on the right; the use cases read top to bottom. Automatic behaviours (regulating speed, the authority stop, failure detection, the station stop) have no bubble, because no user starts them; they sit inside Send power command and Send brake commands, and run in the per-tick sequence in Section 5.6.8. Vital use cases are in orange.

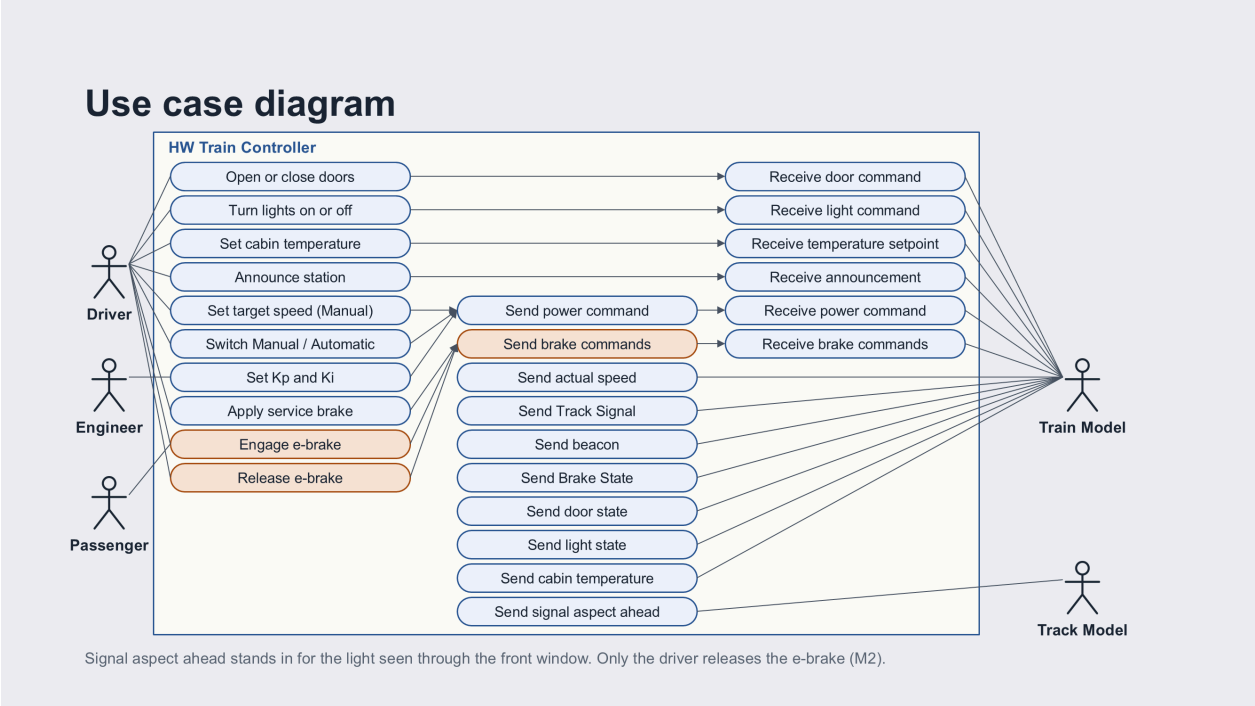

Figure 137. Train Controller (SW and HW): use case diagram

Signal aspect ahead stands in for the light seen through the front window. Only the driver releases the e-brake (M2).

Each description below is followed by the sequence diagram of the use case a user starts. Each command reaches the Train Model on the next tick, when the harness steps the fleet and maps its commands (D005).

Use case 1 — Set target speed (Manual)

System	HW Train Controller
Use case	Set target speed (Manual)
Actors	Driver
Description	• The driver turns the speed dial in 1 mph steps; a press in the bottom gap reads 0 (M14). • Accepted only when the driver is signed in, the train is in Manual, authority is above 0 and no failure is latched. • The target is capped at the commanded speed, the only cap; when the commanded speed drops, the target drops with it (M1). • Each tick the PI law drives power toward the target.

Data	Input: dial value (mph) Output: target speed; power command (W) each tick
Stimulus	The driver moves the speed dial.
Response	The train accelerates or slows to the target; the dial shows the new setpoint.

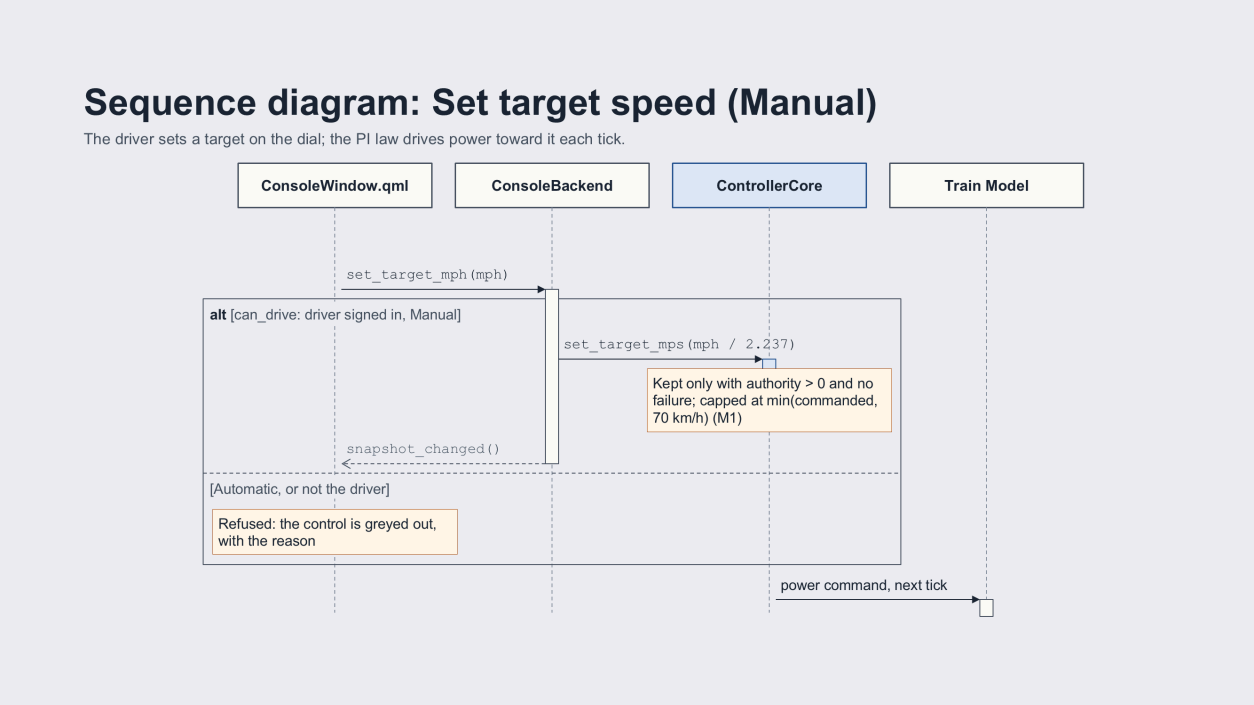

Figure 138. Sequence diagram: Set target speed (Manual)

Use case 2 — Switch Manual / Automatic

System	HW Train Controller
Use case	Switch Manual / Automatic
Actors	Driver

Description	• The signed-in operator picks Manual or Automatic for the selected train. • Allowed with a failure latched, so the driver can reach Manual to try the train; the failure still holds the target at 0 (M3). • Automatic: the target follows the commanded speed, and the cab controls lock except the emergency brake and the mode switch. • Manual: the driver's own target applies (use case 1).
Data	Input: mode selection Output: control mode; which cab controls are enabled
Stimulus	The driver selects a mode.
Response	The mode changes, and controls that are no longer available are disabled.

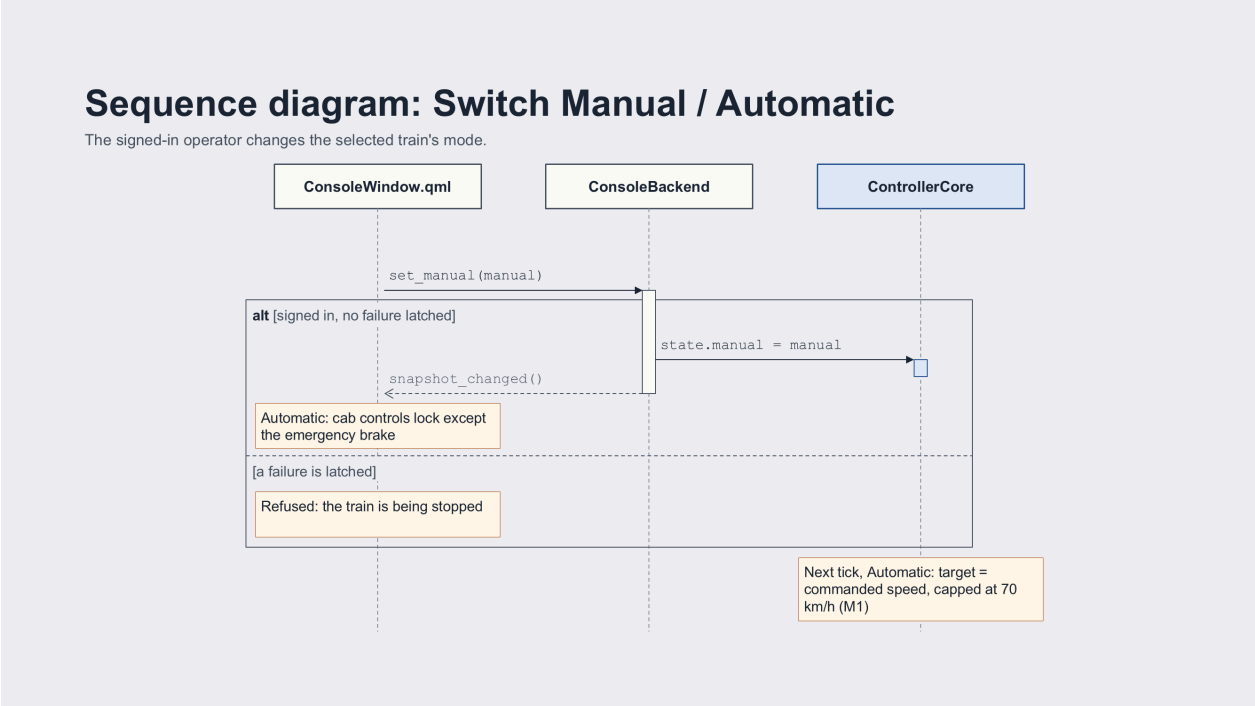

Figure 139. Sequence diagram: Switch Manual / Automatic

Use case 3 — Apply service brake

System	HW Train Controller
Use case	Apply service brake
Actors	Driver

Description	• In Manual, the driver toggles a service brake request. • Also allowed under a brake failure, so the driver can try the brake (M4). • While it is applied, traction is cut to 0 and the integrator holds. • The Train Model reports back whether the brake is actually applied (Brake State).
Data	Input: service brake press Output: brake command [emergency, service] with service on; power 0 W
Stimulus	The driver presses the service brake.
Response	The train slows at the service rate (1.2 m/s^2 on the data sheet) until the driver releases it.

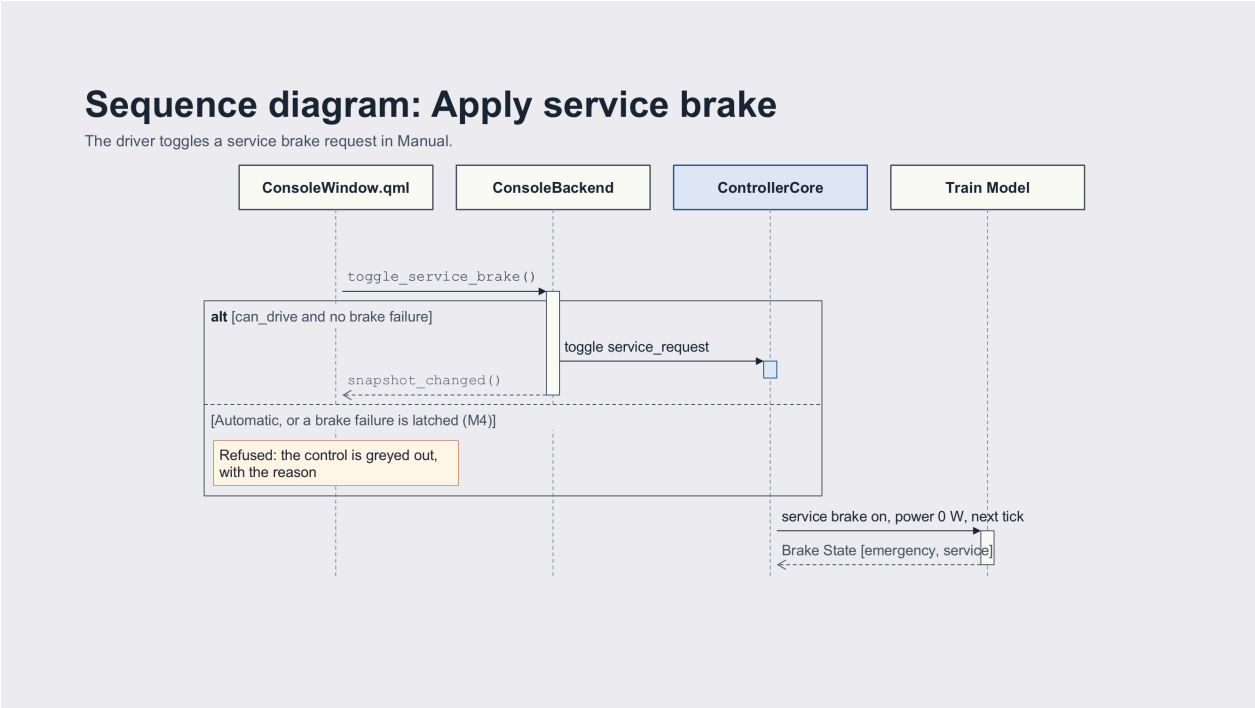

Figure 140. Sequence diagram: Apply service brake

Use case 4 — Doors, lights, temperature, announce

System	HW Train Controller
Use case	Doors, lights, temperature, announce
Actors	Driver (Manual only)

Description	• Doors open only at a stand (under 0.1 m/s), and at a station only on the platform side from the beacon; closing is always allowed (M6). • Interior and exterior lights toggle independently. • Cabin temperature setpoint from 60 to 80 °F, sent to the Train Model in °C. • Announce repeats the last automatic announcement, held 5 s of system time; the Train Model plays it (M7).
Data	Input: tile presses, thermostat (°F) Output: door, light, temperature setpoint and announcement commands
Stimulus	The driver presses a tile or sets the thermostat.
Response	The command goes out; tiles show the state the Train Model reports, and a tile that is not available now is greyed out.

Open or close doors:

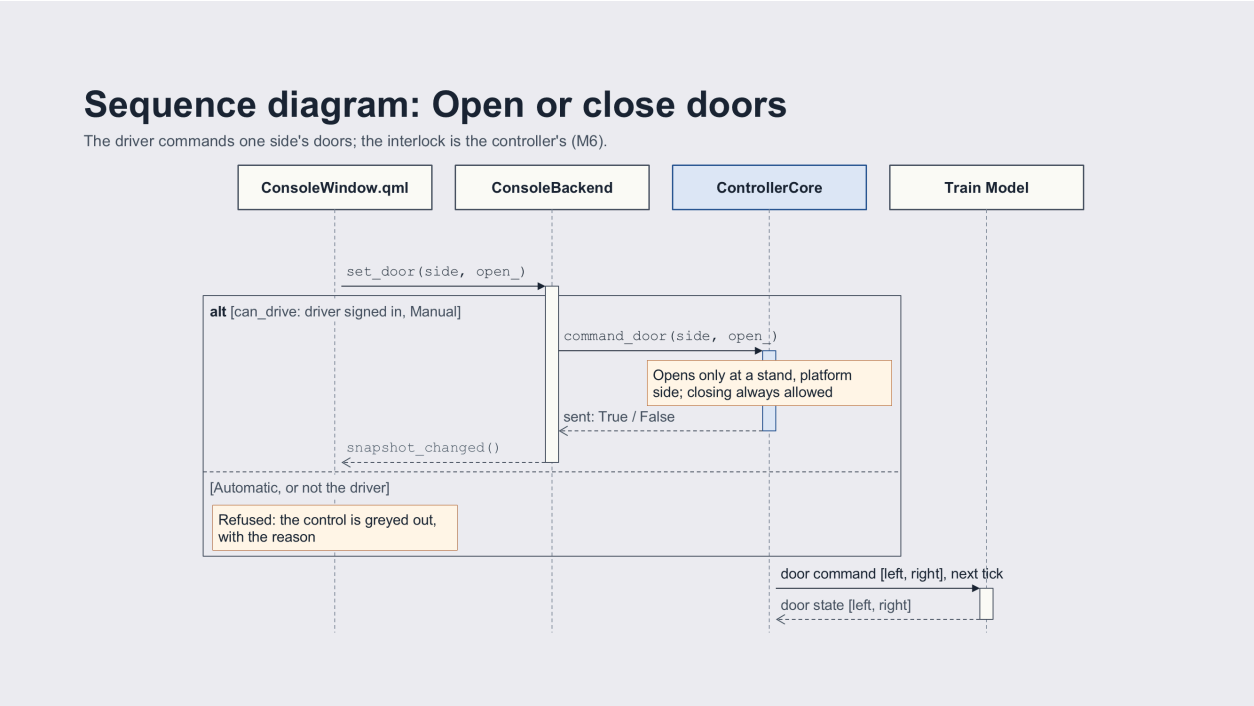

Figure 141. Sequence diagram: Open or close doors

Turn lights on or off:

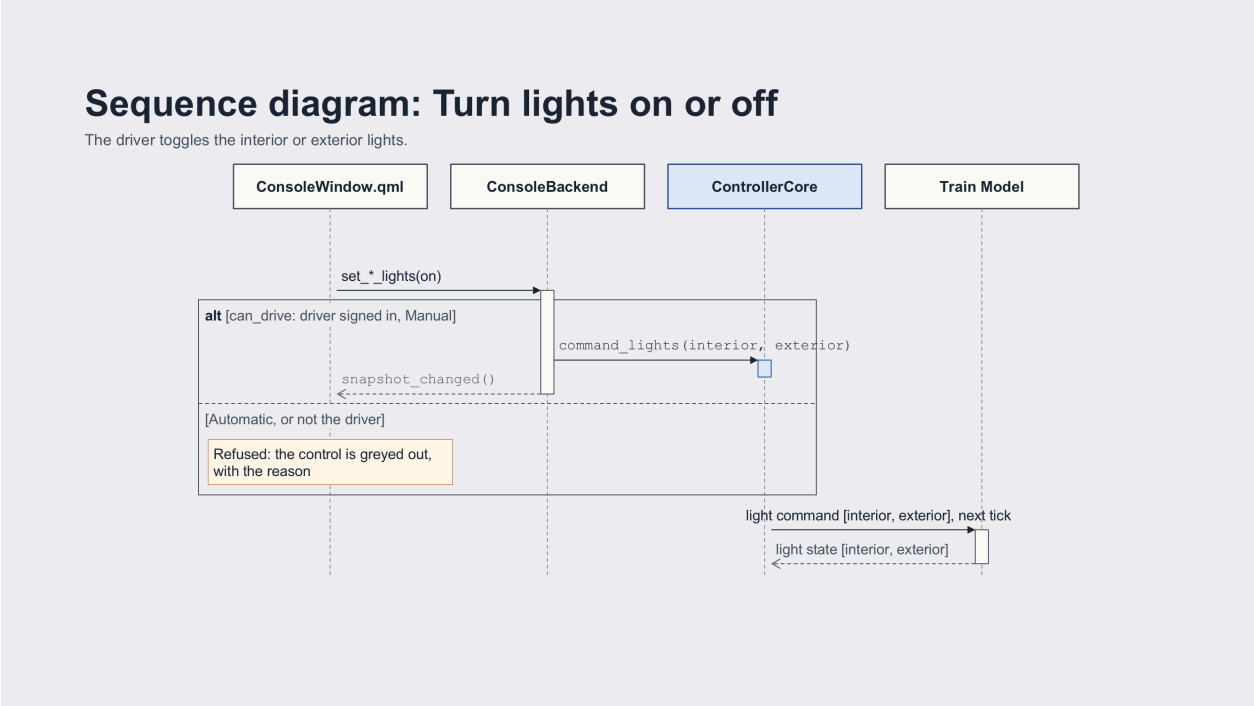

Figure 142. Sequence diagram: Turn lights on or off

Set cabin temperature:

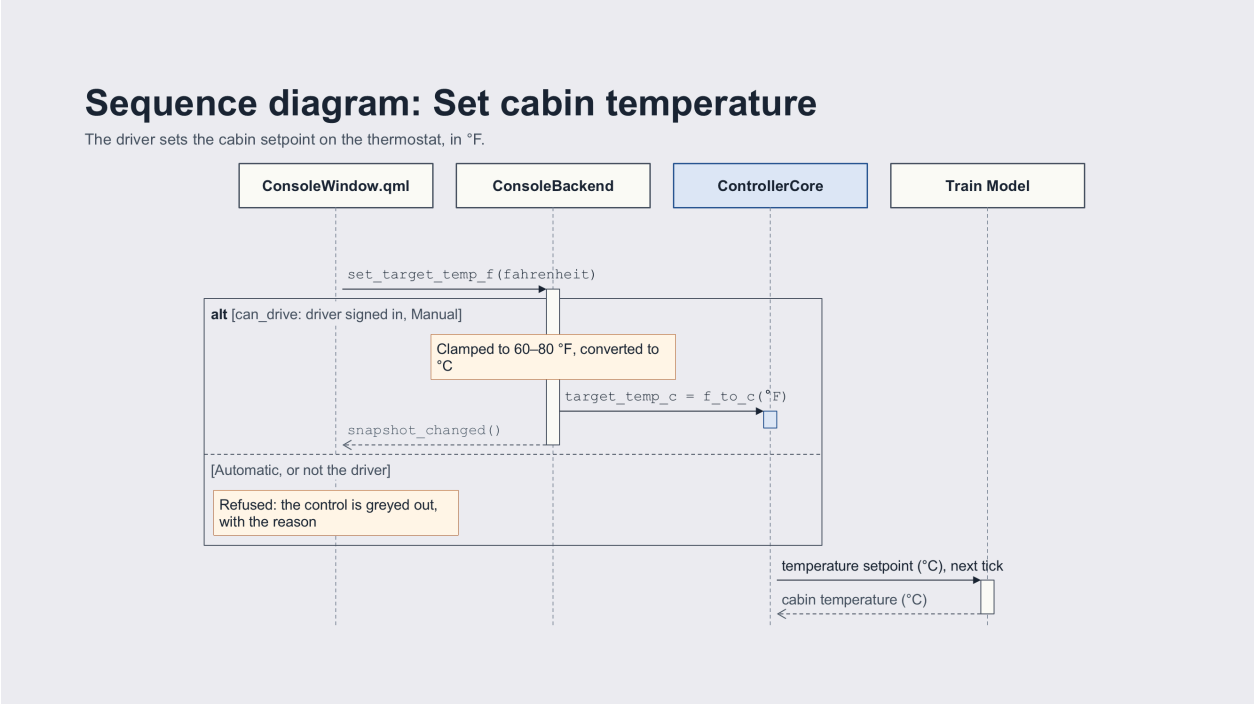

Figure 143. Sequence diagram: Set cabin temperature

Announce station:

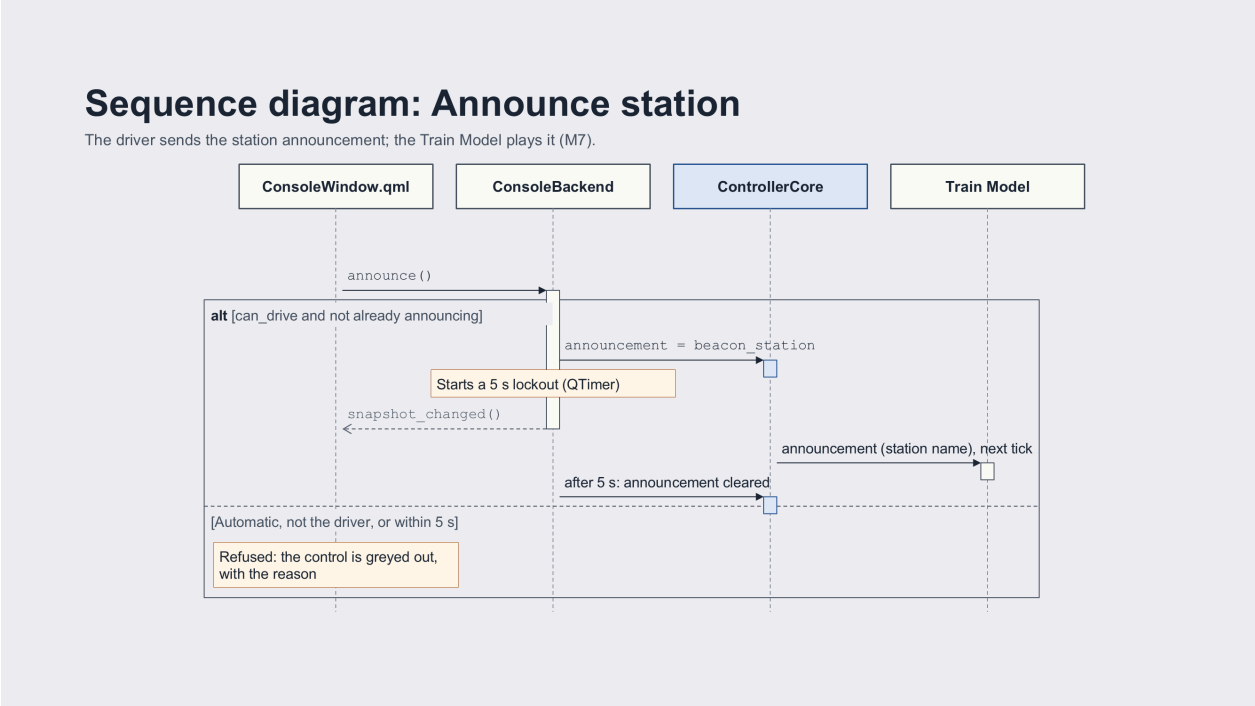

Figure 144. Sequence diagram: Announce station

Use case 5 — Engage the emergency brake

System	HW Train Controller
Use case	Engage the emergency brake
Actors	Driver; Passenger (through the Train Model)
Description	• The driver engages it in any mode; it acts at once and latches. • A passenger pull arrives as Brake State from the Train Model and latches it too. • While it is applied, traction is cut to 0. • Only the driver can release it (use case 6, M2).
Data	Input: emergency brake button; Brake State [emergency] Output: brake command [emergency, service] with emergency on; power 0 W
Stimulus	The driver presses the emergency brake, or a passenger pulls it.
Response	The train stops on the emergency brake (2.73 m/s^2 on the data sheet); the console shows it engaged.

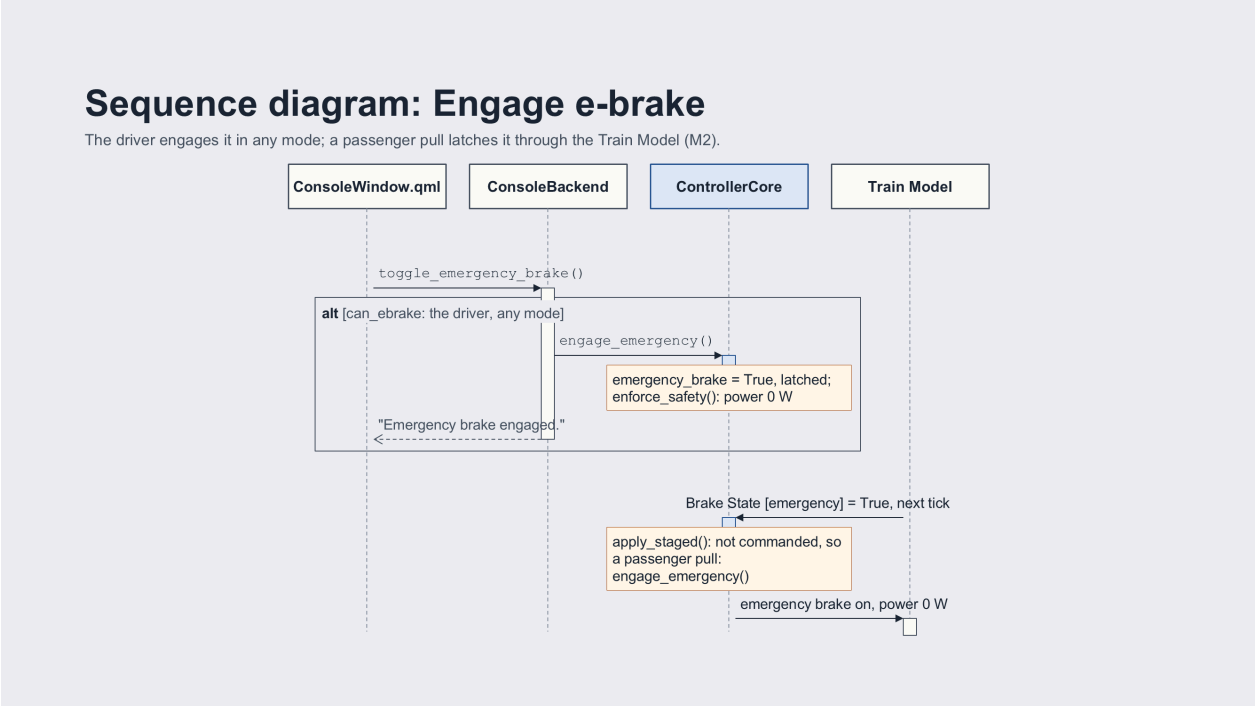

Figure 145. Sequence diagram: Engage e-brake

Use case 6 — Release the emergency brake

System	HW Train Controller
Use case	Release the emergency brake
Actors	Driver
Description	• Only the driver releases it (M2); passengers and the engineer cannot. • Only below 0.1 m/s: with no failure latched, in either mode; with one latched, only in Manual, so the driver can try the train (M2, M3). • A Train Model report of off never releases it. • A refused release says why: still moving, or which failure is active.
Data	Input: emergency brake button Output: brake command with emergency off; brake message
Stimulus	The driver presses the emergency brake while it is engaged.
Response	The brake releases, or the console says why it cannot.

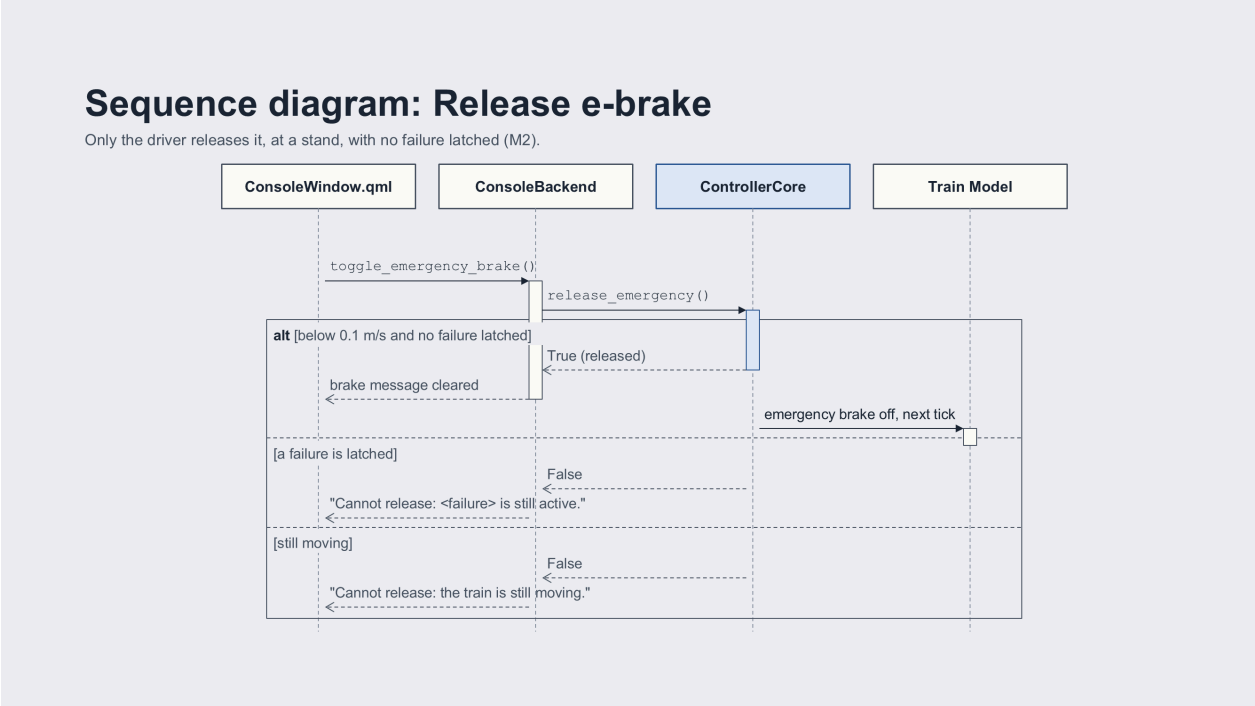

Figure 146. Sequence diagram: Release e-brake

Use case 7 — Commission Kp and Ki

System	HW Train Controller
Use case	Commission Kp and Ki
Actors	Engineer
Description	• The engineer signs in on the selected train; refused if it is already commissioned. • Kp and Ki go in the gains pop-up (defaults 400,000 W per m/s and 8,000 W per m); they must be numbers, and negatives become 0. • The gains are fixed for the run, and the console hands back to the driver after 1.5 s. • Until then the train commands 0 W, but its brakes and safety path still work.
Data	Input: Kp, Ki Output: a commissioned train; a confirmation message
Stimulus	The engineer commits the gains.
Response	The pop-up confirms "Set to Kp …, Ki …. Fixed for this run." and the train can now be driven.

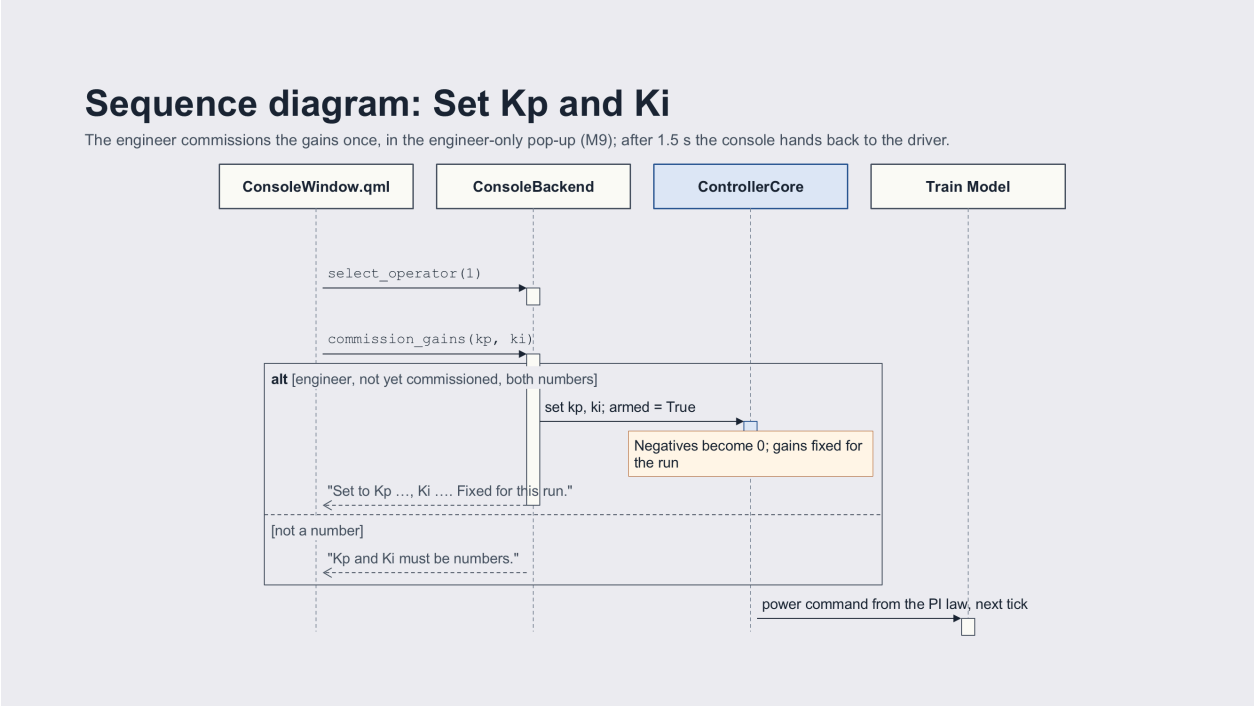

Figure 147. Sequence diagram: Set Kp and Ki

— End of document —

5.6.5 Train Controller architecture (component view)

internal parts, all under `Train-Ctrl_HW/`:

Part	File	Imports Qt	Role
Fleet	`train_ctrl/fleet.py`	No	`TrainControllerFleet`: add, remove, get, `step_all(dt, inputs_by_id, time_s)`
Controller core	`train_ctrl/control.py`	No	`ControllerCore`: PI law, safety path, failure inference, station stop, lights, announcements
Input reader	`train_ctrl/inputs.py`	No	`read_inputs`, `EVERY_TICK`: validates one train's inputs
State and outputs	`train_ctrl/state.py`	No	`State`, `TrainModelCommands`
Errors	`train_ctrl/errors.py`	No	Exception hierarchy
Constants, units	`train_ctrl/constants.py`, `units.py`	No	Tick, limits, thresholds, conversions

Part	File	Imports Qt	Role
Bench simulation	`train_ctrl/bench_sim.py`	No	`SimTrain`: stand-in Train Model and Track Model for the bench only; not part of the controller (M0)
Console bridge	`train_ctrl/backend.py`	Yes	`ConsoleBackend`: driver, engineer and bench actions
App	`train_ctrl/app.py`, `main.py`	Yes	Opens the console and bench windows; `--check` self-test
Views	`ui_parts/*.qml`	Yes	Console and bench screens, built on the shared `ui/` library

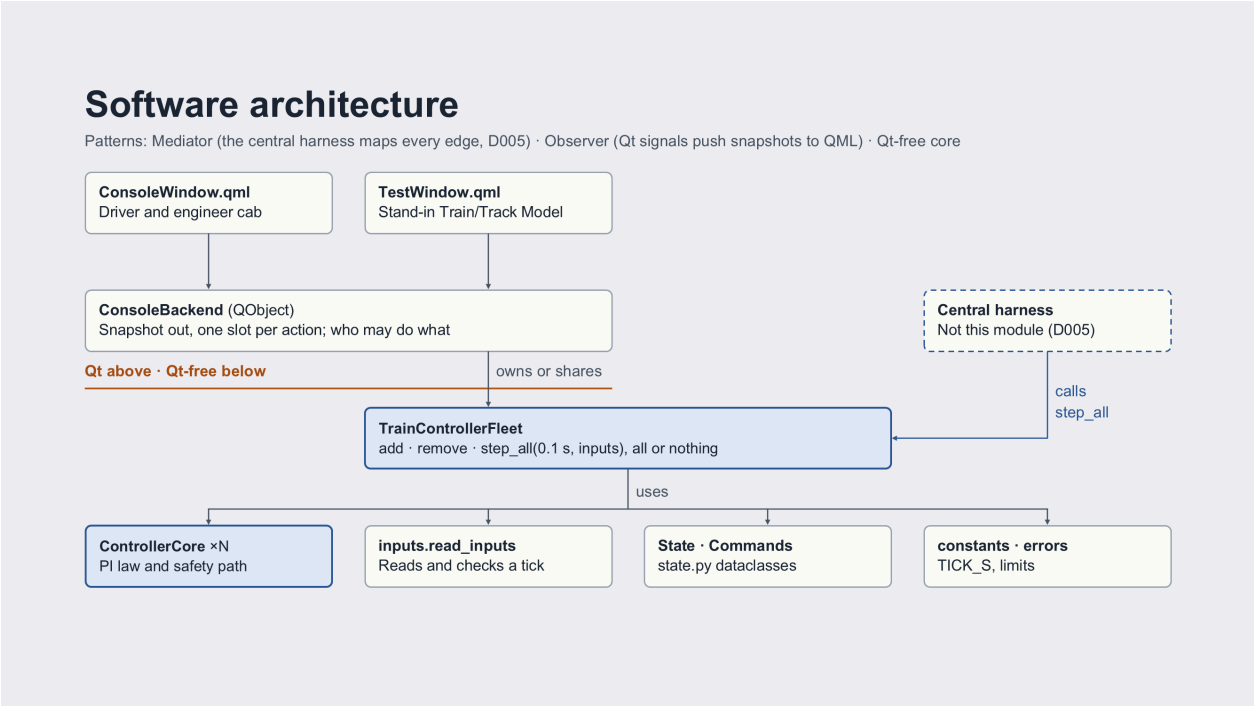

Figure 148. Train Controller (SW and HW) architecture (component view)

Either the harness or the bench drives the fleet, never both, and the controller cannot tell which (M0).

- **More Information:** `HARNESS_INTEGRATION.md` gives the call sequence and import path. `import train_ctrl` imports no Qt and does not re-export `bench_sim`.

  Design choices and rationale:

Choice	Rationale
The controller core imports no Qt; the console is a separate layer	The core runs unchanged with or without a window, so the same code serves the SW variant and, later, the HW variant on the Raspberry Pi, whose window runs on the host PC (D021, D027)
One fleet, stepped once per tick through `step_all`	Mirrors the Train Model's fleet, so one harness loop drives both modules
All-or-nothing input validation, every required input every tick	A bad or stale value never moves one train and not another, and a stale brake state never reads as a brake failure (NFR-003/004)
The commanded speed is the only cap (M1)	The controller cannot know the true speed limit; keeping under it is the wayside's, through the commanded speed
Failures are inferred from returned signals (M3)	Nothing reports failures to the controller; it sees only what the Train Model sends back
Authority is used as given, never counted down (M16)	The Track Model updates the count after each block change the Train Model reports, so the controller keeps no count of its own
Perfect movement assumed; no mass or grade model (M19)	Physics is the Train Model's (D009); the speed error absorbs every outside force
Fixed 0.1 s tick; time from the harness clock (M11)	The same inputs give the same behaviour at 1× and 10× (D006, D017)
The test bench is outside the controller and feeds it through `step_all` (M0)	The controller cannot tell bench from harness, so what is tested on the bench is what runs integrated
Gains set once by the engineer, never defaulted (M9)	A new train applies no power until a person has commissioned it (D024)
Safety rules enforced in the backend, not only in the view	A control that is disabled on screen but called anyway still refuses

5.6.6 Class diagrams

Controller core:

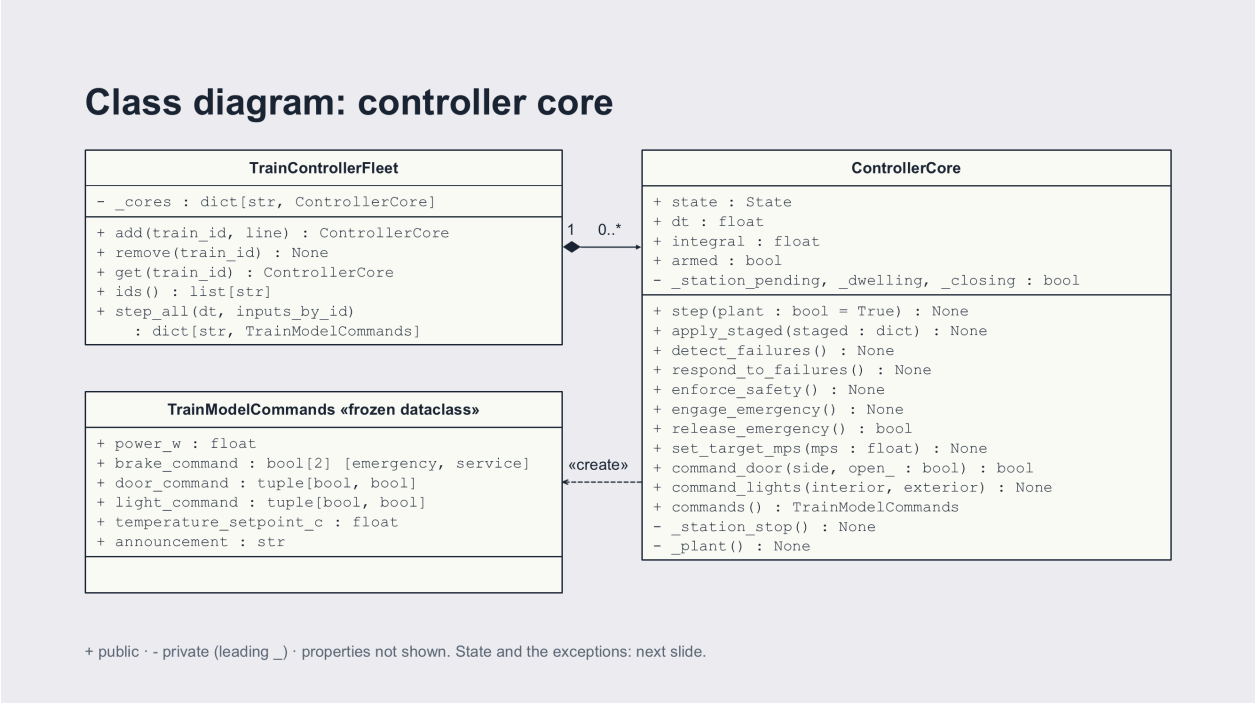

Figure 149. Class diagram: controller core

State and exceptions:

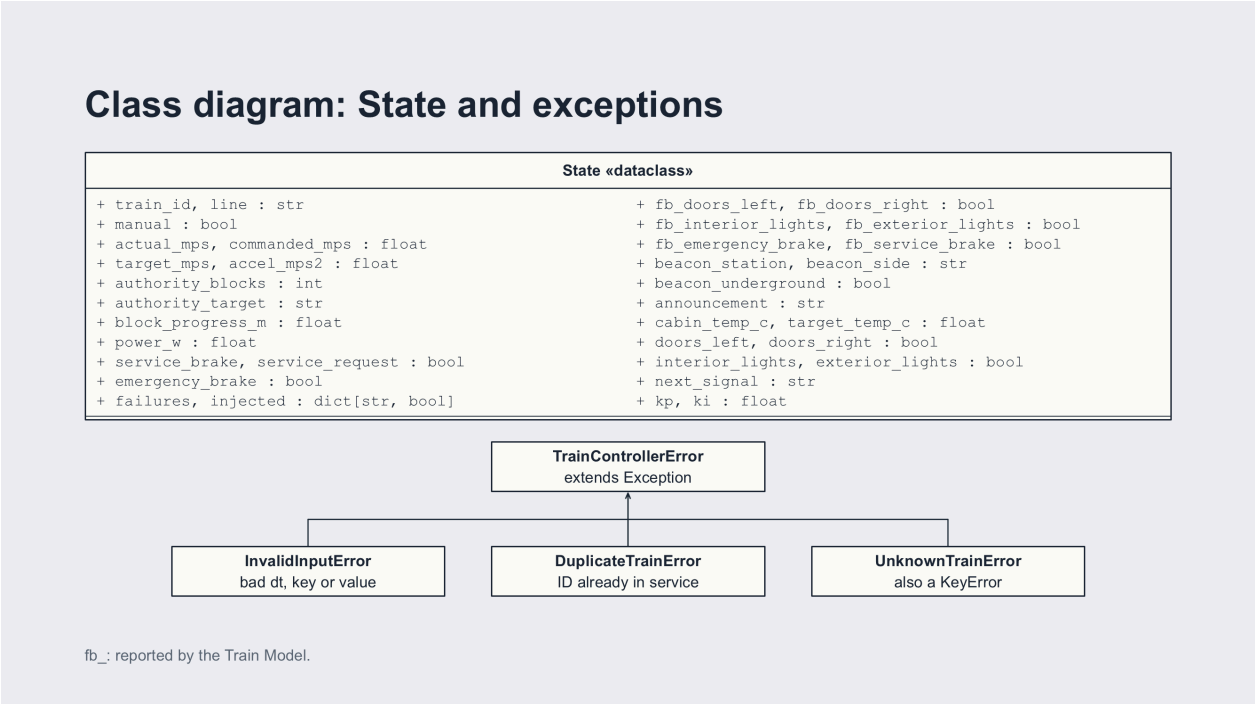

Figure 150. Class diagram: State and exceptions

Console:

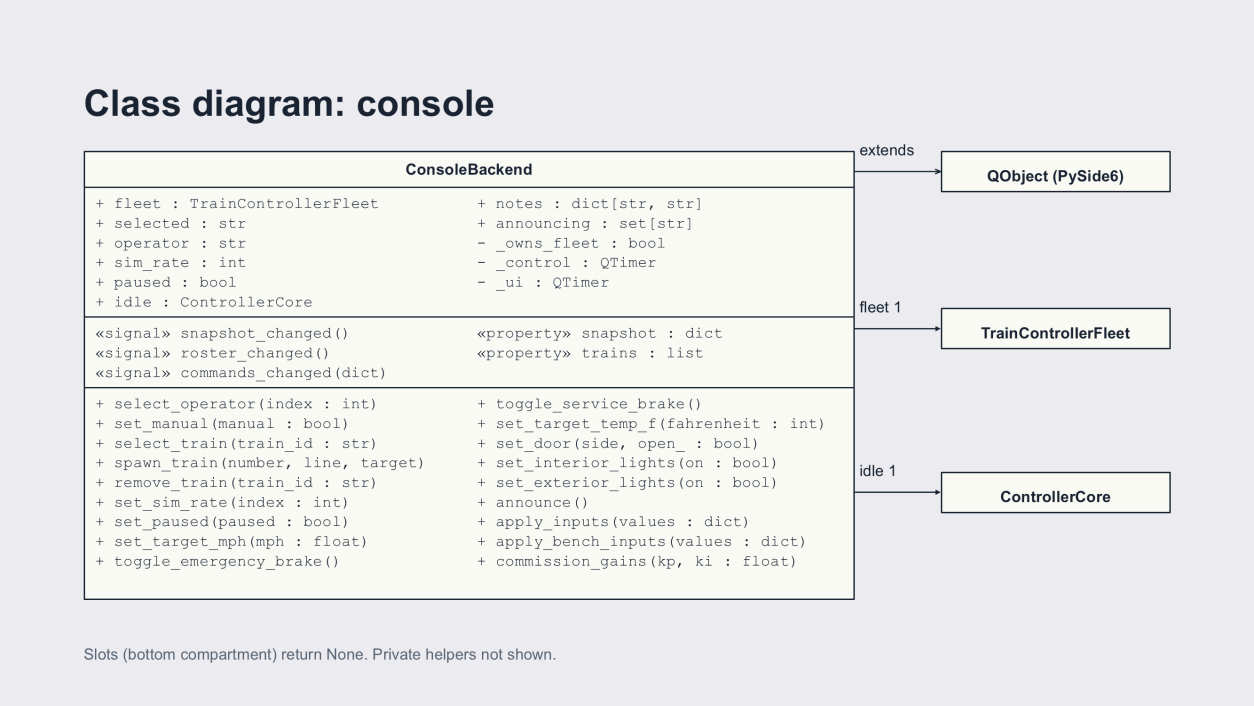

Figure 151. Class diagram: console

Boundary types this module **produces**: `TrainModelCommands` (one per train per tick). Boundary type it **consumes**: a plain `dict` per train with the keys in Section 5.6.7, checked by `read_inputs`. It defines no `typing.Protocol` yet (Section 5.6.12).

Class cards. One card per class: what it is for, what it holds, what it does and with whom.

TrainControllerFleet (`fleet.py`) — Holds one controller per train in service, keyed by train ID. The harness's only entry point.

Attribute	Description
cores	Each train's `ControllerCore`, by train ID, in the order added

Responsibility	Collaborator
Add a train's controller when it enters service	Central harness, ControllerCore
Remove it when it leaves	Central harness
Return one train's controller	ConsoleBackend
Read every train's inputs, then step every train, or none	Central harness, `read_inputs`, ControllerCore
Return each train's commands	Central harness, TrainModelCommands

ControllerCore (`control.py`) — One train's controller: speed law, safety path, failure inference, station stop, lights and announcements. Imports no Qt.

Attribute	Description
state	The train's `State`
dt	The fixed 0.1 s step
integral	The PI law's accumulated speed error
armed	Whether the engineer has commissioned the gains
last_announcement	The last automatic announcement, for the driver's repeat
Station-stop flags	A station beacon heard and not yet served, its name, arrived, held with doors open, closing
Failures braked for	Failures that have already latched the emergency brake, so it is not engaged again after the driver's release

Responsibility	Collaborator
Apply this tick's inputs	TrainControllerFleet
Run one control period: failures, lights, target, station stop, PI law, safety	TrainControllerFleet, State
Detect failures from returned signals	State
Respond to a failure: latch the emergency brake, target 0	State
Enforce safety: service brake at authority 0, traction cut under braking	State
Engage and release the emergency brake	ConsoleBackend
Set the Manual target	ConsoleBackend
Say whether a door side may open: at a stand, and only the platform side at a station (M6)	ConsoleBackend
Command doors and lights	ConsoleBackend

Responsibility	Collaborator
Repeat the last announcement	ConsoleBackend
Clear a failure the bench injected and cleared. Bench only: the one place the bench reaches into the controller (M0); to be removed once the controller detects resolution itself (#23)	ConsoleBackend
Produce this tick's commands	TrainModelCommands

State (`state.py`) — Everything the console shows or the control law needs, for one train.

Attribute	Description
train_id, line, manual	Identity, line and mode
actual_mps, commanded_mps, target_mps	Speeds, m/s
time_s	System time as last received
authority_blocks	Blocks the train may still enter (D013)
power_w, service_brake, service_request, emergency_brake	What the controller commands, and the driver's service brake request
failures	Engine, brake and signal pickup, as detected
fb_*	Doors, lights and brakes as the Train Model reports them
beacon_station, beacon_side, beacon_underground	The last beacon
announcement	The announcement while it plays, else empty
cabin_temp_c, target_temp_c	Cabin temperature and setpoint, °C
doors_left, doors_right, interior_lights, exterior_lights	Door and light commands
next_signal	Signal light ahead, display only
kp, ki	Gains

Responsibility	Collaborator
Hold one train's values	ControllerCore, ConsoleBackend

TrainModelCommands (`state.py`) — One tick's commands to the Train Model, one field per truth signal; frozen. Fields in Section 5.6.7.

Responsibility	Collaborator
Carry the commands to the harness, which maps them onto the Train Model's types	Central harness

ConsoleBackend (`backend.py`) — The console's Qt bridge: one snapshot out, one slot per operator action. Every rule about who may do what lives here; the view only mirrors it.

Attribute	Description
fleet	The fleet shown: its own, or the harness's
selected	The selected train
operator	Driver, engineer or nobody
sim_rate, paused	1× or 10×, and Pause, for a fleet it owns
sims	The bench's `SimTrain` per train
time_s	The clock shown

Responsibility	Collaborator
Sign the driver or engineer in	Driver, Engineer
Commission the gains once	Engineer, ControllerCore
Switch Manual and Automatic	Driver, ControllerCore
Set speed, service brake, temperature, doors, lights, announcement repeat (Manual only)	Driver, ControllerCore
Engage and release the emergency brake (any mode)	Driver, ControllerCore
Step the fleet on the bench, when it owns the fleet	TrainControllerFleet, SimTrain
Publish a snapshot to the views	QML views

SimTrain (`bench_sim.py`, bench only) — The bench's stand-in for the Train Model and Track Model. Not part of the controller (M0).

Attribute	Description
Track Model side	Commanded speed, authority, beacon, signal light, at station
Train Model side	Actual speed, acceleration, cabin temperature, brakes, doors, lights
injected	Failures injected on the bench

Responsibility	Collaborator
Supply the controller's inputs, as the harness would	ConsoleBackend, TrainControllerFleet
Apply the bench's edits and Block change	ConsoleBackend
Step its toy physics from the controller's commands	TrainModelCommands

Errors (`errors.py`): `TrainControllerError` and its subclasses `InvalidInputError`, `DuplicateTrainError`, `UnknownTrainError` (Section 5.6.7).

- **More Information:** `armed` is the commissioning flag; the harness must not set it (`HARNESS_INTEGRATION.md`, Commissioning). Per D005 the module defines only its own types and never imports the Train Model's; the harness maps `TrainModelCommands` onto the Train Model's `ControllerCommands`, and `ControllerOutputs` onto this module's input dict.

5.6.7 Interface

signal dictionary:

Sender	Receiver	Signal	Data items
Central harness	Train Controller	Tick	`dt` (0.1 s), `time_s`
Train Model	Train Controller	Actual speed	Speed, m/s
Train Model	Train Controller	Brake state	Emergency brake engaged, service brake engaged
Train Model	Train Controller	Door state	Left, right
Train Model	Train Controller	Light state	Interior, exterior
Train Model	Train Controller	Cabin temperature	°C

Sender	Receiver	Signal	Data items
Train Model	Train Controller	Track signal (passed through)	Commanded speed, authority (blocks)
Train Model	Train Controller	Beacon	Station name, platform side, underground
Track Model	Train Controller	Signal aspect ahead	Red, Yellow, Green, Super Green
Train Controller	Train Model	Power command	Power, W
Train Controller	Train Model	Brake command	Emergency, service
Train Controller	Train Model	Door command	Left, right
Train Controller	Train Model	Light command	Interior, exterior
Train Controller	Train Model	Temperature setpoint	°C
Train Controller	Train Model	Announcement	Text

Call: `fleet.step_all(TICK_S, inputs_by_id, time_s)` once per tick, one input dict per train in service. All values SI (D002).

Clock inputs (from the harness, D017):

Argument	Unit	Rule
`dt`	s	Must equal `TICK_S` (0.1); anything else raises `InvalidInputError`
`time_s`	s since midnight of day 1, unwrapped	Finite, ≥ 0

Inputs per train, required every tick (`EVERY_TICK`) and their source:

Key	Unit / shape	Truth signal	Harness maps from (`ControllerOutputs`)
`actual_speed`	m/s, signed	`actual-speed`	`actual_speed_mps`
`commanded_speed`	m/s (negative clamped to 0)	`track-signal`	`commanded_speed_mps` (passed through; 0 under pickup failure)
`authority_blocks`	int, blocks (negative clamped to 0)	`track-signal` (D013)	`authority_blocks` (passed through; 0 under pickup failure or before the first step)
`emergency_brake_state`	bool, engaged	`brake-state` [0]	`emergency_brake_active` (commanded or passenger)
`service_brake_state`	bool, engaged	`brake-state` [1]	`service_brake_active` (false while the e-brake is engaged or the brakes have failed)
`door_state`	`[left, right]`	`door-state`	`door_left_open`, `door_right_open`
`light_state`	`[interior, exterior]`	`light-state`	`interior_lights_on`, `exterior_lights_on`
`cabin_temperature`	°C	`cabin-temperature`	`cabin_temp_c`

Inputs per train, optional (left out = last value kept):

Key	Unit / shape	Truth signal	Harness maps from
`beacon_station`	str	`beacon`	`ControllerOutputs.beacon.station_name`, only when `beacon` is not `None`
`beacon_side`	`"L"`, `"R"`, `"LR"`	`beacon`	`beacon.platform_side`
`beacon_underground`	bool	`beacon`	`beacon.underground`
`signal_light_ahead`	`"RED"`, `"YELLOW"`, `"GREEN"`, `"SUPER GREEN"`	`signal-aspect-ahead`	`TrackModelOutputs.signal_seen[train_id]`, when not `None` (display only, M20; not mapped yet, #27)

Not used: `ControllerOutputs.speed_limit_mps` (M1, #24).

Outputs per train (`TrainModelCommands`), all to the Train Model:

Field	Unit / shape	Truth signal	Harness maps to (`ControllerCommands`)
`power_w`	W, 0–480,000	`power-command`	`power_cmd_w`
`brake_command`	`[emergency, service]` bool	`emergency-brake-command`, `service-brake-command`	`emergency_brake`, `service_brake`
`door_command`	`(left, right)`	`door-command`	`door_left_open`, `door_right_open`
`light_command`	`(interior, exterior)`	`light-command`	`interior_lights`, `exterior_lights`
`temperature_setpoint_c`	°C	`temperature-setpoint`	`temp_setpoint_c`
`announcement`	str, empty when none	`announcement`	`announcement`

Code today sends two bools (`service_brake`, `emergency_brake`); to change to `brake_command`.

Errors (all derive from `TrainControllerError`; a failed `step_all` changes no train):

Exception	When
`DuplicateTrainError`	`add` with an ID already in the fleet
`UnknownTrainError` (also `KeyError`)	`get`, `remove` or `step_all` with an unknown ID
`TrainControllerError`	A train in service has no inputs this tick
`InvalidInputError`	Bad `dt` or `time_s`; a missing `EVERY_TICK` key; a value that cannot be read as its type

Lifecycle: the harness calls `fleet.add(train_id, line)` when a train enters service and `fleet.remove(train_id)` when it leaves. Train IDs are opaque strings.

- **More Information:** The edge mapping functions are the harness's (D005); the right-hand columns are what they must do, checked against the Train Model's `interface.py` and `module-interface.md` on `development` `de7a98f`. What the Train Model's side means for this module:

- **One tick of latency.** The Train Model's outputs describe the end of its tick, so the controller sees the result of its commands on its next tick. M4's "next tick's service brake state" relies on this.
- **The Train Model rejects** negative power, a non-`bool` on/off value and a non-text announcement, and caps power at 480 kW itself. The controller never sends these (power is clamped 0–480 kW).
- **No speed governor** in the Train Model (D009): the commanded-speed cap is the controller's alone.
- **No traction cut** in the Train Model: the controller sends 0 W under braking (truth `traction-cut-under-braking`).

5.6.8 Interaction: one tick

The controller's part. The whole chain, from the CTC Office to the train, is the representative interaction in Section 3.5.

When `authority_blocks` reaches 3 the controller sends "arriving soon" (also on a rise to 3); at 0 the target is 0 and the service brake is applied while the train moves, as a fail safe (M17). Stopping at the right block depends on the wayside lowering the commanded speed in time. If commanded speed and authority both drop to 0 from 2 or more blocks in one update, the controller infers a signal pickup failure (Section 5.6.10).

One 0.1 s tick under the harness. Inputs come from the Train Model, commands go back to it (D005). All or nothing: any error before the loop leaves every train unchanged. The automatic behaviours (regulating speed, the authority stop, failure detection, the station stop) run inside `step()`.

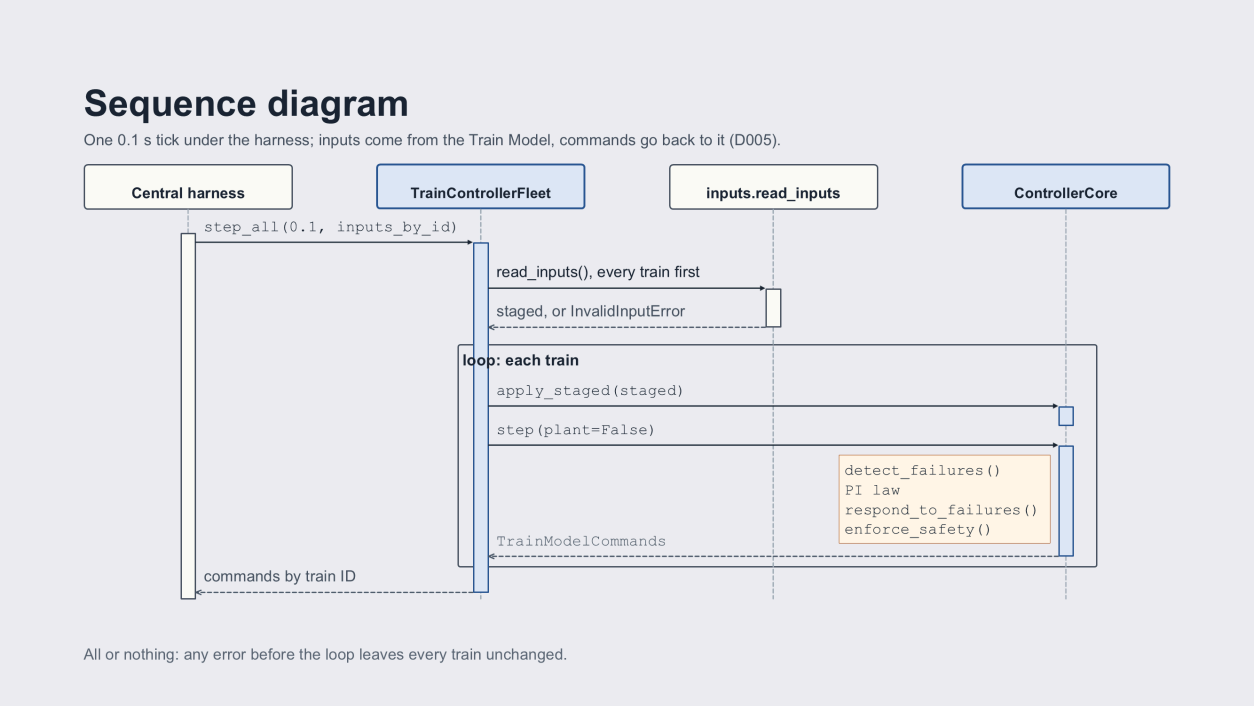

Figure 152. Sequence diagram: one 0.1 s tick

5.6.9 Speed law and safety path

Item	Rule
Target, Automatic	Commanded speed
Target, Manual	Driver's dial setpoint (1 mph steps), capped at the commanded speed: when the commanded speed drops, the setpoint drops with it; when it rises, the setpoint stays (M1). No 70 km/h cap; the dial's 70 km/h scale is display only (M14)
Authority 0	Target 0, and the service brake applied while moving, as a fail safe (M17)
Failure latched	Emergency brake latched once, on detection, and target 0, either mode; mode switching still allowed (M3)
Power	`P = Kp·e + Ki·∫e dt`, e = target − actual speed; clamped 0–480 kW, the integral held at the clamp
Service brake	Applied when the speed is more than 0.05 m/s over the target, on the driver's request, or at a target of 0 until the train stands; the integral holds while braking

Item	Rule
Traction cut	Power 0 while either brake is commanded (truth `traction-cut-under-braking`)
Uncommissioned	Power 0 until the engineer sets Kp, Ki (M9). Defaults offered: Kp 400,000 W per m/s, Ki 8,000 W per m
Emergency brake	Driver engages; a passenger pull (reported state turning on while not commanded) latches it; only the driver releases, at < 0.1 m/s: in either mode with no failure latched; in Manual only with a failure latched, which does not engage it again (M2, M3)
Tick	Fixed 0.1 s; 10× runs more ticks, never a longer step (M11)

5.6.10 Failure inference

Detected only from returned signals (M3); nothing reports failures to the controller.

Failure	Detected when (decided)	Resolved when (decided)
Brake	Last tick commanded the service brake (e-brake off), this tick's `service_brake_state` still off; the service brake stays commanded	Service brake reads applied again when commanded
Engine	**Under review (#28).** Rule decided: power > 0, no brake in the window, speed fallen by more than a threshold over a window a little longer than 2 s, never within the control band. Measured against the Train Model, no window and threshold separate the cases: a dead engine on the flat loses at most 0.10 m/s in 5 s, while a healthy full train climbing +5 % under full power loses up to 0.81 m/s, and the controller has no grade (M19). A replacement rule is undecided	Commanded power gives rising speed again

Failure	Detected when (decided)	Resolved when (decided)
Signal pickup	Commanded speed and authority drop to 0 together from ≥ 2 blocks	A valid non-zero Track Signal resumes

Code today: brake and signal pickup detection as decided. The engine rule is still the mass-based `F = P/v` one, which flags a healthy full train 2.4–2.9 s onto a +2 % or +5 % grade and latches the emergency brake (#28). Resolution is not detected (#23).

While a signal pickup failure is detected, the console shows the commanded speed and authority as "--", not as the zeros that arrive.

5.6.11 Automatic behaviour

Function	Rule
Announcements (M7)	Commands, both modes, each held 5 s of system time: "arriving soon" when authority reaches 3 blocks; "has arrived at *station*" when a station beacon was heard, stopped, authority 0; "is departing from *station*" when doors report closed and the Track Signal resumes. In Manual the driver's press repeats the last one, held 5 s the same way
Doors (M6)	Automatic: open the beacon's platform side on arrival; close when a non-zero Track Signal resumes; depart once both report closed. Manual: either side at a full stop, except at a station (arrived, until it departs), where only the platform side opens
Lights (M8)	On with an underground beacon and at 18:00. Off with a later above-ground beacon, unless it is between 18:00 and 06:00 (leaving underground at night keeps them on), and at 06:00, unless underground. The driver's choice stands until the next transition. A train spawns with nothing on; spawned between 18:00 and 06:00, its lights are set on
Temperature	Driver setpoint 60–80 °F in Manual, sent in °C

5.6.12 Design decisions

The module's position on each system decision of Section 4.

Interface Contract Mechanism

- **ID:** system decision 4.1
- **Context:** The module exposes a plain Python API: `TrainControllerFleet.step_all` taking a `dict` per train and returning `TrainModelCommands`. It has no `interface.py` and no `typing.Protocol`.
- **Options:**
  - `typing.Protocol` in each module's `interface.py`
  - Each module's own boundary dataclasses, harness maps each edge (D005)
  - A shared catalog, `common/interfaces.py` (superseded, D008)
- **Outcome:** follows D005 (own types, harness maps each edge). Adding a Protocol is deferred: under D005 no other module imports our types, so its absence breaks nothing at integration. If adopted, the module adds `train_ctrl/interface.py` declaring the fleet's existing calls, with no behaviour change.

Shared Boundary Data Types

- **ID:** system decision 4.2
- **Context:** Output is a frozen dataclass (`TrainModelCommands`); input is an untyped `dict` keyed by truth signal names, validated at run time by `read_inputs`.
- **Options:**
  - Frozen dataclasses per module
  - Plain dicts with run-time validation (this module's input today)
  - Shared types imported across modules (ruled out by D005)
- **Outcome:** deferred; if typed inputs are chosen, add a frozen `TrainControllerInputs` dataclass with the `EVERY_TICK` and optional keys from Section 5.6.7.

Contract Enforcement in CI

- **ID:** system decision 4.3
- **Context:** The module's checks run locally: `python main.py --check` (offscreen self-test, rule checks, cut-off text scan). It has no `mypy.ini`, no contract tests, and no CI job. `check.py` is timing-sensitive under load (#21).
- **Options:**
  - mypy per module plus contract tests in CI
  - Each module's own self-test, run by hand
  - No enforcement
- **Outcome:** type-checking and contract tests deferred; nothing depends on them at integration. `--check` stays the module's gate.

Stub-First Development

- **ID:** system decision 4.4
- **Context:** No stub with `STUB = True` was committed; the real core was built directly. Its test bench (`bench_sim.py`) stands in for the Train Model and Track Model instead, and `harness_sim.py` stands in for the harness. Both are for testing only.
- **Options:**
  - Stub at the real import path from Day 1
  - Real implementation with a local stand-in for neighbours (this module)
  - No stand-ins; integrate only when every module is real
- **Outcome:** as recorded for the system in Section 4.

Contracts Package Ownership

- **ID:** system decision 4.5
- **Context:** Under D005 each module owns its own boundary types and the harness owns the edge mappings. This module's types are owned by Jonathan Tsang and Ivan Zheng, in `Train-Ctrl_HW/train_ctrl/`. The truth contract shard `truth/modules/train-controller/train-controller.md` is still a placeholder with no owner.
- **Options:**
  - Each module owns its types; harness owns mappings (D005)
  - One owner for a shared contracts package
  - Shared ownership of a shared package
- **Outcome:** as recorded for the system in Section 4.

External Input Validation

- **ID:** system decision 4.6
- **Context:** Validation is at the boundary, all or nothing: every train's inputs are read before any train steps; one bad value raises `InvalidInputError` and leaves the fleet unchanged (NFR-003/004). Every required key is needed every tick, so a stale brake state can never read as a brake failure. Unknown keys are ignored. The console cannot produce bad input; a refused bench entry shows a "Not sent" note.
- **Options:**
  - Validate at each module's boundary, reject the whole tick (this module)
  - Validate in the harness's mapping functions only
  - Trust inputs
- **Outcome:** keeps boundary validation whatever is chosen, because failure inference depends on faithful inputs.

Knowledge Capture and Decision Logging

- **ID:** system decision 4.7
- **Context:** Team decisions live in the truth store (`truth/decisions/`, proposals through `truth/_inbox/`). Module rules live in `Train-Ctrl_HW/REQUIREMENTS.md` (M0–M20, with code status and revision table), open items in `OPEN_ISSUES.md`, changes in `CHANGELOG.md`.
- **Options:**
  - Truth store for team decisions, module docs for module rules (this module)
  - Everything in the truth store
- **Outcome:** as recorded for the system in Section 4.
- **More Information:** The module's rules are not yet in truth (#19).

Rejected Tooling — Summary Table

Tool	Why it was rejected
`common/interfaces.py` (`ISwTrainController`, `TrainState`)	Stale: authority in metres, failures reported by the Train Model; superseded by D008
Trapezoidal PI integration from the course deck	Not taken up for now; the code sums rectangles at the fixed 0.1 s tick. To consider later

5.6.13 Open items

- **Process and USB link — deferred.** D005 runs each module as its own process exchanging newline-delimited JSON; D027 runs the HW core on a Raspberry Pi over USB serial. The module today is an in-process Python API with no message wrapper and no Pi build.
- **Passenger brake release — planned.** Truth `passenger-emergency-brake` expects a one-shot `passenger-brake-release` command; the module's outputs have none yet (#25). The Train Model's `module-interface.md` still says how the controller releases a pull is not settled, and only a test call clears its latch.
- **Door interlock.** The Train Model's `ControllerCommands` table says door commands are obeyed only at 0 mph, while its `ControllerOutputs` table and M6 say the interlock is commented out. The controller opens doors only at a stand either way; which the Train Model does is Kevin's to confirm.
- **Failure resolution** is not detected yet (M3, M5, #23). Engine resolution cannot be seen while the emergency brake is on; once the driver releases it in Manual at a stand, a repaired service brake can be seen.
- **Engine failure rule** (M19, #28): measured against the Train Model, no window and threshold separate a dead engine on the flat from a healthy full train climbing +5 %, because the controller has no grade. The code still uses the mass-based `F = P/v` rule, which flags a healthy full train 2.4–2.9 s onto a +2 % or +5 % grade.

- **E-brake under a failure in Automatic** (#34): after a Manual release, switching to Automatic leaves the emergency brake off; the train is held by the 0 target. Keep it, or re-engage it, after testing.
- **Signal light ahead** not yet mapped from the Track Model (M20, #27).
- **Spelling of the fourth signal aspect** (owner: Track Model team): the code uses `"SUPER GREEN"`, truth uses `SUPER_GREEN`, and `inputs.py` ignores unknown aspects, so a mismatch drops every super-green light.
- **To review (noted, not yet assessed against the requirements):** spawning a train from the bench breaks a harness fleet, as the harness does not know that train; this module's exception classes differ from `TrainModelFleet`'s (`HARNESS_INTEGRATION.md`); the dial cannot set a commanded speed above 43.5 mph (#29).
- **Truth shards** for the Train Controller are placeholders; the module's rules are not in truth or the SRS (#19).
- **Recorded deviations from truth:** no separate speed limit input (M1, against D009 and `signals/track-info.md`); acceleration shown in m/s^2 (M12).

---

6. Appendixes

6.1 Figure Register

Every figure in this document, once. The owner keeps its source beside it.

Figure	Caption	Section	View point	Owner	Source file
1	System architecture: modules, users and the information between them	3.1	Context	Ivan Zheng	uml/system/system_architecture.png
2	Software architecture, part 1: the central harness, shared utilities and the seven module processes	3.2	Composition	Braden Bosilovich	uml/system/system_classes_harness.png
3	Software architecture, part 2: module contracts, implementations, remote proxies and Pi cores	3.2	Composition	Braden Bosilovich	uml/system/system_classes_modules.png
4	Representative interaction: movement authority from the CTC Office to the train, tick by tick	3.5	Interaction	Braden Bosilovich	uml/system/system_seq_authority.png
5	Deployment: the host PC, its processes and data files, and the two Raspberry Pi 5 boards	3.6	Deployment	Braden Bosilovich	uml/system/system_deployment.png
6	Use cases of the Dispatcher	5.1.3	Context	Landis	uml/ctc_office/uc_dispatcher.png
7	Use cases of the Central Harness	5.1.3	Context	Landis	uml/ctc_office/uc_harness.png
8	Use cases of the Track Controller	5.1.3	Context	Landis	uml/ctc_office/uc_track_controller.png
9	Use cases of the Track Model	5.1.3	Context	Landis	uml/ctc_office/uc_track_model.png

Figure	Caption	Section	View point	Owner	Source file
10	CTC Office architecture (component view)	5.1.4	Composition	Landis	uml/ctc_office/architecture.png
11	Core classes: module and tracking	5.1.6	Logical	Landis	uml/ctc_office/class_core.png
12	Core classes: scheduling, routing and data	5.1.6	Logical	Landis	uml/ctc_office/class_services.png
13	CTC Office: boundary types	5.1.6	Logical	Landis	uml/ctc_office/class_boundary.png
14	Process and window classes	5.1.6	Logical	Landis	uml/ctc_office/class_host.png
15	SD-01 Control the simulation clock	5.1.7	Interaction	Landis	uml/ctc_office/sd01_clock.png
16	SD-02 Change operating mode	5.1.7	Interaction	Landis	uml/ctc_office/sd02_mode.png
17	SD-03 Dispatch a train to a station	5.1.7	Interaction	Landis	uml/ctc_office/sd03_dispatch_station.png
18	SD-04 Dispatch a train to a block	5.1.7	Interaction	Landis	uml/ctc_office/sd04_dispatch_block.png
19	SD-05 Reroute or cancel a train's order	5.1.7	Interaction	Landis	uml/ctc_office/sd05_reroute_cancel.png
20	SD-06 Return a train to the yard	5.1.7	Interaction	Landis	uml/ctc_office/sd06_return_to_yard.png
21	SD-07 Load a schedule	5.1.7	Interaction	Landis	uml/ctc_office/sd07_load_schedule.png
22	SD-08 Monitor and pause automatic dispatch	5.1.7	Interaction	Landis	uml/ctc_office/sd08_auto_dispatch.png
23	SD-09 Close or reopen a block	5.1.7	Interaction	Landis	uml/ctc_office/sd09_close_block.png

Figure	Caption	Section	View point	Owner	Source file
24	SD-10 Set a switch position	5.1.7	Interaction	Landis	uml/ctc_office/sd10_set_switch.png
25	SD-11 Monitor the track map	5.1.7	Interaction	Landis	uml/ctc_office/sd11_track_map.png
26	SD-12 Monitor train occupancy	5.1.7	Interaction	Landis	uml/ctc_office/sd12_occupancy.png
27	SD-13 View the selected train	5.1.7	Interaction	Landis	uml/ctc_office/sd13_selected_train.png
28	SD-14 View throughput metrics	5.1.7	Interaction	Landis	uml/ctc_office/sd14_throughput.png
29	SD-15 Handle notices and reversal alerts	5.1.7	Interaction	Landis	uml/ctc_office/sd15_notices_alerts.png
30	SD-16 Run one step	5.1.7	Interaction	Landis	uml/ctc_office/sd16_step.png
31	SD-17 Receive Track Controller reports	5.1.7	Interaction	Landis	uml/ctc_office/sd17_reports.png
32	SD-18 Follow trains from occupancy	5.1.7	Interaction	Landis	uml/ctc_office/sd18_tracking.png
33	SD-19 Run the schedule	5.1.7	Interaction	Landis	uml/ctc_office/sd19_schedule_run.png
34	SD-20 Release and retire trains	5.1.7	Interaction	Landis	uml/ctc_office/sd20_release_retire.png
35	SD-21 Send speed and authority suggestions	5.1.7	Interaction	Landis	uml/ctc_office/sd21_suggestions.png
36	SD-22 Send closures and switch commands	5.1.7	Interaction	Landis	uml/ctc_office/sd22_closures_switches.png
37	SD-23 Receive ticket sales	5.1.7	Interaction	Landis	uml/ctc_office/sd23_ticket_sales.png
38	SD-24 Hold a line that is not ready	5.1.7	Interaction	Landis	uml/ctc_office/sd24_line_not_ready.png

Figure	Caption	Section	Viewpoint	Owner	Source file
39	Track Controller (SW): use case diagram	5.2.3	Context	Harry Tye	uml/track_ctrl_sw/track_controller_use_cases.png
40	Track Controller (SW) architecture (component view)	5.2.4	Composition	Harry Tye	uml/track_ctrl_sw/architecture.png
41	Core classes	5.2.5	Logical	Harry Tye	uml/track_ctrl_sw/class_core.png
42	Track layout	5.2.5	Logical	Harry Tye	uml/track_ctrl_sw/class_layout.png
43	Track Controller (SW): boundary types	5.2.5	Logical	Harry Tye	uml/track_ctrl_sw/class_boundary.png
44	Host-side classes	5.2.5	Logical	Harry Tye	uml/track_ctrl_sw/class_host.png
45	SD-01 Load track layout (UC-01)	5.2.6	Interaction	Harry Tye	uml/track_ctrl_sw/sequence/sd01_load_track_layout.png
46	SD-02 Select wayside (UC-02)	5.2.6	Interaction	Harry Tye	uml/track_ctrl_sw/sequence/sd02_select_wayside.png
47	SD-03 Load PLC program (UC-03)	5.2.6	Interaction	Harry Tye	uml/track_ctrl_sw/sequence/sd03_load_plc_program.png
48	SD-04 Write PLC program (UC-04)	5.2.6	Interaction	Harry Tye	uml/track_ctrl_sw/sequence/sd04_write_plc_program.png
49	SD-05 Run PLC program (UC-05)	5.2.6	Interaction	Harry Tye	uml/track_ctrl_sw/sequence/sd05_run_plc_program.png
50	SD-06 Commit PLC program (UC-06)	5.2.6	Interaction	Harry Tye	uml/track_ctrl_sw/sequence/sd06_commit_plc_program.png

Figure	Caption	Section	View point	Owner	Source file
51	SD-07 Monitor wayside state (UC-07)	5.2.6	Interaction	Harry Tye	uml/track_ctrl_sw/sequence/sd07_monitor_wayside_state.png
52	SD-08 Hand-set switch (UC-08)	5.2.6	Interaction	Harry Tye	uml/track_ctrl_sw/sequence/sd08_hand_set_switch.png
53	SD-09 Run scan cycle (UC-09)	5.2.6	Interaction	Harry Tye	uml/track_ctrl_sw/sequence/sd09_run_scan_cycle.png
54	SD-10 Receive speed and authority suggestions (UC-10)	5.2.6	Interaction	Harry Tye	uml/track_ctrl_sw/sequence/sd10_receive_suggestions.png
55	SD-11 Receive closures and switch commands (UC-11)	5.2.6	Interaction	Harry Tye	uml/track_ctrl_sw/sequence/sd11_receive_closures_switch_commands.png
56	SD-12 Receive track state (UC-12)	5.2.6	Interaction	Harry Tye	uml/track_ctrl_sw/sequence/sd12_receive_track_state.png
57	SD-13 Command switches (UC-13)	5.2.6	Interaction	Harry Tye	uml/track_ctrl_sw/sequence/sd13_command_switches.png
58	SD-14 Set signal aspects (UC-14)	5.2.6	Interaction	Harry Tye	uml/track_ctrl_sw/sequence/sd14_set_signal_aspects.png
59	SD-15 Control railway crossings (UC-15)	5.2.6	Interaction	Harry Tye	uml/track_ctrl_sw/sequence/sd15_control_railway_crossings.png
60	SD-16 Send speed and authority on track circuits (UC-16)	5.2.6	Interaction	Harry Tye	uml/track_ctrl_sw/sequence/sd16_send_track_circuits.png

Figure	Caption	Section	View point	Owner	Source file
61	SD-17 Report occupancy, read-back and readiness (UC-17)	5.2.6	Interaction	Harry Tye	uml/track_ctrl_sw/sequence/sd17_report_to_ctc.png
62	SD-18 Transmit nothing while not ready (UC-18)	5.2.6	Interaction	Harry Tye	uml/track_ctrl_sw/sequence/sd18_transmit_nothing_while_not_ready.png
63	Track Controller (HW): use case diagram	5.3.3	Context	Braden Bosilovich	uml/track_ctrl_hw/uc_track_ctrl_hw.png
64	Track Controller (HW) architecture (component view)	5.3.4	Composition	Braden Bosilovich	uml/track_ctrl_hw/architecture.png
65	Core classes: control	5.3.5	Logical	Braden Bosilovich	uml/track_ctrl_hw/class_core_control.png
66	Core classes: PLC engine	5.3.5	Logical	Braden Bosilovich	uml/track_ctrl_hw/class_core_plc.png
67	Boundary types: inputs and outputs	5.3.5	Logical	Braden Bosilovich	uml/track_ctrl_hw/class_boundary_io.png
68	Boundary types: territory	5.3.5	Logical	Braden Bosilovich	uml/track_ctrl_hw/class_boundary_territory.png
69	Boundary types: snapshot	5.3.5	Logical	Braden Bosilovich	uml/track_ctrl_hw/class_boundary_snapshot.png
70	Host-side classes: presentation	5.3.5	Logical	Braden Bosilovich	uml/track_ctrl_hw/class_host_ui.png

Figure	Caption	Section	Viewpoint	Owner	Source file
71	Host-side classes: link to the Pi	5.3.5	Logical	Braden Bosilovich	uml/track_ctrl_hw/class_host_link.png
72	SD-01 Load wayside database (UC-01)	5.3.6	Interaction	Braden Bosilovich	uml/track_ctrl_hw/sd01_load_database.png
73	SD-02 Load PLC program (UC-02)	5.3.6	Interaction	Braden Bosilovich	uml/track_ctrl_hw/sd02_load_program.png
74	SD-03 Select wayside (UC-03)	5.3.6	Interaction	Braden Bosilovich	uml/track_ctrl_hw/sd03_select_wayside.png
75	SD-04 Monitor wayside state (UC-04)	5.3.6	Interaction	Braden Bosilovich	uml/track_ctrl_hw/sd04_monitor.png
76	SD-05 View PLC program details (UC-05)	5.3.6	Interaction	Braden Bosilovich	uml/track_ctrl_hw/sd05_plc_details.png
77	SD-06 View last report to the CTC Office (UC-06)	5.3.6	Interaction	Braden Bosilovich	uml/track_ctrl_hw/sd06_last_report.png
78	SD-07 Run scan cycle (UC-07)	5.3.6	Interaction	Braden Bosilovich	uml/track_ctrl_hw/sd07_run_scan_cycle.png
79	SD-08 Receive speed and authority suggestions (UC-08)	5.3.6	Interaction	Braden Bosilovich	uml/track_ctrl_hw/sd08_suggestions.png
80	SD-09 Receive closures and maintenance switch commands (UC-09)	5.3.6	Interaction	Braden Bosilovich	uml/track_ctrl_hw/sd09_closures_switch_commands.png

Figure	Caption	Section	View point	Owner	Source file
81	SD-10 Receive track state (UC-10)	5.3.6	Interaction	Braden Bosilovich	uml/track_ctrl_hw/sd10_track_state.png
82	SD-11 Command switches (UC-11)	5.3.6	Interaction	Braden Bosilovich	uml/track_ctrl_hw/sd11_command_switches.png
83	SD-12 Set signal aspects (UC-12)	5.3.6	Interaction	Braden Bosilovich	uml/track_ctrl_hw/sd12_signal_aspects.png
84	SD-13 Control railway crossings (UC-13)	5.3.6	Interaction	Braden Bosilovich	uml/track_ctrl_hw/sd13_crossings.png
85	SD-14 Send speed and authority on track circuits (UC-14)	5.3.6	Interaction	Braden Bosilovich	uml/track_ctrl_hw/sd14_track_circuits.png
86	SD-15 Report occupancy, state and readiness (UC-15)	5.3.6	Interaction	Braden Bosilovich	uml/track_ctrl_hw/sd15_report.png
87	SD-16 Hold fail-safe outputs (UC-16)	5.3.6	Interaction	Braden Bosilovich	uml/track_ctrl_hw/sd16_hold_fail_safe.png
88	Use case diagram: Track Model, all actors	5.4.2	Context	Deni Ephraim	uml/track_model/03_usecase.png
89	Use cases of the Track Builder	5.4.2	Context	Deni Ephraim	uml/track_model/uc_track_builder.png
90	Use cases of Murphy	5.4.2	Context	Deni Ephraim	uml/track_model/uc_murphy.png
91	Use cases of the Track Controller	5.4.2	Context	Deni Ephraim	uml/track_model/uc_track_controller.png
92	Use cases of the Train Model	5.4.2	Context	Deni Ephraim	uml/track_model/uc_train_model.png

Figure	Caption	Section	Viewpoint	Owner	Source file
93	Use cases of the Train Controller	5.4.2	Context	Deni Ephraim	uml/track_model/uc_train_controller.png
94	Use cases of the CTC Office	5.4.2	Context	Deni Ephraim	uml/track_model/uc_ctc_office.png
95	Track Model architecture (component view)	5.4.3	Composition	Deni Ephraim	uml/track_model/06_component.png
96	Boundary types of the Track Model (track_model/interface.py)	5.4.4	Logical	Deni Ephraim	uml/track_model/16_types.png
97	Class diagram 1: module core	5.4.5	Logical	Deni Ephraim	uml/track_model/04_class_core.png
98	Class diagram 2: window and test link	5.4.5	Logical	Deni Ephraim	uml/track_model/05_class_ui.png
99	The TrackModel Protocol and the harness edge functions	5.4.6	Interface	Deni Ephraim	uml/track_model/17_protocols.png
100	SD-01 Load a track layout (UC-1)	5.4.12	Interaction	Deni Ephraim	uml/track_model/sd01_load.png
101	SD-02 Display track properties (UC-2)	5.4.12	Interaction	Deni Ephraim	uml/track_model/sd02_display.png
102	SD-03 Track train position and occupancy (UC-3)	5.4.12	Interaction	Deni Ephraim	uml/track_model/sd03_position.png
103	SD-04 Report state to the Track Controller (UC-4)	5.4.12	Interaction	Deni Ephraim	uml/track_model/sd04_report.png
104	SD-05 Apply a switch position (UC-5)	5.4.12	Interaction	Deni Ephraim	uml/track_model/sd05_switch.png
105	SD-06 Apply signal lights and crossing gates (UC-6)	5.4.12	Interaction	Deni Ephraim	uml/track_model/sd06_signal.png

Figure	Caption	Section	Viewpoint	Owner	Source file
106	SD-07 Deliver the track circuit signal (UC-7)	5.4.12	Interaction	Deni Ephraim	uml/track_model/sd07_circuit.png
107	SD-08 Send beacon data (UC-8)	5.4.12	Interaction	Deni Ephraim	uml/track_model/sd08_beacon.png
108	SD-09 Send the signal light to the Train Controller (UC-9)	5.4.12	Interaction	Deni Ephraim	uml/track_model/sd09_signalseen.png
109	SD-10 Sell tickets and board passengers (UC-10)	5.4.12	Interaction	Deni Ephraim	uml/track_model/sd10_boarding.png
110	SD-11 Report ticket sales (UC-11)	5.4.12	Interaction	Deni Ephraim	uml/track_model/sd11_tickets.png
111	SD-12 Set the ambient temperature (UC-12)	5.4.12	Interaction	Deni Ephraim	uml/track_model/sd12_temperature.png
112	SD-13 Switch the track heaters (UC-13)	5.4.12	Interaction	Deni Ephraim	uml/track_model/sd13_heaters.png
113	SD-14 Inject or clear a failure (UC-14)	5.4.12	Interaction	Deni Ephraim	uml/track_model/sd14_failure.png
114	SD-15 One tick in the Track Model (overview)	5.4.12	Interaction	Deni Ephraim	uml/track_model/sd15_tick.png
115	SD-16 Dispatch a train (system level)	5.4.12	Interaction	Deni Ephraim	uml/track_model/sd16_dispatch.png
116	Block state machine	5.4.13	State dynamics	Deni Ephraim	uml/track_model/13_state_block.png
117	Train Model use case diagram	5.5.4	Context	Kevin Schillinger	uml/train-model-uml(2)(1)-png/01_use-case.png
118	Train Model: use cases by role	5.5.4	Context	Kevin Schillinger	uml/train-model-uml(2)(1)-png/02_use-cases-by-role.png

Figure	Caption	Section	View point	Owner	Source file
119	Train Model: use case descriptions	5.5.4	Context	Kevin Schillinger	uml/train-model-uml(2)(1)-png/03_use-case-descriptions.png
120	Sequence diagram: validate step inputs	5.5.4	Interaction	Kevin Schillinger	uml/train-model-uml(2)(1)-png/15_sequence-validate-step-inputs.png
121	Sequence diagram: compute train motion	5.5.4	Interaction	Kevin Schillinger	uml/train-model-uml(2)(1)-png/16_sequence-compute-train-motion.png
122	Sequence diagram: update passengers	5.5.4	Interaction	Kevin Schillinger	uml/train-model-uml(2)(1)-png/17_sequence-update-passengers.png
123	Sequence diagram: detect block change	5.5.4	Interaction	Kevin Schillinger	uml/train-model-uml(2)(1)-png/18_sequence-detect-block-change.png
124	Sequence diagram: update cabin temperature	5.5.4	Interaction	Kevin Schillinger	uml/train-model-uml(2)(1)-png/19_sequence-update-cabin-temperature.png
125	Sequence diagram: publish outputs	5.5.4	Interaction	Kevin Schillinger	uml/train-model-uml(2)(1)-png/20_sequence-publish-outputs.png
126	Sequence diagram: view train status	5.5.4	Interaction	Kevin Schillinger	uml/train-model-uml(2)(1)-png/06_sequence-view-train-status.png
127	Sequence diagram: view announcement	5.5.4	Interaction	Kevin Schillinger	uml/train-model-uml(2)(1)-png/07_sequence-view-announcement.png
128	Sequence diagram: apply controller commands	5.5.4	Interaction	Kevin Schillinger	uml/train-model-uml(2)(1)-png/12_sequence-apply-controller-commands.png

Figure	Caption	Section	View point	Owner	Source file
129	Sequence diagram: receive track data	5.5.4	Interaction	Kevin Schillinger	uml/train-model-uml(2)(1)-png/13_sequence-receive-track-data.png
130	Sequence diagram: board and disembark passengers	5.5.4	Interaction	Kevin Schillinger	uml/train-model-uml(2)(1)-png/09_sequence-board-and-disembark-passengers.png
131	Sequence diagram: pull passenger emergency brake	5.5.4	Interaction	Kevin Schillinger	uml/train-model-uml(2)(1)-png/08_sequence-pull-passenger-emergency-brake.png
132	Sequence diagram: inject clear failure	5.5.4	Interaction	Kevin Schillinger	uml/train-model-uml(2)(1)-png/10_sequence-inject-clear-failure.png
133	Train Model class diagram	5.5.6	Logical	Kevin Schillinger	uml/train-model-uml(2)(1)-png/04_class.png
134	One accepted or rejected Train Model tick	5.5.8	Interaction	Kevin Schillinger	uml/train-model-uml(2)(1)-png/05_sequence-advance-one-simulation-step.png
135	Sequence diagram: report state to train controller	5.5.8	Interaction	Kevin Schillinger	uml/train-model-uml(2)(1)-png/11_sequence-report-state-to-train-controller.png
136	Sequence diagram: report position to track model	5.5.8	Interaction	Kevin Schillinger	uml/train-model-uml(2)(1)-png/14_sequence-report-position-to-track-model.png
137	Train Controller (SW and HW): use case diagram	5.6.4	Context	Jonathan Tsang, Ivan Zheng	sdd-figures/train-controller/03-use-case.png

Figure	Caption	Section	View point	Owner	Source file
138	Sequence diagram: Set target speed (Manual)	5.6.4	Interaction	Jonathan Tsang, Ivan Zheng	sdd-figures/train-controller/07-seq-01-set-target-speed.png
139	Sequence diagram: Switch Manual / Automatic	5.6.4	Interaction	Jonathan Tsang, Ivan Zheng	sdd-figures/train-controller/07-seq-02-switch-mode.png
140	Sequence diagram: Apply service brake	5.6.4	Interaction	Jonathan Tsang, Ivan Zheng	sdd-figures/train-controller/07-seq-03-service-brake.png
141	Sequence diagram: Open or close doors	5.6.4	Interaction	Jonathan Tsang, Ivan Zheng	sdd-figures/train-controller/07-seq-04-doors.png
142	Sequence diagram: Turn lights on or off	5.6.4	Interaction	Jonathan Tsang, Ivan Zheng	sdd-figures/train-controller/07-seq-05-lights.png
143	Sequence diagram: Set cabin temperature	5.6.4	Interaction	Jonathan Tsang, Ivan Zheng	sdd-figures/train-controller/07-seq-06-cabin-temperature.png
144	Sequence diagram: Announce station	5.6.4	Interaction	Jonathan Tsang, Ivan Zheng	sdd-figures/train-controller/07-seq-07-announce.png
145	Sequence diagram: Engage e-brake	5.6.4	Interaction	Jonathan Tsang, Ivan Zheng	sdd-figures/train-controller/07-seq-09-engage-ebrake.png

Figure	Caption	Section	View point	Owner	Source file
146	Sequence diagram: Release e-brake	5.6.4	Interaction	Jonathan Tsang, Ivan Zheng	sdd-figures/train-controller/07-seq-10-release-ebrake.png
147	Sequence diagram: Set Kp and Ki	5.6.4	Interaction	Jonathan Tsang, Ivan Zheng	sdd-figures/train-controller/07-seq-08-set-gains.png
148	Train Controller (SW and HW) architecture (component view)	5.6.5	Composition	Jonathan Tsang, Ivan Zheng	sdd-figures/train-controller/02-software-architecture.png
149	Class diagram: controller core	5.6.6	Logical	Jonathan Tsang, Ivan Zheng	sdd-figures/train-controller/04-class-controller-core.png
150	Class diagram: State and exceptions	5.6.6	Logical	Jonathan Tsang, Ivan Zheng	sdd-figures/train-controller/05-class-state-exceptions.png
151	Class diagram: console	5.6.6	Logical	Jonathan Tsang, Ivan Zheng	sdd-figures/train-controller/06-class-console.png
152	Sequence diagram: one 0.1 s tick	5.6.8	Interaction	Jonathan Tsang, Ivan Zheng	sdd-figures/train-controller/07-seq-00-one-tick.png

6.2 Requirements Traceability Index

Each module traces its requirements in its own chapter.

Module	Traceability
CTC Office	Section 5.1.2
Track Controller (SW)	Section 5.2.2
Track Controller (HW)	Section 5.3.2
Track Model	Section 5.4.15
Train Model	Section 5.5.11
Train Controller (SW and HW)	No table in the module's contribution; its rules M0 to M20 are traced in `Train-Ctrl_HW/REQUIREMENTS.md`

6.3 Module Figures Replaced by System Figures

These module figures showed the whole system. Each system concept is drawn once, in Section 3, so these are not reproduced; they stay in their owners' folders.

Module figure	Replaced by
`uml/track_model/01_architecture.png`	3.1 System context (system architecture)
`uml/track_model/15_composition.png`	3.2 Software architecture
`uml/track_model/18_seq_authority.png`	3.5 Representative interaction
`uml/track_model/14_deployment.png`	3.6 Deployment
`sdd-figures/train-controller/01-system-architecture.png`	3.1 System context (system architecture)
`uml/ctc_office/deployment.png`	3.6 Deployment
`uml/track_ctrl_sw/deployment.png`	3.6 Deployment
`uml/train-model-uml(2)(1)-png/21_deployment.png`	3.6 Deployment

End of document.